

بازیابی اطلاعات

دانشگاه آزاد شیراز • دانشکده مهندسی کامپیوتر

ترم دوم • سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل نوزدهم: Web Search Basics

استاد درس

دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی)

امیرمحمد اسدجو

علی نیکوان

علی مجاهد

امیرحسین همتی

حمید نامجو

فهرست مطالب

19.1 Background and history

19.2 Web characteristics

19.2.1 The web graph

19.2.2 Spam

19.3 Advertising as the economic model

19.4 The search user experience

19.4.1 User query needs

19.5 Index size and estimation

19.6 Near-duplicates and shingling

19.7 Chapter 19 Summary

19.8 End of Chapter Exercises

وب جهانی (Web) از سه جهت بی‌سابقه است: (۱) مقیاس عظیم، (۲) عدم هماهنگی در ایجاد محتوا، و (۳) تنوع بی‌نظیر نویسندگان (از افراد عادی تا سازمان‌های بزرگ).

برای درک این موضوع، تصور کنید یک کتابخانه‌ی سنتی را با یک بازار بزرگ و شلوغ مقایسه می‌کنید. در کتابخانه، همه چیز سازمان‌یافته، طبقه‌بندی شده و توسط متخصصان مدیریت می‌شود. اما در بازار، هر فروشنده (وب‌سایت) هر چیزی که بخواهد را به هر شکلی که بخواهد ارائه می‌دهد، بدون هماهنگی با دیگران. این دقیقاً همان چیزی است که وب را از مجموعه‌های سنتی اطلاعات متمایز می‌کند و بازیابی اطلاعات در آن را چالش‌برانگیزتر می‌سازد.

🏗️ معماری ساده‌ی وب: چگونه سادگی منجر به پیچیدگی شد؟

طراحی ساده‌ی وب – متشکل از پروتکل HTTP و زبان نشانه‌گذاری HTML – به‌صورت عمدی غیرسخت‌گیرانه طراحی شد. این سادگی، مانند قوانین ساده‌ی یک زبان جدید بود که به هر کسی اجازه می‌داد بدون نیاز به آموزش فنی، با مشاهده‌ی کد صفحات دیگر، صفحات خود را بسازد.

مثلاً وقتی Tim Berners-Lee اولین مرورگر وب را طراحی کرد، ویژگی "View Source" را اضافه کرد که به کاربران اجازه می‌داد کد HTML صفحات را ببینند. این ویژگی ساده، مانند این بود که یک آشپز حرفه‌ای اجازه دهد هر کسی دستور پخت غذاهایش را ببیند و از آن الگو بگیرد. در نتیجه، انتشار وب به یک فعالیت جمعی تبدیل شد – نه انحصار برنامه‌نویسان حرفه‌ای.

ویژگی کلیدی دیگر این بود که مرورگرها هر چیزی را که نمی‌فهمند، نادیده می‌گرفتند. این مانند این بود که یک قاضی به جای رد کردن یک سند به خاطر یک خطای جزئی، صرفاً آن بخش را نادیده بگیرد و به بقیه سند توجه کند. این رویکرد به کاربران اجازه داد بدون ترس از "خراب کردن" سیستم، آزمایش و خطا کنند.

🔍 جستجو در وب: دو رویکرد اولیه

با انفجار محتوای وب، یک چالش جدید ظهور کرد: چگونه این اقیانوس اطلاعات را قابل کشف کنیم؟ دو رویکرد اولیه برای این هدف عبارت بودند از:

1. **موتورهای جستجوی مبتنی بر فهرست معکوس** (مثل AltaVista): این رویکرد مانند یک کتابدار بود که تمام کلمات تمام کتاب‌ها را فهرست می‌کند تا بتواند سریعاً کتاب‌های مرتبط با هر کلمه را پیدا کند.
2. **دسته‌بندی‌های دستی** (مثل Yahoo!): این رویکرد مانند یک فروشگاه بزرگ بود که محصولات را در قفسه‌های مختلف طبقه‌بندی می‌کند تا مشتریان بتوانند با گشت و گذار در قفسه‌ها، چیزی که نیاز دارند را پیدا کنند.

🏠 چالش‌های دسته‌بندی دستی: داستان Yahoo!

هرچند دسته‌بندی‌های دستی در ظاهر شهودی به‌نظر می‌رسیدند، اما دو مشکل اساسی داشتند:

- **عدم مقیاس‌پذیری:** تصور کنید بخواهید تمام کتاب‌های یک کتابخانه‌ی ملی را دستی طبقه‌بندی کنید. حالا این کتابخانه هر روز هزاران کتاب جدید دریافت کند! این دقیقاً چالشی بود که Yahoo! با آن روبه‌رو بود. با رشد سریع وب، دسته‌بندی دستی میلیون‌ها صفحه وب، نیازمند نیروی انسانی غیرعملی بود.
- **اختلاف در درک معنایی:** تصور کنید دنبال اطلاعات در مورد "سگ‌های نگهبان" هستید. آیا باید به بخش "حیوانات خانگی" بروید یا "امنیت و حفاظت"؟ این مشکل در Yahoo! وجود داشت که کاربر و ویراستار ممکن است

تفسیرهای متفاوتی از جایگاه یک موضوع در درخت دسته‌بندی داشته باشند. در واقع، درخت دسته‌بندی Yahoo! به سرعت به بیش از 1000 گره رسید و پیدا کردن مسیر مناسب برای کاربران دشوار شد.

با وجود این چالش‌ها، برخی پروژه‌ها مانند Open Directory Project تلاش کردند با استفاده از داوطلبان متخصص، این رویکرد را زنده نگه دارند، اما محبوبیت کلی دسته‌بندی‌های دستی کاهش یافت.

🚀 نسل اول موتورهای جستجو: غول‌های فناوری در حال شکل‌گیری

در مقابل، موتورهای جستجوی نسل اول مانند AltaVista، Excite و Infoseek با مقیاس‌های بی‌سابقه (ده‌ها میلیون سند) روبه‌رو شدند. این موتورها مانند کارخانه‌های عظیمی بودند که باید میلیون‌ها محصول را هر روز پردازش می‌کردند. آن‌ها با ترکیب صدها ماشین، سیستم‌هایی با **زمان پاسخ زیر ثانیه** ساختند – یک دستاورد فنی چشمگیر در آن زمان.

با این حال، کیفیت نتایج پایین ماند. دلیل آن: محتوای وب توسط هر کسی و با هر انگیزه‌ای (از جمله اسپم و محتوای بی‌کیفیت) ایجاد می‌شود. این مانند این بود که در یک کتابخانه، هر کسی بتواند هر چیزی را بنویسد و در قفسه‌ها قرار دهد، از مقالات علمی گرفته تا تبلیغات و اطلاعات نادرست.

🔗 فراتر از مرتبط بودن: نیاز به سنجش اعتبار

بنابراین، روش‌های کلاسیک بازیابی (مثل شباهت کسینوس) دیگر کافی نبودند. برای جبران این کاستی، نیاز به معیارهایی فراتر از «مرتبط‌بودن» احساس شد – معیارهایی که **اعتبار یک سند** را بر اساس منبع آن (مثلاً دامنه میزبان) ارزیابی کنند.

برای مثال، وقتی شما در مورد "سرطان" جستجو می‌کنید، آیا می‌خواهید یک مقاله از سایت Mayo Clinic یا یک وبلاگ شخصی ناشناس را در نتایج اول ببینید؟ این ایده، پایه‌های **الگوریتم‌هایی مانند PageRank** (فصل ۲۱) را فراهم کرد که نه تنها مرتبط بودن، بلکه اعتبار و اقتدار منابع اطلاعاتی را نیز در نظر می‌گیرند.

این تحول، نقطه عطفی در تاریخ جستجوی وب بود که موتورهای جستجو را از صرفاً "پیداکننده‌های کلمات کلیدی" به "ارزیاب‌های کیفیت اطلاعات" تبدیل کرد و راه را برای ظهور گوگل به عنوان غالب‌ترین موتور جستجو هموار کرد.

19.2 ویژگی‌های ذاتی وب: چالش‌های پیش روی موتورهای جستجو 🌐

🌐 جهانی بدون مرز: چالش زبانی و ساختاری

ویژگی‌ای که وب را به شدت گسترش داد – **عدم وجود کنترل مرکزی بر ایجاد محتوا** – همان ویژگی‌ای بود که بزرگ‌ترین چالش را برای موتورهای جستجو ایجاد کرد. نویسندگان وب از ده‌ها زبان طبیعی و هزاران لهجه استفاده می‌کنند، که نیازمند **روش‌های متنوع ریشه‌یابی** و پردازش زبانی است.

برای درک این چالش، تصور کنید موتور جستجوی گوگل باید بتواند همزمان با یک مقاله علمی انگلیسی از سایت Nature، یک وبلاگ فارسی درباره آشپزی، یک صفحه تجاری چینی برای فروش الکترونیک، و یک پست توییت عربی با املا و قواعد غیراستاندارد کار کند. این فقط چند نمونه از تنوع زبانی است که موتورهای جستجو با آن روبرو هستند.

🎨 هرج و مرج بصری: از طراحی حرفه‌ای تا وب‌سایت‌های شخصی

انتشار آزادانه محتوا منجر به **هتروژنی شدید** در جنبه‌های کلیدی شد:

- **سبک و دستور زبان:** بسیاری از صفحات فاقد ساختار زبانی شناخته‌شده هستند.
- **قالب‌بندی بصری:** ترکیب‌های بی‌رویه رنگ، فونت و طرح‌بندی که خوانایی را کاهش می‌دهد.
- **محتوای غیرمتمنی:** صفحاتی که کاملاً از تصویر ساخته شده‌اند (بدون متن قابل ایندکس).

برای مثال، صفحه اصلی شرکت اپل در سال ۱۹۹۷ تقریباً کاملاً از تصاویر تشکیل شده بود که برای موتورهای جستجوی آن زمان مانند AltaVista یک چالش بزرگ بود، زیرا هیچ متن قابل ایندکسی برای درک محتوای صفحه وجود نداشت. در مقابل، یک وبلاگ شخصی ممکن است حاوی متن فراوانی باشد اما با ساختار و دستور زبان بسیار ضعیف که درک آن برای الگوریتم‌ها دشوار است.

🔍 حقیقت در مقابل دروغ: چالش اعتماد در وب

اما مشکل عمیق‌تر، **کیفیت محتوا** است: وب هم‌زمان حاوی **حقیقت، دروغ، تناقض و حدس** است. این امر سؤال کلیدی را مطرح می‌کند: **به کدام صفحات باید اعتماد کرد؟**

برای مثال، اگر جستجو کنید "آیا واکسن کووید بی‌خطر است؟"، ممکن است با نتایج متناقضی روبرو شوید: از مقالات علمی سایت CDC و WHO گرفته تا پست‌های وبلاگ‌هایی که ادعاهای ضدواکسین را ترویج می‌کنند. موتور جستجو باید بتواند بین این منابع تمایز قائل شود و نتایج معتبرتر را در اولویت قرار دهد.

در نشر سنتی، کاربران منابع مورد اعتماد خود را **انتخاب می‌کنند** (مثل The New York Times یا The Wall Street Journal). اما در وب، کاربر بیشتر اوقات **از طریق موتور جستجو با منبع آشنا می‌شود** – بنابراین، مسئولیت انتخاب منبع معتبر به‌طور غیرمستقیم به موتور جستجو واگذار می‌شود. علاوه بر این، «اعتماد» ممکن است **وابسته به کاربر** باشد: صفحه‌ای برای یک فرد قابل اعتماد است، برای دیگری نه.

🌐 صفحات ایستا در مقابل پویا: چالش مقیاس‌پذیری

موتورهای جستجو همچنین با چالش **مقیاس وب** روبه‌رو هستند. در سال ۱۹۹۵، AltaVista ادعا کرد که حدود **۳۰ میلیون صفحه ایستا** را ایندکس کرده است. صفحه **ایستا** (Static) به صفحه‌ای گفته می‌شود که محتوای آن بین درخواست‌های متوالی تغییر نمی‌کند – حتی اگر هفته‌ای یک‌بار به‌صورت دستی به‌روزرسانی شود. در مقابل، صفحات **پویا** (Dynamic) – مثل وضعیت پرواز فرودگاه – به‌صورت زنده از پایگاه داده تولید می‌شوند و معمولاً در URL آن‌ها کاراکتر `?` دیده می‌شود (مثل `flight?num=AA129/`).

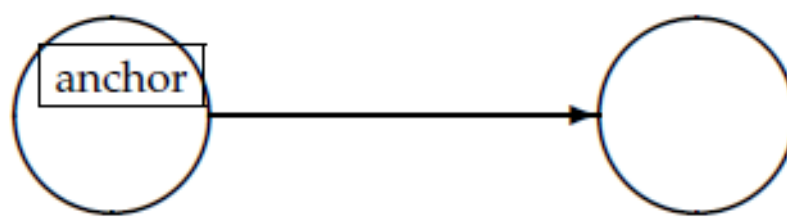
این تقسیم‌بندی برای موتورهای جستجو اهمیت دارد: صفحات پویا دامنه‌ی بی‌پایانی از نتایج ممکن دارند که **اندکس کردن آن‌ها نه تنها غیرعملی، بلکه گمراه‌کننده** است (چون هر درخواست خروجی متفاوتی تولید می‌کند). بنابراین، موتورهای جستجو معمولاً **صفحات ایستا را ترجیح می‌دهند** – یا حداقل، مکانیزم‌هایی برای فیلتر کردن صفحات پویای بی‌معنی اعمال می‌کنند.

برای مثال، سایت آمازون میلیون‌ها صفحه پویا دارد که هر کدام ممکن است با تغییر یک پارامتر در URL ایجاد شوند (مثل صفحه محصولات با فیلترهای مختلف قیمت یا رنگ). این در حالی است که یک صفحه شخصی دانشگاهی که هر چند ماه یک‌بار به‌روز می‌شود، یک صفحه ایستا محسوب می‌شود.



► **Figure 19.1** A dynamically generated web page. The browser sends a request for flight information on flight AA129 to the web application, that fetches the information from back-end databases then creates a dynamic web page that it returns to the browser.

شکل 19.1: یک صفحه وب پویا. مرورگر درخواستی برای اطلاعات پرواز AA129 به برنامه وب ارسال می‌کند، برنامه اطلاعات را از پایگاه‌های داده پشتیبان دریافت کرده و یک صفحه وب پویا ایجاد می‌کند که به مرورگر بازمی‌گرداند.



► **Figure 19.2** Two nodes of the web graph joined by a link.

شکل 19.2: دو گره از گراف وب که توسط یک پیوند به هم متصل شده‌اند.

🚀 نتیجه‌گیری: وب به عنوان یک اکوسیستم پیچیده

وب جهانی به دلیل ماهیت غیرمتمرکز و بدون کنترل خود، مجموعه‌ای بی‌نظیر از چالش‌ها را برای موتورهای جستجو ایجاد کرده است. از تنوع زبانی و ساختاری گرفته تا مسائل مربوط به اعتماد و قابلیت اطمینان، همگی نیازمند راه‌حل‌های نوآورانه‌ای هستند که فراتر از تکنیک‌های سنتی بازیابی اطلاعات عمل کنند.

در فصل‌های بعدی، خواهیم دید که موتورهای جستجوی مدرن چگونه با استفاده از الگوریتم‌های پیچیده مانند PageRank و تکنیک‌های یادگیری ماشین، این چالش‌ها را برطرف کرده و توانسته‌اند در این اکوسیستم پیچیده، اطلاعات مرتبط و معتبر را به کاربران ارائه دهند.

🔗 19.2.1 گراف وب: شبکه‌ای از ارتباطات

🌐 وب به عنوان یک گراف جهت‌دار

وب ایستا را می‌توان به عنوان یک گراف جهت‌دار در نظر گرفت:

- **گره‌ها (Nodes):** صفحات HTML.
- **یال‌های جهت‌دار (Directed Edges):** لینک‌های هائپر متنی (hyperlinks).

برای درک بهتر، تصور کنید هر صفحه وب مانند یک ایستگاه در مترو و هر لینک مانند یک خط قطار است که شما را از یک ایستگاه به ایستگاه دیگر منتقل می‌کند. این شبکه‌ی عظیم از ایستگاه‌ها و خطوط، گراف وب را تشکیل می‌دهد.

هر لینک از صفحه A به صفحه B ، یک **out-link** برای A و یک **in-link** برای B محسوب می‌شود. **درجه ورودی** (in-degree) یک صفحه = تعداد لینک‌های ورودی به آن. **درجه خروجی** (out-degree) = تعداد لینک‌های خروجی از آن. مطالعات نشان می‌دهند میانگین درجه ورودی بین ۸ تا ۱۵ است.

برای مثال، صفحه اصلی گوگل میلیون‌ها لینک ورودی از سایت‌های مختلف دارد (in-degree بسیار بالا)، در حالی که وبلاگ شخصی یک دانشجو ممکن است تنها چند لینک ورودی از دوستان و همکلاسی‌هایش داشته باشد (in-degree پایین).

🇮🇷 قانون توانی: عدم توازن در محبوبیت

توزیع درجه ورودی از یک **قانون توانی** (Power Law) پیروی می‌کند:

$$i \propto 1 / i^{\alpha} \quad (\alpha \approx 2.1)$$

تعداد صفحات با درجه ورودی

این یعنی تعداد کمی از صفحات (مثل Google یا Wikipedia) **درجه ورودی بسیار بالا** دارند، در حالی که اکثر صفحات تنها چند لینک ورودی دارند.

این پدیده را می‌توان در دنیای واقعی نیز مشاهده کرد: تعداد کمی از سلبریتی‌ها میلیون‌ها دنبال‌کننده دارند، در حالی که اکثر افراد در شبکه‌های اجتماعی تنها چند صد یا چند هزار دنبال‌کننده دارند. این عدم توازن در وب نیز به همین شکل وجود دارد.

🎀 ساختار پاپیونی وب: چگونه اطلاعات جریان می‌یابد؟

ساختار کلی گراف وب به شکل **Bowtie** (پاپیون) است و سه مؤلفه اصلی دارد:

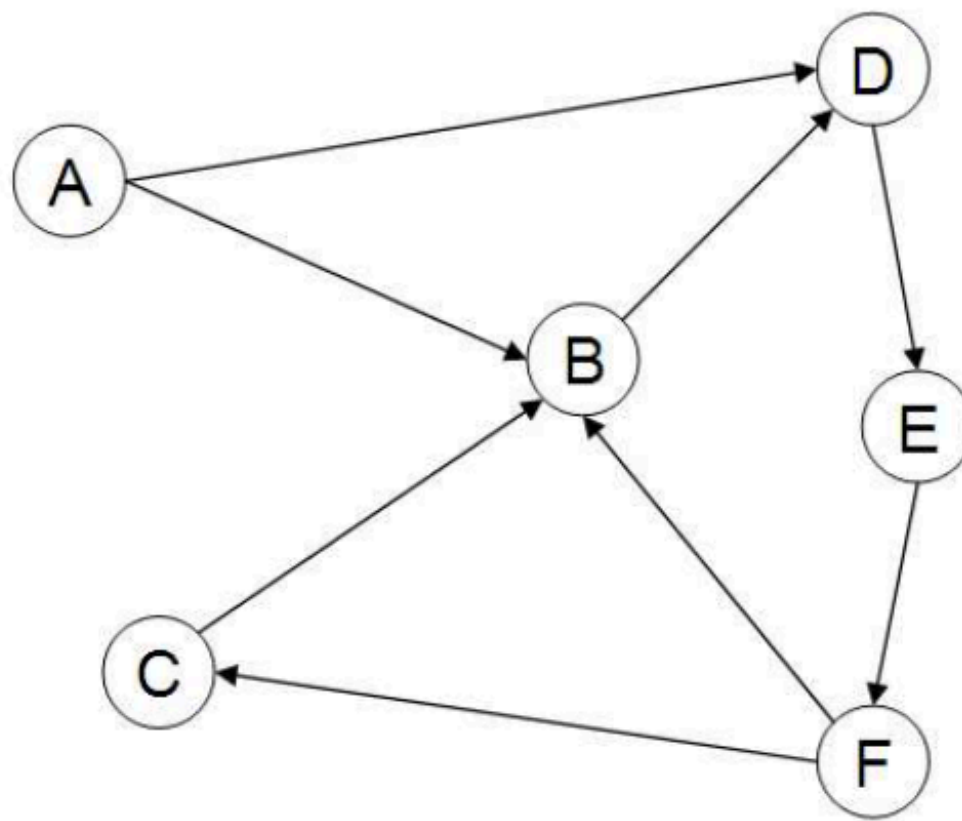
1. **SCC** (Strongly Connected Component): هر صفحه در این بخش به هر صفحه دیگر در همین بخش قابل دسترسی است. این مانند مرکز یک شهر بزرگ است که می‌توانید از هر نقطه‌ای به هر نقطه‌ی دیگر در آن حرکت کنید.

2. **IN**: صفحاتی که فقط به SCC لینک دارند (اما از آن‌ها خروجی به بیرون نیست). این مانند حومه‌های شهری است که فقط مسیرهای ورودی به مرکز شهر دارند.

3. **OUT**: صفحاتی که فقط از SCC لینک دریافت می‌کنند (اما به بیرون لینک نمی‌دهند). این مانند خروجی‌های شهر است که فقط مسیرهای خروجی از مرکز شهر دارند.

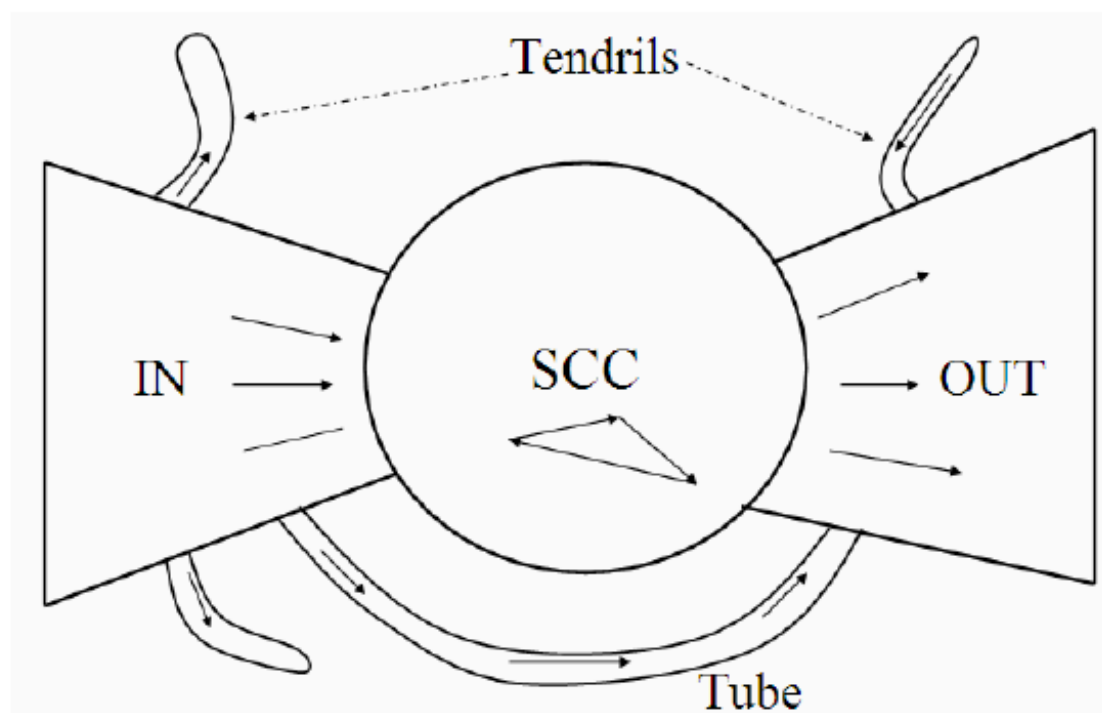
علاوه بر این، بخش‌های کوچکی به نام **tubes** (لوله‌ها) و **tendrils** (رشته‌ها) وجود دارند که به ترتیب مسیرهای مستقیم از IN به OUT و شاخه‌های بن‌بست هستند.

این ساختار نشان می‌دهد که چرا برخی صفحات وب بسیار محبوب‌تر از دیگران هستند: صفحات در SCC مرکز توجه هستند و از هر دو طرف (IN و OUT) ترافیک دریافت می‌کنند، در حالی که صفحات در بخش‌های دیگر ممکن است ترافیک کمتری دریافت کنند.



► **Figure 19.3** A sample small web graph. In this example we have six pages labeled A-F. Page B has in-degree 3 and out-degree 1. This example graph is not strongly connected: there is no path from any of pages B-F to page A.

شکل 19.3: یک نمونه گراف وب کوچک. در این مثال ما شش صفحه با برچسب A-F داریم. صفحه B دارای درجه ورودی 3 و درجه خروجی 1 است. این گراف نمونه قویاً متصل نیست: هیچ مسیری از هیچ یک از صفحات B-F به صفحه A وجود ندارد.



► **Figure 19.4** The bowtie structure of the Web. Here we show one tube and three tendrils.

شکل 19.4: ساختار پایبونی وب. در اینجا یک لوله و سه رشته را نشان می‌دهیم.

متن لنگر (Anchor Text): پلی بین صفحات

یکی از جنبه‌های مهم گراف وب که در شکل 19.2 نیز نمایش داده شده، **متن لنگر** است. این متنی است که در صفحه مبدأ (A) در اطراف لینک به صفحه مقصد (B) قرار دارد و معمولاً در کد HTML در ویژگی href تگ قرار می‌گیرد.

متن لنگر برای موتورهای جستجو بسیار ارزشمند است، زیرا اغلب توصیفی دقیق از محتوای صفحه مقصد ارائه می‌دهد. برای مثال، اگر بسیاری از سایت‌ها با متن لنگر "بهترین رستوران‌های تهران" به صفحه‌ی شما لینک دهند، موتورهای جستجو احتمالاً صفحه شما را برای این عبارت رتبه‌بندی خواهند کرد.

این ویژگی به شرکت‌هایی مانند گوگل کمک کرده تا الگوریتم‌های بسیار دقیقی برای رتبه‌بندی صفحات توسعه دهندند، زیرا متن لنگر اغلب یک "رأی اعتماد" از طرف صفحه مبدأ به صفحه مقصد محسوب می‌شود.

```
In [1]: import matplotlib.pyplot as plt
import networkx as nx
import numpy as np
import sys

# بازگشتی DFS افزایش محدودیت بازگشت برای پیاده‌سازی
sys.setrecursionlimit(10000)

class WebGraph:
    def __init__(self):
        """
        نمایش گراف وب با دو مدل نگاشت برای دسترسی سریع
        - out_links: لیست صفحات مقصد (لینک‌های خروجی)
        - in_links: لیست صفحات مبدأ (لینک‌های ورودی)
        این ساختار بهینه برای محاسبات درجه و تحلیل ساختاری است.
        """
        self.out_links = {}
        self.in_links = {}

    def add_page(self, page):
        """افزودن یک صفحه جدید (گره) به گراف بدون لینک اولیه"""
        if page not in self.out_links:
            self.out_links[page] = []
            self.in_links[page] = []

    def add_link(self, from_page, to_page):
        """
        افزودن یک لینک جهت‌دار از from_page به to_page
        است to_page برای in-link و from_page برای out-link این یک
        لینک‌های تکراری اضافه نمی‌شوند.
        """
        self.add_page(from_page)
        self.add_page(to_page)
        if to_page not in self.out_links[from_page]:
            self.out_links[from_page].append(to_page)
            self.in_links[to_page].append(from_page)

    def in_degree(self, page):
        """یک صفحه (in-degree) بازگرداندن تعداد لینک‌های ورودی"""
        return len(self.in_links.get(page, []))

    def out_degree(self, page):
        """یک صفحه (out-degree) بازگرداندن تعداد لینک‌های خروجی"""
        return len(self.out_links.get(page, []))

    def analyze_degrees(self):
        """
        تحلیل توزیع درجه‌های ورودی و خروجی با محاسبه مقادیر آماری
        این توزیع‌ها نشان‌دهنده ساختار قدرت و محبوبیت در وب هستند.
        """
        if not self.out_links:
            print("گراف خالی است")
            return

        pages = list(self.out_links.keys())

        in_degrees = [self.in_degree(p) for p in pages]
        out_degrees = [self.out_degree(p) for p in pages]

        print("=== تحلیل توزیع درجه‌ها ===")
        print("میانگین: میانگین درجه ورودی: {:.2f}، انحراف معیار: {:.2f}، حداکثر: {}".format(
            np.mean(in_degrees), np.std(in_degrees), max(in_degrees)))
        print("میانگین: میانگین درجه خروجی: {:.2f}، انحراف معیار: {:.2f}، حداکثر: {}".format(
            np.mean(out_degrees), np.std(out_degrees), max(out_degrees)))
        print("\nجزئیات هر صفحه:")
        for p in sorted(pages):
            print("ورودی: {}, خروجی: {}".format(p, self.in_degree(p), self.out_degree(p)))

    def dfs_forward(self, node, visited, order):
        """
        Kosaraju برای مرتب‌سازی توپولوژیکی - پاس اول (DFS) پیمایش عمق‌اول
        """
        visited.add(node)
        for neighbor in self.out_links.get(node, []):
            if neighbor not in visited:
                self.dfs_forward(neighbor, visited, order)
        order.append(node)

    def dfs_reverse(self, node, visited, scc):
        """
        Kosaraju روی گراف معکوس - پاس دوم
        """
        visited.add(node)
        scc.add(node)
        for neighbor in self.in_links.get(node, []):
            if neighbor not in visited:
                self.dfs_reverse(neighbor, visited, scc)

    def find_all_sccs(self):
        """
        Kosaraju با الگوریتم (SCC) یافتن تمام مؤلفه‌های قویاً همبند
        پیچیدگی زمانی: O(V+E)
        """
```

```

"""
if not self.out_links:
    return []

# و پر کردن پشته مرتب‌سازی DFS: پاس اول
visited = set()
order = []
for node in self.out_links:
    if node not in visited:
        self.dfs_forward(node, visited, order)

# order روی گراف معکوس به ترتیب معکوس DFS: پاس دوم
visited.clear()
sccs = []
for node in reversed(order):
    if node not in visited:
        scc = set()
        self.dfs_reverse(node, visited, scc)
        sccs.append(scc)

return sccs

def analyze_bowtie_structure(self):
    """
    SCC بزرگترین) اصلی SCC بر اساس Bowtie تحلیل ساختار
    .این ساختار جریان اطلاعات در وب را به صورت دسته‌بندی شده نشان می‌دهد.
    """

    if not self.out_links:
        print("\nگراف خالی است\n")
        return

    sccs = self.find_all_sccs()
    if not sccs:
        print("\nهیچ SCC یافت نشد\n")
        return

    # SCC اصلی: بزرگترین SCC
    main_scc = max(sccs, key=len)
    all_pages = set(self.out_links.keys())

    # قابل دسترسی نیستند SCC برسند اما از SCC می‌توانند به IN بخش
    reachable_to_scc = set()
    for node in main_scc:
        stack = [node]
        visited = set()
        while stack:
            current = stack.pop()
            if current not in visited:
                visited.add(current)
                for neighbor in self.in_links.get(current, []):
                    if neighbor not in visited:
                        stack.append(neighbor)
        reachable_to_scc |= visited
    in_set = reachable_to_scc - main_scc

    # نمی‌رسند SCC قابل دسترسی‌اند اما به SCC از OUT بخش
    reachable_from_scc = set()
    for node in main_scc:
        stack = [node]
        visited = set()
        while stack:
            current = stack.pop()
            if current not in visited:
                visited.add(current)
                for neighbor in self.out_links.get(current, []):
                    if neighbor not in visited:
                        stack.append(neighbor)
        reachable_from_scc |= visited
    out_set = reachable_from_scc - main_scc

    # Tubes, Tendrils, Isolated) بقیه صفحات
    others = all_pages - (in_set | main_scc | out_set)

    # نمایش نتایج
    print("\n=== تحلیل ساختار Bowtie ===")
    print("SCC (صفحه {}): {}".format(
        sorted(main_scc), len(main_scc)))
    print("IN (SCC لینک‌دهنده به {}): {}".format(
        sorted(in_set), len(in_set)))
    print("OUT (SCC قابل دسترسی از {}): {}".format(
        sorted(out_set), len(out_set)))
    if others:
        print("OTHER (لوله‌ها/ریشه‌های جدا): {}".format(
            sorted(others), len(others)))

    # توزیع درصدی
    total = len(all_pages)
    print("\ntوزیع درصدی:")
    print(" SCC: {:.1%}".format(len(main_scc) / total))
    print(" IN: {:.1%}".format(len(in_set) / total))
    print(" OUT: {:.1%}".format(len(out_set) / total))
    if others:
        print(" OTHER: {:.1%}".format(len(others) / total))

    return main_scc, in_set, out_set, others

def demonstrate_power_law_theory(self, max_degree=100, alpha=2.1):
    """
    نمایش تئوری توزیع قانون توان برای درجه ورودی
    دارای لینک‌های بسیار زیادی هستند (Hub) این نشان می‌دهد چرا چند صفحه محبوب
    در حالی که اکثریت صفحات جدا افتاده‌اند.
    """

    degrees = np.arange(1, max_degree + 1)

```

```

probs = 1.0 / (degrees ** alpha)
probs = probs / probs.sum()

print("\n=== شبیه‌سازی قانون توان ( $\alpha=\{ }\)$  ===".format(alpha))
print("(درجه | احتمال | صفحات مورد انتظار (از ۱۰۰۰))")
print("-" * 45)
for i in [1, 2, 3, 5, 10, 20, 50, 100]:
    if i <= max_degree:
        prob = probs[i-1]
        expected = prob * 1000
        print("{:6d} | {:.84f} | {:.72f}".format(i, prob, expected))

# رسم توزیع لگاریتمی برای نمایش
plt.figure(figsize=(10, 6))
plt.loglog(degrees, probs, 'b-', linewidth=2, label='Power Law ( $\alpha=\{ }\)$ '.format(alpha))
plt.xlabel('In-degree (i)', fontsize=12)
plt.ylabel('P(i) (Probability)', fontsize=12)
plt.title('Power Law Distribution of Web Page In-degrees', fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3, which='both')
plt.legend()
plt.tight_layout()
plt.show()

def plot_actual_degree_distribution(self):
    """
    رسم توزیع درجه‌های واقعی گراف و مقایسه با قانون توان
    """
    if not self.out_links:
        print("گراف خالی است")
        return

    pages = list(self.out_links.keys())
    in_degrees = [self.in_degree(p) for p in pages]

    # محاسبه فراوانی
    max_degree = max(in_degrees) if in_degrees else 0
    degree_counts = np.zeros(max_degree + 1)
    for d in in_degrees:
        degree_counts[d] += 1

    # فیلتر کردن درجه‌های صفر برای نمودار لگاریتمی
    non_zero = np.arange(1, max_degree + 1)
    probs = degree_counts[1:] / len(pages)

    plt.figure(figsize=(12, 5))

    # نمودار 1: فراوانی
    plt.subplot(1, 2, 1)
    plt.bar(non_zero, degree_counts[1:], color='skyblue', edgecolor='black')
    plt.xlabel('In-degree')
    plt.ylabel('Number of Pages')
    plt.title('Actual Degree Frequency Distribution')

    # نمودار 2: لگاریتمی
    plt.subplot(1, 2, 2)
    if len(probs) > 0:
        plt.loglog(non_zero, probs, 'ro-', markersize=6, label='Actual Data')
        # خط راهنمای قانون توان
        x = np.linspace(1, max_degree, 100)
        y = 1.0 / (x ** 2.1)
        y = y / sum(y) * sum(probs) # نرمالیزه کردن برای مقایسه
        plt.loglog(x, y, 'b--', label='Power Law ( $\alpha=2.1$ )')

    plt.xlabel('In-degree (log scale)')
    plt.ylabel('Probability (log scale)')
    plt.title('Comparison with Power Law')
    plt.legend()
    plt.grid(True, alpha=0.3, which='both')

    plt.tight_layout()
    plt.show()

def visualize(self, layout='spring'):
    """
    رسم بصری گراف با رنگ‌بندی خودکار بخش‌های
    """
    if not self.out_links:
        print("گراف خالی است")
        return

    # ایجاد گراف NetworkX
    G = nx.DiGraph()
    G.add_nodes_from(self.out_links.keys())
    for from_page, to_pages in self.out_links.items():
        for to_page in to_pages:
            G.add_edge(from_page, to_page)

    # برای رنگ‌بندی Bowtie تحلیل
    result = self.analyze_bowtie_structure()
    if result is None:
        return

    main_scc, in_set, out_set, others = result

    # رنگ‌بندی خودکار
    color_map = {
        'scc': 'gold',
        'in': 'lightgreen',
        'out': 'lightcoral',
        'other': 'lightgray',
        'default': 'lightblue'
    }

```

```
node_colors = []
for node in G.nodes():
    if node in main_scc:
        node_colors.append(color_map['scc'])
    elif node in in_set:
        node_colors.append(color_map['in'])
    elif node in out_set:
        node_colors.append(color_map['out'])
    elif node in others:
        node_colors.append(color_map['other'])
    else:
        node_colors.append(color_map['default'])

# انتخاب الگوریتم چیدمان
if layout == 'spring':
    pos = nx.spring_layout(G, seed=42, k=2, iterations=50)
elif layout == 'circular':
    pos = nx.circular_layout(G)
elif layout == 'kamada':
    pos = nx.kamada_kawai_layout(G)
else:
    pos = nx.random_layout(G)

fig, ax = plt.subplots(figsize=(12, 8))

# رسم گراف (غیرفعال کردن برجسب‌های پیش‌فرض)
nx.draw(G, pos, with_labels=False, node_color=node_colors,
        node_size=2500, edge_color='gray', linewidths=1.5,
        font_size=11, font_weight='bold', arrowsize=20,
        arrowstyle='->', connectionstyle='arc3,rad=0.1')

# افزودن برجسب‌های سفارشی (فقط یک بار)
labels = {page: f"{page}\n(in={self.in_degree(page)}, out={self.out_degree(page)})"
          for page in G.nodes()}
nx.draw_networkx_labels(G, pos, labels, font_size=9, font_color='black')

# افزودن لیوان رنگ
import matplotlib.patches as mpatches
legend_elements = [
    mpatches.Patch(color=color_map['scc'], label='SCC'),
    mpatches.Patch(color=color_map['in'], label='IN'),
    mpatches.Patch(color=color_map['out'], label='OUT'),
    mpatches.Patch(color=color_map['other'], label='OTHER')
]
ax.legend(handles=legend_elements, loc='upper right')

plt.title("Web Graph Structure with Bowtie Coloring", fontsize=14, fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.show()
```

واقعی SCC نمونه گراف بهبود یافته با
web = WebGraph()

واقعی SCC ایجاد یک
web.add_link("A", "B")
web.add_link("B", "C")
web.add_link("C", "A")

لینک می‌دهند SCC صفحاتی که به IN اضافه کردن
web.add_link("X", "A")
web.add_link("Y", "B")

قابل دسترسی‌اند SCC صفحاتی که از OUT اضافه کردن
web.add_link("A", "D")
web.add_link("B", "E")
web.add_link("D", "F")

SCC بدون عبور از OUT به IN مسیر از Tube اضافه کردن یک
web.add_link("Y", "G")
web.add_link("G", "F")

اضافه کردن یک صفحه منزوی
web.add_page("H")

تحلیل‌ها
web.analyze_degrees()
web.analyze_bowtie_structure()
web.demonstrate_power_law_theory(max_degree=50, alpha=2.1)

نمایش داده‌های واقعی و مقایسه با تئوری
web.plot_actual_degree_distribution()

نمایش بصری
print("\n=== نمایش بصری با رنگبندی Bowtie ===")
web.visualize(layout='spring')

=== تحلیل توزیع درجه‌ها ===
درجه ورودی: میانگین=1.00، انحراف معیار=0.77، حداکثر=2
درجه خروجی: میانگین=1.00، انحراف معیار=0.77، حداکثر=2

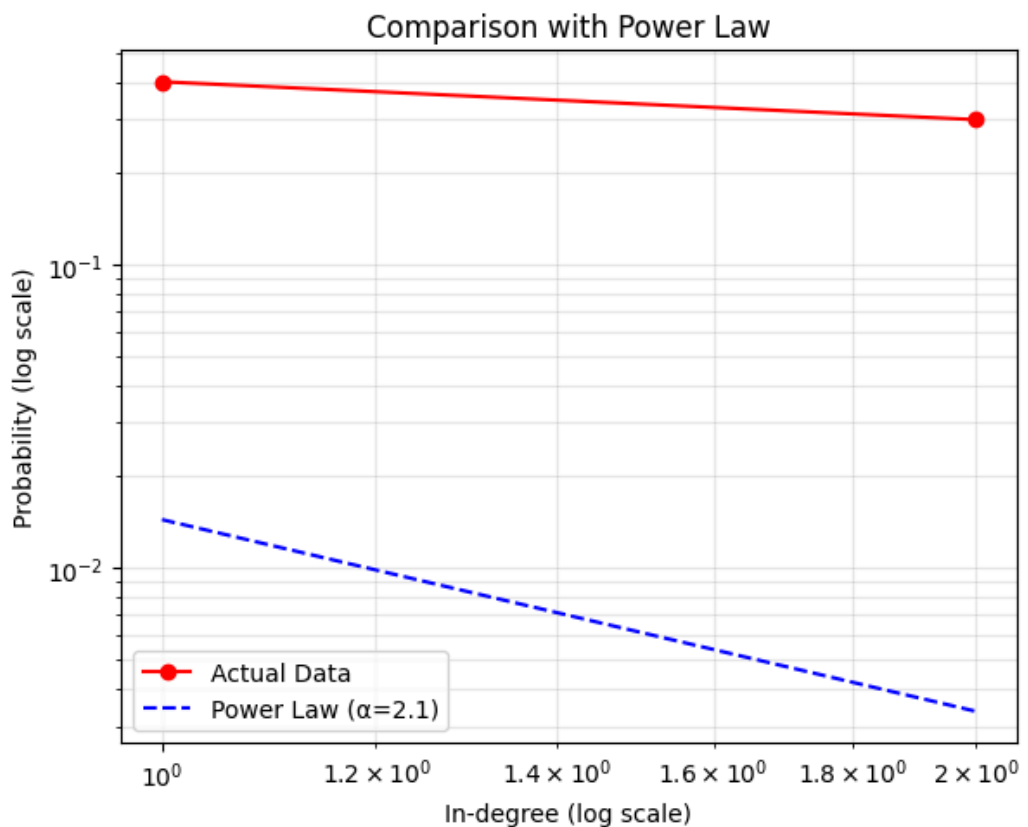
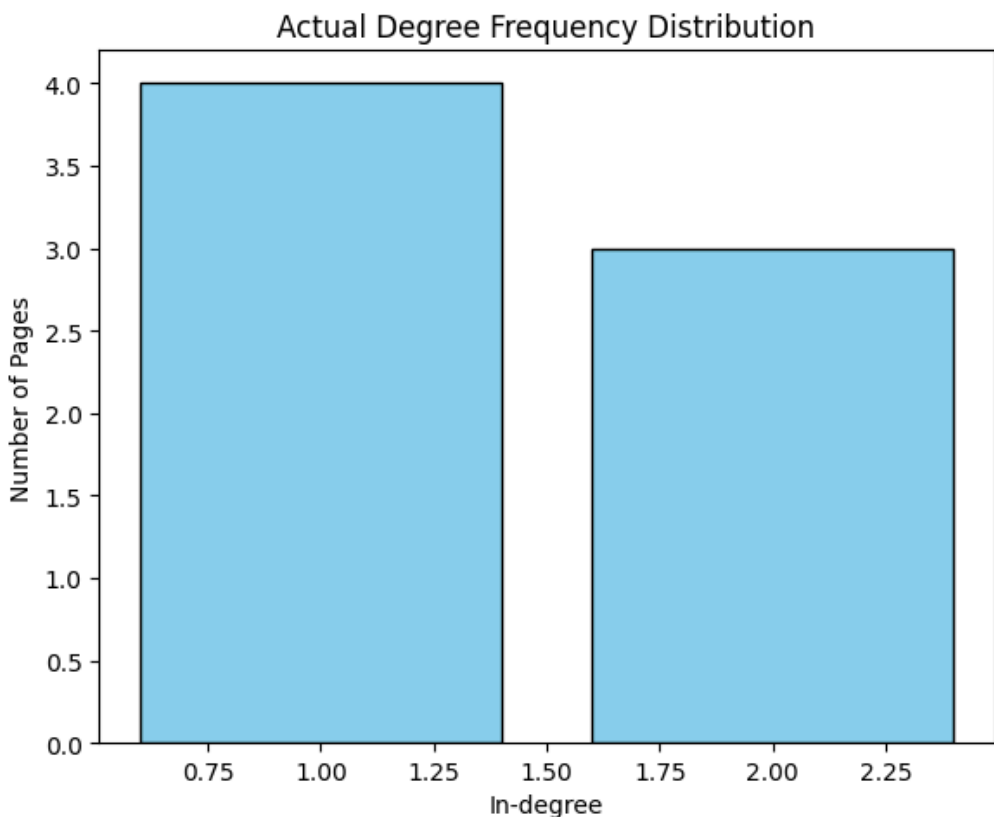
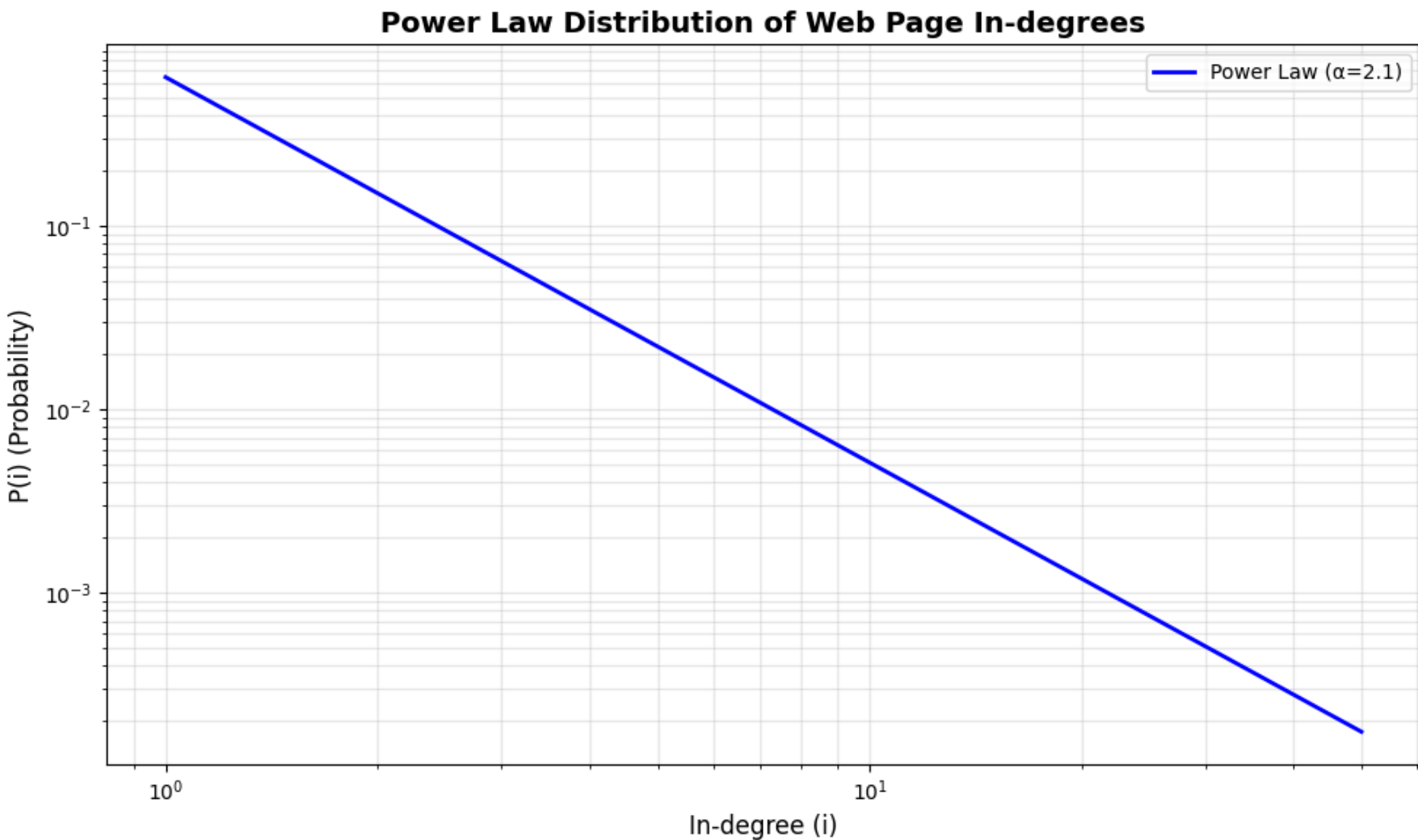
جزئیات هر صفحه
ورودی=2، خروجی=2 A: 2 صفحه
ورودی=2، خروجی=2 B: 2 صفحه
ورودی=1، خروجی=1 C: 1 صفحه
ورودی=1، خروجی=1 D: 1 صفحه
ورودی=1، خروجی=0 E: 0 صفحه
ورودی=2، خروجی=0 F: 0 صفحه
ورودی=1، خروجی=1 G: 1 صفحه
ورودی=0، خروجی=0 H: 0 صفحه
ورودی=0، خروجی=1 X: 1 صفحه
ورودی=0، خروجی=2 Y: 2 صفحه

=== تحلیل ساختار Bowtie ===
صفحه 3 (['A', 'B', 'C']): SCC (قویاً همبند)
صفحه 2 (['X', 'Y']): IN (SCC لینک‌دهنده به)
صفحه 3 (['D', 'E', 'F']): OUT (SCC قابل دسترسی از)
صفحه 2 (['G', 'H']): OTHER (لوله‌ها/ریشه‌های جدا)

توزیع درصدی:
SCC: 30.0%
IN: 20.0%
OUT: 30.0%
OTHER: 20.0%

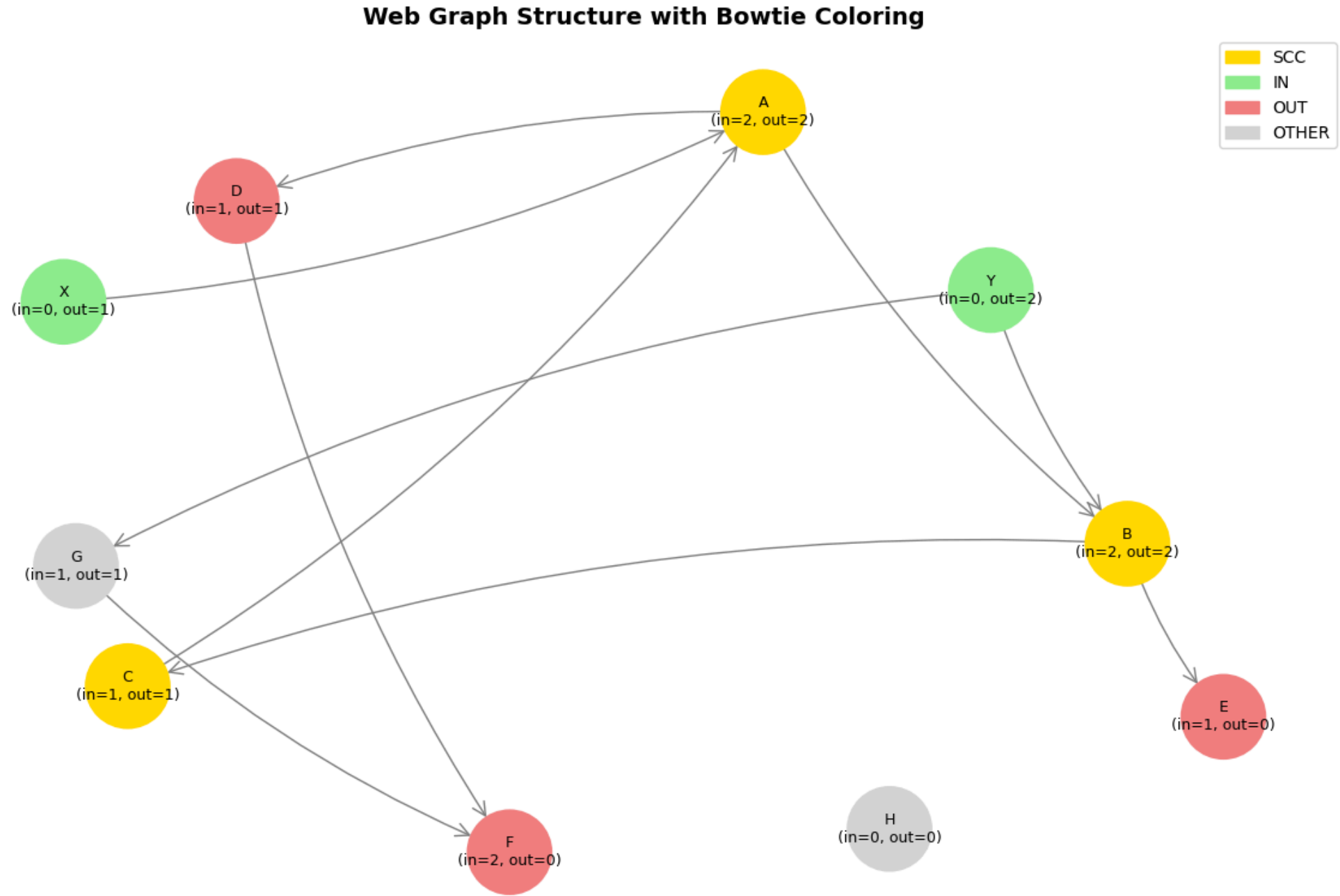
=== (α=2.1) شبیه‌سازی قانون توان ===
درجه | احتمال | صفحات مورد انتظار (از ۱۰۰۰)

1	0.6460	645.97
2	0.1507	150.68
3	0.0643	64.31
5	0.0220	22.00
10	0.0051	5.13
20	0.0012	1.20
50	0.0002	0.17



=== Bowtie نمایش بصری با رنگبندی ===
SCC (صفحه 3): ['A', 'B', 'C'] (فویاً همبند)
IN (صفحه 2): ['X', 'Y'] (لینکدهنده به SCC)
OUT (صفحه 3): ['D', 'E', 'F'] (قابل دسترسی از SCC)
OTHER (صفحه 2): ['G', 'H'] (لوله‌ها/ریشه‌های جدا)

توزیع درصدی:
SCC: 30.0%
IN: 20.0%
OUT: 30.0%
OTHER: 20.0%



🕷️ 19.2.2 اسپم: جنگ پنهان در جستجوی وب

💰 انگیزه اقتصادی: ریشه اصلی اسپم

از همان ابتدای تاریخ جستجوی وب، مشخص شد که رتبه‌بندی در نتایج جستجو می‌تواند مستقیماً بر فروش تأثیر بگذارد. تصور کنید کاربری عبارت «آپارتمان در سعادت‌آباد» را جستجو می‌کند؛ این کاربر به احتمال زیاد به دنبال خرید یا اجاره ملک است، نه صرفاً مطالعه درباره این منطقه. همین انگیزه اقتصادی، زمینه را برای بروز **اسپم** فراهم کرد: دستکاری محتوای صفحات برای رتبه‌بندی بالا در کلمات کلیدی خاص.

این پدیده مشابه شرکت‌هایی است که در دفترچه تلفن‌های قدیمی (Yellow Pages) نام خود را با چند حرف «آ» در ابتدا ثبت می‌کردند تا در صدر فهرست قرار گیرند. اما در دنیای وب، این دستکاری هزینه بسیار کمتری دارد و تأثیر بسیار بیشتری بر نتایج جستجو می‌گذارد.

👤 نسل اول اسپم: تکرار کلمات کلیدی

اولین نسل اسپم، تکرار بی‌رویه کلمات کلیدی بود. در موتورهای جستجوی اولیه که بر اساس فراوانی کلمات کار می‌کردند، صفحه‌ای که عبارت «خرید لپ‌تاپ ارزان» را ۱۰۰ بار تکرار می‌کرد، در نتایج جستجو رتبه بالاتری کسب می‌کرد. برای پنهان‌کردن

این تکرار از کاربر، اسپمرها از ترفندهایی مانند **رنگ متن مشابه پس‌زمینه** استفاده می‌کردند – که برای انسان نامرئی، اما برای کرالر قابل ایندکس بود.

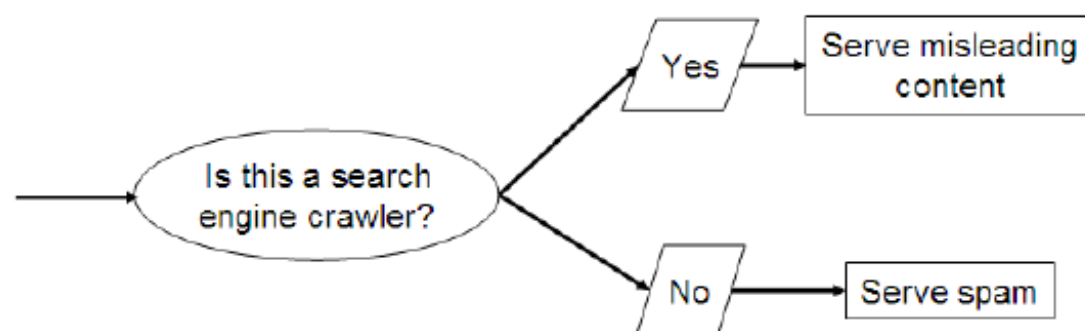
مثلاً در سال‌های اولیه ظهور دیجی‌کالا، رقبا صفحاتی ایجاد می‌کردند که عبارت «خرید موبایل» را با رنگ سفید روی پس‌زمینه سفید تکرار می‌کردند. کاربران این متن را نمی‌دیدند، اما موتورهای جستجو این صفحات را برای این عبارت رتبه‌بندی می‌کردند.

🏠 تکنیک‌های پیشرفته اسپم

با پیشرفت موتورهای جستجو، اسپمرها نیز تکنیک‌های پیچیده‌تری را توسعه دادند:

- **Cloaking (پوشاندن):** سرور، محتوای متفاوتی به کرالر موتور جستجو و کاربر انسانی ارائه می‌دهد. کرالر، صفحه‌ای با کلمات کلیدی فریبنده را ایندکس می‌کند، اما کاربر چیزی کاملاً متفاوت می‌بیند.
- **Doorway Pages (صفحات ورودی):** صفحاتی که فقط برای رتبه‌بندی بالا ساخته شده‌اند و هنگام دسترسی کاربر، وی را به صفحه تجاری واقعی هدایت می‌کنند.
- **Link Spam (اسپم لینک):** ایجاد لینک‌های مصنوعی برای دستکاری معیارهای اعتبارسنجی مبتنی بر گراف وب.

برای مثال، یک شرکت فروش آنلاین ممکن است صفحه‌ای ایجاد کند که برای موتور جستجو پر از کلمات کلیدی «تخفیف ویژه لباس» است، اما وقتی کاربر روی لینک کلیک می‌کند، به صفحه فروشگاهی هدایت می‌شود که هیچ ارتباطی با محتوای ایندکس شده ندارد.



► Figure 19.5 Cloaking as used by spammers.

شکل 19.5: تکنیک Cloaking در اسپم. در این روش، سرور محتوای متفاوتی را به کرالر موتور جستجو و کاربر ارائه می‌دهد.

✂ جنگ بین اسپمرها و موتورهای جستجو

این رقابت منجر به شکل‌گیری صنعت **بهینه‌سازی موتورهای جستجو (SEO)** شد. برخی شرکت‌های SEO به‌صورت قانونی (بهبود محتوا و ساختار سایت) کار می‌کنند، اما برخی دیگر به‌صورت خصمانه تلاش می‌کنند الگوریتم‌های اختصاصی موتورهای جستجو را دور بزنند.

در مقابل، موتورهای جستجو سیاست‌های سخت‌گیرانه‌ای برای مبارزه با اسپم اعمال می‌کنند، از جمله تعلیق کامل وب‌سایت‌های اسپمر. این جنگ دائمی بین اسپمرها و موتورهای جستجو، زمینه‌ی شکل‌گیری حوزه تحقیقاتی **بازیابی اطلاعات خصمانه (Adversarial IR)** را فراهم کرده است.

برای مثال، گوگل در سال ۲۰۱۲ آپدیت پنگوئن را منتشر کرد که به طور خاص با وب‌سایت‌هایی که از تکنیک‌های اسپم لینک استفاده می‌کردند مبارزه می‌کرد. این آپدیت باعث شد بسیاری از وب‌سایت‌هایی که با خرید لینک رتبه خود را artificially بالا برده بودند، افت رتبه شدیدی را تجربه کنند.

یکی از مؤثرترین راه‌های مبارزه با اسپمرها، **تحلیل ساختار لینک‌های وب** (Link Analysis) است. این تکنیک که پایه‌ی الگوریتم‌هایی مانند Google PageRank است، به موتورهای جستجو اجازه می‌دهد تا کیفیت و اعتبار صفحات را بر اساس لینک‌های طبیعی دریافت شده از سایر صفحات ارزیابی کنند.

برای مثال، اگر یک صفحه فروش لپ‌تاپ، لینک‌های طبیعی و باکیفیتی از منابع معتبر مانند زومیت، دیجی‌کالا و وبلاگ‌های تخصصی دریافت کند، احتمالاً در نتایج جستجو رتبه بهتری خواهد داشت، حتی اگر محتوای آن صفحه به اندازه یک صفحه اسپم، کلمات کلیدی را تکرار نکرده باشد.

این رویکرد باعث شده است تا اسپمرها نیز تلاش خود را بر روی دستکاری لینک‌ها متمرکز کنند که به آن **اسپم لینک** می‌گویند. این مبارزه همچنان ادامه دارد و هر دو طرف در حال توسعه تکنیک‌های جدید برای غلبه بر دیگری هستند.

19.3 تبلیغات: مدل اقتصادی که وب را متحول کرد 💰

📊 از بنرهای گرافیکی تا انقلاب تبلیغات کلیک

در روزهای اولیه وب، شرکت‌ها مانند نایکی و کوکاکولا، بنرهای گرافیکی در سایت‌های محبوبی مانند CNN و Yahoo! قرار می‌دادند. این تبلیغات بر اساس مدل **CPM** (هزینه به ازای هر هزار نمایش) قیمت‌گذاری می‌شدند و هدف اصلی، ایجاد حس مثبت در مخاطب نسبت به برند بود - نه فروش مستقیم.

با افزایش تعاملی بودن وب، مدل **CPC** (هزینه به ازای هر کلیک) متولد شد. در این مدل، مانند آنچه امروز در گوگل ادز می‌بینیم، تبلیغ‌دهنده فقط زمانی هزینه می‌پردازد که کاربر واقعاً روی تبلیغ کلیک کند. این تغییر، تمرکز را از «تبلیغ برند» به «ایجاد تراکنش» منتقل کرد.

برای درک تفاوت، تصور کنید دیجی‌کالا بنری در سایت ورزش سه نمایش می‌دهد که هر هزار بار نمایش ۱۰۰ هزار تومان هزینه دارد (CPM). در مقابل، وقتی در گوگل «خرید گوشی سامسونگ» را جستجو می‌کنید، تبلیغات نمایش داده‌شده فقط در صورت کلیک هزینه دارند (CPC).

🚀 انقلاب Goto: وقتی جستجو به منبع درآمد تبدیل شد

شرکت **Goto** (که بعداً به Overture تغییر نام داد و در نهایت توسط یاهو خریداری شد)، ایده‌ای انقلابی ارائه داد: برای هر عبارت جستجو، از شرکت‌ها پیشنهاد مالی دریافت می‌کرد و نتایج را بر اساس میزان پیشنهاد رتبه‌بندی می‌کرد.

برای مثال، وقتی کاربری عبارت «هتل مشهد» را جستجو می‌کرد، هتل‌هایی که برای این عبارت بیشتر پیشنهاد داده بودند، در نتایج بالاتر نمایش داده می‌شدند. این مدل به دو دلیل بسیار مؤثر بود:

- کاربران با جستجوی عبارت خاص، **قصد خرید فعال** نشان می‌دادند.
- تبلیغ‌دهندگان فقط برای نتایج واقعی هزینه می‌پرداختند.

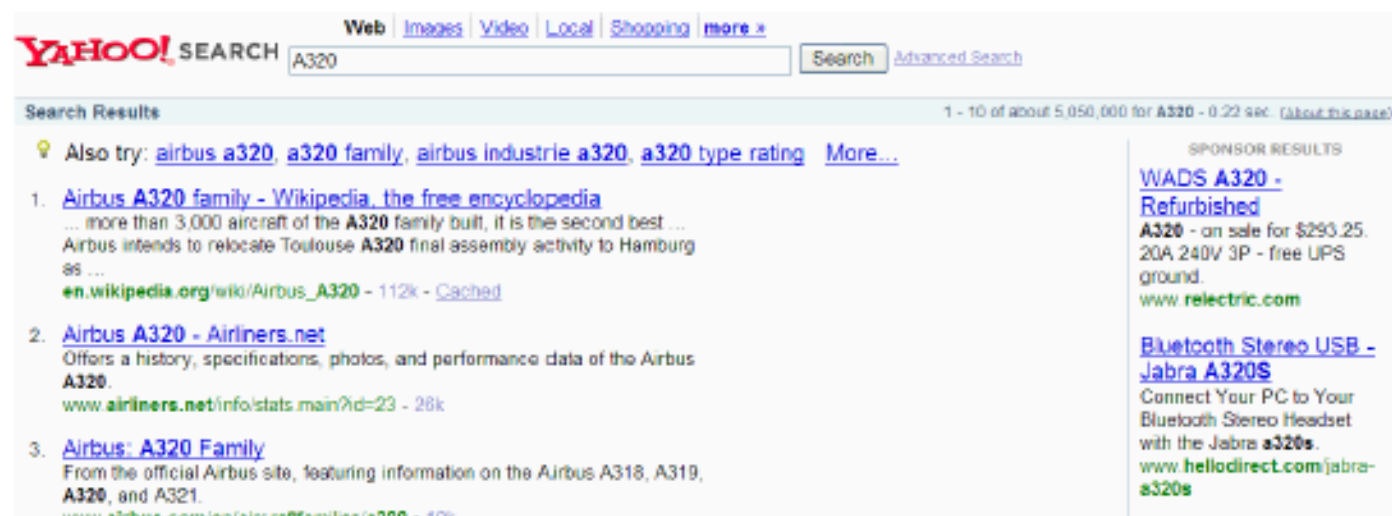
این مدل به سرعت درآمد سالانه Goto/Overture را به صدها میلیون دلار افزایش داد و پایه و اساس مدل تبلیغات مدرن در موتورهای جستجو را شکل داد.

امروزه، موتورهای جستجو مانند گوگل دو نوع نتیجه ارائه می‌دهند:

- **نتایج الگوریتمی:** نتایج اصلی که بر اساس معیارهای فنی مانند PageRank و ارتباط با عبارت جستجو رتبه‌بندی شده‌اند.

- **نتایج اسپانسرشده:** تبلیغات که معمولاً در بالای صفحه یا در کنار نتایج اصلی نمایش داده می‌شوند و با برچسب «تبلیغ» مشخص شده‌اند.

این ترکیب، بهترین هر دو جهان را فراهم می‌کند: کاربران به نتایج مرتبط دسترسی دارند و تبلیغ‌دهندگان می‌توانند به مخاطبان هدف خود دسترسی پیدا کنند.



► **Figure 19.6** Search advertising triggered by query keywords. Here the query A320 returns algorithmic search results about the Airbus aircraft, together with advertisements for various non-aircraft goods numbered A320, that advertisers seek to market to those querying on this query. The lack of advertisements for the aircraft reflects the fact that few marketers attempt to sell A320 aircraft on the web.

شکل 19.6: تبلیغات جستجو triggered by query keywords. در این مثال، جستجوی A320 نتایج الگوریتمی درباره هواپیمای ایرباس A320 را به همراه تبلیغات برای کالاهای غیرمرتبط با هواپیما نمایش می‌دهد که تبلیغ‌کنندگان به دنبال بازاریابی برای کاربرانی هستند که این عبارت را جستجو می‌کنند.

📈 **بازاریابی موتورهای جستجو (SEM): حرفه‌ای نوظهور**

با پیچیده‌تر شدن فرآیند رتبه‌بندی تبلیغات در موتورهای جستجو، که ترکیبی از بازاریابی اطلاعات و اقتصاد خرد است، حرفه‌ای جدید به نام **بازاریابی موتورهای جستجو (SEM)** ظهور کرد.

متخصصان SEM به شرکت‌ها کمک می‌کنند تا بودجه بازاریابی خود را بهینه تخصیص دهند، کلمات کلیدی مؤثر را انتخاب کنند و کمپین‌های تبلیغاتی موفق‌تری ایجاد کنند. در ایران نیز شرکت‌هایی مانند وب‌سازان و آژانس‌های دیجیتال مارکتینگ متعددی، خدمات SEM را به کسب‌وکارهای مختلف ارائه می‌دهند.

⚠️ **چالش کلیک اسپم: وقتی رقابت غیرمنصفانه می‌شود**

همان‌طور که هر سیستم اقتصادی با چالش‌هایی روبرو است، تبلیغات در موتورهای جستجو نیز با مشکلی به نام **کلیک اسپم** مواجه است. کلیک اسپم به کلیک‌های غیراصولی روی تبلیغات گفته می‌شود که از طرف کاربران واقعی نیستند.

برای مثال، یک رقیب ممکن است با استفاده از ربات‌ها، به طور مکرر روی تبلیغات شما کلیک کند تا بودجه تبلیغاتی شما سریع‌تر تمام شود. این چالش، موتورهای جستجو را مجبور کرده است الگوریتم‌های پیچیده‌ای برای شناسایی و نادیده گرفتن کلیک‌های اسپم توسعه دهند.

این جنگ میان اسپمرها و موتورهای جستجو، نمونه‌ای دیگر از رقابت خصمانه در اکوسیستم وب است که به «بازیابی اطلاعات خصمانه» معروف است.

19.4 تجربه کاربری در جستجوی وب: انقلابی در تعامل با اطلاعات

از متخصصان به عموم: تحول کاربران جستجو

در سیستم‌های بازیابی اطلاعات سنتی، کاربران معمولاً متخصصانی بودند که با ساختار مجموعه‌های اطلاعاتی آشنا بودند و در نگارش پرس‌وجوها آموزش دیده بودند. اما در جستجوی وب، با میلیاردها کاربر عادی روبرو هستیم که:

- به‌طور میانگین فقط ۲ تا ۳ کلمه کلیدی در جستجوی خود استفاده می‌کنند
- به ندرت از عملگرهای پیچیده (مانند AND, OR, *) استفاده می‌کنند
- درباره ساختار و ناهمگونی محتوای وب اطلاعی ندارند و اهمیتی هم نمی‌دهند

برای درک این تفاوت، تصور کنید یک کتابدار متخصص که با استفاده از دستورات پیچیده در کاتالوگ کتابخانه جستجو می‌کند در مقابل یک دانشجو که در گوگل "نظریه نسبیت اینشتین" را تایپ می‌کند. این تغییر بنیادی، نیازمند طراحی موتورهای جستجویی بود که برای افراد عادی، نه متخصصان، مناسب باشند.

🚀 اصول موفقیت گوگل: دقت و سادگی

در رقابت شدید بین موتورهای جستجو، گوگل با دو اصل کلیدی توانست بر رقبای خود پیشی بگیرد:

1. **تمرکز بر دقت (Precision) به جای جامعیت (Recall):** گوگل متوجه شد کاربران عادی به نتایج دقیق در ۳-۵ جایگاه اول اهمیت می‌دهند، نه لیست‌های طولانی از نتایج مرتبط. این اصل باعث صرفه‌جویی در زمان کاربر و افزایش رضایت او می‌شود.

2. **تجربه کاربری سبک و سریع:** در حالی که رقبای گوگل مانند یاهو و AltaVista صفحه‌های شلوغی با محتوای خبری، ایمیل و بنرهای تبلیغاتی داشتند، گوگل صفحه‌ای ساده، متنی و با بارگذاری سریع ارائه داد. این طراحی مینیمالیستی باعث شد کاربران به سرعت به نتیجه برسند.

این دو اصل در کنار هم، ترکیبی قدرتمند ایجاد کردند: کاربران سریع‌تر به اطلاعات دقیق‌تر دسترسی پیدا کردند و این باعث افزایش ترافیک و در نتیجه درآمد گوگل از تبلیغات شد.

🔍 سه دسته اصلی پرس‌وجوهای کاربران

تحقیقات نشان داده است که پرس‌وجوهای جستجوی وب را می‌توان در سه دسته اصلی طبقه‌بندی کرد:

پرس وجوهای اطلاعاتی
(Informational)

کاربر به دنبال اطلاعات عمومی درباره یک موضوع است و معمولاً از چندین منبع استفاده می‌کند.

مثال‌های واقعی:

- "علائم بیماری کووید-۱۹ چیست؟"
- "تاریخچه جنگ جهانی دوم"
- "آموزش پخت کیک شکلاتی"

پرس وجوهای ناوبری
(Navigational)

کاربر به دنبال وبسایت یا صفحه خاصی است که از قبل نام آن را می‌داند و انتظار دارد نتیجه اول همان سایت باشد.

مثال‌های واقعی:

- "فیس‌بوک"
- "دیجی‌کالا"
- "بانک ملی"

پرس وجوهای تراکنشی
(Transactional)

کاربر قصد انجام یک عمل آنلاین مانند خرید، دانلود یا رزرو را دارد و به دنبال سایتی برای انجام این تراکنش است.

مثال‌های واقعی:

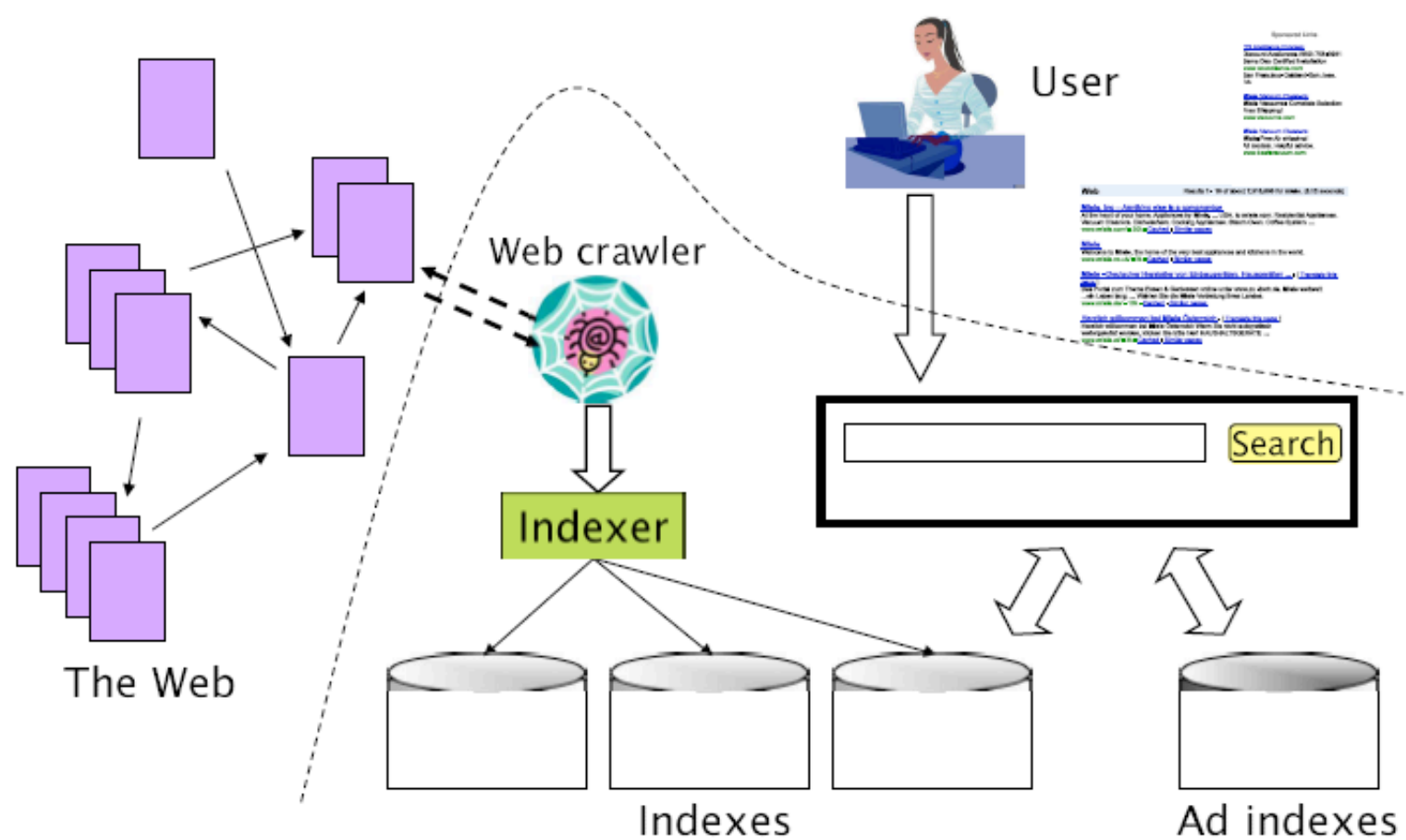
- "خرید گوشی آیفون ۱۴"
- "دانلود نرم‌افزار فتوشاپ"
- "رزرو هتل در مشهد"

اهمیت تشخیص نوع پرس وجو

درک نوع پرس وجو برای موتورهای جستجو از دو جهت حیاتی است:

- بهبود نتایج الگوریتمی:** برای پرس وجوهای ناوبری، دقت در رتبه اول ($Precision@1$) مهم‌ترین شاخص موفقیت است. وقتی کاربر "یوتوب" را جستجو می‌کند، انتظار دارد اولین نتیجه، سایت اصلی یوتیوب باشد.
- بهینه‌سازی تبلیغات:** پرس وجوهای تراکنشی اغلب نشان‌دهنده نیت خرید هستند و بنابراین برای نمایش تبلیغات اسپانسرشده ایده‌آل هستند. برای مثال، وقتی کاربر "خرید لپ‌تاپ ایسوس" را جستجو می‌کند، نمایش تبلیغات فروشگاه‌های مختلف لپ‌تاپ بسیار مؤثرتر از نمایش تبلیغات برای مقالات علمی درباره لپ‌تاپ است.

این درک عمیق از نیازهای کاربران، موتورهای جستجو را قادر می‌سازد تا تجربه‌ای شخصی‌سازی شده و مفید ارائه دهند که کاربران را به استفاده مداوم از سرویس خود تشویق کند.



► **Figure 19.7** The various components of a web search engine.

شکل 19.7: تصویر ترکیبی از یک موتور جستجوی وب شامل کرالر، ایندکس صفحات وب و ایندکس تبلیغات. بخش زیر خط منحنی ممتد داخلی موتور جستجو است.

19.5 اندازه ایندکس: چالش‌های اندازه‌گیری وب نامحدود

🌐 چرا اندازه‌گیری ایندکس پیچیده است؟

در نگاه اول، ممکن است فکر کنید اندازه ایندکس یک موتور جستجو معیار ساده‌ای برای ارزیابی جامعیت آن است. اما واقعیت بسیار پیچیده‌تر است. وب شامل تعداد نامحدودی صفحات پویا است که هر بار با بارگذاری مجدد، محتوای جدیدی تولید می‌کند.

برای درک این چالش، به این مثال توجه کنید: آدرس `http://www.amazon.com/search?keyword=any_string` برای هر عبارت دلخواهی که جایگزین `any_string` شود، یک صفحه معتبر برمی‌گرداند - حتی اگر محصولی با آن نام وجود نداشته باشد. این صفحات "Soft 404" که به جای خطای 404، صفحه‌ای معتبر نمایش می‌دهند، فضای جستجو را به صورت تئوری نامحدود می‌کنند.

علاوه بر این، برخی سایت‌ها عمداً "دام‌های کرالر" (Spider Traps) ایجاد می‌کنند - صفحاتی که کرالرهای موتورهای جستجو را در حلقه‌ای از لینک‌های بی‌پایان گرفتار می‌کنند تا ایندکس آن‌ها را با صفحات بی‌ارزش پر کنند.

📊 روش Capture-Recapture: از زیست‌شناسی تا موتورهای جستجو

به جای اندازه مطلق، معمولاً نسبت اندازه ایندکس دو موتور جستجو (مثلاً گوگل در مقابل بینگ) مورد علاقه است. برای تخمین این نسبت، محققان از روشی به نام **Capture-Recapture** استفاده کرده‌اند که اصل آن از زیست‌شناسی گرفته شده است.

در زیست‌شناسی، برای تخمین جمعیت ماهیان در یک دریاچه، زیست‌شناسان ابتدا تعدادی ماهی صید و علامت‌گذاری می‌کنند (Capture). سپس ماهیان را آزاد کرده و در صید دوم، تعداد ماهیان علامت‌دار در نمونه جدید را می‌شمارند (Recapture). با این اطلاعات، می‌توانند تخمین بزنند که چند درصد از کل جمعیت در نمونه اول حضور داشته‌اند.

در دنیای موتورهای جستجو، این روش به صورت زیر عمل می‌کند:

- نمونه‌ای از ایندکس موتور E_1 انتخاب می‌شود و بررسی می‌شود چه کسری از آن در E_2 وجود دارد $x \rightarrow$
- نمونه‌ای از ایندکس E_2 انتخاب می‌شود و بررسی می‌شود چه کسری در E_1 وجود دارد $y \rightarrow$

با این فرضیات، نسبت اندازه ایندکس‌ها به صورت زیر تخمین زده می‌شود:

$$|E_1| / |E_2| \approx y / x$$

البته این روش با فرض‌های ساده‌کننده‌ای کار می‌کند که در عمل کاملاً دقیق نیستند: اینکه هر موتور جستجو یک زیرمجموعه تصادفی و مستقل از وب را ایندکس می‌کند.

روش‌های نمونه‌برداری و چالش‌های آن‌ها

در عمل، نمونه‌برداری واقعی از ایندکس موتورهای جستجو چالش‌برانگیز است. چندین روش پیشنهاد شده است، اما هر کدام با سوگیری‌های خاص خود همراه است:

آدرس‌های IP تصادفی

در این روش، آدرس‌های IP تصادفی تولید شده و به سرورهای وب درخواست ارسال می‌شود. این روش با مشکلاتی مانند میزبانی مجازی (چند سایت در یک IP) و سوگیری به سمت سایت‌های کوچک مواجه است.

جستجوهای تصادفی

در این روش، از لاگ جستجوی کاربران واقعی استفاده می‌شود و یک جستجوی تصادفی انتخاب می‌شود. مشکل این روش این است که نمونه‌برداری به سمت علایق گروه خاصی از کاربران (مثلاً محققان یک دانشگاه) سوگیری دارد.

قدم‌زدن تصادفی (Random Walk)

این روش بر پایه حرکت تصادفی در گراف وب عمل می‌کند، اما با این چالش که وب یک گراف قویاً همبند نیست و همگرایی به توزیع پایا بسیار کند است.

? پرس‌وجوهای تصادفی

رایج‌ترین روش، استفاده از پرس‌وجوهای تصادفی با کلمات هم‌ایستا است. این روش نیز با سوگیری به سمت اسناد بلند و تأثیر الگوریتم رتبه‌بندی موتور جستجو مواجه است.

برای مثال، اگر برای نمونه‌برداری از گوگل از پرس‌وجوی تصادفی استفاده کنیم، ممکن است به طور ناخواسته به سمت صفحاتی سوگیری داشته باشیم که گوگل آن‌ها را رتبه بالاتری داده است، نه نمونه‌ای واقعاً تصادفی از کل ایندکس.

پیشرفت‌های اخیر در تخمین اندازه ایندکس

پژوهش‌های اخیر سعی کرده‌اند این سوگیری‌ها را با روش‌های آماری پیشرفته‌تر جبران کنند. یکی از رویکردهای نوین، **نمونه‌برداری با قدم‌زدن تصادفی در گراف مجازی اسناد** است.

در این روش، یک گراف مجازی ایجاد می‌شود که در آن گره‌ها اسناد هستند و دو سند با یک یال به هم متصل می‌شوند اگر حداقل دو کلمه مشترک داشته باشند. سپس یک قدم‌زدن تصادفی در این گراف انجام می‌شود بدون اینکه گراف به طور کامل ساخته شود.

این رویکردهای جدید، اگرچه کامل نیستند، اما درک دقیق‌تری از اندازه نسبی ایندکس موتورهای جستجو ارائه می‌دهند. این موضوع نشان می‌دهد که حتی ساده‌ترین سؤالات درباره وب، پاسخ‌های پیچیده‌ای دارند و نیازمند ترکیبی از دانش از حوزه‌های مختلف مانند آمار، علوم کامپیوتر و نظریه گراف هستیم.

In [15]:

```
import matplotlib.pyplot as plt
import numpy as np

# -----
# Capture-Recapture شبیه‌سازی روش
# برای تخمین نسبت اندازه ایندکس موتورهای جستجو
# -----

class SearchEngine:
    """
    نمایش ساده یک موتور جستجو
    مجموعه صفحاتی که ایندکس شده‌اند: pages
    """

    def __init__(self, name, pages):
        self.name = name
        self.pages = set(pages)

    def sample(self, n):
        """نمونه‌برداری یکنواخت (فرض ایده‌آل)"""
        return np.random.choice(list(self.pages), min(n, len(self.pages)), replace=False)

    def contains(self, page):
        """بررسی وجود صفحه در ایندکس"""
        return page in self.pages

def capture_recapture(engine1, engine2, sample_size):
    """
    |E1| / |E2| : تخمین نسبت اندازه ایندکس

    فرمول:
    ratio = (E2 در نمونه E1 درصد صفحات) / (E1 در نمونه E2 درصد صفحات)
    """

    # E2 و بررسی وجود در E1 نمونه‌گیری از
    sample_e1 = engine1.sample(sample_size)
    overlap_in_e2 = sum(p in engine2.pages for p in sample_e1)
    pct_e1_in_e2 = overlap_in_e2 / len(sample_e1)

    # E1 و بررسی وجود در E2 نمونه‌گیری از
    sample_e2 = engine2.sample(sample_size)
    overlap_in_e1 = sum(p in engine1.pages for p in sample_e2)
    pct_e2_in_e1 = overlap_in_e1 / len(sample_e2)

    # محاسبه نسبت
    if pct_e1_in_e2 == 0:
        return float('inf'), pct_e1_in_e2, pct_e2_in_e1

    ratio = pct_e2_in_e1 / pct_e1_in_e2
    return ratio, pct_e1_in_e2, pct_e2_in_e1

# -----
# سناریوی شبیه‌سازی
# -----

# برای نتایج قابل تکرار seed تنظیم
np.random.seed(42)

# ایجاد یک وب سنتتیک: 100,000 صفحه
all_pages = [f"page_{i}" for i in range(100000)]

# ایندکس 60% وب - Google :موتور جستجو 1
google = SearchEngine("Google", all_pages[:60000])

# Google ایندکس 40% وب با 30% همپوشانی با - Bing :موتور جستجو 2
bing = SearchEngine("Bing", all_pages[30000:70000])

# نسبت واقعی (برای مقایسه)
true_ratio = len(google.pages) / len(bing.pages)

print("Capture-Recapture شبیه‌سازی روش")
print("=" * 45)
print(f"|Google|/|Bing|: {true_ratio:.3f}")
print(f"Google: {len(google.pages):,}")
print(f"Bing: {len(bing.pages):,}")
```

```
print(f"تعداد صفحات مشترک: {len(google.pages & bing.pages):,}")
print()

# -----
# بررسی با اندازه‌های نمونه مختلف
# -----

sample_sizes = [100, 500, 1000, 5000, 10000]
estimates = []

for n in sample_sizes:
    ratio, x, y = capture_recapture(google, bing, n)
    error = abs(ratio - true_ratio) / true_ratio * 100
    estimates.append((ratio, error))
    print(f"نمونه: {n:6d} | تخمین: {ratio:6.3f} | خطا: {error:5.2f}% | x={x:.3f} y={y:.3f}")

# -----
# نمایش نتایج به صورت بصری
# -----

ratios = [e[0] for e in estimates]
errors = [e[1] for e in estimates]

plt.figure(figsize=(12, 5))

# نمودار 1: نسبت تخمین‌شده
plt.subplot(1, 2, 1)
plt.plot(sample_sizes, ratios, 'bo-', linewidth=2, markersize=6, label='Capture-Recapture Estimation')
plt.axhline(y=true_ratio, color='r', linestyle='--', linewidth=2, label=f'Real Ratio: {true_ratio:.3f}')
plt.xscale('log')
plt.xlabel('Sample Size (log scale)', fontsize=11)
plt.ylabel('Estimated Ratio |E1| / |E2|', fontsize=11)
plt.title('Capture-Recapture Estimation', fontsize=12, fontweight='bold')
plt.legend(fontsize=10)
plt.grid(True, which='both', ls='--', alpha=0.5)

# نمودار 2: خطای درصدی
plt.subplot(1, 2, 2)
plt.plot(sample_sizes, errors, 'ro-', linewidth=2, markersize=6, label='Estimation Error')
plt.xscale('log')
plt.xlabel('Sample Size (log scale)', fontsize=11)
plt.ylabel('Error (%)', fontsize=11)
plt.title('Estimation Error vs Sample Size', fontsize=12, fontweight='bold')
plt.legend(fontsize=10)
plt.grid(True, which='both', ls='--', alpha=0.5)

plt.tight_layout()
plt.show()

print("\n" + "=" * 60)
print("نکات کلیدی:")
print("1. افزایش اندازه نمونه → خطای تخمین کمتر")
print("2. این روش فرض می‌کند نمونه‌برداری یکنواخت از ایندکس ممکن است")
print("3. در واقعیت، نمونه‌برداری از طریق پرس‌وجوهای تصادفی انجام می‌شود")
print("4. پرس‌وجوها سوگیری دارند (مثلاً به سمت صفحات بلند یا محبوب)")
print("5. نیاز است (Horvitz-Thompson مثل) برای اصلاح سوگیری به روش‌های آماری پیشرفته")
print("نکته: نتایج برای نمونه‌های کوچک دارای واریانس بالایی هستند")
```

Capture-Recapture شبیه‌سازی روش

=====

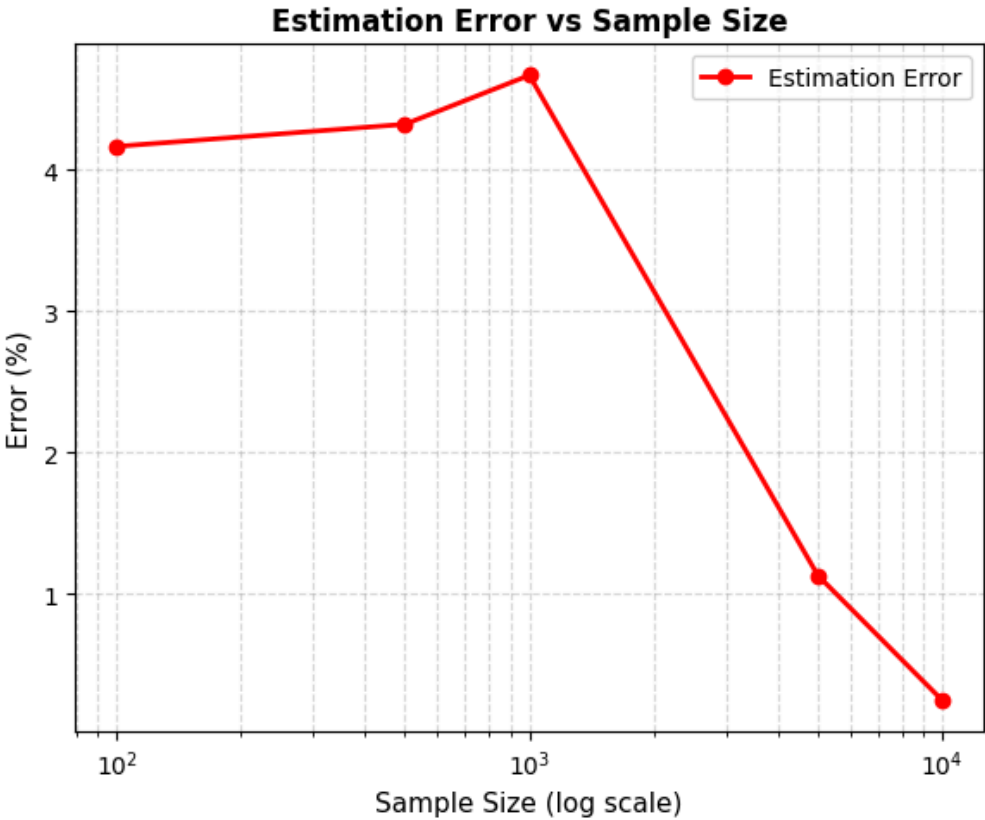
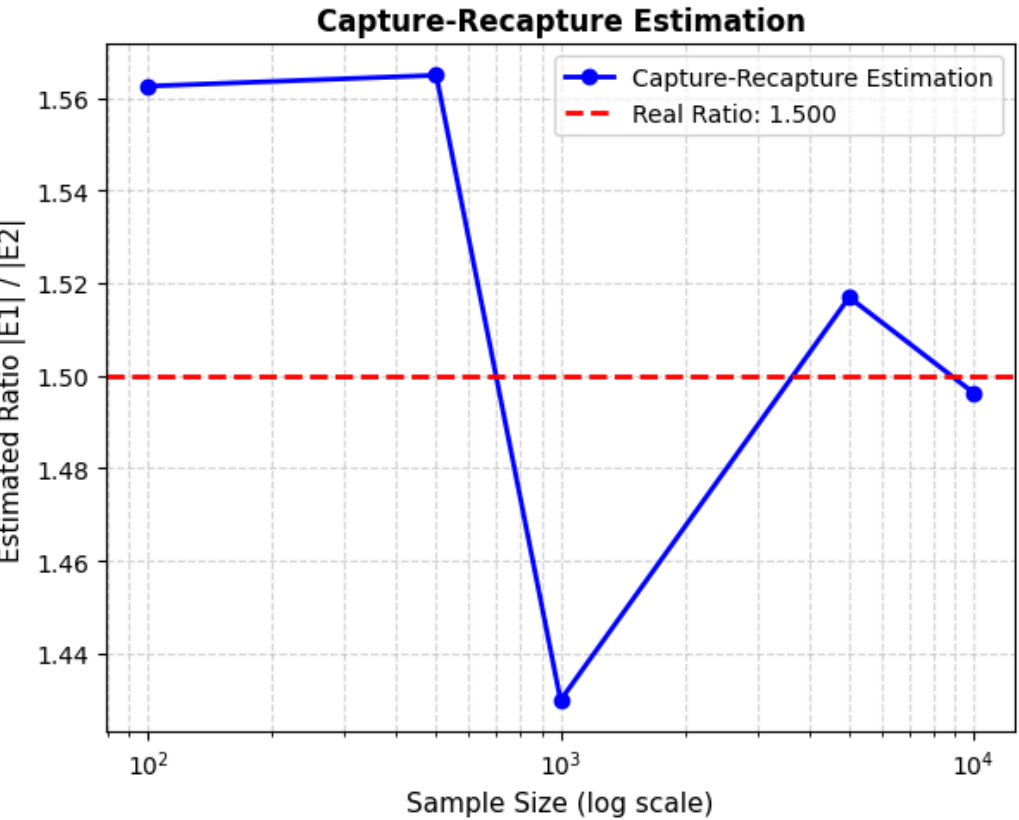
نسبت واقعی |Google|/|Bing|: 1.500

تعداد صفحات Google: 60,000

تعداد صفحات Bing: 40,000

تعداد صفحات مشترک: 30,000

4.17	خطا:	1.562	تخمین:	100	%نمونه:	x=0.480	y=0.750
4.32	خطا:	1.565	تخمین:	500	%نمونه:	x=0.478	y=0.748
4.67	خطا:	1.430	تخمین:	1000	%نمونه:	x=0.521	y=0.745
1.13	خطا:	1.517	تخمین:	5000	%نمونه:	x=0.490	y=0.744
0.25	خطا:	1.496	تخمین:	10000	%نمونه:	x=0.503	y=0.752



=====

نکات کلیدی:

- افزایش اندازه نمونه → خطای تخمین کمتر
- این روش فرض می‌کند نمونه‌برداری یکنواخت از ایندکس ممکن است
- در واقعیت، نمونه‌برداری از طریق پرس‌وجوهای تصادفی انجام می‌شود
- پرس‌وجوها سوگیری دارند (مثلاً به سمت صفحات بلند یا محبوب)
- نیاز است (Horvitz-Thompson مثل) برای اصلاح سوگیری به روش‌های آماری پیشرفته

نکته: نتایج برای نمونه‌های کوچک دارای واریانس بالایی هستند #

19.6 صفحات تقریباً تکراری و روش Shingling

چالش تکرار در وب

تخمین اندازه ایندکس بدون در نظر گرفتن **تکرارها** گمراه‌کننده است: تا ۴۰٪ **صفحات وب تکراری** یا **تقریباً تکراری** (Near-Duplicate) هستند. این پدیده دلایل مختلفی دارد:

- میرورهای قانونی:** سایت‌های خبری مانند CNN یا BBC محتوای خود را در چند سرور مختلف برای افزایش دسترسی و قابلیت اطمینان کپی می‌کنند.
- محتوای پویا:** صفحاتی که فقط در بخش‌های کوچکی مانند زمان آخرین به‌روزرسانی یا شمارنده بازدیدکنندگان تفاوت دارند.
- سامانه‌های مدیریت محتوا:** قالب‌های استاندارد که باعث ایجاد صفحات با ساختار بسیار مشابه اما محتوای کمی متفاوت می‌شوند.

برای مثال، خبر یک رویداد ورزشی در سایت‌های مختلف اغلب با متن اصلی یکسان است، اما با عناوین متفاوت یا با افزودن تاریخ و زمان انتشار. موتورهای جستجو باید بتوانند این صفحات را به‌عنوان نسخه‌های مختلف یک محتوای واحد شناسایی کنند تا از ایندکس کردن چندباره آن جلوگیری شود.

از اثر انگشت تا Shingling

برای تشخیص تکرارهای دقیق، می‌توان از **اثر انگشت** (Fingerprint) استفاده کرد - یک خلاصه ۶۴-بیتی از کل محتوای صفحه. اما این روش برای صفحاتی که فقط در چند کاراکتر تفاوت دارند، شکست می‌خورد.

برای حل این مشکل، از تکنیک **Shingling** استفاده می‌شود. در این روش، هر سند به‌عنوان مجموعه‌ای از **k-shingle** در نظر گرفته می‌شود که دنباله‌های پیوسته از k واژه در سند هستند.

برای مثال، متن خبری زیر را در نظر بگیرید:

"تیم ملی فوتبال ایران امروز در دیداری حساس به مصاف ژاپن رفت و با نتیجه ۲ بر یک به پیروزی رسید."

اگر $k=4$ باشد، shingle-4های این متن عبارتند از:

"تیم ملی فوتبال" و "ملی فوتبال ایران" و "فوتبال ایران امروز" و "ایران امروز در" و "امروز در دیداری" و "در دیداری حساس" و "دیداری حساس به" و "حساس به مصاف" و "به مصاف ژاپن" و "مصاف ژاپن رفت" و "ژاپن رفت و" و "رفت و با نتیجه" و "و با نتیجه ۲" و "با نتیجه ۲ بر" و "نتیجه ۲ بر یک" و "۲ بر یک به" و "بر یک به پیروزی" و "یک به پیروزی رسید"

حالا اگر خبر دیگری با متن زیر وجود داشته باشد:

"تیم ملی فوتبال ایران امروز در دیداری حساس به مصاف ژاپن رفت و با نتیجه ۲ بر یک به پیروزی رسید. این پیروزی در تاریخ ۱۴۰۲/۰۲/۱۰ ثبت شد."

ضرب جاکارد و الگوریتم کارآمد

برای اندازه گیری شباهت بین مجموعه shingleهای دو سند، از **ضرب جاکارد** استفاده می شود:

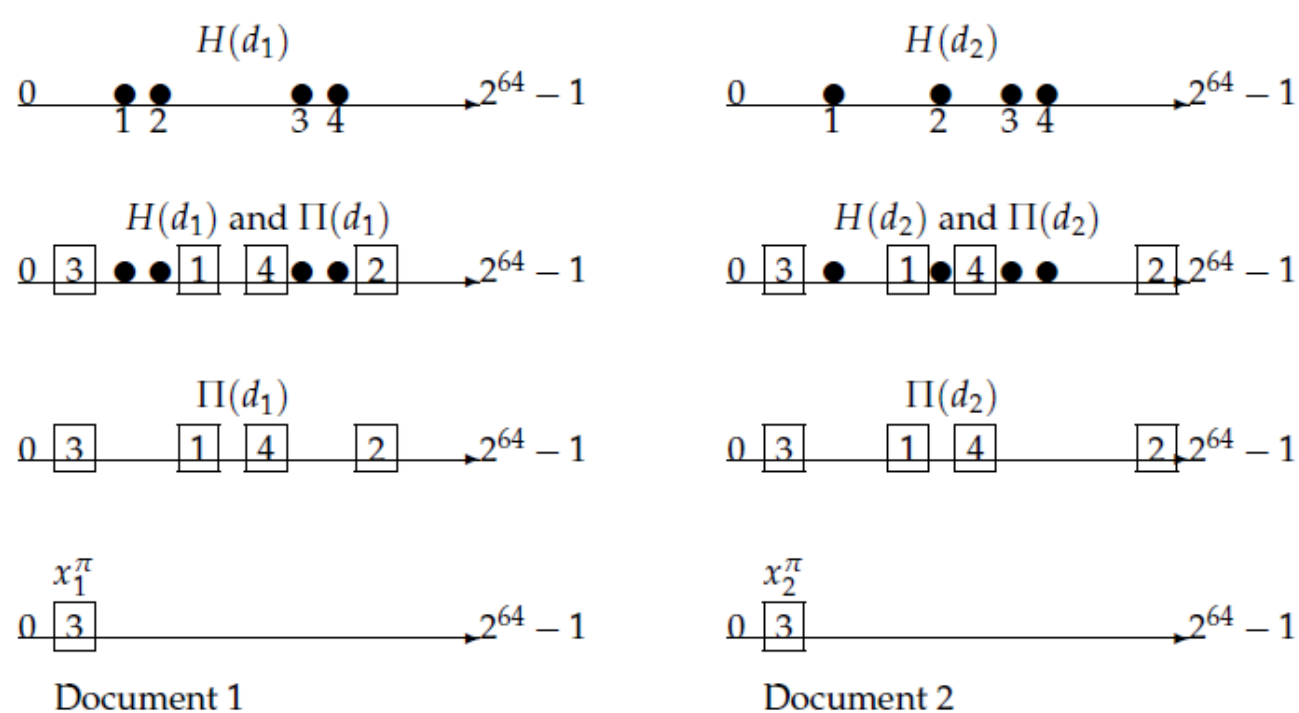
$$J(A, B) = |A \cap B| / |A \cup B|$$

اما محاسبه این ضرب برای میلیاردها سند غیرعملی است. راه حل، استفاده از تکنیک های مبتنی بر **درهم سازی (Hashing)** و **جایگشت تصادفی (Random Permutation)** است:

1. هر shingle را به یک عدد ۶۴-بیتی درهم سازی می کنیم.
2. یک جایگشت تصادفی روی فضای درهم سازی اعمال می کنیم.
3. کوچک ترین مقدار درهم سازی شده پس از جایگشت را به عنوان **امضا (Signature)** سند ذخیره می کنیم.

یک نتیجه شگفت انگیز (قضیه ۱۹.۱) نشان می دهد که احتمال برابری امضاهای دو سند، دقیقاً برابر با ضرب جاکارد آنهاست!

در عمل، این کار را برای **۲۰۰ جایگشت مستقل** تکرار می کنیم تا یک **اسکچ (Sketch)** از ۲۰۰ عدد برای هر سند بسازیم. سپس تعداد اشتراک اسکچ های دو سند را می شماریم و اگر این مقدار از آستانه ای (مثلاً ۱۶۰ از ۲۰۰) بیشتر بود، صفحات را تقریباً تکراری می دانیم.



► **Figure 19.8** Illustration of shingle sketches. We see two documents going through four stages of shingle sketch computation. In the first step (top row), we apply a 64-bit hash to each shingle from each document to obtain $H(d_1)$ and $H(d_2)$ (circles). Next, we apply a random permutation Π to permute $H(d_1)$ and $H(d_2)$, obtaining $\Pi(d_1)$ and $\Pi(d_2)$ (squares). The third row shows only $\Pi(d_1)$ and $\Pi(d_2)$, while the bottom row shows the minimum values x_1^π and x_2^π for each document.

شکل 19.8: تصویرسازی از محاسبه اسکچ shingle. دو سند در چهار مرحله پردازش می شوند: ابتدا هر shingle به یک مقدار ۶۴-بیتی درهم سازی می شود (دایره ها)، سپس یک جایگشت تصادفی اعمال می شود (مربع ها)، در مرحله سوم فقط مقادیر جایگشت یافته نمایش داده می شود، و در نهایت کوچک ترین مقادیر برای هر سند مشخص می شود.

بهینه سازی با Super-Shingles ⚡

حتی با این روش، مقایسه تمام جفت های ممکن از اسناد هنوز بسیار پرهزینه است. برای کاهش هزینه محاسباتی، از تکنیک **Super-Shingles** استفاده می شود:

1. اسکچ ۲۰۰ تایی هر سند را مرتب می‌کنیم.

2. از این اسکچ مرتب‌شده، shingleهای جدیدی (Super-Shingles) می‌سازیم.

3. فقط جفت‌های اسنادی که حداقل یک Super-Shingle مشترک دارند را با دقت کامل بررسی می‌کنیم.

این روش مانند یک فیلتر اولیه عمل می‌کند که اکثر جفت‌های غیرمشابه را حذف می‌کند و فقط به بررسی جفت‌های

امیدبخش می‌پردازد. با این بهینه‌سازی، موتورهای جستجو می‌توانند با کارایی بالا صفحات تقریباً تکراری را شناسایی کرده و از

ایندکس کردن چندباره آن‌ها جلوگیری کنند.

In [19]:

```
import random
import hashlib
import numpy as np
import matplotlib.pyplot as plt

# -----
# برای تشخیص صفحات تقریباً تکراری MinHash و Shingling پیاده‌سازی
# -----

class Shingling:
    """از متن k-shingles استخراج"""
    def __init__(self, k=4):
        self.k = k

    def get_shingles(self, text):
        """k-shingles تبدیل متن به مجموعه"""
        words = text.split()
        if len(words) < self.k:
            return set()
        return {tuple(words[i:i+self.k]) for i in range(len(words) - self.k + 1)}

class MinHash:
    """های ثابت permutation با MinHash پیاده‌سازی"""
    def __init__(self, num_permutations=200, seed=42):
        self.num_permutations = num_permutations
        # permutation ها یکبار تولید می‌شوند و ثابت می‌مانند
        random.seed(seed)
        self.permutations = [(random.randint(1, 2**64 - 1), random.randint(0, 2**64 - 1))
                               for _ in range(num_permutations)]

    def _hash_shingle(self, shingle):
        """به عدد 64 بیتی shingle درهم‌سازی یک"""
        shingle_str = " ".join(shingle)
        hash_obj = hashlib.md5(shingle_str.encode())
        return int.from_bytes(hash_obj.digest()[:8], byteorder='little', signed=False)

    def create_signature(self, shingles):
        """shingle برای یک مجموعه MinHash ساخت امضای"""
        if not shingles:
            return [0] * self.num_permutations

        hashed = [self._hash_shingle(s) for s in shingles]
        signature = []

        # های ثابت برای همه اسناد استفاده می‌شود permutation از همان
        for a, b in self.permutations:
            min_hash = min((a * h + b) % (2**64) for h in hashed)
            signature.append(min_hash)

        return signature

    def estimate_similarity(self, sig1, sig2):
        """تخمین ضریب جاکارد از روی دو امضا"""
        matches = sum(1 for a, b in zip(sig1, sig2) if a == b)
        return matches / len(sig1)

class NearDuplicateDetector:
    """MinHash و Shingling تشخیص صفحات تقریباً تکراری با استفاده از"""
    def __init__(self, k=4, threshold=0.7, num_perm=200, seed=42):
        self.k = k
        self.threshold = threshold
        self.shingler = Shingling(k)
        self.minhash = MinHash(num_perm, seed)

    def is_near_duplicate(self, text1, text2):
        """
        تشخیص اینکه آیا دو متن تقریباً تکراری هستند
        (تخمینی_jaccard, دقیق_jaccard, boolean): خروجی
        """
        # shingles محاسبه
        shingles1 = self.shingler.get_shingles(text1)
        shingles2 = self.shingler.get_shingles(text2)

        # محاسبه جاکارد دقیق برای مقایسه
        union = shingles1 | shingles2
        intersection = shingles1 & shingles2
        exact_jaccard = len(intersection) / len(union) if union else 0

        # MinHash محاسبه تخمین

```

```
sig1 = self.minhash.create_signature(shingles1)
sig2 = self.minhash.create_signature(shingles2)
est_jaccard = self.minhash.estimate_similarity(sig1, sig2)

is_duplicate = est_jaccard >= self.threshold
return is_duplicate, exact_jaccard, est_jaccard

# -----
# سناریوی آموزشی
# -----

# خبر اصلی
news1 = "تیم ملی فوتبال ایران امروز در دیدی حساس به مصاف ژاپن رفت و با نتیجه ۲ بر یک به پیروزی رسید"
```

خبر تقریباً تکراری: افزودن تاریخ و زمان

```
news2 = "تیم ملی فوتبال ایران امروز در دیدی حساس به مصاف ژاپن رفت و با نتیجه ۲ بر یک به پیروزی رسید این پیروزی در تاریخ ۱۴۰۲ ثبت شد"
```

خبر کاملاً متفاوت

```
news3 = "قیمت طلا در بازار جهانی با افزایش ۵ درصدی روبرو شد و به ۲۰۰۰ دلار در هر اونس رسید"
```

```
print("Shingling و MinHash تشخیص صفحات تقریباً تکراری با")
print("=" * 60)

# detector تنظیم (k=4, threshold=0.7, 200 permutation)
detector = NearDuplicateDetector(k=4, threshold=0.7, num_perm=200, seed=42)

# مقایسه 1 و 2
is_dup, exact, est = detector.is_near_duplicate(news1, news2)
print(f"\nمقایسه خبر اصلی و خبر با تاریخ اضافی:")
print(f"تعداد کلمات: {len(news1.split())} vs {len(news2.split())}")
print(f"جاکارد دقیق: {exact:.3f} (|A∩B|/|A∪B|)")
print(f"MinHash تخمین: {est:.3f} ({200} permutation)")
print(f"نتیجه: {'تقریباً تکراری ✓' if is_dup else 'X متفاوت'}")

# مقایسه 1 و 3
is_dup, exact, est = detector.is_near_duplicate(news1, news3)
print(f"\nمقایسه خبر اصلی و خبر متفاوت:")
print(f"جاکارد دقیق: {exact:.3f}")
print(f"MinHash تخمین: {est:.3f}")
print(f"نتیجه: {'تقریباً تکراری ✓' if is_dup else 'X متفاوت'}")

# -----
# ها permutation بررسی تجربی: تاثیر تعداد
# -----

def experiment_permutations():
    """ها، دقت افزایش می‌یابد permutation تایید قضیه ۱۹.۱: با افزایش"""
    perm_counts = [10, 20, 50, 100, 200, 500]
    results = []

    for count in perm_counts:
        detector = NearDuplicateDetector(k=4, threshold=0.7, num_perm=count, seed=42)
        _, exact, est = detector.is_near_duplicate(news1, news2)
        error = abs(est - exact) / exact * 100 if exact > 0 else 0
        results.append((est, error))

    # رسم نمودار
    plt.figure(figsize=(10, 6))
    estimates = [r[0] for r in results]
    errors = [r[1] for r in results]

    plt.plot(perm_counts, estimates, 'bo-', linewidth=2, markersize=6, label='MinHash Estimation')
    plt.axhline(y=exact, color='r', linestyle='--', linewidth=2, label=f'Exact Jaccard: {exact:.3f}')
    plt.xlabel('Number of Permutations', fontsize=12)
    plt.ylabel('Estimated Similarity', fontsize=12)
    plt.title('Theorem 19.1: Permutations vs Estimation Accuracy', fontsize=14, fontweight='bold')
    plt.legend(fontsize=10)
    plt.grid(True, which='both', ls='--', alpha=0.5)
    plt.xscale('log')
    plt.tight_layout()
    plt.show()

    # چاپ نتایج
    print("\nها بر دقت (قضیه ۱۹.۱) تاثیر تعداد:")
    print("=" * 50)
    for count, (est, error) in zip(perm_counts, results):
        print(f"count:4d} permutations: estimate={est:.3f}, error={error:.2f}%")

# اجرا
experiment_permutations()

# -----
# تحلیل Super-Shingles
# -----

def analyze_super_shingles(band_size=10):
    """فیلتر اولیه برای کاهش ۹۹٪ مقایسه‌ها Super-Shingles: نمایش کار مشترک دارند بررسی می‌شوند Super-Shingle فقط اسنادی که حداقل یک"""

    detector = NearDuplicateDetector(k=4, threshold=0.7, num_perm=200, seed=42)

    # محاسبه signatures
    sig1 = detector.minhash.create_signature(detector.shingler.get_shingles(news1))
    sig2 = detector.minhash.create_signature(detector.shingler.get_shingles(news2))
    sig3 = detector.minhash.create_signature(detector.shingler.get_shingles(news3))

    # (های متوالی band به signature تقسیم) Super-Shingles ساخت
    num_bands = len(sig1) // band_size
    super1 = {tuple(sig1[i*band_size:(i+1)*band_size]) for i in range(num_bands)}
```

```
super2 = {tuple(sig2[i*band_size:(i+1)*band_size]) for i in range(num_bands)}
super3 = {tuple(sig3[i*band_size:(i+1)*band_size]) for i in range(num_bands)}

# محاسبه اشتراک
common_12 = len(super1 & super2)
common_13 = len(super1 & super3)

print("\n" + "=" * 50)
print("Super-Shingles: بهینه‌سازی با")
print(f"در هر متن Super-Shingles تعداد: {num_bands}")
print(f"مشترک (خبر ۱ و ۲): {common_12}")
print(f"مشترک (خبر ۱ و ۳): {common_13}")

# تصمیم‌گیری بر اساس فیلتر
print(f"Super-Shingles: نتیجه فیلتر:")
if common_12 > 0:
    print("MinHash مشترک → بررسی دقیق با Super-Shingle خبر ۱ و ۲: دارای ✓")
else:
    print("مشترک → بدون بررسی دقیق رد می‌شوند Super-Shingle خبر ۱ و ۲: بدون X")

if common_13 > 0:
    print("مشترک → بررسی دقیق می‌شود Super-Shingle خبر ۱ و ۳: دارای ✓")
else:
    print("مشترک → بدون بررسی دقیق رد می‌شود Super-Shingle خبر ۱ و ۳: بدون X")

print("\n💡 نکته: Super-Shingles ۹۹٪ بدون محاسبه دقیق رد می‌کند")

# اجرا
analyze_super_shingles(band_size=10)

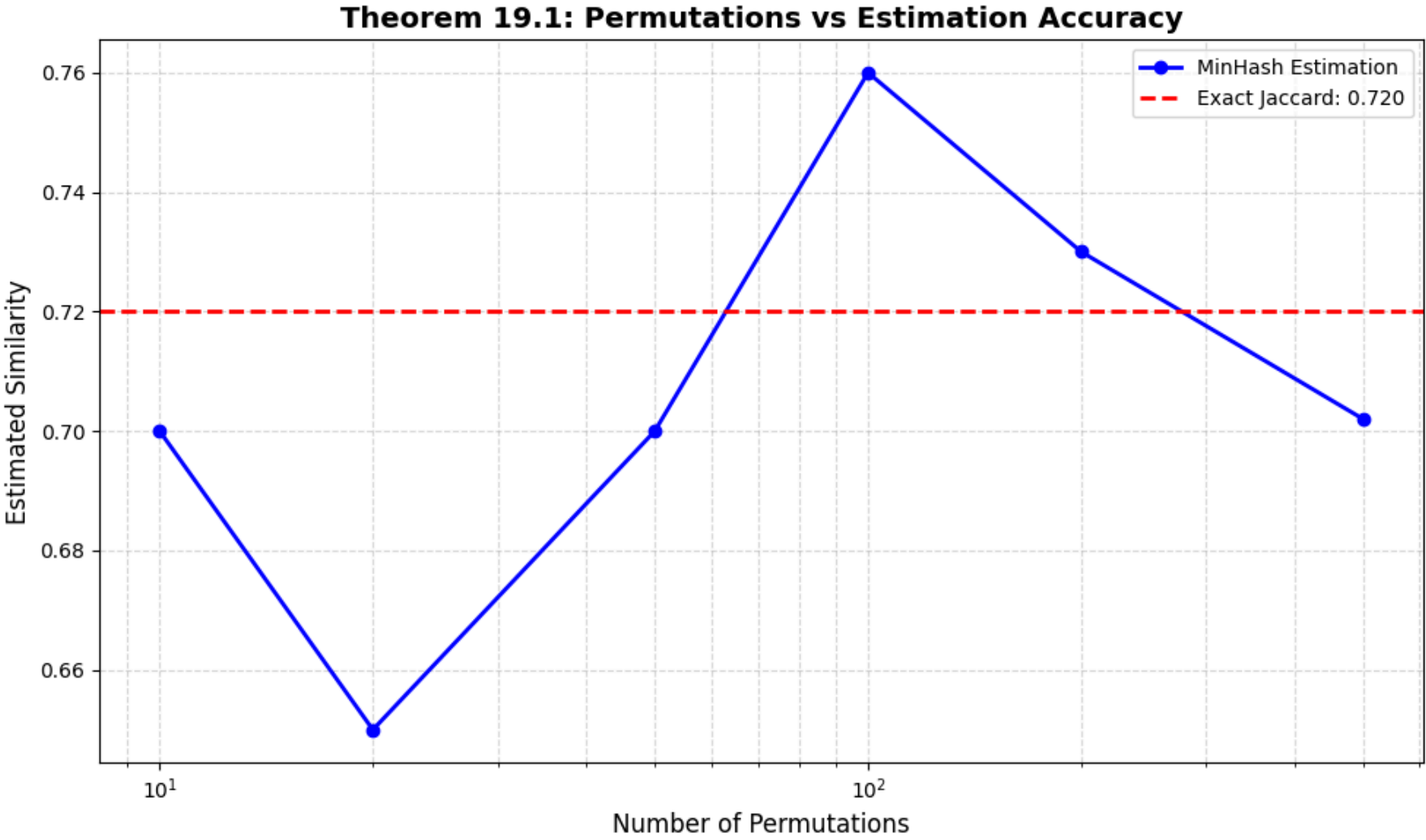
# -----
# نکات کلیدی آموزشی
# -----

print("\n" + "=" * 70)
print("نکات کلیدی:")
print("1. Shingling: تبدیل متن به مجموعه k-shingles (k-gramها)")
print("2. ضریب جاکارد: معیار دقیق شباهت = |AnB|/|AUB|")
print("3. MinHash: 200 permutation تخمین سریع جاکارد با (۱۹.۱ قضیه)")
print("4. ها، خطا کاهش می‌یابد permutation دقت: با افزایش")
print("5. Super-Shingles: ۹۹٪ مقایسه‌ها را حذف می‌کند (فیلتر اولیه،)")
print("6. کاربرد: گوگل و بینگ برای حذف ۴۰٪ صفحات تقریباً تکراری استفاده می‌کنند")
print("7. Trade-off: (band size) و سرعت (permutation تعداد) بین دقت")
```

MinHash و Shingling تشخیص صفحات تقریباً تکراری با
=====

مقایسه خبر اصلی و خبر با تاریخ اضافی
28 vs تعداد کلمات: 21
(|AnB|/|AUB|) جاکارد دقیق: 0.720
MinHash تخمین (200 permutation): 0.730
نتیجه: ✓ تقریباً تکراری

مقایسه خبر اصلی و خبر متفاوت
جاکارد دقیق: 0.000
MinHash تخمین: 0.000
نتیجه: X متفاوت



ها بردقت (قضیه ۱۹.۱) permutation تاثیر تعداد
=====

10 permutations: estimate=0.700, error=2.78%
20 permutations: estimate=0.650, error=9.72%
50 permutations: estimate=0.700, error=2.78%
100 permutations: estimate=0.760, error=5.56%
200 permutations: estimate=0.730, error=1.39%
500 permutations: estimate=0.702, error=2.50%

=====

Super-Shingles بهینه‌سازی با
در هر متن: Super-Shingles 20 تعداد
مشترک (خبر ۱ و ۲): Super-Shingles 1
مشترک (خبر ۱ و ۳): Super-Shingles 0

Super-Shingles: نتیجه فیلتر
MinHash مشترک → بررسی دقیق با Super-Shingle خبر ۱ و ۲: دارای ✓
مشترک → بدون بررسی دقیق رد می‌شود Super-Shingle خبر ۱ و ۳: بدون X

!جفت‌های غیرمشابه را بدون محاسبه دقیق رد می‌کند ۹۹٪ Super-Shingles نکته 🟡

=====

نکات کلیدی:

- Shingling: تبدیل متن به مجموعه k-shingles (k-gramها)
- $|A \cap B| / |A \cup B|$ = ضریب جاکارد: معیار دقیق شباهت
- MinHash: 200 تخمین سریع جاکارد با
- ها، خطا کاهش می‌یابدpermutation دقت: با افزایش
- فیلتر اولیه، ۹۹٪ مقایسه‌ها را حذف می‌کند Super-Shingles:
- کاربرد: گوگل و بینگ برای حذف ۴۰٪ صفحات تقریباً تکراری استفاده می‌کنند
- band size) و سرعت (permutation تعداد) بین دقت Trade-off:

جمع‌بندی فصل ۱۹: مبانی جستجوی وب 🌐

چگونه می‌توان در یک محیط بی‌نهایت، پراکنده و شلوغ و در حال تغییر دائمی، اطلاعات مرتبط را پیدا کرد؟ فصل ۱۹ ما را به دنیای پیچیده و منحصر به فرد ****جستجوی وب**** می‌برد؛ جایی که در آن، نه تنها فناوری بازیابی اطلاعات، بلکه اقتصاد، روانشناسی کاربران و حتی نبردهای تجاری در کار است. این فصل نشان می‌دهد که چرا جستجوی وب چیزی فراتر از یک «کتابخانه دیجیتالی» بزرگ است و چه چالش‌های منحصر به فردی آن را از بازیابی اطلاعات سنتی متمایز می‌کند.

- با ****پیشینه و تاریخچه وب**** آشنا شدیم. وب با سه ویژگی اصلی تعریف می‌شود: مقیاس بی‌سابقه، عدم وجود مرکزیت و تنوع فوق‌العاده در انگیزه‌های نویسندگان. این ویژگی‌ها وب را به یک «غرب وحشی» اطلاعات تبدیل کرد که مدیریت آن نیازمند رویکردهای کاملاً جدید بود.

- ساختار ****گراف وب (Web Graph)**** را به عنوان مدل بنیادی برای درک روابط بین صفحات بررسی کردیم. هر صفحه یک گره (گره) و هر پیوند یک یال (لبه) است. با استفاده از این مدل، مفاهیمی مانند ****توانگرایی (شاخص رتبه صفحه PageRank)**** و ساختار ****کمان و پاپیون (Bowtie)**** قابل توضیح هستند.

- با ****چالش هرزنامه‌نگاری (Spam)**** روبرو شدیم. انگیزه‌های تجاری باعث به وجود محتوای دستکاری شده برای فریب دادن به موتورهای جستجو شده است. تکنیک‌هایی مانند ****پنهان‌کاری (Cloaking)**** که در آن یک صفحه متفاوت به ربات جستجو و کاربر نمایش داده می‌شود، نمونه‌ای از این نبردهای هوشمندانه است.

- مدل ****اقتصادی و تجارت الکترونیک وب**** را بررسی کردیم. مدل‌های اولیه مبتنی بر نمایش آگهی (CPM) بودند، اما امروز مدل‌های مبتنی بر ****هزینه به ازای هر کلیک (CPC)**** و ****جستجوی حمایت‌شده (Sponsored Search)**** غالب هستند. این مدل‌ها به شرکت‌ها اجازه می‌دهند تا مستقیماً در مقابل کاربرانی که به دنبال محصولات یا خدماتشان هستند، ظاهر شوند.

- با ****تجربه کاربر در جستجوی وب**** آشنا شدیم. برخلاف کاربران حرفه‌ای پایگاه‌های اطلاعاتی، کاربران وب معمولاً جستارهای کوتاه و بدون عملگرهای پیچیده انجام می‌دهند. درک نیازهای آن‌ها به سه دسته اصلی تقسیم می‌شوند: ****اطلاعاتی**** (مانند جستجوی «لوکمی خون»)، ****ناوبری**** (مانند جستجوی نام یک وب‌سایت خاص)، و ****تراکنشی یا معاملتی**** (مانند جستجوی «خرید بلیط هوایی»).

- در نهایت، به چالش ****اندازه‌گیری نمایه (Index Size Estimation)**** و ****تشخیص نسخه‌های نزدیک به هم (Near-Duplicates)**** با استفاده از تکنیکی ****شینگلینگ (Shingling)**** پرداختیم این تکنیک‌ها برای مدیریت حجم عظیم وب و جلوگیری از ایندکس‌سازی محتوای تکراری ضروری هستند.

بیاید این مفاهیم را با چند مثال واقعی و کاربردی زنده کنیم:

❖ **ساختار گراف وب و توانگرایی:** تصور کنید هر وبسایت یک شهر کوچک و هر پیوند یک جاده یکطرفه است.

وبسایت‌های بزرگی مانند ویکی‌پدیا یا خبرگزاری‌های معتبر، «میدان‌های» بزرگی هستند که تعداد زیادی جاده به آن‌ها وارد می‌شود (in-degree بالا). موتورهای جستجو مانند گوگل از این مفهوم استفاده می‌کنند و به صفحاتی که از منابع معتبر زیادی لینک گرفته‌اند، امتیاز بالاتری می‌دهند. این همان منطق ****توانگرایی**(شاخص رتبه صفحه PageRank) ****** است: یک صفحه تا زمانی معتبر است که از صفحات معتبر دیگر به آن ارجاع داده شود.

💰 **هرزنامه‌نگاری و پنهان‌کاری:** یک آگهی املاک در «گلف کوچ مائویی» را در نظر بگیرید. یک هرزنامه‌نگار ممکن است وبسایتی بسازد که عبارت «گلف کوچ مائویی املاک» را صدها بار تکرار کند تا در نتایج جستجوی این عبارت رتبه بالایی بگیرد. وقتی کاربر روی این لینک کلیک می‌کند، به صفحه‌ای کاملاً متفاوت هدایت می‌شود که تبلیغات گران‌قیمت‌تری دارد. اما یک موتور جستجو ممکن است با استفاده از تکنیک ****پنهان‌کاری** (Cloaking) ****** متوجه این تقلب شود. در این تکنیک، سرور وبسایت به ربات جستجو یک صفحه با محتوای مرتبط با گلف (که برای رتبه بالا بهینه شده) نمایش می‌دهد، اما وقتی یک کاربر واقعی روی همان لینک کلیک می‌کند، او را به صفحه‌ای با تبلیغات و پیشنهادهای خرید هدایت می‌دهد. این مانند این است که یک مغازه‌دار دو چهره متفاوت به بازرس و مشتری نشان دهد!

👤 **مدل تجاری و جستجوی حمایت‌شده:** یک فروشگاه کوچک آنلاین که «کیف چرمی دست‌دوز» می‌فروشد، می‌خواهد وقتی کاربری عبارت «خرید کیف چرمی» را جستجو می‌کند، در بالاترین نتایج ظاهر شود. در مدل ****جستجوی حمایت‌شده****، این فروشگاه می‌تواند به موتور جستجو (مانند گوگل) مبلغی پرداخت کند تا وقتی کاربر این عبارت را جستجو می‌کند، آگهی فروشگاه او در بالای لیست (معمولاً با علامت «Ad») نمایش داده شود. اگر کاربر روی آگهی کلیک کند، فروشگاه هزینه‌ای اندک (معمولاً چند سنت) به موتور جستجو می‌پردازد. این مدل به کاربران اجازه می‌دهد تا خدمات مرتبط را پیدا کنند و به کسب‌وکارها اجازه می‌دهد تا دقیقاً در مقابل مشتریان بالقوه ظاهر شوند.

🔍 **اندازه‌گیری نمایه وب:** چگونه می‌توان تخمین زد که گوگل چند میلیارد صفحه را ایندکس کرده است؟ این کار مستقیم نیست، زیرا وب دائماً در حال تغییر است و صفحات پویا (Dynamic Pages) بی‌نهایت هستند. یک روش ساده، ****نمونه‌گیری مجدد** (Capture-Recapture) ******، این است: یک موتور جستجو ۱۰۰۰ صفحه تصادفی از وب برمی‌دارد و «علامت‌گذاری» می‌کند (مثلاً با افزودن یک رشته خاص به URL). سپس چند ماه بعد، دوباره ۱۰۰۰ صفحه تصادفی برمی‌دارد و بررسی می‌کند که چند مورد از صفحات علامت‌گذاری‌شده در نمونه جدید وجود دارند. اگر از ۱۰۰۰ صفحه اولیه، ۱۰ صفحه در نمونه دوم وجود داشته باشد، می‌توان تخمین زد که نمایه موتور جستجو حدوداً ۱۰ برابر کل وب است. این روش ساده، اما به ما درکی از مقیاس وب کمک می‌کند.

🌿 **تشخیص نسخه‌های نزدیک به هم با شینگلینگ:** دو خبرگزاری مختلف، یکی در تهران و دیگری در نیویورک، ممکن است یک گزارش کامل از یک بازی فوتبال را پوشش دهند. متن اصلی گزارش در هر دو سایت تقریباً یکسان است، اما نام خبرنگار، تاریخ و برخی جزئیات کوچک متفاوت است. الگوریتم ****شینگلینگ** (Shingling) ****** به جای مقایسه کل متن، فقط «اثر انگشتان» (شینگل‌ها) را مقایسه می‌کند. برای مثال، هر ۴ کلمه متوالی در متن یک شینگل تشکیل می‌شود. اگر دو مقاله تعداد زیادی شینگل مشترک داشته باشند، به احتمال زیاد نسخه‌ای از یکدیگر هستند. این روش به موتورهای جستجو اجازه می‌دهد تا با ذخیره کردن فقط یکی از نسخه‌ها در نمایه خود، از اتلاف حافظه و پردازش بیهوده جلوگیری کنند.

این فصل نشان داد که جستجوی وب یک حوزه چندرشته‌ای است که در آن، فناوری اطلاعات با اقتصاد دیجیتال و رفتار کاربران در هم می‌آمیزد. درک موفقیت یک موتور جستجوی مدرن نه تنها در بهینه‌سازی الگوریتم‌های بازیابی، بلکه در درک این اکوسیستم پیچیده و طراحی سیستم‌هایی است که بتوانند با هرزنامه‌نگاری هوشمندانه، مدل‌های تجاری منصفانه و تجربه کاربری بهینه مقابله کنند.

End of Chapter Exercises

مسئله 19.1: تفاوت وب و بازیابی سنتی (Web vs. Traditional IR)

محیط وب 3 تفاوت عمده با سیستم‌های سنتی بازیابی اطلاعات (مثل سیستم کتابخانه‌ها) دارد. این 3 تفاوت را نام برده و به صورت خلاصه توضیح دهید که چرا این تفاوت‌ها کار موتورهای جستجو را سخت‌تر می‌کنند؟

مسئله 19.2: ماهیت پویای وب (Dynamic Nature of Web)

در بخش تاریخچه و ویژگی‌های وب، اشاره شده است که وب یک موجودیت ایستا نیست. تفاوت «صفحات استاتیک» و «صفحات دینامیک» چیست و صفحات دینامیک چه چالشی برای خزنده‌های وب (Crawlers) ایجاد می‌کنند؟

مسئله 19.3: مدل پایونی گراف وب (Bow-tie Structure of the Web)

با توجه به ساختار گراف وب، 3 مولفه اصلی IN، SCC و OUT را تعریف کنید. اگر یک صفحه وب هیچ لینک خروجی نداشته باشد اما لینک‌های ورودی متعددی داشته باشد، در کدام بخش از این مدل قرار می‌گیرد؟

مسئله 19.4: لوله‌ها و شاخک‌ها در گراف وب (Tendrils and Tubes in Web Graph)

در مدل Bow-tie، مفاهیم Tendril (شاخک) و Tube (لوله) به چه معنا هستند؟ مسیر یک Tube از کجا به کجا ختم می‌شود؟

مسئله 19.5: تعریف اسپم در جستجو (Definition of Search Spam)

اسپم وب (Web Spam) یا Adversarial IR چیست؟ چرا در محیط وب، تولیدکنندگان محتوا انگیزه اقتصادی بالایی برای دور زدن الگوریتم‌های موتور جستجو دارند؟

مسئله 19.6: تکنیک پنهان‌کاری (Cloaking Technique)

مفهوم Cloaking (پنهان‌کاری) در اسپم وب را توضیح دهید. در این روش، اسپمر چگونه موتور جستجو و کاربر را به صورت همزمان فریب می‌دهد؟

مسئله 19.7: انواع تبلیغات اینترنتی (Types of Internet Advertising)

تفاوت بین تبلیغات نمایشی (Display Ads) و تبلیغات جستجوی حمایت‌شده (Sponsored Search) چیست؟ موتورهای جستجو عموماً بر کدام یک متکی هستند؟

مسئله 19.8: مدل‌های قیمت‌گذاری تبلیغات (Ad Pricing Models)

در مدل اقتصادی موتورهای جستجو، دو واژه **CPM** و **CPC** به چه معنا هستند؟ تفاوت محاسبه هزینه در این دو مدل را در یک خط بیان کنید.

مسئله 19.9: انواع نیازهای اطلاعاتی کاربر (User Query Needs)

طبق دسته‌بندی آندری برودر (**Andrei Broder**)، کوئری‌های کاربران به 3 دسته اصلی تقسیم می‌شوند. آن‌ها را نام برده و برای هر کدام یک مثال واقعی بنویسید:

- Informational**: مثال مانند جستجوی "تاریخچه ماشین‌های برقی".
- Navigational**: مثال مانند جستجوی "ورود به سایت دانشگاه".
- Transactional**: مثال مانند جستجوی "خرید لپ‌تاپ مدل X".

مسئله 19.10: نتایج ارگانیک در برابر پولی (Organic vs. Paid Results)

در صفحه نتایج جستجو (**SERP**)، چه تفاوتی بین نتایج ارگانیک (**Algorithmic**) و نتایج اسپانسر شده وجود دارد؟ موتور جستجو چگونه رتبه هر کدام را تعیین می‌کند؟

مسئله 19.11: چالش تخمین اندازه ایندکس (Index Size Estimation Challenge)

چرا محاسبه دقیق تعداد کل صفحات وب (**The size of the Web**) عملاً غیرممکن است؟ 2 دلیل فنی برای این محدودیت ذکر کنید.

مسئله 19.12: مفهوم شینگل (Concept of Shingling)

شینگل (**Shingle**) به زیرمجموعه‌ای از 3 توکن متوالی در یک متن گفته می‌شود که برای تشخیص شباهت میان اسناد استفاده می‌گردد. اگر عبارت **The sky is blue** را داشته باشیم، مجموعه 3-shingle های آن را استخراج کنید

مسئله 19.13: تحلیل مسیر در گراف وب (Path Analysis in Web Graph)

یک **Crawler** خزش خود را از گره‌ای در بخش **IN** در مدل **Bow-tie** آغاز کرده است. آیا این **Crawler** با دنبال کردن لینک‌های خروجی می‌تواند به گره‌های بخش **OUT** برسد؟ مسیر حرکت (از طریق چه بخش‌هایی) را رسم کنید.

مسئله 19.14: شناسایی مزرعه لینک (Link Farm Detection Scenario)

شما در حال بررسی گراف وب یک شبکه از سایت‌ها هستید. متوجه می‌شوید 1000 دامنه مختلف وجود دارند که هیچ محتوای مفیدی ندارند اما همگی به صورت متقاطع به یکدیگر لینک داده‌اند و در نهایت همگی به سایت **X** لینک خروجی دارند. این ساختار چه نام دارد و چه تاثیری بر الگوریتم‌هایی نظیر **PageRank** می‌گذارد؟

مسئله 19.15: مبارزه با کلمات کلیدی پنهان (Combating Hidden Keywords)

یک اسپمر برای بالا بردن رتبه سایت خود، هزاران کلمه کلیدی پرجستجو را با رنگ سفید روی پس‌زمینه سفید در صفحه خود قرار داده است. به عنوان طراح موتور جستجو، چه سناریویی (الگوریتمی) پیاده می‌کنید تا صفحات دارای این نوع اسپم (

مسئله 19.16: حراج قیمت دوم (Second-Price Auction Calculation)

سه تبلیغ‌کننده برای کلمه کلیدی «خرید موبایل» در موتور جستجو بید (Bid) داده‌اند:

کاربر A: پیشنهاد 3 دلار

کاربر B: پیشنهاد 2.5 دلار

کاربر C: پیشنهاد 1 دلار

با فرض استفاده از سیستم Generalized Second-Price (GSP) و اینکه تنها 2 جایگاه تبلیغاتی وجود دارد، کدام

تبلیغ‌کنندگان برنده می‌شوند و هر کدام بابت هر کلیک دقیقاً چه مبلغی پرداخت می‌کنند؟

مسئله 19.17: محاسبه بازگشت سرمایه تبلیغات (Ad ROI Scenario)

یک فروشگاه اینترنتی برای هر کلیک روی تبلیغ خود در موتور جستجو 0.5 دلار پرداخت می‌کند. نرخ تبدیل (Conversion Rate)

(این سایت 2% است. اگر سود خالص فروشگاه از فروش هر محصول 40 دلار باشد، آیا این کمپین تبلیغاتی سودآور است؟ سود

یا زیان خالص به ازای هر 1000 کلیک را محاسبه کنید.

مسئله 19.18: ابهام در کوئری کاربر (Ambiguous Query Resolution)

کاربری کلمه Jaguar را جستجو می‌کند. این کوئری می‌تواند به ماشین جگوار، حیوان جگوار، یا سیستم عامل مک (

Mac OS X Jaguar) اشاره داشته باشد. طراحی یک SERP (صفحه نتایج) را پیشنهاد دهید که ریسک نارضایتی کاربر در این

کوئری مبهم را به حداقل برساند.

مسئله 19.19: تصحیح املائی بر اساس لاگ جستجو (Spell Correction via Query Logs)

موتور جستجوی شما روزانه میلیون‌ها کوئری دریافت می‌کند. چگونه می‌توانید از «رفتار کاربران در اصلاح کوئری‌های خود» (

Query Reformulation Logs) برای ساخت یک سیستم پیشنهاد تصحیح املائی (Did you mean) به صورت خودکار

استفاده کنید؟ یک سناریو بنویسید.

مسئله 19.20: تخمین اندازه وب با روش Capture-Recapture (Capture-Recapture Index Estimation)

موتور جستجوی X دارای $N_1 = 40 \times 10^9$ صفحه است. موتور Y دارای $N_2 = 30 \times 10^9$ صفحه است. با استفاده از

نمونه‌گیری تصادفی، متوجه می‌شویم که احتمال اینکه یک صفحه تصادفی ایندکس شده توسط Y در X نیز وجود داشته باشد،

0.6 است (یعنی 60% اشتراک). تخمین شما از اندازه کل وب (N) چقدر است؟

مسئله 19.21: چالش تولید کوئری تصادفی (Random Query Generation Challenge)

برای اجرای روش Capture-Recapture نیاز به انتخاب صفحات «کاملاً تصادفی» داریم. موتورهای جستجو اجازه دسترسی به

دیتابیس خود را نمی‌دهند و ما مجبوریم از طریق ارسال کوئری جستجو این کار را بکنیم. چرا ساختن یک «کوئری تصادفی» برای

رسیدن به «صفحه تصادفی» در وب به شدت چالش‌برانگیز و دارای سوگیری (Bias) است؟ 2 دلیل بیاورید.

مسئله 19.22: محاسبه شباهت ژاکارد (Jaccard Similarity Calculation)

دو سند A و B به صورت مجموعه‌ای از کلمات داده شده‌اند:

A = {apple, banana, cherry, date}

B = {banana, cherry, fig, grape}

ضریب شباهت ژاکارد $J(A,B)$ را محاسبه کنید. فرمول و مراحل را بنویسید.

مسئله 19.23: تاثیر اندازه شینگل در کشف کپی‌کاری (Impact of Shingle Size k)

فرض کنید می‌خواهیم اسناد تکراری را با استفاده از **k-shingling** پیدا کنیم. اگر $k=1$ (کلمات تکی) باشد چه مشکلاتی به وجود می‌آید؟ اگر k خیلی بزرگ باشد (مثلاً $k=15$) چه خطایی در تشخیص صفحات **Near-duplicate** رخ می‌دهد؟ اندازه مناسب برای متون انگلیسی معمولاً چقدر در نظر گرفته می‌شود؟

مسئله 19.24: تله‌های عنکبوتی (Spider Traps Scenario)

تله عنکبوتی (**Spider Trap**) چیست؟ فرض کنید یک وب‌سایت با استفاده از اسکریپت، تقویمی طراحی کرده که کاربر با کلیک روی هر ماه، به ماه بعدی می‌رود (تا بی‌نهایت). خزنده شما چگونه ممکن است در این تله گرفتار شود و چه محدودیت مهندسی ساده‌ای می‌توان برای جلوگیری از آن در نظر گرفت؟

مسئله 19.25: معماری خزنده مقاوم در برابر گراف‌های خصمانه (Adversarial-Resistant Crawler Architecture) ★

یک شبکه از اسپمرها، یک گراف وب خصمانه طراحی کرده‌اند که شامل میلیون‌ها صفحه داینامیک است که به صورت متقاطع با معماری گراف کامل (**Complete Graph**) به هم لینک شده‌اند تا پهنای باند **Crawler** موتور جستجو را فلج کنند. به عنوان معمار ارشد سیستم، مکانیزمی مبتنی بر تحلیل دامنه و **URL Pattern** پیشنهاد دهید که **Crawler** به صورت درنگام (Real-time) این گره‌های مخرب را تشخیص داده و **Prune** (هرس) کند.

مسئله 19.26: تقلب در کلیک و یادگیری ماشین (Click Fraud Detection System) ★

در مدل اقتصادی **CPC**، رقبای تجاری انگیزه دارند تا با کلیک‌های رباتی و جعلی روی تبلیغ شما، بودجه روزانه شما را تمام کنند (Click Fraud). به عنوان یک مهندس داده، 5 فیچر (**Feature**) رفتاری و شبکه‌ای استخراج کنید که سیستم یادگیری ماشین شما بتواند کلیک‌های واقعی کاربران (**Human**) را از بات‌نت‌های توزیع شده (**Botnets**) متمایز کند.

مسئله 19.27: اثبات سوگیری در تخمین اندازه وب (Proof of Bias in Web Estimation) ★

در فرمول استاندارد **Capture-Recapture**، فرض بر این است که احتمال ایندکس شدن یک صفحه در موتور جستجوی **X** کاملاً مستقل از موتور **Y** است.

- الف) به صورت منطقی اثبات کنید که چرا این فرض در وب واقعی نقض می‌شود (پدیده **Preferential Attachment**).

- (ب) نشان دهید که این وابستگی (همپوشانی بالای موتورهای در ایندکس صفحات معروف)، باعث می‌شود فرمول استاندارد، اندازه واقعی وب را بیش‌برآورد (**Overestimate**) کند یا کم‌برآورد (**Underestimate**)؟

مسئله 19.28: طراحی مکانیزم حراج با احتساب CTR (GSP Auction with CTR Variation) ★

در موتورهای جستجوی واقعی (مثل گوگل)، حراج فقط بر اساس قیمت پیشنهادی (**Bid**) نیست، بلکه احتمال کلیک شدن تبلیغ (**Expected CTR**) نیز دخیل است تا درآمد موتور جستجو ماکزیمم شود.

فرض کنید:

تبلیغ‌دهنده **A** : پیشنهاد 3 دلار، **CTR** تخمینی: 1%

تبلیغ‌دهنده **B** : پیشنهاد 2 دلار، **CTR** تخمینی: 3%

تبلیغ‌دهنده **C** : پیشنهاد 1 دلار، **CTR** تخمینی: 2%

- (الف) موتور جستجو این تبلیغات را بر اساس چه متریکی (فرمول) مرتب می‌کند؟
- (ب) ترتیب نمایش تبلیغات را محاسبه کنید.
- (ج) بر اساس **GSP** اصلاح‌شده، تبلیغ‌دهنده‌ای که در جایگاه اول قرار می‌گیرد، بابت هر کلیک دقیقاً چقدر باید بپردازد؟

مسئله 19.29: پیاده‌سازی مفهومی MinHash (MinHash Conceptual Implementation) ★

شما باید شباهت یک میلیون سند را حساب کنید. محاسبه ماتریس کامل ژاکارد به $O(N^2)$ زمان نیاز دارد که غیرممکن است. مکانیزم الگوریتم **MinHash** را در 4 مرحله (به‌صورت شبه‌کد یا فلوچارت دقیق) شرح دهید. توضیح دهید چگونه استفاده از توابع هش تصادفی (**Random Hash Functions**) روی اندیس‌های ماتریس ویژگی‌ها، به ما اجازه می‌دهد شباهت ژاکارد را با مقایسه بردارهایی بسیار کوچک‌تر (**Signatures**) تخمین بزنیم.

مسئله 19.30: معضل هش کردن محلی با LSH (Locality Sensitive Hashing - LSH) ★

حتی با داشتن **Signature** های **MinHash**، مقایسه دو به دو امضاها برای یک میلیارد صفحه همچنان $O(N^2)$ است. تکنیک **LSH** (**Locality Sensitive Hashing**) چگونه این مشکل را حل می‌کند؟ مکانیزم تقسیم امضاها به بندها (**Bands**) و ردیف‌ها (**Rows**) و هش کردن آن‌ها در سطل‌ها (**Buckets**) را برای یافتن **Candidates** (نامزدهای شباهت) به دقت تحلیل کنید.

مسئله 19.31: اسپم توزیع شده و TrustRank (Distributed Spam and TrustRank) ★

الگوریتم **PageRank** در برابر مزارع لینک (**Link Farms**) به‌شدت آسیب‌پذیر است. محققان برای رفع این مشکل **TrustRank** را معرفی کردند.

- (الف) ایده اصلی **TrustRank** چیست و مجموعه هسته (**Seed Set**) در آن چگونه انتخاب می‌شود؟
- (ب) فرمول یا مکانیزم انتشار «اعتماد» (**Trust**) در گراف وب را بنویسید و توضیح دهید چگونه یک سایت با هزاران بکلینک اسپم، رتبه اعتماد پایینی دریافت می‌کند.

مسئله 19.32: معماری موتور جستجوی درنگام (Real-time Search Engine Architecture) ★

برای رویدادهای زنده (مثل مسابقات فینال جام جهانی یا اخبار فوری سیل)، موتور جستجو باید صفحاتی که یک ثانیه پیش تولید شده‌اند را ایندکس و در رتبه اول نشان دهد. در حالی که فرآیند خزش گراف وب و محاسبه **PageRank** هفته‌ها زمان می‌برد. سیستمی طراحی کنید که شامل دو لایه **Indexing** باشد (لایه **Freshness** و لایه **Deep**). چگونه نتایج این دو لایه را در کسری از ثانیه در **SERP** با هم ادغام (**Merge**) می‌کنید؟

مسئله 19.33: دور زدن Shingling توسط اسپمرها (Bypassing Shingling by Spammers) ★

اسپمرها می‌دانند که موتورهای جستجو از تکنیک **Shingling** برای حذف صفحات کپی استفاده می‌کنند. آن‌ها از روش **Content Spinning** (جایگذاری کلمات با مترادف‌هایشان با استفاده از دیکشنری) استفاده می‌کنند.

مثال: "The fast car" -> "The quick automobile".

آیا الگوریتم **k-shingling** مبتنی بر **Jaccard** و **MinHash** می‌تواند این نوع اسپم معنایی را شناسایی کند؟ اگر خیر، چه معماری جایگزینی (مبتنی بر **NLP** یا **Embedding**) برای کشف این اسناد تکراری پنهان پیشنهاد می‌دهید؟

مسئله 19.34: مهندسی معکوس کوئری‌های تصادفی در تخمین ایندکس (Reverse-engineering Random Queries) ★

در بخش 19.5 خواندیم که برای ساختن استخر کوئری جهت آزمایش **Capture-Recapture**، محققان از یک فرهنگ لغت (**Dictionary**) استفاده کرده و کلمات را با هم ترکیب می‌کنند.

اگر فرکانس کلمات در فرهنگ لغت با فرکانس واقعی آن‌ها در اسناد وب متفاوت باشد، چه خطایی در نمونه‌گیری رخ می‌دهد؟ یک فرمول اصلاحی (**Correction Weight**) مبتنی بر احتمال رخداد ترم‌ها $P(w)$ پیشنهاد دهید تا سوگیری کوئری‌ها در استخراج اسناد برطرف شود.

مسئله 19.35: شخصی‌سازی نتایج جستجو و حباب فیلتر (Personalization and Filter Bubbles) ★

موتورهای جستجو از تاریخچه کاربر (**User History**) و کوئری‌های قبلی او برای شخصی‌سازی نتایج در دسته‌بندی **Navigational** استفاده می‌کنند.

اما این کار باعث ایجاد پدیده **Filter Bubble** (حباب فیلتر) می‌شود که در آن کاربر فقط اطلاعات همسو با عقاید خود را می‌بیند. به‌عنوان یک طراح الگوریتم رتبه‌بندی، یک تابع هزینه (**Cost Function**) جدید طراحی کنید که تعادلی ریاضی بین «میزان ارتباط شخصی‌سازی شده» (**Relevance**) و «تنوع و اکتشاف» (**Diversity/Serendipity**) در **SERP** ایجاد کند.

مسئله 19.36: کدنویسی MinHash (MinHash Python Implementation Scenario) ★

فرض کنید ماتریس ترم-سند (**Term-Document Matrix**) بولی برای 3 سند و 5 کلمه به شرح زیر است (ردیف‌ها کلمات، ستون‌ها اسناد):

$$D_1 = [1, 0, 1, 1, 0]^T$$

$$D_2 = [1, 1, 0, 1, 1]^T$$

$$D_3 = [0, 0, 1, 1, 0]^T$$

سه تابع هش به صورت $h_1(x) = (x+1) \bmod 5$ و $h_2(x) = (3x+1) \bmod 5$ و $h_3(x) = (2x+2) \bmod 5$ در اختیار دارید (x اندیس ردیف از 0 تا 4 است).

ماتریس **Signature** (امضا) نهایی که ابعاد آن 3×3 خواهد بود را مرحله به مرحله محاسبه کنید. سپس شباهت ژاکارد تقریبی بین D_1 و D_3 را از روی ماتریس امضا محاسبه کرده و با شباهت واقعی آن‌ها مقایسه کنید.

پاسخنامه مسئله 19.1: تفاوت وب و بازیابی سنتی

بخش 1: تحلیل صورت مسئله

هدف از این سوال، درک تغییر پارادایم از «سیستم‌های بسته و کنترل‌شده» به «محیط باز و بی‌نظم وب» است. برای حل این مسئله، دانشجو باید محیط یک کتابخانه دیجیتال یا پایگاه داده مقالات (به عنوان نماینده بازیابی سنتی) را با محیط اینترنت (به عنوان بازیابی وب) مقایسه کند و چالش‌های محاسباتی و الگوریتمی که این تغییر محیط برای موتورهای جستجو ایجاد می‌کند را استخراج نماید.

بخش 2: مبانی و پیش‌نیازها

در سیستم‌های بازیابی اطلاعات سنتی (**Traditional IR**)، فرض بر این است که مجموعه اسناد (**Corpus**) دارای حجم مشخص، کیفیت تضمین‌شده و ساختاری همگن است. در مقابل، وب (**Web IR**) شبکه‌ای گراف‌گونه و درهم‌تنیده از اطلاعات است که هیچ‌گونه مرجع مرکزی برای مدیریت، یکپارچه‌سازی و کنترل آن وجود ندارد. در این محیط، توزیع داده‌ها به شدت از قوانین توانی پیروی می‌کند.

بخش 3: شرح گام‌به‌گام و پاسخ تفصیلی (ابعاد سه‌گانه تفاوت‌ها)

سه تفاوت عمده محیط وب با سیستم‌های سنتی و چالش‌های ناشی از آن‌ها به شرح زیر است:

1. مقیاس عظیم و پویایی داده‌ها (**Scale and Dynamism**)

توضیح تفاوت: در سیستم‌های سنتی (مانند سیستم جستجوی یک کتابخانه دانشگاهی)، با تعداد محدودی سند (مثلاً چند میلیون مقاله) روبه‌رو هستیم که به ندرت تغییر می‌کنند یا حذف می‌شوند. اما وب شامل میلیاردها صفحه است که در هر لحظه ایجاد می‌شوند، تغییر محتوا می‌دهند و یا برای همیشه از دسترس خارج می‌شوند (مانند خطای **404**).

چرا کار موتور جستجو را سخت‌تر می‌کند؟ موتورهای جستجو برای مواجهه با این مقیاس، نیازمند زیرساخت‌های سخت‌افزاری و نرم‌افزاری بسیار توزیع‌شده هستند. چالش اصلی در اینجا **خزش (Crawling)** و **نمایه‌سازی (Indexing)** است. موتور جستجو باید با استفاده از سیاست‌های زمان‌بندی بهینه (مثل مدل‌های پواسون)، مدام تصمیم بگیرد که کدام صفحات را در چه بازه‌های زمانی دوباره بررسی کند تا پایگاه داده‌اش همیشه به‌روز (**Fresh**) بماند، بدون اینکه منابع شبکه و سرورها بیش از حد مصرف شوند.

2. فقدان کنترل کیفیت و ماهیت خصمانه (**Lack of Quality Control & Adversarial IR**)

توضیح تفاوت: اسناد در سیستم‌های سنتی توسط متخصصان، ویراستاران یا داوران تایید می‌شوند؛ بنابراین کیفیت اطلاعات بالا است و نویسنده قصد فریب سیستم را ندارد. در محیط وب، هر فردی می‌تواند هر محتوایی منتشر کند. این آزادی باعث ایجاد پدیده‌ای به نام **اسپم وب (Web Spam)** و فعالیت‌های خصمانه (مثل سئوی کلاه‌سیاه) می‌شود که در آن سایت‌ها با تکرار مصنوعی کلمات کلیدی یا تکنیک‌های پنهان‌کاری (**Cloaking**) سعی می‌کنند سیستم جستجو را فریب دهند تا در رتبه‌های بالا قرار گیرند.

چرا کار موتور جستجو را سخت تر می کند؟ موتور جستجو دیگر نمی تواند صرفاً به توزیع کلمات کلیدی (مثل **TF-IDF**) در داخل یک صفحه اعتماد کند. الگوریتم ها باید به شدت هوشمند باشند تا صفحات باکیفیت و معتبر را از صفحات اسپم، کپی شده (**Duplicate Content**) یا فریبنده تشخیص دهند. این یعنی موتور جستجو وارد یک «بازی موش و گربه» دائمی با اسپمرها می شود که به پیچیدگی محاسبات رتبه بندی (**Ranking**) می افزاید.

3. ساختار پیوندی و تنوع کوئری های کاربران (**Link Structure & Query Heterogeneity**)

توضیح تفاوت: در بازیابی سنتی، اسناد معمولاً مستقل از یکدیگرند و کاربران آن اغلب افراد متخصصی هستند که از عبارات جستجوی (**Query**) طولانی، دقیق و منطقی (Boolean) استفاده می کنند. در مقابل، وب ساختاری گراف گونه دارد (صفحات با ابرپیوندها یا **Hyperlinks** به هم متصل اند) و کاربران وب، افراد عادی با سطح دانش متفاوت هستند که معمولاً کوئری های بسیار کوتاه (به طور متوسط 2 تا 3 کلمه)، مبهم و دارای غلط املایی را جستجو می کنند.

چرا کار موتور جستجو را سخت تر می کند؟ به دلیل کوتاه و مبهم بودن کوئری ها، موتور جستجو به سختی می تواند «نیت کاربر» (**User Intent**) را درک کند (مثلاً کلمه "شیر" به حیوان اشاره دارد، خوراکی یا شیر آب؟). برای جبران این کمبود اطلاعات در کوئری، موتور جستجو مجبور است ساختار لینک های وب را تحلیل کند (مانند الگوریتم **PageRank**) تا ببیند کدام صفحات از نظر سایر سایت ها معتبرترند.

پردازش گراف عظیم وب برای استخراج این اعتبارات، یک چالش ریاضیاتی سنگین است که به طور کلاسیک از رابطه زنجیره مارکوف زیر برای محاسبه احتمال حضور یک کاربر تصادفی در هر صفحه وب استفاده می کند:

$$PR(u) = \frac{1 - d}{N} + d \sum_{v \in B(u)} \frac{PR(v)}{L(v)}$$

بخش 4: جدول تطبیقی تفاوت های بنیادین

معیار مقایسه	بازیابی اطلاعات سنتی (Traditional IR)	بازیابی اطلاعات وب (Web IR)
حجم و پویایی	محدود و عموماً ایستا (Static)	بسیار عظیم و دائماً متغیر (Dynamic)
کیفیت و اعتماد	بالا، بررسی شده توسط متخصصین	نامشخص، وجود اسپم و محتوای مخرب
ساختار داده ها	تخت (Flat) و مستقل از هم	گراف گونه و مبتنی بر ابرپیوند (Hyperlinks)
نحوه پرس و جو	دقیق، طولانی و دارای عملگرهای منطقی	بسیار کوتاه (2 الی 3 کلمه)، مبهم و محاوره ای

بخش 5: جمع بندی

به زبان ساده، موتور جستجوی سنتی شبیه به پیدا کردن یک کتاب در یک کتابخانه منظم با قفسه بندی مشخص است؛ اما جستجو در وب مانند پیدا کردن یک تکه کاغذ مهم در یک بازار شلوغ، دائماً در حال تغییر و پر از اطلاعات نادرست

پاسخنامه مسئله 19.2: ماهیت پویای وب و چالش‌های خزنده‌ها

بخش 1: تحلیل صورت مسئله

هدف از این سوال، بررسی معماری تولید محتوا در وب (تفاوت رندر ثابت و تولید در لحظه) و درک محدودیت‌های عملیاتی خزنده‌های وب (**Web Crawlers / Spiders**) در مواجهه با محتوای غیرثابت است. دانشجو باید بتواند نشان دهد که چرا الگوریتم‌های ساده‌ی تعقیب لینک (**Link Following**) که بر مبنای پیمایش گراف‌های متناهی طراحی شده بودند، در برخورد با تکنولوژی‌های مدرن وب با شکست، افت عملکرد یا حلقه‌های بی‌نهایت مواجه می‌شوند.

بخش 2: مبانی و پیش‌نیازها

خزنده وب (**Crawler**)، یک ربات نرم‌افزاری است که از یک مجموعه آدرس اولیه (**Seed URLs**) شروع به کار کرده، صفحات را دانلود می‌کند، لینک‌های داخل آن‌ها را استخراج کرده و این فرآیند را به صورت بازگشتی تکرار می‌کند. پیش‌فرض اولیه یک خزنده ساده این است که هر **URL** معادل یک «سند فیزیکی و مستقل» در گراف وب است؛ فرضی که با ظهور صفحات داینامیک و برنامه‌نویسی سمت سرور به طور کامل نقض شد.

بخش 3: شرح گام‌به‌گام و پاسخ تفصیلی

برای پاسخ به این سوال، ابتدا دو مفهوم پایه را تفکیک کرده و سپس به سراغ چالش‌های ناشی از آن‌ها می‌رویم.

گام اول: تفاوت صفحات استاتیک و داینامیک

1. صفحات استاتیک (Static Web Pages):

تعریف: این صفحات شامل فایل‌های از پیش آماده‌شده‌ای (غالباً با پسوند **.html**) هستند که دقیقاً به همان شکلی که در هارد دیسک سرور ذخیره شده‌اند، برای کاربر ارسال می‌شوند. زمان پاسخ‌گویی سرور در این حالت از مرتبه زمان ثابت (**O(1)**) برای خواندن دیسک است.

ویژگی بارز: محتوای آن‌ها ثابت است. هر کاربری، در هر زمانی و از هر مکانی که آن **URL** را درخواست کند، دقیقاً یک خروجی یکسان را مشاهده خواهد کرد.

2. صفحات داینامیک (Dynamic Web Pages):

تعریف: این صفحات به صورت فیزیکی به عنوان یک فایل ثابت روی سرور وجود ندارند. بلکه در لحظه‌ی درخواست کاربر (**On-the-fly**)، توسط کدهای سمت سرور (مثل **PHP, Python, Node.js**) و با استخراج اطلاعات از یک پایگاه داده (**Database**) ساخته و رندر می‌شوند.

ویژگی بارز: محتوای این صفحات می‌تواند بر اساس فاکتورهای مختلفی تغییر کند؛ از جمله پارامترهای موجود در **URL** (مثل متغیرهای جستجو)، زمان درخواست، کوکی‌ها (**Cookies**)، وضعیت لاگین کاربر یا موقعیت جغرافیایی او.

ویژگی ساختاری	صفحات استاتیک (Static)	صفحات داینامیک (Dynamic)
محل تولید محتوا	از پیش در هارد سرور ذخیره شده است	در لحظه درخواست (On-the-fly) تولید می‌شود
بار پردازشی سرور	بسیار کم (فقط ارسال فایل)	بالا (نیاز به کوئری پایگاه داده و رندر کد)
تعداد URLهای ممکن	محدود و متناهی (برابر با تعداد فایل‌ها)	نامتناهی (وابسته به پارامترهای ورودی)

گام دوم: چالش‌های صفحات داینامیک برای خزنده‌های وب

تولید محتوا در لحظه، موتورهای جستجو را با چهار چالش اساسی و پیچیده مواجه می‌کند که نیازمند الگوریتم‌های پیشرفته برای مهار آن‌هاست:

1. تله‌های خزنده و فضای بی‌نهایت آدرس‌ها (Spider Traps & Infinite URL Space)

توضیح: در صفحات داینامیک، با اضافه کردن پارامتر به URL می‌توان بی‌نهایت صفحه ساخت. به عنوان مثال، یک تقویم داینامیک آنلاین را در نظر بگیرید که برای هر ماه در سال‌های آینده یک لینک "ماه بعد" دارد (مثلاً ?year=2026&month=8). از نظر ریاضی، فضای حالت (State Space) گراف وب در این سناریو به سمت بی‌نهایت میل می‌کند.

چالش: خزنده ممکن است وارد یک حلقه بی‌نهایت (Infinite Loop) شود و منابع سخت‌افزاری سرور خود و سایت مقصد را برای خزشِ میلیاردها URL بی‌ارزش یا تکراری هدر دهد. سیستم‌های مدرن برای مقابله با این مشکل از الگوریتم‌های تشخیص الگو در URL استفاده می‌کنند.

2. پدیده وب پنهان یا وب عمیق (Deep Web / Hidden Web)

توضیح: بسیاری از صفحات داینامیک تنها پس از پر کردن یک فرم جستجو، لاگین کردن، یا اعمال فیلترهای خاص (مثلاً انتخاب بازه قیمتی 100 تا 200 دلار در یک فروشگاه اینترنتی) در پایگاه داده تولید می‌شوند و لینک مستقیمی (Anchor Tag) در صفحات دیگر برای رسیدن به آن‌ها وجود ندارد.

چالش: خزنده‌ها به طور سنتی برنامه‌ریزی شده‌اند که فقط روی لینک‌ها با متد درخواست GET کلیک کنند. آن‌ها توانایی وارد کردن متن در فرم‌های جستجو، استدلال برای فیلترها، یا ارسال درخواست‌های مبتنی بر POST را ندارند. در نتیجه، حجم عظیمی از داده‌های باارزش وب هرگز توسط آن‌ها کشف نمی‌شود و پنهان می‌ماند.

3. تکراری بودن محتوا در آدرس‌های متفاوت (Duplicate Content)

توضیح: سیستم‌های داینامیک ممکن است یک محتوای واحد را با چندین URL مختلف نمایش دهند. به عنوان مثال، یک محصول ممکن است با آدرس‌های زیر در دسترس باشد:

```
example.com/product?id=123
example.com/product?id=123&sort=price
example.com/product?id=123&session_id=abc
```

چالش: موتور جستجو باید پردازش‌های هوشمند و پیچیده‌ای (مانند الگوریتم‌های Simhash یا بررسی تگ‌های Canonical) انجام دهد تا متوجه شود این URLها در واقع یک سند واحد هستند و نباید آن‌ها را به عنوان صفحات مجزا ایندکس کند؛ در غیر این صورت، حافظه ذخیره‌سازی ایندکس به شدت اتلاف شده و رتبه صفحه (PageRank) بین لینک‌های مختلف پخش می‌شود.

4. رندرینگ سمت کاربر (Client-Side Rendering - Javascript)

توضیح: وب مدرن به شدت به صفحات داینامیکی وابسته است (معروف به **SPA - Single Page Applications**) که محتوای آن‌ها نه در سرور، بلکه توسط کدهای جاوا اسکریپت در مرورگر کاربر (مانند فریم‌ورک‌های **React** یا **Vue**) پردازش و ساخته می‌شود.

چالش: خزنده‌های سنتی فقط کدهای اولیه **HTML** را دانلود می‌کردند که در این سایت‌ها تقریباً خالی است (فقط شامل یک تگ `<div id="root">`). امروزه موتورهای جستجو (مثل گوگل از طریق سیستم **Web Rendering Service - WRS**) مجبورند برای دیدن محتوای واقعی، یک مرورگر واقعی و بدون رابط کاربری (**Headless Browser**) را برای اجرای جاوا اسکریپت شبیه‌سازی کنند که این امر **هزینه پردازشی (Computational Cost)** سیستم‌های خزنده را به شدت افزایش می‌دهد.

شماتیک تفاوت رفتار خزنده‌ها در مواجهه با صفحات جاوا اسکریپتی (چالش 4)

خزنده سنتی (Traditional Crawler)

Request URL → Download Raw HTML → **Result: Empty Page (No Data)**

خزنده مدرن (Modern Crawler with WRS)

Request URL → Download HTML → Execute Javascript (Headless Browser) →
Result: Full DOM Extracted

بخش 4: جمع‌بندی

گذار از معماری صفحات استاتیک به رویکرد تولید داینامیک و در لحظه، نقطه عطفی در تاریخ تکامل موتورهای جستجو به شمار می‌رود. خزنده‌های وب از نرم‌افزارهای ساده‌ای که صرفاً به دانلود فایل‌های متنی می‌پرداختند، به سیستم‌های هوشمند و پیچیده‌ای بدل شده‌اند که باید توانایی شبیه‌سازی مرورگرهای واقعی (مانند اجرای کدهای جاوا اسکریپت)، تشخیص الگوهای تکراری برای فرار از تله‌های خزنده، و مدیریت بهینه منابع پردازشی را داشته باشند. این تغییر پارادایم نشان می‌دهد که در مهندسی وب مدرن، موفقیت یک سیستم بازیابی اطلاعات عمیقاً به درک ساختارهای سمت کلاینت و سرور، و توانایی مقابله با فضای حالت بی‌نهایت و پدیده‌ی وب پنهان گره خورده است.

پاسخنامه مسئله 19.3: مدل پاپیونی گراف وب (Bow-tie Model)

بخش 1: تحلیل صورت مسئله

در این سوال، گراف جهت‌دار وب (**Web Graph**) از دیدگاه ساختار کلان و مسیریابی مورد بررسی قرار می‌گیرد. هدف این است که دانشجو ابتدا بخش‌های اصلی مدل معروف «پاپیونی» (**Bow-tie Model**) که در سال 2000 توسط محققان (**Broder et al.**) با

بررسی حدود 200 میلیون صفحه وب ارائه شد را تعریف کند؛ و سپس با تحلیل منطق لینک‌های ورودی (In-links) و خروجی (Out-links)، جایگاه یک صفحه وب خاص را در این توپولوژی مشخص نماید.

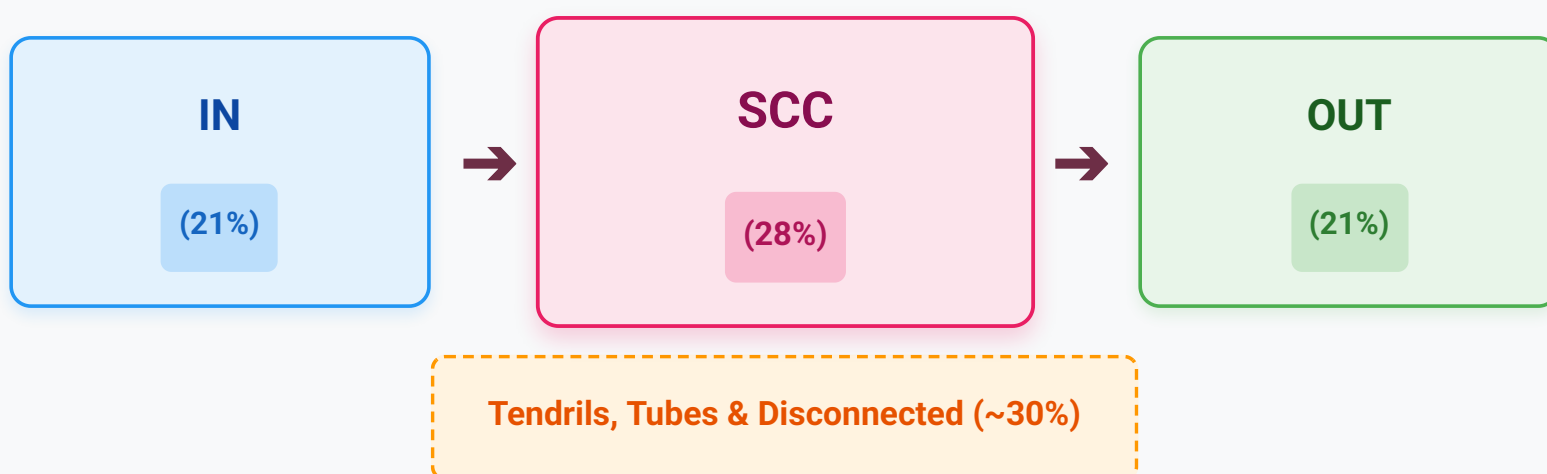
بخش 2: مبانی و پیش‌نیازهای ریاضی گراف

محیط وب را می‌توان به عنوان یک گراف جهت‌دار مدل‌سازی کرد که با رابطه زیر نمایش داده می‌شود:

$$G = (V, E)$$

در این گراف، گره‌ها (V) همان صفحات وب هستند و یال‌های جهت‌دار (E) نشان‌دهنده ابرپیوندها (Hyperlinks) از یک صفحه به صفحه دیگر می‌باشند. بر اساس قابلیت دسترسی (Reachability) مسیرها در این گراف، ساختار کلان وب شبیه به یک پایپون دسته‌بندی می‌شود.

نمودار شماتیک ساختار کلان وب (مدل پایپونی)



بخش 3: شرح گام‌به‌گام و پاسخ تفصیلی

گام اول: تعریف مولفه‌های اصلی ساختار پایپونی

این گراف کلان دارای سه ناحیه پردازشی اصلی (و نواحی فرعی برای تکمیل ساختار) است:

ناحیه در گراف	نام کامل و تخصصی	تعریف در گراف وب
هسته مرکزی (SCC)	Strongly Connected Component	قلب وب است. مجموعه‌ای از صفحات که از هر صفحه در آن، می‌توان با دنبال کردن لینک‌ها به هر صفحه دیگری در همان مجموعه رسید (مانند سایت‌های خبری معتبر و ویکی‌پدیا).
بخش ورودی (IN)	In-component	صفحاتی که می‌توانند با دنبال کردن لینک‌ها به هسته مرکزی (SCC) برسند، اما هیچ مسیری از هسته مرکزی به سمت آن‌ها وجود ندارد. این‌ها معمولاً سایت‌های نوپا یا صفحات پنهان هستند که به سایت‌های مرجع لینک داده‌اند.
بخش خروجی (OUT)	Out-component	صفحاتی که از هسته مرکزی (SCC) قابل دسترس هستند، اما هیچ مسیری از آن‌ها به سمت هسته مرکزی بر نمی‌گردد. وبلاگ‌های قدیمی، اسناد مستقل (PDFها) یا صفحات بن‌بست در این دسته جای می‌گیرند.

ناحیه در گراف	نام کامل و تخصصی	تعریف در گراف وب
شاخک‌ها و لوله‌ها (تکمیلی)	Tendrils & Tubes	گره‌هایی که به بخش IN متصلند (اما راهی به SCC ندارند) یا از OUT لینک می‌گیرند. اگر مسیری مستقیماً بخش IN را به OUT متصل کند و از SCC عبور نکند، به آن لوله (Tube) می‌گویند.

گام دوم: مکان‌یابی صفحه وب با ویژگی‌های داده‌شده

مسئله صفحه‌ای را توصیف می‌کند که دارای دو مشخصه ریاضی زیر در نظریه گراف است:

$$\deg^-(v) > 0, \quad \deg^+(v) = 0$$

- هیچ لینک خروجی ندارد: یعنی درجه خروجی آن صفر است ($\deg^+ = 0$) و به هیچ گره دیگری متصل نمی‌شود.
- لینک‌های ورودی متعددی دارد: یعنی درجه ورودی آن بزرگتر از صفر است ($\deg^- > 0$) و از سایر صفحات قابل دسترسی است.

تحلیل و استدلال استنتاجی:

چون صفحه مسیری به بیرون ندارد، هرگز نمی‌تواند در بخش IN قرار گیرد (زیرا شرط قطعی حضور در IN، داشتن حداقل یک مسیر پیش‌رو به سمت SCC است).

به همین دلیل، نمی‌تواند عضوی از SCC نیز باشد (زیرا شرط حضور در SCC، قابلیت رفت‌وبرگشت و داشتن لینک خروجی برای رسیدن به سایر صفحات همان مجموعه است).

از آنجایی که صفحه دارای لینک‌های ورودی است، مشخصاً مسیری از سایر بخش‌های گراف وب (غالباً از سمت SCC یا IN) به آن منتهی شده است و یک صفحه ایزوله نیست.

نتیجه‌گیری نهایی:

صفحه‌ای که صرفاً دریافت‌کننده ترافیک است و به عنوان یک «مقصد نهایی» یا «بن‌بست» (Sink Node / Dead End) عمل می‌کند، به طور قطعی در بخش خروجی (OUT) قرار می‌گیرد.

بخش 4: جمع‌بندی

گذار از نگاه ساده‌انگارانه به وب به عنوان یک گراف کاملاً همبند، به سمت درک ساختار نامتقارن و جهت‌دار مدل پایبونی، یکی از مهم‌ترین دستاوردهای تحلیلی در حوزه بازیابی اطلاعات محسوب می‌شود. این توپولوژی به وضوح نشان می‌دهد که جریان توزیع اطلاعات و اعتبار (نظیر آنچه در الگوریتم‌های مبتنی بر لینک مانند PageRank محاسبه می‌شود) در بخش‌های گسترده‌ای از وب، جریانی کاملاً یک‌طرفه است. برای مهندسان سیستم‌های جستجو و طراحان خزنده‌های وب (Web Crawlers)، درک دقیق این معماری، به‌ویژه شناخت گره‌های بن‌بست (Dead Ends) در ناحیه خروجی (OUT-component)، از اهمیتی استراتژیک برخوردار است. هدایت کورکورانه خزنده به سمت این صفحات، بدون پیش‌بینی مکانیزم‌های خروج هوشمند (مانند جهش‌های تصادفی یا Teleportation)، منجر به اتلاف شدید منابع پردازشی، گیر افتادن در تله‌های ساختاری و کاهش چشمگیر نرخ پوشش‌دهی (Coverage) هسته مرکزی (SCC) خواهد

شد. بنابراین، تخصیص هوشمندانه بودجه خزش (**Crawl Budget**) در موتورهای جستجوی مدرن، مستقیماً به توانایی سیستم در تشخیص زودهنگام جایگاه هر صفحه در این ساختار پاپیونی بستگی دارد.

پاسخنامه مسئله 19.4: لوله‌ها و شاخک‌ها در گراف وب (Tubes and Tendrils)

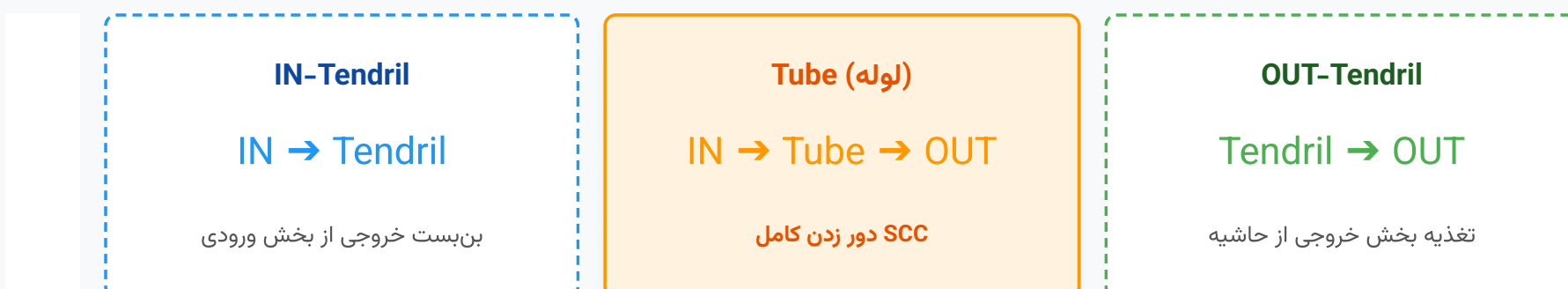
بخش 1: تحلیل صورت مسئله

هدف از این سوال، تکمیل شناخت دانشجو از توپولوژی پیچیده وب و بررسی اجزای فرعی آن است. در حالی که بخش‌های **IN**، **SCC** و **OUT** استخوان‌بندی و جریان اصلی اطلاعات را تشکیل می‌دهند، حجم عظیمی از صفحات وب (حدود **30%** از کل گراف وب) در حاشیه‌ی این جریان اصلی قرار دارند. دانشجو در این مسئله باید بتواند این اجزای حاشیه‌ای (شاخک‌ها و لوله‌ها) را که از هسته مرکزی کاملاً منزوی هستند، به دقت از منظر نظریه گراف تعریف کند و مسیر حرکت داده‌ها در آن‌ها را ردیابی نماید.

بخش 2: مبانی و پیش‌نیازها

همان‌طور که در مسئله قبل (مدل پاپیونی) بررسی شد، مسیر طبیعی و همگرا در گراف وب عموماً از بخش ورودی (**IN**) به سمت هسته مرکزی (**SCC**) و سپس به سمت بخش خروجی (**OUT**) است. اما گراف وب دارای بی‌نظمی‌ها و ساختارهای واگرایی است که باعث می‌شود برخی مسیرها هرگز به هسته مرکزی نرسند یا به طور کامل از آن عبور نکنند. این انحرافات ساختاری، گره‌های حاشیه‌ای شامل شاخک‌ها (**Tendrils**) و لوله‌ها (**Tubes**) را به وجود می‌آورند.

نمودار مفهومی مسیرهای حاشیه‌ای (انحراف از هسته مرکزی)



بخش 3: شرح گام‌به‌گام و پاسخ تفصیلی

در مدل گراف جهت‌دار وب، علاوه بر سه قطعه اصلی، اجزای فرعی زیر نیز برای پوشش دادن گره‌های منزوی و حاشیه‌ای تعریف می‌شوند:

1. شاخک‌ها (Tendrils)

تعریف ریاضی و ساختاری: شاخک‌ها شامل گره‌هایی (صفحاتی) هستند که به بخش‌های **IN** یا **OUT** متصل‌اند، اما هیچ‌گونه ارتباط مسیریابی با هسته مرکزی (**SCC**) ندارند (نه به آن وارد می‌شوند و نه از آن خارج می‌شوند).

انواع شاخک‌ها:

شاخک‌های خروجی از IN (In-Tendrils): مسیرهایی که از صفحات بخش **IN** شروع می‌شوند، اما به جای رفتن به سمت هسته **SCC**، به سمت صفحاتی می‌روند که در نهایت به بن‌بست می‌رسند. (صفحاتی که از **IN** قابل دسترسی‌اند اما به **SCC** راه ندارند).

شاخک‌های ورودی به OUT (Out-Tendrils): صفحاتی که از **SCC** قابل دسترسی نیستند، اما لینک‌هایی به سمت

صفحات بخش **OUT** ایجاد می‌کنند. (مسیرهایی که به **OUT** سرازیر می‌شوند اما ریشه در جریان اصلی و معتبر **SCC** ندارند).

علت شکل‌گیری: این ساختارها معمولاً زمانی شکل می‌گیرند که یک سایت نوپا (در بخش IN) به سایت‌های ناشناخته یا محلی دیگری لینک می‌دهد، اما هیچ‌کدام توانایی و اعتبار کافی برای اتصال به جریان اصلی و مرکزی اینترنت (مثل سایت‌های خبری یا مراجع بزرگ) را ندارند.

2. لوله‌ها (Tubes)

تعریف و عملکرد ساختاری: اگر یک شاخک متصل به **IN**، در امتداد مسیر خود مستقیماً به یک شاخک متصل به **OUT** گره بخورد (به آن لینک بدهد)، یک مسیر ممتد و جدید به نام «لوله» شکل می‌گیرد. در تئوری گراف، این یک مسیر مستقیم از مؤلفه‌های ورودی به مؤلفه‌های خروجی است.

پاسخ صریح به بخش دوم سوال (مسیر Tube از کجا به کجا ختم می‌شود؟)

یک **Tube** مسیری کاملاً یک‌طرفه است که مستقیماً از **بخش ورودی (IN)** آغاز شده و مستقیماً به **بخش خروجی (OUT)** ختم می‌شود، در حالی که **هسته مرکزی (SCC)** را کاملاً دور می‌زند (می‌کند Bypass). در این مسیر، کاربر یا خزنده وب می‌تواند بدون هیچ تماس و ارجاعی به سایت‌های معتبر مرکزی، ترافیک را از یک سمت وب به سمت دیگر منتقل کند.

بخش 4: جمع‌بندی

ساختارهای **Tendril** و **Tube** نشان می‌دهند که بخش قابل توجهی از وب، محیطی «حاشیه‌ای»، «منزوی» و خارج از جریان اصلی اطلاعات است. برای یک موتور جستجوی مدرن، کشف صفحاتی که در این نواحی قرار دارند بسیار دشوارتر و پرهزینه‌تر است؛ زیرا خزنده‌ها (**Crawlers**) که معمولاً فعالیت خود را بر پایه هسته مرکزی (**SCC**) و الگوریتم‌های رتبه‌بندی پیوند بنا می‌کنند، هیچ مسیر مستقیمی برای ورود به **Tendril**-های متصل به **OUT** ندارند و لوله‌ها (**Tubes**) نیز ترافیک را از دید هسته پنهان نگه می‌دارند.

پاسخنامه مسئله 19.5: تعریف اسپم در جستجو و انگیزه‌های اقتصادی

(Web Spam & Economic Motivations)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، بررسی رفتار متقابل موتورهای جستجو و تولیدکنندگان محتوا در یک محیط رقابتی و غیرمتمرکز است. دانشجو باید بتواند مفهوم «اسپم» را از معنای رایج و سنتی آن (ایمیل‌های مزاحم) جدا کرده و آن را به عنوان یک حمله الگوریتمی در حوزه بازیابی اطلاعات درک کند. همچنین بررسی ریشه‌های تجاری این رفتار، دیدگاه واقع‌بینانه‌تری از چالش‌های مهندسی جستجو به دانشجو می‌دهد و نشان می‌دهد که چرا طراحی موتورهای جستجو امروزه بیشتر به «نظریه بازی‌ها» (**Game Theory**) شباهت دارد تا صرفاً پردازش متن.

الگوریتم‌های رتبه‌بندی (**Ranking Algorithms**) در سیستم‌های بازیابی اطلاعات، بر پایه اندازه‌گیری سیگنال‌های خاصی کار می‌کنند؛ مانند سنجش چگالی کلمات کلیدی (مدل **TF-IDF**) یا شمارش لینک‌های ورودی به عنوان رای اعتماد (مدل **PageRank**). پیش‌فرض این الگوریتم‌ها، توزیع طبیعی و ارگانیک این سیگنال‌ها است. پدیده اسپم زمانی رخ می‌دهد که عامل انسانی با دستکاری عمدی این سیگنال‌ها، پیش‌فرض‌های ریاضی موتور جستجو را نقض کند.

بخش 3: شرح گام‌به‌گام و پاسخ تفصیلی

برای درک کامل این پدیده، پاسخ را در دو محور «مفهوم‌شناسی» و «ریشه‌یابی اقتصادی» بررسی می‌کنیم:

1. تعریف اسپم وب و بازیابی اطلاعات خصمانه (**Adversarial IR**)

بازیابی اطلاعات خصمانه (Adversarial IR**):** محیط وب یک سیستم اطلاعاتی ایستا و بی‌طرف نیست؛ بلکه یک میدان

نبرد الگوریتمی است. بازیابی خصمانه به شاخه‌ای از علم جستجو اشاره دارد که در آن طراحان سیستم باید

الگوریتم‌هایی توسعه دهند که در برابر کاربرانی که قصد فریب و مهندسی معکوس سیستم را دارند، مقاوم باشند. این

یک «بازی موش و گربه» دائمی است که در آن هر به‌روزرسانی الگوریتم موتور جستجو، با تاکتیک‌های جدید اسپم‌ها

پاسخ داده می‌شود.

اسپم وب (Web Spam**):** هرگونه اقدام عمدی، تکنیک کلاه‌سیاه (**Black-hat SEO**) یا دستکاری ساختاری است که با

هدف گمراه کردن الگوریتم‌های موتور جستجو انجام می‌شود. هدف نهایی اسپم، این است که یک صفحه وب در نتایج

جستجو، رتبه‌ای (**Rank**) بسیار بالاتر از ارزش واقعی، ارتباط (**Relevance**) و کیفیت ذاتی خود کسب کند. این اسپم‌ها

معمولاً به دو دسته اصلی تقسیم می‌شوند:

اسپم محتوا (Content Spam**):** مانند تکرار پنهانی کلمات کلیدی (**Keyword Stuffing**) در تگ‌های نامرئی یا

متن‌های هم‌رنگ پس‌زمینه.

اسپم پیوند (Link Spam**):** مانند ساخت شبکه‌های لینک‌سازی جعلی (**Link Farms**) برای دستکاری نمرات الگوریتم

پیج‌رنک.

2. انگیزه اقتصادی و دلیل دور زدن الگوریتم‌ها

انگیزه اصلی تولیدکنندگان اسپم، یک معادله کاملاً تجاری است که بر پایه‌ی سه عامل زیر استوار است:

اقتصاد توجه و تمرکز کلیک‌ها (Attention Economy**):** در فضای وب، ترافیک (بازدیدکننده) معادل با پول است.

آمارهای موتورهای جستجو نشان می‌دهد که بیش از **90** درصد کلیک‌های کاربران تنها روی صفحه اول نتایج (و به

خصوص **3** لینک اول) انجام می‌شود. صفحات دوم به بعد، عملاً از دید کاربران پنهان هستند و ترافیکی دریافت

نمی‌کنند.

کیفیت بالای ترافیک جستجو (High Conversion Rate**):** کاربری که عبارت "خرید لپ‌تاپ" را در موتور جستجو وارد

می‌کند، یک مشتری بالقوه و آماده خرید است. هدایت این کاربر به یک فروشگاه، برخلاف تبلیغات بیلبوردی یا بنری،

نیازمند متقاعدسازی اولیه نیست. قرار گرفتن در رتبه اول برای کلمات کلیدی تجاری، روزانه هزاران مشتری رایگان و

هدفمند را به همراه می‌آورد.

هزینه پایین در برابر سود نجومی (High ROI of Spam): تولید محتوای ارزشمند، استخدام متخصصان و برندسازی

(سئوی کلاه سفید)، فرآیندی بسیار زمان‌بر و پرهزینه است. در مقابل، یک تولیدکننده اسپم می‌تواند با استفاده از ربات‌ها، خرید لینک‌های ارزان و تولید محتوای خودکار، با هزینه‌ای ناچیز الگوریتم‌ها را فریب دهد و در مدت زمان کوتاهی به صدر نتایج برسد. سود حاصل از این جایگاه (فروش مستقیم، نمایش تبلیغات یا هدایت کاربر به سایت‌های دیگر) به قدری بالاست که حتی اگر سایت پس از مدتی توسط موتور جستجو شناسایی و جریمه (Ban/Penalize) شود، اسپمر باز هم سود کلانی به دست آورده است و به راحتی با یک دامنه جدید کار را مجدداً آغاز می‌کند.

بخش 4: تحلیل تطبیقی سرمایه‌گذاری (اسپم در برابر رشد ارگانیک)

ویژگی سرمایه‌گذاری	رشد ارگانیک (سئوی کلاه سفید)	اسپم الگوریتمی (سئوی کلاه سیاه)
هزینه اولیه تولید	بسیار بالا (تولید محتوای تخصصی)	بسیار پایین (استفاده از ربات‌ها)
زمان رسیدن به رتبه 1	طولانی (معمولاً 6 تا 12 ماه)	بسیار سریع (چند روز تا چند هفته)
ریسک جریمه (Penalty)	نزدیک به صفر (ایمن)	قطعی و بسیار بالا (حذف دامنه)
بازگشت سرمایه (ROI)	کند اما پایدار و بلندمدت	نجومی اما کوتاه‌مدت و ناپایدار

بخش 5: جمع‌بندی

اسپم در وب، ناشی از یک نقص فنی ساده در برنامه‌نویسی موتور جستجو نیست؛ بلکه یک پیامد اجتناب‌ناپذیر اقتصادی است. تا زمانی که حضور در رتبه‌های بالای موتورهای جستجو (مانند گوگل) ارزش مالی و تجاری بالایی تولید می‌کند، همواره افرادی وجود خواهند داشت که تلاش کنند سیستم‌های رتبه‌بندی مبتنی بر متن و لینک را فریب دهند.

پاسخنامه مسئله 19.6: تکنیک پنهان‌کاری (Cloaking)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، بررسی یکی از پیچیده‌ترین و مخرب‌ترین تکنیک‌های اسپم وب (Web Spam) در دسته «اسپم محتوایی» است. در این مسئله، دانشجو باید مکانیزم ارتباطی کلاینت-سرور را در پروتکل اینترنت درک کند و نشان دهد که چگونه یک وب‌سرور می‌تواند هویت درخواست‌کننده (خزنده یا انسان) را در همان لایه شبکه تشخیص داده و با ارائه دو نسخه کاملاً متفاوت از یک صفحه واحد (برای یک URL ثابت)، هم سیستم رتبه‌بندی الگوریتمی موتور جستجو و هم اعتماد کاربر نهایی را مورد سوءاستفاده قرار دهد.

بخش 2: مبانی شبکه‌ای و پیش‌نیازها

زمانی که یک درخواست (HTTP Request) برای مشاهده یک صفحه وب به سرور ارسال می‌شود، هدرهای این درخواست (HTTP Headers) حاوی اطلاعات هویتی خاصی هستند. سرورهای اسپم برای اجرای تکنیک پنهان‌کاری، عموماً روی دو متغیر کلیدی در این هدرها متمرکز می‌شوند:

1. **رشته هویتی (User-Agent):** رشته‌ای متنی است که نوع مرورگر، نسخه آن، سیستم‌عامل و ماهیت نرم‌افزار

درخواست‌کننده را مشخص می‌کند. به عنوان مثال، مشخص می‌کند که درخواست از سمت مرورگر کروم روی

سیستم‌عامل ویندوز است یا از سمت ربات خزنده‌ی گوگل به نام **Googlebot**.

2. **آدرس شبکه (IP Address):** از آنجایی که هدر **User-Agent** به راحتی قابل جعل (Spoofing) است، اسپمرهای

حرفه‌ای آدرس شبکه درخواست‌کننده را نیز بررسی می‌کنند. از طریق تطبیق IP با پایگاه‌های داده معتبر (یا اجرای

Reverse DNS Lookup)، می‌توان بازه آدرس‌های متعلق به دیتاسنترهای موتورهای جستجو را با دقت بالا شناسایی

کرد.

نمودار شماتیک جریان کنترل در تکنیک Cloaking (منطق سمت سرور)



بخش 3: شرح گام‌به‌گام فرآیند فریب (The Cloaking Pipeline)

تکنیک پنهان‌کاری بر پایه یک رفتار دوگانه و شرطی برنامه‌ریزی می‌شود. فرآیند فریب همزمان در سه گام مجزا به شکل زیر انجام می‌گیرد:

گام اول: مکانیزم تشخیص هویت در سرور

اسپمر یک پایگاه داده عظیم و به‌روز از **User-Agent**-ها و آدرس‌های IP متعلق به خزنده‌های معروف موتورهای جستجو (مانند ربات‌های گوگل، بینگ و یاهو) جمع‌آوری می‌کند. اسکریپت سمت سرور (مثلاً کدهای نوشته شده در فایل **htaccess** یا کدهای PHP) برنامه‌ریزی می‌شود تا هر درخواست ورودی را به صورت درنگ در لحظه با این لیست تطبیق دهد.

گام دوم: نحوه فریب موتور جستجو (The Search Engine View)

شرط اجرایی: اگر سرور تطبیق موفقیت‌آمیزی داشته باشد و تشخیص دهد که درخواست‌کننده یک **خزنده وب (Crawler)** است.

واکنش سرور: یک صفحه HTML کاملاً سفارشی و بهینه‌سازی‌شده برای الگوریتم‌ها را تولید و ارسال می‌کند. این صفحه فاقد استایل‌های سنگین CSS یا جاوا اسکریپت است و اغلب شامل متن‌های طولانی، ساختار تگ‌های H1 تا H3 کاملاً استاندارد و سرشار از کلمات کلیدی هدف (مرتبط با یک موضوع پرجستجو) است.

نتیجه فریب: موتور جستجو این صفحه باکیفیت و مرتبط (اما جعلی) را می‌خواند و نمایه‌سازی می‌کند. به دلیل رعایت تمام اصول کلاسیک سئو، الگوریتم به این URL رتبه (**Rank**) بسیار بالایی در صفحه اول نتایج جستجو (**SERP**) اختصاص می‌دهد.

گام سوم: نحوه فریب کاربر انسانی (The User View)

شرط اجرایی: اگر سرور متوجه شود IP یا رشته هویتی متعلق به یک مرورگر تجاری استاندارد (فرد عادی) است.

واکنش سرور: دقیقاً برای همان آدرس اینترنتی (**URL واحد**)، یک محتوای کاملاً متفاوت ارسال می‌شود. این محتوا معمولاً هیچ ربطی به کلمات کلیدی ایندکس‌شده توسط موتور جستجو ندارد و مستقیماً با هدف کسب درآمد نامشروع طراحی شده است. این صفحات می‌توانند شامل تبلیغات آزاردهنده، هدایت اجباری (Redirect) به صفحات فروش محصولات غیرمجاز، سایت‌های کلاهبرداری یا حتی اجرای کدهای مخرب (**Malware**) باشند.

نتیجه فریب: کاربر با جستجوی یک عبارت کاملاً مشروع در موتور جستجو، روی یک لینک معتبر در رتبه اول کلیک می‌کند، اما به جای دریافت پاسخ مورد انتظار، در دام یک سیستم کلاهبرداری یا تبلیغاتی گرفتار می‌شود.

بخش 4: جمع‌بندی و نکات پیشرفته معماری

در تکنیک **Cloaking**، اسپمر تضاد ذاتی بین «آنچه ماشین (الگوریتم) می‌خواند» و «آنچه انسان مصرف می‌کند» را به ابزاری برای تقلب تبدیل می‌کند. این روش به عنوان یکی از شدیدترین تخلفات کلاه‌سیاه شناخته می‌شود و در صورت شناسایی، دامنه متخلف فوراً و به طور دائم از پایگاه داده موتور جستجو حذف (**De-indexed**) می‌شود.

تکنیک‌های پیشرفته موتورهای جستجو برای شناسایی Cloaking:

برای مقابله با این پدیده، مهندسان سیستم‌های جستجو الگوریتم‌های پیچیده‌ای را در سطح شبکه پیاده‌سازی کرده‌اند:

- خزش ناشناس (Anonymous/Honeypot Crawling):** موتورهای جستجو گاهی ربات‌های خود را با IP‌های عمومی (خارج از دیتاسنتر رسمی خود) و با User-Agent‌های جعلی شبیه‌ساز موبایل یا مرورگرهای دسکتاپ ارسال می‌کنند. سپس، کدهای دریافتی توسط این ربات‌های مخفی را با کدهای دریافتی توسط ربات رسمی مقایسه می‌کنند؛ هرگونه مغایرت هش‌کد (Hash mismatch) به معنای کشف Cloaking است.
- پردازش رفتار کاربر (Pogo-Sticking Analysis):** اگر کاربران متعددی روی یک لینک رتبه بالا کلیک کنند و بلافاصله (به دلیل مواجهه با محتوای نامربوط یا مخرب) دکمه Back مرورگر را بزنند و به صفحه جستجو برگردند، سیستم این بازگشت سریع را به عنوان یک سیگنال منفی شدید ثبت کرده و به صورت خودکار رتبه سایت را تنزل می‌دهد.

پاسخنامه مسئله 19.7: انواع تبلیغات اینترنتی و مدل درآمدی موتورهای جستجو

بخش 1: تحلیل صورت مسئله

هدف از این سوال، بررسی اقتصاد اطلاعات در محیط وب و درک دقیق نحوه درآمدزایی سرویس‌های به ظاهر رایگان (مانند موتورهای جستجو) است. دانشجو باید بتواند تفاوت ساختاری و رفتاری بین مدل‌های سنتی «تبلیغات مبتنی بر نمایش» و مدل نوین «تبلیغات مبتنی بر نیت» را درک کرده و دلیل برتری بلامنازع یکی بر دیگری را در پلتفرم‌های جستجو با استفاده از مفاهیم بازگشت سرمایه استدلال کند.

بخش 2: مبانی و پیش‌نیازهای اقتصاد وب

موتورهای جستجو برای تامین هزینه‌های نجومی و سنگین زیرساخت‌های خزش (Crawling)، نمایه‌سازی (Indexing) و پردازش درنگ‌درلحظه کوئری‌ها، نیازمند یک مدل درآمدی به شدت پایدار و مقیاس‌پذیر هستند. از آنجایی که در فرهنگ وب، کاربران تمایلی به پرداخت حق اشتراک مستقیم برای جستجو ندارند، تبلیغات به عنوان موتور محرک اقتصاد وب شکل گرفت. در این فضا، موفقیت یک سیستم تبلیغاتی مستقیماً به «نرخ تبدیل» (Conversion Rate) یا همان احتمال کلیک کاربر و انجام یک کنش نهایی (خرید، ثبت‌نام و...) بستگی دارد.

بخش 3: شرح گام‌به‌گام و پاسخ تفصیلی

برای درک تفاوت‌های بنیادین، ابتدا هر دو مدل تبلیغاتی را کالبدشکافی کرده و سپس به استراتژی انحصاری موتورهای جستجو می‌پردازیم:

1. تبلیغات نمایشی (Display Ads)

تعریف و ساختار: این تبلیغات معمولاً به شکل بنرهای گرافیکی، تصاویر متحرک، ویدیوها یا پاپ‌آپ‌ها (Pop-ups) در حاشیه صفحات وبسایت‌های مختلف (مانند خبرگزاری‌ها، وبلاگ‌ها و شبکه‌های اجتماعی) نمایش داده می‌شوند. مدل قیمت‌گذاری این آگهی‌ها معمولاً بر اساس CPM (Cost Per Mille) یا هزینه به ازای هر 1000 بازدید تنظیم می‌شود.

نحوه هدف‌گیری: نمایش این آگهی‌ها عموماً بر اساس محتوای صفحه (Contextual) یا پروفایل و تاریخچه مرور کاربر (Demographic / Behavioral) انجام می‌شود (مثلاً نمایش تبلیغ کفش ورزشی در یک سایت اخبار ورزشی).

چالش اصلی (مدل برون‌گرا یا Push): در این مدل، تبلیغ‌دهنده پیام خود را به سمت کاربر «هل می‌دهد». کاربری که در حال خواندن یک خبر سیاسی است، در آن لحظه نیت و قصد خرید ندارد؛ بنابراین این تبلیغات ماهیتی وقفه‌انداز (Interruptive) دارند. به همین دلیل، پدیده «نابینایی بنری» (Banner Blindness) رخ داده و نرخ کلیک (CTR) آن‌ها نسبتاً پایین است.

2. تبلیغات جستجوی حمایت‌شده (Sponsored Search)

تعریف و ساختار: این تبلیغات به صورت متن و لینک در صفحه نتایج موتور جستجو (SERP) ظاهر می‌شوند. آن‌ها از نظر ظاهری بسیار شبیه به نتایج ارگانیک (طبیعی) هستند، با این تفاوت که برچسب کوچکی مانند "Ad" یا "Sponsored" در کنار آن‌ها درج می‌شود. مدل قیمت‌گذاری اینجا عمدتاً CPC (Cost Per Click) است؛ یعنی تبلیغ‌دهنده تنها زمانی پول می‌پردازد که کاربر روی آگهی کلیک کند.

نحوه هدف‌گیری: نمایش این آگهی‌ها صرفاً بر اساس **کلمه کلیدی (Keyword)** است که کاربر دقیقاً در همان کسر از ثانیه در کادر جستجو وارد کرده است.

نقطه قوت (مدل درون‌گرا یا Pull): در این مدل، تبلیغ پاسخی مستقیم به نیاز کاربر است. کاربری که عبارت «خرید لپ‌تاپ لنوو» را جستجو می‌کند، نیت (Intent) قطعی و شفاف برای خرید دارد. در اینجا تبلیغ، کاربر را متوقف نمی‌کند، بلکه او را دقیقاً به سمت هدفش هدایت می‌کند. به همین دلیل، نرخ تبدیل این تبلیغات به شدت بالاست.

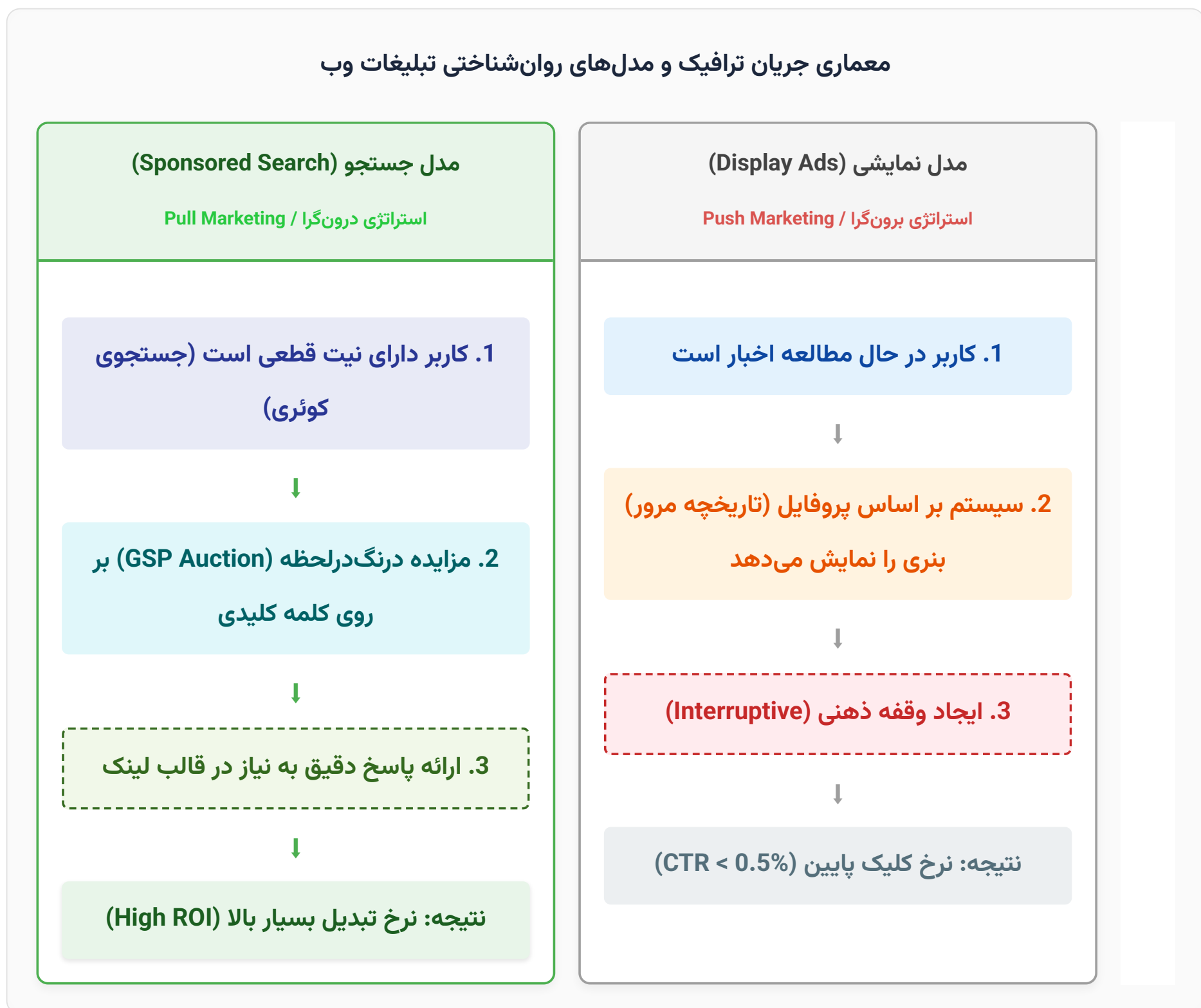
3. اتکای موتورهای جستجو به تبلیغات مبتنی بر نیت

موتورهای جستجو (مانند گوگل یا بینگ) به طور عمده بر **تبلیغات جستجوی حمایت‌شده** متکی هستند (بخش اعظم درآمد هلدینگ آلفابت/گوگل صرفاً از همین بخش تامین می‌شود).

دلیل این اتکای استراتژیک:

موتور جستجو تنها پلتفرمی در جهان است که به صورت لحظه‌ای و بی‌واسطه از «نیاز دقیق و صریح» کاربر مطلع می‌شود. این آگاهی از نیت کاربر (**User Intent**)، به موتور جستجو اجازه می‌دهد تا ترافیک بسیار ارزشمند و هدفمندی را به سمت کسب‌وکارها روانه کند. به دلیل نرخ بازگشت سرمایه (**ROI**) بالا، تبلیغ‌دهندگان حاضرند در یک سیستم مزایده‌ای درنگ‌درلحظه (**Real-time Auction**)، مبالغ قابل‌توجهی (گاهی تا ده‌ها دلار) برای هر کلیک روی کلمات کلیدی رقابتی پرداخت کنند.

بخش 4: نمودار شماتیک پیشرفته معماری سیستم‌های تبلیغاتی



ویژگی	تبلیغات نمایشی (Display Ads)	تبلیغات جستجو (Sponsored Search)
مکان نمایش	سایت‌های ناشر (وبلاگ‌ها، مجلات خبری)	صفحه نتایج موتور جستجو (SERP)
فرمت رایج	بنرهای گرافیکی، ویدیو، انیمیشن	لینک‌های متنی (شبیه نتایج ارگانیک)
مبنای هدف‌گیری	محتوای صفحه یا تاریخچه مرورگر	کوئری و کلمه کلیدی در لحظه جستجو
رویکرد کاربر	منفعل (در حال مطالعه وب‌سایت)	فعال (در حال جستجو برای رفع نیاز)

بخش 6: جمع‌بندی و نکات پیشرفته معماری سیستم‌های تبلیغاتی

ارزش بنیادین موتورهای جستجو در اقتصاد دیجیتال، در درک و تجاری‌سازی «نیت کاربر» نهفته است. در حالی که تبلیغات نمایشی سعی در «ایجاد نیاز» در ذهن مخاطب منفعل دارند، تبلیغات جستجو دقیقاً در کسری از ثانیه که «نیاز شکل گرفته است»، پاسخ را ارائه می‌دهند.

نکته پیشرفته معماری (الگوریتم مزایده و Quality Score):

برای جلوگیری از افت کیفیت نتایج و حفظ اعتماد کاربر، موتورهای جستجو جایگاه تبلیغات را صرفاً بر اساس مبلغ پیشنهادی تبلیغ‌دهنده نمی‌فروشند. آن‌ها از یک مدل ریاضی موسوم به مزایده قیمت دوم تعمیم‌یافته (Generalized Second Price - GSP) در نظریه بازی‌ها استفاده می‌کنند. در این معماری، رتبه نهایی هر تبلیغ با فرمول زیر محاسبه می‌شود:

$$Ad_Rank = Max_Bid \times Quality_Score$$

استفاده از فاکتور امتیاز کیفیت (Quality Score) (که بر پایه میزان ارتباط تبلیغ با کوئری و نرخ کلیک تاریخی محاسبه می‌شود) تضمین می‌کند که یک کسب‌وکار نمی‌تواند صرفاً با پرداخت پول بیشتر، نتایج نامربوط را به کاربر تحمیل کند. این ظرافت ریاضی، منافع هر سه ضلع مثلث وب (موتور جستجو، کاربر و تبلیغ‌دهنده) را در حالت تعادل نش (Nash Equilibrium) حفظ می‌کند.

پاسخنامه مسئله 19.8: مدل‌های قیمت‌گذاری تبلیغات (Pricing Models: CPM vs CPC)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، آشنایی دقیق با مکانیزم‌های پرداخت، انتقال ریسک و محاسبه هزینه در اکوسیستم پیچیده تبلیغات دیجیتال (Computational Advertising) است. دانشجو باید بتواند تفاوت بنیادین بین ارزش‌گذاری بر اساس «نمایش صرف» و ارزش‌گذاری بر اساس «تعامل قطعی» را درک کند و بداند هر مدل در چه سناریویی از قیف بازاریابی (Marketing Funnel) کاربرد دارد و چگونه موتورهای جستجو این مدل‌ها را در معماری سیستم درآمدزایی خود ترکیب می‌کنند.

بخش 2: مبانی و پیش‌نیازهای اقتصادی

در پلتفرم‌های تبلیغاتی پیشرفته (مانند **Google Ads** یا شبکه‌های **RTB**)، تبلیغ‌دهندگان بسته به هدف کمپین خود (ایجاد آگاهی از برند در سطح کلان یا فروش مستقیم و خرد)، استراتژی‌های متفاوتی برای پرداخت بودجه انتخاب می‌کنند. موتورهای جستجو برای پوشش دادن انواع نیازهای تجاری و بهینه‌سازی درآمد خود، این مدل‌های استاندارد را ارائه می‌دهند تا تعادل بین «موجودی تبلیغاتی ناشر» و «بودجه تبلیغ‌دهنده» حفظ شود.

بخش 3: شرح گام‌به‌گام و کالبدشکافی مدل‌ها

برای درک مکانیزم تسویه‌حساب در شبکه‌های تبلیغاتی، دو مدل اصلی را به صورت تحلیلی بررسی می‌کنیم:

1. مدل قیمت‌گذاری (CPM (Cost Per Mille

تعریف ساختاری: این عبارت به معنای «هزینه به ازای 1000 بار نمایش» است (کلمه Mille در لاتین به معنای هزار است). در این مدل، تبلیغ‌دهنده توافق می‌کند که به ازای هر 1000 باری که آگهی او در صفحه کاربران بارگذاری و به اصطلاح رندر (**Impression**) شود، مبلغ مشخصی را پرداخت کند.

نحوه محاسبه و ریسک: در اینجا، اینکه کاربر روی آگهی کلیک کرده است یا خیر، هیچ تاثیری در محاسبه هزینه ندارد. رویداد منجر به کسر بودجه، صرفاً **نمایش (View)** است. در این مدل، ریسک عملکرد (اینکه کسی کلیک نکند) کاملاً بر عهده تبلیغ‌دهنده است.

کاربرد اصلی: کمپین‌های آگاهی از برند (**Brand Awareness**)؛ جایی که هدف صرفاً دیده شدن لوگو و پیام توسط بیشترین افراد ممکن است.

2. مدل قیمت‌گذاری (CPC (Cost Per Click

تعریف ساختاری: این عبارت به معنای «هزینه به ازای کلیک» است (که در ادبیات بازاریابی گاهی **PPC** یا **Pay Per Click** نیز نامیده می‌شود). در این مدل، صرف بارگذاری و نمایش دادن تبلیغ کاملاً رایگان است و هیچ هزینه‌ای در بر ندارد.

نحوه محاسبه و ریسک: سیستم تبلیغاتی تنها زمانی از بودجه تبلیغ‌دهنده مبلغی کسر می‌کند که کاربر عملاً روی لینک آگهی کلیک کرده و یک رویداد قطعی رخ دهد. در این مدل، ریسک دیده نشدن و کلیک نخوردن به پلتفرم نمایش‌دهنده (ناشر/موتور جستجو) منتقل می‌شود؛ زیرا اگر کاربری کلیک نکند، ناشر با وجود اشغال کردن فضای سایت خود، درآمدی کسب نخواهد کرد.

کاربرد اصلی: بازاریابی عملکردمحور (**Performance Marketing**)؛ جایی که هدف، هدایت ترافیک به صفحه فرود (**Landing Page**) و ایجاد فروش مستقیم است.

بخش 4: نمودار شماتیک پیشرفته معماری رویدادهای صدور صورت‌حساب (Billing Triggers)

جریان کنترل و تریگرهای کسر بودجه در سیستم تبلیغاتی

CPM

Ad Request



Ad Rendered (DOM)



Impression Event



Advertiser Billed (\$)

CPC

Ad Rendered



Impression (Free)



User Clicks Link



Advertiser Billed (\$)

تفاوت کلیدی: در مدل CPC، رویداد Impression (نمایش) هیچ تریگر مالی در سیستم ایجاد نمی‌کند.

تفاوت محاسبه هزینه (پاسخ استنتاجی):

در مدل **CPM** هزینه صرفاً برای تضمین «**دیده شدن**» آگهی و اجاره فضای وبسایت پرداخت می‌شود، اما در مدل **CPC** هزینه منحصرأ در صورت ایجاد «**تعامل قطعی و ورود کاربر**» به سایت مقصد محاسبه می‌گردد.

بخش 5: جدول مقایسه‌ای تطبیقی

ویژگی مورد بررسی	مدل CPM (هزینه در هزار نمایش)	مدل CPC (هزینه در کلیک)
رویداد ایجادکننده هزینه	بارگذاری گرافیکی آگهی در صفحه	کلیک موفق و انتقال به صفحه فرود
تحمل‌کننده ریسک	تبلیغ‌دهنده (ممکن است کلیک دریافت نکند)	ناشر/موتور جستجو (ممکن است کلیک نشود)
مکان متداول استفاده	شبکه‌های تبلیغات بنری و ویدئویی (Display)	صفحه نتایج موتور جستجو (Sponsored Search)

بخش 6: جمع‌بندی و نکات پیشرفته معماری (الگوریتم eCPM)

به عنوان یک قاعده کلی در سیستم‌های بازیابی اطلاعات تجاری: موتورهای جستجو در بخش نتایج جستجوی خود (**Sponsored Search**) از مدل **CPC** استفاده می‌کنند تا پرداخت را مشروط به هدایت ترافیک قطعی کنند؛ اما در تبلیغات نمایشی (**Display Ads**) که هدف اصلی حک شدن نام در ذهن مخاطب است، از مدل **CPM** استفاده می‌شود.

نکته پیشرفته مهندسی (یکسان‌سازی در مزایده):

هنگامی که یک موتور جستجو (مثل شبکه نمایش گوگل) بخواهد همزمان تبلیغات **CPM** و **CPC** را در یک جایگاه واحد به مزایده بگذارد، با یک چالش ریاضی مواجه می‌شود: چگونه پیشنهاد یک دلار برای 1000 نمایش را با پیشنهاد نیم دلار برای 1 کلیک مقایسه کند؟

برای حل این مشکل، الگوریتم‌ها از مفهومی به نام **eCPM (Effective CPM)** یا «هزینه مؤثر در هزار نمایش» استفاده می‌کنند. سیستم با استفاده از یادگیری ماشین، نرخ کلیک احتمالی (**pCTR**) آگهی CPC را پیش‌بینی کرده و آن را با فرمول زیر به یک استاندارد مشترک (eCPM) تبدیل می‌کند تا مزایده عادلانه برگزار شود:

$$eCPM_{CPC} = Bid_{CPC} \times pCTR \times 1000$$

این فرمول نشان می‌دهد که چگونه سیستم‌های اطلاعاتی مدرن، معیارهای تعاملی را به معیارهای نمایشی ترجمه می‌کنند تا درآمد خود را به حداکثر برسانند.

بخش 1: تحلیل صورت مسئله

هدف از این سوال، آشنایی با مفهوم بسیار مهم و بنیادین «نیت کاربر» (**User Intent**) در علم بازیابی اطلاعات است. برای طراحی یک موتور جستجوی کارآمد و مدرن، صرفاً تطبیق کلمات کلیدی (**Keyword Matching**) و محاسبه امتیاز **TF-IDF** کافی نیست؛ سیستم باید از طریق پردازش زبان طبیعی بداند کاربر از وارد کردن این کلمات چه هدف غایی و پنهانی در سر دارد تا نوع درستی از محتوا (مقاله تحلیلی، فروشگاه آنلاین، یا صفحه ورود یک سازمان) را در قالب مناسب به او نمایش دهد.

بخش 2: مبانی و پیش‌نیازها

در سال 2002، آندری برودر (**Andrei Broder**) در یکی از مقالات مرجع حوزه جستجوی وب (با عنوان *A taxonomy of web search*)، استدلال کرد که جستجوهای محیط وب اساساً با جستجوهای کلاسیک کتابخانه‌ای متفاوت‌اند. او با استخراج و تحلیل حجم عظیمی از لاگ‌های موتور جستجو (**Search Logs**)، کوئری‌های کاربران را بر اساس «قصد و هدف نهایی» به سه دسته کلان و مجزا تقسیم‌بندی کرد. این معماری سه‌گانه، پایه و اساس طراحی رابط کاربری نتایج جستجو (**SERP**) در موتورهای جستجوی امروزی را شکل داد.

بخش 3: نمودار شماتیک پیشرفته از معماری طبقه‌بندی نیت کاربر



هدف: یادگیری و	هدف: دسترسی به
کشف پاسخ	URL خاص
استراتژی رتبه‌بندی:	استراتژی رتبه‌بندی:
Topic Authority	Exact Domain
فیچر SERP: گراف	Match
دانش، Snippet	فیچر SERP:
	لینک‌های سایت
	(Sitelinks)
هدف: انجام عملیات	
/ خرید	
استراتژی رتبه‌بندی:	
Service / Product	
Quality	
فیچر SERP: لیست	
محصولات، تبلیغات	
CPC	

بخش 4: شرح گام‌به‌گام و پاسخ تفصیلی دسته‌ها

سه دسته اصلی کوئری‌ها در طبقه‌بندی برودر به همراه منطق الگوریتمی و مثال‌های آن‌ها به شرح زیر است:

1. کوئری‌های ناوبری یا هدایتی (Navigational Queries)

تعریف: هدف کاربر در این نوع جستجو، رسیدن به یک وب‌سایت یا صفحه وب خاص است که از قبل می‌داند وجود دارد. در واقع کاربر قصد کشف و کسب اطلاعات جدید را ندارد، بلکه تنها آدرس دقیق (**URL**) سایت را نمی‌داند، به خاطر نمی‌آورد یا صرفاً ترجیح می‌دهد از کادر جستجوی مرورگر به عنوان یک «میانبر و بوک‌مارک» برای رسیدن به مقصد استفاده کند.

مثال واقعی: جستجوی عبارت «سامانه آموزشی دانشگاه تهران» یا «صفحه ورود اینستاگرام».

چالش موتور جستجو: در این حالت، کاربر دقیقاً یک مقصد مشخص و واحد در ذهن دارد. موتور جستجو باید دقیقاً همان سایت رسمی را در رتبه اول نشان دهد و سایر نتایج (حتی اگر محتوای غنی‌تری داشته باشند یا مقالات طولانی درباره آن سازمان نوشته باشند) برای کاربر کاملاً بی‌ارزش هستند. **حساسیت به رتبه اول در این کوئری‌ها مطلق است.**

2. کوئری‌های اطلاعاتی (Informational Queries)

تعریف: هدف کاربر، یادگیری، کسب آگاهی یا یافتن پاسخ برای یک پرسش مشخص است. در این حالت، کاربر مقصد (وب‌سایت) خاصی در ذهن ندارد و برایش تفاوتی نمی‌کند که این اطلاعات را از کدام سایت معتبر به دست بیاورد. او آماده است محتوای چندین سایت مختلف را مطالعه، ارزیابی و مقایسه کند.

مثال واقعی: جستجوی عبارت «نحوه کارکرد الگوریتم PageRank» یا «جمعیت کشور ژاپن چقدر است؟».

چالش موتور جستجو: موتور جستجو باید صفحاتی را در اولویت قرار دهد که دارای محتوای متنی غنی، دقیق، جامع و از منابع معتبر (مثل ویکی‌پدیا، مقالات علمی یا وبلاگ‌های تخصصی) باشند. موتورهای مدرن تلاش می‌کنند پاسخ‌های کوتاه این دسته را مستقیماً در قالب **Featured Snippets** در خود صفحه نتایج ارائه دهند.

3. کوئری‌های تراکنشی یا عملیاتی (Transactional Queries)

تعریف: هدف کاربر انجام یک فعالیت خاص در محیط وب است که نیازمند تعامل دوطرفه با یک سامانه است. این فعالیت منجر به یک «تراکنش» (مالی یا غیرمالی) می‌شود؛ مانند خرید یک کالا، دانلود یک فایل رایگان، ثبت‌نام در یک وب‌سایت، رزرو بلیت هواپیما یا گوش دادن به یک موسیقی.

مثال واقعی: جستجوی عبارت «خرید لپ‌تاپ لنوو مدل ThinkPad» یا «دانلود نرم‌افزار Python 3.12».

چالش موتور جستجو: در اینجا کاربر به دنبال خواندن مقاله، تاریخچه برند یا نقد و بررسی صرف نیست، بلکه به یک پلتفرم خدماتی و امن نیاز دارد. موتور جستجو باید فروشگاه‌های اینترنتی، درگاه‌های رزرو یا سایت‌های دانلود معتبر (و دارای موجودی کالا) را در صدر نتایج قرار دهد. این کوئری‌ها بالاترین ارزش تجاری را برای نمایش تبلیغات دارند.

بخش 5: جدول مقایسه‌ای و معیارهای سنجش

نوع کوئری	هدف اصلی کاربر	حساسیت به رتبه 1	معیار موفقیت جستجو
ناوبری	رسیدن به یک آدرس از پیش تعیین شده	مطلق (بسیار بالا)	کلیک سریع روی نتیجه اول و عدم بازگشت
اطلاعاتی	کشف دانش، حل مسئله و یادگیری	متوسط	مدت زمان ماندگاری بالا (Dwell Time) در سایت
تراکنشی	انجام کنش نهایی (خرید، رزرو، دانلود)	متوسط	نرخ تبدیل و انجام موفقیت‌آمیز تراکنش

بخش 6: جمع‌بندی و نکات پیشرفته معماری سیستم

درک معماری این طبقه‌بندی نشان می‌دهد که چرا موتورهای جستجوی مدرن (الگوریتم‌های **Universal Search**) فرمت‌های نمایش متفاوتی را برای یک صفحه واحد اتخاذ می‌کنند. اگر نیت سیستمیک کاربر تراکنشی تشخیص داده شود، رابط کاربری بلافاصله تغییر کرده و لیست قیمت‌ها و عکس محصولات جایگزین متن می‌شود؛ اما اگر نیت اطلاعاتی باشد، یک گراف دانش (**Knowledge Graph**) در سمت راست صفحه بارگذاری می‌شود.

نکته پیشرفته معماری (ابهام کوئری و نیت چندگانه):

امروزه در معماری موتورهای جستجو (استفاده از مدل‌های هوش مصنوعی مانند **BERT** و **RankBrain**)، مفهوم **ابهام کوئری (Query Ambiguity)** یک چالش بزرگ است. به عنوان مثال، اگر کاربر کلمه "Apple" را جستجو کند، نیت او می‌تواند ناوبری (سایت رسمی اپل)، اطلاعاتی (میوه سیب یا ارزش سهام) یا تراکنشی (خرید آیفون) باشد. در پاسخ به این نیت‌های چندگانه (**Multi-intent Queries**)، موتور جستجو از مدل‌های احتمالی برای ترکیب (**Blending**) نتایج استفاده می‌کند؛ به این معنا که صفحه اول نتایج را به بلوک‌های مختلفی تقسیم کرده و سهمی را به هر نیت اختصاص می‌دهد تا احتمال رضایت کاربر را به حداکثر برساند.

پاسخنامه مسئله 19.10: نتایج ارگانیک در برابر پولی (Organic vs. Paid Results)

بخش 1: تحلیل صورت مسئله

در این مسئله، ساختار دوگانه صفحه نتایج موتور جستجو (**SERP - Search Engine Results Page**) مورد بررسی قرار می‌گیرد. هدف این است که دانشجو تفاوت مرزهای «ارزش‌گذاری اطلاعاتی» و «ارزش‌گذاری تجاری» را به درستی درک کند و بداند که موتور جستجو برای چینش نتایج در هر کدام از این 2 بخش، از چه فرمول‌ها و منطق کاملاً متفاوتی استفاده می‌کند.

بخش 2: مبانی و پیش‌نیازها

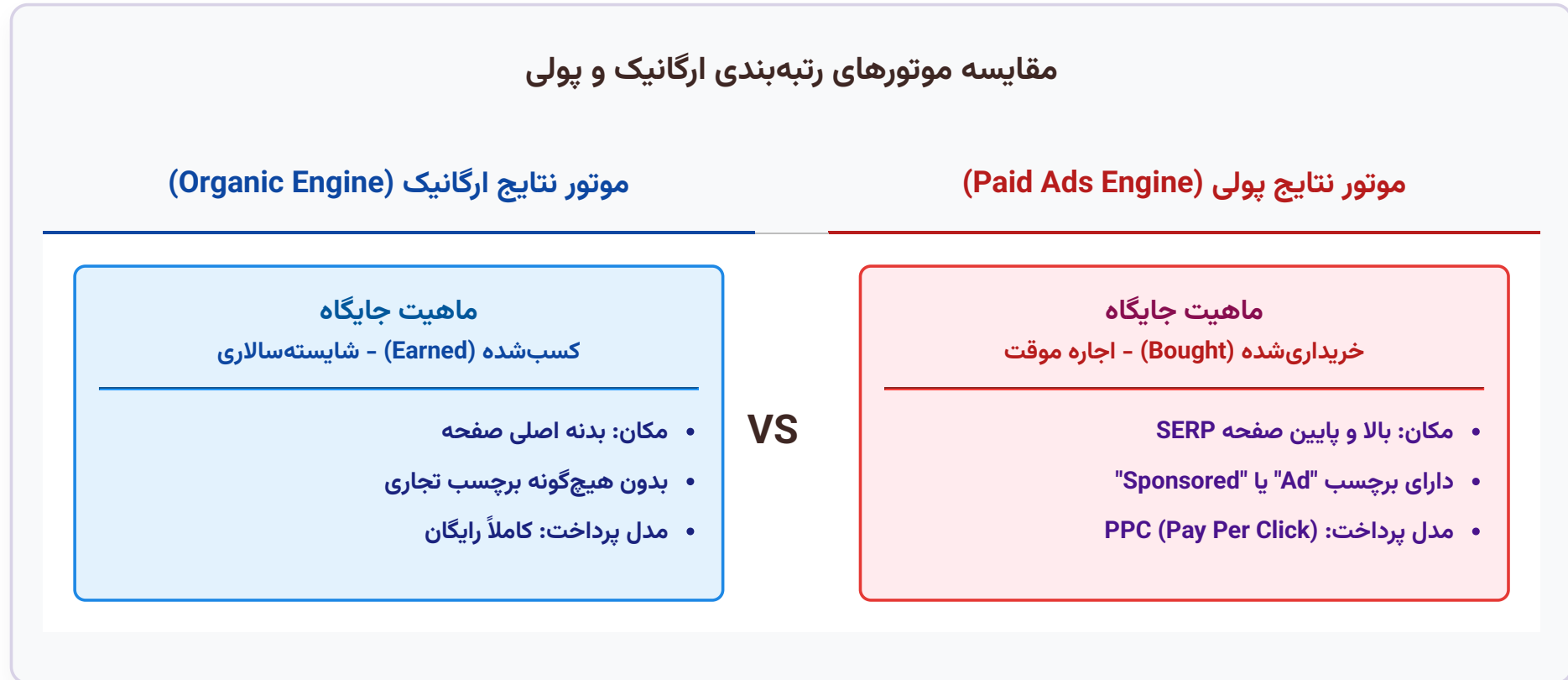
صفحه نتایج جستجو به عنوان محصول نهایی یک موتور جستجو، باید 2 هدف موازی و گاه متضاد را همزمان برآورده کند:

اول (بخش ارگانیک): ارائه بهترین و نامطلوب‌ترین اطلاعات به کاربر برای حفظ اعتماد و وفاداری او.

دوم (بخش پولی): کسب درآمد برای پوشش هزینه‌های عظیم زیرساخت و تضمین بقای پلتفرم.

برای جلوگیری از فریب کاربر، قوانین شفافیت و اخلاق مهندسی ایجاب می‌کند که این 2 بخش از نظر بصری و ماهوی کاملاً از یکدیگر تفکیک شوند تا کاربر بداند کدام نتیجه بر اساس شایستگی به دست آمده و کدام نتیجه خریداری شده است.

بخش 3: نمودار شماتیک معماری دوگانه در SERP



بخش 4: شرح گام‌به‌گام و پاسخ تفصیلی

برای پاسخ به این سوال، ابتدا تفاوت‌های ماهوی این 2 نوع نتیجه را بیان کرده و سپس مکانیزم رتبه‌بندی هر کدام را به صورت مجزا کالبدشکافی می‌کنیم:

1. تفاوت‌های نتایج ارگانیک و پولی

نتایج ارگانیک (Organic / Algorithmic Results): این نتایج، پاسخ‌های طبیعی الگوریتم موتور جستجو به کوئری کاربر هستند. حضور در این بخش کاملاً رایگان است و کسب‌وکارها تحت هیچ شرایطی نمی‌توانند با پرداخت پول به موتور جستجو، جایگاه خود را در این لیست بهبود بخشند. این نتایج بدنه اصلی و هسته مرکزی صفحه جستجو را تشکیل می‌دهند.

نتایج پولی یا حمایت‌شده (Paid / Sponsored Results): این نتایج در واقع تبلیغات (Ads) هستند. صاحبان وب‌سایت‌ها با پرداخت هزینه (معمولاً به روش **CPC** یا پرداخت به ازای هر کلیک) این جایگاه‌ها را اجاره می‌کنند. این نتایج معمولاً در بالاترین یا پایین‌ترین بخش صفحه قرار می‌گیرند و موتور جستجو قانوناً موظف است با درج برچسب‌هایی نظیر "Ad"، "Sponsored" یا "تبلیغ"، آن‌ها را به وضوح از نتایج طبیعی متمایز کند.

2. مکانیزم تعیین رتبه در نتایج ارگانیک

رتبه‌بندی در بخش ارگانیک، محصول پردازش‌های فوق‌العاده پیچیده الگوریتم‌های بازایی اطلاعات (**IR**) است. موتور جستجو برای تعیین اینکه کدام صفحه رتبه 1 را کسب کند، صدها فاکتور (**Ranking Factors**) را ارزیابی می‌کند که 2 ستون اصلی آن عبارتند از:

ارتباط متنی و معنایی: (Relevance) چقدر محتوای صفحه با نیت کاربر و کلمات کلیدی جستجو شده تطابق دارد؟ این مورد با بررسی تگ‌های **HTML**، چگالی کلمات، و الگوریتم‌های مدرن پردازش زبان طبیعی (**NLP**) محاسبه می‌شود.

اعتبار و کیفیت: (Authority & Quality) این صفحه چقدر در محیط وب معتبر است؟ این اعتبار عمدتاً از طریق تحلیل گراف پیوندها (مثل الگوریتم **PageRank**) و شمارش لینک‌های ورودی (**Backlinks**) از سایت‌های معتبر دیگر، و همچنین فاکتورهای تجربه کاربری (مثل سرعت بارگذاری سایت و امنیت) محاسبه می‌شود.

3. مکانیزم تعیین رتبه در نتایج پولی

برخلاف تصور رایج، رتبه‌بندی در بخش تبلیغات صرفاً با «پول بیشتر» تعیین نمی‌شود. موتورهای جستجو (مانند سیستم **Google Ads**) برای تعیین رتبه یک تبلیغ، از یک سیستم مزایده هوشمند به نام **مزایده تبلیغات (Ad Auction)** استفاده می‌کنند. رتبه تبلیغ (**Ad Rank**) بر اساس ترکیبی از **2** متغیر اصلی محاسبه می‌شود:

پیشنهاد قیمت: (Bid) حداکثر مبلغی که تبلیغ‌دهنده حاضر است برای یک کلیک روی تبلیغ خود پرداخت کند.

نمره کیفیت: (Quality Score) موتور جستجو نمی‌خواهد یک تبلیغ بی‌ربط را فقط به خاطر پول زیاد به کاربر نشان دهد (چون اعتماد کاربر سلب می‌شود). بنابراین، میزان مرتبط بودن متن تبلیغ با کلیدواژه، نرخ کلیک مورد انتظار (**Expected CTR**)، و همچنین کیفیت و سرعت صفحه فرود (**Landing Page**) سایت تبلیغ‌دهنده را از **1** تا **10** نمره‌دهی می‌کند.

$$Ad_Rank = Max_Bid \times Quality_Score$$

نکته اقتصادی الگوریتم: در این سیستم، یک کسب‌وکار کوچک با یک صفحه وب بسیار باکیفیت و مرتبط (نمره کیفیت بالا)، می‌تواند با پرداخت پول کمتر، یک شرکت بزرگ اما با محتوای نامربوط (نمره کیفیت پایین) را در مزایده شکست داده و در جایگاه بالاتری قرار گیرد.

بخش 5: جمع‌بندی

معماری دوگانه صفحات نتایج موتور جستجو (**SERP**)، تجلی مهندسی هوشمندانه برای برقراری تعادل میان «ارزش‌آفرینی اطلاعاتی» و «پایداری اقتصادی» است. نتایج ارگانیک جایگاهی است که باید با تولید محتوای ارزشمند، رعایت اصول سئو (**SEO**) و کسب اعتبار، آن را «به‌دست آورد» (**Earned**). در نقطه مقابل، نتایج پولی به عنوان شریان حیاتی درآمدزایی موتورهای جستجو عمل می‌کنند و جایگاهی هستند که می‌توان آن را برای مدتی محدود «خریداری کرد» (**Bought**). با این حال، شاهکار طراحی این سیستم در مکانیزم مزایده تبلیغات نهفته است؛ جایی که موتور جستجو صرفاً به بالاترین پیشنهاد مالی تکیه نمی‌کند، بلکه با دخالت دادن اهرم قدرتمند «نمره کیفیت» (**Quality Score**)، معیار بنیادین ارتباط و شایستگی محتوایی را به عنوان شرط اساسی و غیرقابل‌مذاکره در هر دو بخش حفظ می‌نماید. این رویکرد تضمین می‌کند که حتی در فضای کاملاً تجاری نیز، تجربه کاربر (**UX**) و اعتماد وی به پلتفرم دچار افت نگردد و اکوسیستم جستجو در یک تعادل ساختاری پایدار به حیات خود ادامه دهد.

پاسخنامه مسئله 19.11: چالش تخمین اندازه ایندکس (The Challenge of Estimating Web Index Size)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، به چالش کشیدن تصورات سنتی درباره مرزهای اطلاعات است. دانشجو باید درک کند که وب یک پایگاه داده متمرکز و ایستا (مانند یک هارد دیسک، سیستم فایل سنتی یا یک کتابخانه فیزیکی) نیست که بتوان فایل‌های آن را به صورت مطلق شمارش کرد؛ بلکه یک موجودیت ارگانیک، توزیع‌شده، گراف‌گونه و دائماً در حال تغییر است. در این مسئله، محدودیت‌های ذاتی معماری وب که مانع از شمارش دقیق و قطعی آن می‌شوند، به لحاظ فنی و الگوریتمی مورد بررسی قرار می‌گیرد.

بخش 2: مبانی و پیش‌نیازها

وقتی موتورهای جستجو مانند گوگل آماری از تعداد صفحات ارائه می‌دهند (مثلاً اعلام می‌کنند که ده‌ها میلیارد صفحه را می‌شناسند و در کسری از ثانیه در میان آن‌ها جستجو می‌کنند)، این عدد صرفاً نشان‌دهنده «اندازه ایندکس خود آن‌ها» (تعداد صفحاتی که با موفقیت خزش و نمایه‌سازی کرده‌اند) است، نه اندازه واقعی کل وب. در واقعیت، اندازه واقعی وب به دلیل تولید متغیرها و مسیرهای بی‌پایان، به صورت ریاضی «نامتناهی» ($\lim_{t \rightarrow \infty} |V| = \infty$) در نظر گرفته می‌شود.

بخش 3: نمودار شماتیک پیشرفته از معماری فضا و مرزهای وب



بخش 4: شرح گام‌به‌گام و کالبدشکافی چالش‌های بنیادین

محاسبه دقیق اندازه وب به دلیل دو چالش بنیادین و فنی زیر عملاً غیرممکن است و هرگونه ادعایی مبنی بر شمارش مطلق، از منظر علوم کامپیوتر فاقد اعتبار است:

1. تولید داینامیک محتوا و فضای بی‌نهایت URLها (Infinite URL Space & Dynamic Generation)

توضیح فنی: همان‌طور که در مباحث معماری کلاینت-سرور و صفحات داینامیک بررسی شد، وب مدرن از صفحاتی تشکیل شده که به جای فایل‌های ثابت، توسط کدهای برنامه‌نویسی و کوئری‌های پایگاه داده درنگ‌درلحظه (**On-the-fly**) تولید می‌شوند. با استفاده از ترکیب پارامترهای URL (مثلاً **?sort=price** یا **?date=2026-07-14**) می‌توان برای یک مجموعه داده مشخص، فضای حالت بی‌نهایتی از آدرس‌های اینترنتی یکتا خلق کرد.

تاثیر بر شمارش: سیستم‌هایی مانند تقویم‌های آنلاین، ماشین‌های حساب تحت وب یا فرم‌های فیلتر محصولات با متغیرهای پیوسته، در صورت درخواست عامل کاربر (یا ربات) می‌توانند تا بی‌نهایت صفحه جدید بسازند. از طرفی، وجود تله‌های خرنده (**Spider Traps**) باعث می‌شود خرنده‌ها در حلقه‌های بی‌نهایتی از لینک‌های تولیدشده‌ی خودکار گیر بیفتند. از نظر ریاضی و فنی، وقتی ماشین می‌تواند با یک حلقه **While** ساده بی‌نهایت **URL** معتبر خلق کند، تعیین یک «سقف مشخص» یا «تعداد کل» کاملاً بی‌معناست.

2. فقدان مرجع مرکزی مدیریت و ماهیت سیال شبکه (**Lack of Central Registry & Fluidity**)

توضیح فنی: برخلاف شبکه‌های کنترل‌شده و متمرکز (مثل دیتابیس یک بانک) که در آن‌ها ثبت هر رکورد جدید باید به یک هسته مرکزی گزارش و تایید شود، وب بر پایه پروتکل‌های غیرمتمرکز (**Decentralized**) کار می‌کند. هر فردی در هر کجای دنیا می‌تواند بدون نیاز به اجازه، هماهنگی یا ثبت‌نام در یک دایرکتوری مرکزی، یک سرور راه‌اندازی کرده و میلیون‌ها صفحه را به شبکه متصل کند.

تاثیر بر شمارش: موتور جستجو تنها صفحاتی را می‌تواند بشمارد که به آن‌ها «لینک» داده شده باشد (از طریق پیمایش گراف). میلیون‌ها صفحه وب پنهان (**Hidden / Deep Web**) یا اجزای قطع‌شده (**Disconnected Components**) وجود دارند که هیچ لینکی به آن‌ها اشاره نمی‌کند و برای خرنده‌ها کاملاً نامرئی هستند. علاوه بر این، سرعت ایجاد صفحات جدید و حذف صفحات قدیمی (خطای **404**) در هر ثانیه به قدری بالاست که هرگونه عملیات شمارش، در همان کسری از ثانیه که انجام می‌شود، فوراً منقضی و نامعتبر خواهد بود.

بخش 5: جدول مقایسه پارادایم‌ها (پایگاه‌داده کلاسیک در برابر وب)

ویژگی محیطی	پایگاه‌داده متمرکز (کتابخانه دیجیتال)	محیط وب (Web Environment)
تولید داده‌ها	متناهی و فیزیکی (فایل‌های ذخیره‌شده)	نامتناهی و پویا (رندر داینامیک)
معماری نظارتی	متمرکز و دارای ساختار ثبت (Registry)	کاملاً غیرمتمرکز و بدون کنترل واحد
قابلیت دسترسی (Visibility)	شفاف (تمام رکوردها با یک کوئری قابل استخراجند)	کدر (وجود بخش‌های پنهان، Dark Web و یتیم)

بخش 6: جمع‌بندی و نکات پیشرفته معماری سیستم

تلاش برای شمارش دقیق صفحات وب، مانند تلاش برای شمردن قطرات آب در یک رودخانه خروشان است؛ هم مرزهای آن نامشخص است (به دلیل وب پنهان و داینامیک بودن) و هم محتوای آن در هر لحظه تغییر می‌کند. به همین دلیل، دانشمندان حوزه بازیابی اطلاعات (**IR**) به جای «شمارش مطلق»، از تکنیک‌های آماری برای «تخمین نسبی» استفاده می‌کنند.

نکته پیشرفته مهندسی (الگوریتم تخمین Mark and Recapture):

برای غلبه بر مشکل فضای نامتناهی، محققان برای تخمین اندازه نسبی وب از مدلی آماری در زیست‌شناسی به نام **صید و بازصید (Capture-Recapture)** استفاده می‌کنند. در این روش، هم‌پوشانی (**Overlap**) نتایج دو موتور جستجوی مستقل (مثلاً موتور A و موتور B) بررسی می‌شود. اگر A تعداد صفحات مشخصی را بشناسد، B نیز تعداد مشخصی را بشناسد و C تعداد صفحات مشترک بین این دو باشد، حجم کل وب قابل‌دسترسی (N) با فرمول احتمالاتی زیر تقریب زده می‌شود:

$$N = \frac{|A| \times |B|}{|C|}$$

این فرمول که بر مبنای استقلال آماری دو سیستم خزشگر بنا شده است، نشان می‌دهد که مهندسی جستجو برای حل مسائل ذاتاً بی‌کران وب، به جای قطعیت مطلق، به تئوری احتمالات تکیه می‌کند.

پاسخنامه مسئله 19.12: مفهوم شینگل (Shingling)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، آشنایی با یکی از مهم‌ترین تکنیک‌های پیش‌پردازش متن برای شناسایی «صفحات تقریباً تکراری» (Near-Duplicate Web Pages) در موتورهای جستجو است. دانشجو باید مفهوم استخراج زیررشته‌های همپوشان (Overlapping Substrings) را درک کرده و بتواند به صورت عملی و با استفاده از الگوریتم پنجره لغزان، مجموعه‌ای از شینگل‌ها را از یک متن نمونه استخراج کند.

بخش 2: مبانی و پیش‌نیازها

همان‌طور که در مسائل قبلی اشاره شد، حجم زیادی از صفحات وب تکراری هستند. اگر موتور جستجو بخواهد دو صفحه را کلمه به کلمه (به صورت یک رشته واحد) مقایسه کند، با تغییر حتی 1 کلمه (مثلاً تغییر تاریخ انتشار صفحه یا یک تگ HTML تبلیغاتی)، دو سند از دیدگاه ماشین کاملاً متفاوت به نظر می‌رسند. برای حل این مشکل، آندری برودر (Andrei Broder) مفهوم شینگلینگ را معرفی کرد. در این روش، به جای مقایسه کل سند به صورت یکجا، سند به قطعات کوچک‌تر و متوالی تقسیم می‌شود تا مقاومت الگوریتم در برابر تغییرات جزئی (Robustness) افزایش یابد.

بخش 3: شرح گام به گام و پاسخ تفصیلی

1. تعریف مفهوم شینگل (Shingle)

یک **k-shingle** (یا شینگل مرتبه **k**)، به توالی پیوسته‌ای از دقیقاً **k** کلمه (توکن) در یک سند متنی گفته می‌شود. برای تبدیل یک سند به مجموعه‌ای از شینگل‌ها، یک پنجره به طول **k** کلمه در نظر می‌گیریم و آن را کلمه به کلمه روی متن حرکت می‌دهیم (Sliding Window). با این کار، سند متنی به یک «مجموعه» (Set) ریاضی از توالی‌های کوتاه تبدیل می‌شود.

نکته: در برخی متون، شینگلینگ در سطح کاراکتر نیز انجام می‌شود، اما در بازیابی اطلاعات وب، استاندارد معمولاً سطح کلمه (Word-level) است.

2. استخراج 3-shingle ها برای عبارت نمونه

عبارت داده شده در صورت سوال: "The sky is blue"

گام اول: توکنایز کردن (جداسازی کلمات)

ابتدا متن را به کلمات تشکیل‌دهنده‌اش تفکیک می‌کنیم:

1. The
2. sky
3. is
4. blue

گام دوم: حرکت دادن پنجره به طول $k = 3$

از آنجایی که در صورت سوال **shingle-3** خواسته شده است، باید توالی‌های سه‌کلمه‌ای پیوسته را استخراج کنیم:

شینگل اول: از کلمه اول شروع می‌کنیم و **3** کلمه متوالی را برمی‌داریم:

$$\text{Shingle}_1 = (\text{The}, \text{sky}, \text{is})$$

شینگل دوم: پنجره را یک کلمه به جلو می‌بریم (از کلمه دوم شروع می‌کنیم):

$$\text{Shingle}_2 = (\text{sky}, \text{is}, \text{blue})$$

پایان: چون بعد از کلمه دوم، کلمات کافی برای ساخت یک توالی سه‌تایی دیگر وجود ندارد، فرآیند در همین نقطه متوقف می‌شود.

پاسخ نهایی:

مجموعه **shingle-3** های استخراج شده از سند **D** به صورت مجموعه ریاضی زیر ارائه می‌گردد:

$$S(D, 3) = \{(\text{The}, \text{sky}, \text{is}), (\text{sky}, \text{is}, \text{blue})\}$$

بخش 4: جمع‌بندی و نکات پیشرفته معماری سیستم

دلیل اصلی همپوشانی (**Overlap**) کلمات در استخراج شینگل‌ها این است که اگر در یک سند متنی طولانی، جابه‌جایی، حذف یا تغییر کوچکی در یک جمله رخ دهد، تنها چند شینگل محدود تغییر کرده و اکثر قریب به اتفاق شینگل‌های دیگر دست‌نخورده باقی می‌مانند. با استفاده از این تکنیک، موتور جستجو می‌تواند با محاسبه میزان اشتراک شینگل‌های دو صفحه، درصد شباهت آن‌ها را با دقت بسیار بالایی اندازه‌گیری کند.

نکته پیشرفته معماری (الگوریتم‌های هشینگ و MinHash):

در سیستم‌های مقیاس‌پذیر موتورهای جستجو، ذخیره مستقیم رشته‌های متنی (String) به عنوان شینگل، حافظه رم را به سرعت دچار سرریز می‌کند. برای حل این مشکل، مهندسان سیستم هر شینگل را با استفاده از توابع هشینگ قدرتمند (مانند **64-bit Rabin Fingerprints**) به یک عدد صحیح تبدیل می‌کنند.

سپس برای مقایسه دو مجموعه شینگل عظیم بدون انجام عملیات تقاطع با مرتبه $O(n^2)$ ، از الگوریتم احتمالی **MinHash** استفاده می‌شود. این الگوریتم می‌تواند ضریب شباهت ژاکارد (**Jaccard Similarity**) دو صفحه را در زمان خطی و با مصرف حافظه‌ای ناچیز تخمین بزند. این معماری تضمین می‌کند که شناسایی محتوای تکراری (**Deduplication**) روی گراف وبِ میلیارد صفحه‌ای کاملاً امکان‌پذیر و مقرون‌به‌صرفه باشد.

بخش 1: تحلیل صورت مسئله

در این مسئله، مفهوم قابلیت دسترسی (**Reachability**) در گراف جهت‌دار وب مورد ارزیابی قرار می‌گیرد. دانشجو باید با استفاده از ویژگی‌های توپولوژیک مدل پایونی (**Bow-tie Model**)، مسیرهای ممکن برای یک الگوریتم پیمایش وب (خزنده یا **Crawler**) را از نقطه‌ی شروع در بخش **IN** به نقطه‌ی پایان در بخش **OUT** به صورت تحلیلی اثبات کند.

بخش 2: مبانی و پیش‌نیازها

گراف وب را می‌توان به صورت یک گراف جهت‌دار $G = (V, E)$ در نظر گرفت. در مدل پایونی، جریان پیمایش در این گراف اساساً یک‌طرفه است. گره‌های بخش ورودی (**IN**) دارای مسیرهایی به سمت مؤلفه قویاً همبند مرکزی (**SCC**) هستند و **SCC** نیز دارای مسیرهایی به سمت گره‌های بخش خروجی (**OUT**) است. از آنجا که خزنده‌های وب نیز دقیقاً از همین منطق ساختاری (حرکت در امتداد یال‌های خروجی) پیروی می‌کنند، قابلیت رسیدن از یک بخش به بخش دیگر، مستقیماً به وجود این مسیرهای جهت‌دار در گراف کلان بستگی دارد.

بخش 3: نمودار شماتیک جریان اطلاعات در مدل پایونی



بخش 4: شرح گام‌به‌گام و پاسخ تفصیلی

پاسخ صریح به این سوال «بله» است. الگوریتم خزش می‌تواند با دنبال کردن لینک‌های خروجی (**Out-links**) مسیر خود را از بخش ورودی به بخش خروجی به صورت کامل طی کند. از منظر تئوری گراف، این پیمایش موفقیت‌آمیز می‌تواند از طریق **2** مسیر کاملاً مجزا و با واسطه‌های متفاوت انجام پذیرد:

مسیر 1: عبور از هسته مرکزی (جریان اصلی اینترنت)

این مسیر رایج‌ترین شکل شارش اطلاعات در محیط وب است که طی آن، داده‌ها از سایت‌های معتبر و متصل به هم عبور می‌کنند. مراحل این مسیر به شرح زیر است:

آغاز از بخش ورودی (IN): الگوریتم جستجو کار خود را از گره‌ای در بخش ورودی آغاز می‌کند. تمامی مسیرهای پیش‌رو در این بخش، ذاتاً به سمت مرکز گراف همگرا می‌شوند.

گذر از مؤلفه قویاً همبند (SCC): با دنبال کردن یال‌های جهت‌دار، خزنده وارد هسته مرکزی گراف وب می‌شود. در این مؤلفه امکان گردش نامتناهی بین سایت‌های مرجع وجود دارد (زیرا هر گره به گره دیگر مسیر دارد)، تا زمانی که خزنده یک یال متصل به بیرون از هسته پیدا کند.

توقف در بخش خروجی (OUT): خزنده از هسته خارج شده و به صفحاتی وارد می‌شود که دیگر یال بازگشتی به هسته مرکزی ندارند. در اینجا مسیر به پایان می‌رسد.

$IN \rightarrow SCC \rightarrow OUT$

مسیر 2: دور زدن هسته (مسیر مستقیم یا میان‌بر)

خزنده وب برای رسیدن به بخش خروجی، لزوماً نیازی به ورود به هسته مرکزی وب ندارد. در توپولوژی حاشیه‌ای این گراف، ساختاری به نام «لوله» (Tube) تعریف می‌شود.

اگر یک صفحه (شاخک متصل به بخش ورودی) یالی مستقیم به صفحه‌ای دیگر در بخش خروجی داشته باشد، یک لوله تشکیل می‌گردد. در این حالت استثنایی، خزنده با استفاده از این لینک مستقیم، مستقیماً از بخش **IN** به بخش **OUT** پرتاب می‌شود، بدون آنکه هرگز هسته مرکزی وب (**SCC**) را ببیند و وارد آن شود.

$IN \rightarrow Tube \rightarrow OUT$

بخش 5: جمع‌بندی

تحلیل مسیرهای دسترسی در گراف وب بر پایه توپولوژی پایبونی، نقابی از پیچیدگی‌های پنهان در ساختار ظاهراً همبند اینترنت برمی‌دارد. جریان یک‌طرفه اطلاعات اثبات می‌کند که گراف وب رفتاری کاملاً نامتقارن و مشابه با شبکه‌های شارهای (Flow Networks) دارد: حرکت در مسیر موافق (دنبال کردن لینک‌های خروجی) همواره از **IN** به سمت **OUT** امکان‌پذیر است، اما حرکت معکوس به دلیل ماهیت گراف‌های جهت‌دار غیرمدور حاشیه‌ای (**DAG**) از نظر ساختاری محال است.

درک این دینامیک جریانی برای مهندسی و ارزیابی سیستم‌های بازیابی اطلاعات حیاتی است. جهت سنجش صحت پیاده‌سازی دانشجویان و بررسی عملکرد الگوریتم‌های پیمایش گراف در سامانه‌های داوری خودکار (**Autograders**) نظیر پلتفرم کوئرا (Quera)، طراحی دقیق فایل‌های ورودی (**Test Cases**) الزامی است. این تست‌ها باید به گونه‌ای ساختاریافته ماتریس مجاورت گراف را شبیه‌سازی کنند که خزنده‌ی طراحی‌شده توسط دانشجو، هم با مسیرهای پرهزینه‌ی درگیر در چرخه‌های **SCC** و هم با مسیرهای میان‌بر و خطی مبتنی بر **Tube** به درستی مواجه شده و مورد ارزیابی جامع قرار گیرد.

پاسخنامه مسئله 19.14: شناسایی مزرعه لینک و تاثیر آن بر رتبه‌بندی

(Link Farming & PageRank Manipulation)

بخش 1: تحلیل صورت مسئله

در این سناریو، با یک الگوی غیرطبیعی در توپولوژی گراف وب روبه‌رو هستیم: مجموعه‌ای متراکم از گره‌ها (سایت‌ها) که بدون داشتن ارزش محتوایی، صرفاً برای ارجاع به یکدیگر و در نهایت ارجاع متمرکز به یک هدف واحد (گره **X**) ایجاد شده‌اند. هدف این

است که دانشجو بتواند این پدیده را به عنوان یک حمله الگوریتمی و سوءاستفاده از معماری موتور جستجو نام‌گذاری کرده و مکانیزم ریاضیاتی تاثیر آن را بر الگوریتم‌های تحلیل پیوند (**Link Analysis**) مانند **PageRank** به دقت کالبدشکافی کند.

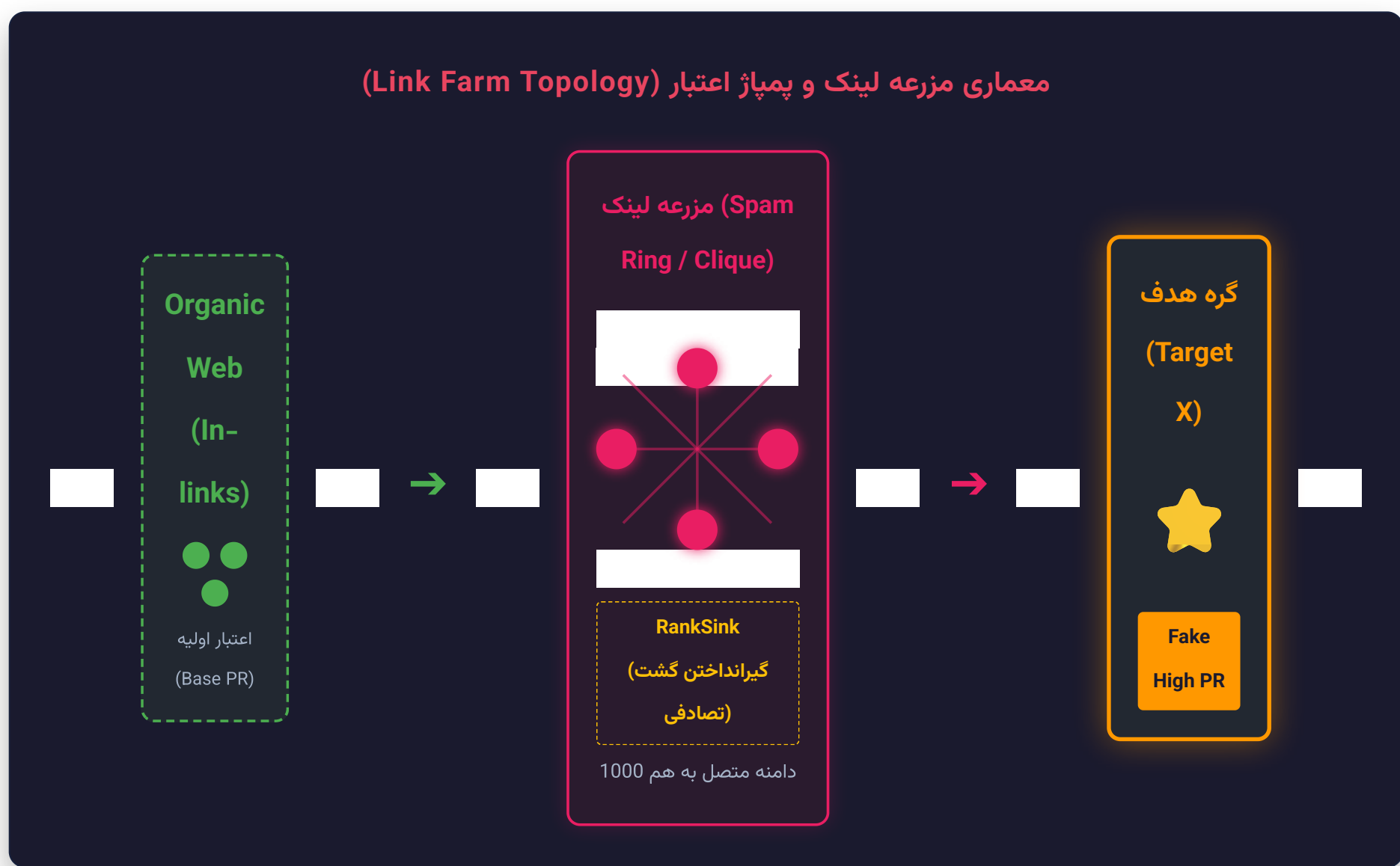
بخش 2: مبانی و پیش‌نیازهای ریاضی الگوریتم

الگوریتم **PageRank** بر یک فرض بنیادین استوار است: هر لینک خروجی، یک «رای اعتماد» (**Vote of Confidence**) است. اگر صفحه‌ای از صفحات متعدد و معتبر لینک دریافت کند، خودش نیز معتبر می‌شود. در مدل ریاضی «گشت تصادفی» (**Random Surfer**)، کاربر مجازی با کلیک تصادفی روی لینک‌ها در گراف حرکت می‌کند و رتبه هر صفحه، در واقع احتمال حضور این کاربر در آن صفحه در درازمدت (حالت پایا یا **Steady-State**) است که با رابطه مارکوفی زیر محاسبه می‌گردد:

$$PR(A) = \frac{1 - d}{N} + d \sum_{i=1}^n \frac{PR(T_i)}{C(T_i)}$$

در این فرمول، d فاکتور میرایی (معمولاً **0.85**) و $C(T_i)$ تعداد لینک‌های خروجی صفحه ارجاع‌دهنده است. اسپمرها با درک این فرمول، ساختارهایی طراحی می‌کنند که مخرج کسر را کنترل کرده و جریان توزیع احتمال را به نفع خود مصادره نمایند.

بخش 3: نمودار شماتیک پیشرفته از توپولوژی حمله



بخش 4: شرح گام‌به‌گام و کالبدشکافی منطق حمله

1. نام‌گذاری ساختار: مزرعه لینک (**Link Farm**)

به این ساختار متراکم و مصنوعی در ادبیات بازیابی اطلاعات، مزرعه لینک (**Link Farm**) یا حلقه هرزنامه (**Spam Ring**) گفته می‌شود. هدف از ایجاد این شبکه‌های کلاه‌سیاه (**Black-hat SEO**)، دستکاری تعمدی گراف وب برای فریب دادن موتورهای جستجو و مهندسی معکوس الگوریتم رتبه‌بندی است.

برای درک اینکه مزرعه لینک چگونه کار می‌کند، باید تغییر مسیر شارش اعتبار (**Authority Flow**) در گراف را در سه مرحله بررسی کنیم:

گام اول: هم‌افزایی درون شبکه‌ای (تله RankSink)

آن 1000 دامنه با لینک دادن متقاطع به یکدیگر، یک زیرگراف به شدت متصل (**Dense Subgraph**) یا اصطلاحاً یک گروهک (**Clique**) تشکیل می‌دهند. وقتی اعتبار اولیه (حتی به مقدار بسیار کم) از گراف طبیعی به یکی از این سایت‌ها وارد می‌شود، به جای اینکه از شبکه خارج شود، در یک حلقه بی‌نهایت بین این 1000 دامنه دست‌به‌دست می‌شود. در مدل ریاضی، این کار باعث می‌شود گشت تصادفی در این زیرگراف گیر بیفتد (پدیده **RankSink**) و در نتیجه، امتیاز تمام این سایت‌ها به صورت مصنوعی و تصاعدی بالا برود تا زمانی که توسط فاکتور میرایی (**d**) متوقف شود.

گام دوم: پمپاژ اعتبار به گره هدف (Target Node X)

پس از آنکه این 1000 دامنه از نظر الگوریتم به سایت‌هایی «به ظاهر معتبر» و قدرتمند تبدیل شدند، همگی یک لینک خروجی متمرکز به سایت **X** می‌دهند. الگوریتم **PageRank**، به صورت خطی عمل کرده و این لینک‌ها را به عنوان 1000 رای اعتماد قدرتمند از طرف سایت‌های دارای رتبه بالا تفسیر می‌کند و تمام آن اعتبار انباشته شده را به گره **X** تزریق می‌نماید.

نتیجه نهایی حمله:

سایت **X** که در واقعیت ممکن است محتوای بسیار بی‌ارزشی داشته باشد، ناگهان به دلیل دریافت **جرم اسپم (Spam Mass)**، در صدر نتایج جستجو قرار می‌گیرد؛ زیرا الگوریتم فریب خورده و وزن ریاضی بسیار بالایی به بردارهای ورودی آن اختصاص داده است.

بخش 5: جمع‌بندی و نکات پیشرفته معماری سیستم (راه‌حل‌های دفاعی)

در پیاده‌سازی و طراحی ساختارهای داوری خودکار برای پروژه‌های گراف وب (نظیر آنچه در سامانه‌های بررسی کد دانشجویان و اتوگریدرها **Autograders** ارزیابی می‌شود)، تشخیص مزرعه لینک نیازمند محاسبه معیارهای گراف‌کاوی فراتر از صرف بررسی درجه ورودی (**In-degree**) است.

نکات پیشرفته معماری (الگوریتم‌های مقابله):

- **تحلیل توپولوژی و ضریب خوشه‌بندی (Clustering Coefficient):** موتورهای جستجو برای شناسایی این حملات، به دنبال زیرگراف‌های متراکم غیرطبیعی می‌گردند. اگر ضریب خوشه‌بندی شبکه‌ای از سایت‌ها به عدد 1 نزدیک باشد (یعنی همه به هم متصلند) و همگی به یک سایت خارجی لینک دهند، سیستم آن را به عنوان یک آنومالی (ناهنجاری) علامت‌گذاری می‌کند.
- **الگوریتم تراسترنگ (TrustRank):** یکی از زیباترین پاسخ‌های مهندسی به این چالش، الگوریتم تراسترنگ بود. در این معماری، به جای اینکه اعتبار در کل گراف پخش شود، شارش اعتبار صرفاً از یک **مجموعه بذر کاملاً مورد اعتماد (Trusted Seed Sets)** - مانند دامنه‌های دانشگاهی (edu) و دولتی (gov) - آغاز می‌شود. از آنجا که مزارع لینک راهی برای دریافت لینک طبیعی از این مراجع امن ندارند، فاصله آن‌ها (Shortest Path) از هسته اعتماد افزایش یافته و اثر پمپاژ آن‌ها خنثی می‌گردد.

پاسخنامه مسئله 19.15: مبارزه با کلمات کلیدی پنهان (Combating Hidden Keyword Stuffing)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، طراحی یک مکانیزم دفاعی در برابر یکی از تکنیک‌های کلاسیک و بسیار مخرب اسپم محتوایی (**Content Spam**) در سئوی کلاه‌سیاه (**Black-hat SEO**) است. در این سناریو، اسپمر سعی دارد با پنهان کردن کلمات کلیدی از دید کاربر انسانی (برای حفظ زیبایی بصری و جلوگیری از تجربه کاربری نامطلوب) و همزمان نمایش آن‌ها در سورس‌کد به خزنده موتور جستجو، الگوریتم‌های محاسبه چگالی کلمات کلیدی (**Keyword Density**) و مدل‌های **TF-IDF** را فریب دهد. دانشجو باید راهکاری مهندسی ارائه دهد که تفاوت معنادار بین «سورس‌کد خام» و «خروجی رندر شده نهایی» را در سمت سرور جستجوگر تشخیص دهد.

بخش 2: مبانی و پیش‌نیازهای معماری

خزنده‌های وب در نسل‌های اولیه، تنها فایل متنی **HTML** را به عنوان یک رشته کاراکتری پردازش می‌کردند. در آن زمان، اگر کلمه‌ای در سورس‌کد وجود داشت، به طور مطلق به عنوان محتوای معتبر صفحه در نظر گرفته می‌شد. اسپمرها با سوءاستفاده از کدهای **CSS** (مانند یکسان کردن رنگ متن و پس‌زمینه) این ضعف بنیادین را هدف قرار دادند. امروزه موتورهای جستجوی مدرن (مانند گوگل با سیستم **WRS - Web Rendering Service**) دیگر به کدهای خام اعتماد نمی‌کنند؛ بلکه صفحه را دقیقاً همان‌گونه که مرورگر کلاینت برای کاربر انسانی تفسیر و ترسیم می‌کند، «رندر» و ارزیابی بصری می‌کنند.

بخش 3: نمودار شماتیک پیشرفته از خط‌لوله تحلیل بصری (Visual Analysis Pipeline)



Phase 3: Visual Heuristics

1. Contrast Ratio Calc

2. Viewport Bounds Check

3. Z-Index Overlap Scan

4. Opacity & Display Rules

بخش 4: شرح گام به گام و سناریوی الگوریتمی

به عنوان طراح سیستم بازیابی اطلاعات، برای کشف این تقلب باید از رویکرد پردازش مبتنی بر رندر (**Rendering-based Processing**) استفاده کنیم. سناریوی الگوریتمی مرحله به مرحله به شرح زیر پیاده سازی می شود:

مرحله اول: بارگذاری و ساخت درخت DOM و CSSOM

خزنده به جای خواندن ساده ی تگ های متنی **HTML**، دقیقاً مانند یک مرورگر واقعی بدون رابط کاربری (**Headless Browser**) عمل می کند. تمامی فایل های جانبی شامل استایل شیت ها (**CSS**) و اسکریپت ها (**JS**) از سرور هدف دانلود شده و مدل شی گرای سند (**DOM**) و مدل شی گرای کدهای استایل (**CSSOM**) در حافظه سرور موتور جستجو ساخته می شوند.

مرحله دوم: استخراج استایل های محاسبه شده (Computed Styles)

الگوریتم، درخت رندر (**Render Tree**) را تشکیل داده و ویژگی های نهایی اعمال شده بر هر گره (**Text Node**) را محاسبه می کند. در این مرحله با قطعیت مشخص می شود که هر کلمه دقیقاً چه رنگی، چه اندازه ای و چه موقعیتی روی مختصات هندسی صفحه (**Screen Coordinates**) دارد.

مرحله سوم: اجرای فیلترهای هندسی و بصری (Heuristic Rules)

روی استایل های استخراج شده، مجموعه ای از شروط منطقی بررسی می شود. الگوریتم به دنبال الگوهای تقلب زیر می گردد:

1. **بررسی نسبت کنتراست (Contrast Ratio):** روشنایی نسبی (Luminance) رنگ متن (L_1) با رنگ پس زمینه (L_2) مقایسه می شود. فرمول استاندارد دسترسی پذیری وب (**WCAG**) برای محاسبه این کنتراست به شکل زیر است:

$$CR = \frac{L_1 + 0.05}{L_2 + 0.05}$$

اگر این نسبت به عدد 1 بسیار نزدیک باشد (مثل رنگ سفید روی پس زمینه سفید)، متن به عنوان «پنهان شده» نشانه گذاری می گردد.

2. **بررسی مختصات مکانی هندسی:** اگر باکسِ دربرگیرنده متن (**Bounding Box**) در مختصات منفی بسیار دور از صفحه نمایش قرار داشته باشد (مثلاً **left: -9999px**) یا سایز فونت آن از نظر هندسی صفر باشد (**font-size: 0**).
3. **بررسی شفافیت و پوشش:** تحلیل مقادیر **opacity: 0** یا **display: none**، و همچنین بررسی قرار گرفتن یک لایه دیگر (المان‌های div) روی متن اصلی با استفاده از دستور **z-index**.

مرحله چهارم: محاسبه نرخ جریمه (Penalization)

اگر کلماتی کشف شوند که در مرحله قبل به عنوان اسپم پنهان نشانه‌گذاری شده‌اند، الگوریتم نه تنها آن کلمات را از بردار وزن‌دهی (**Index**) حذف می‌کند تا تاثیری در رتبه صفحه نداشته باشند، بلکه به دلیل تلاش واضح سایت برای فریب سیستم، کل دامنه را با اعمال یک جریمه سنگین الگوریتمی (**Algorithmic Penalty**) مواجه کرده و یا در موارد شدید، دامنه را به طور کامل از پایگاه داده موتور جستجو حذف (**De-index**) می‌کند.

بخش 5: جمع‌بندی

تکامل موتورهای جستجو از مدل ساده‌انگارانه‌ی «پردازش متن محض» (**Text Processing**) به پارادایم نوین «درک بصری صفحه» (**Visual Page Analysis**)، نقطه‌ی عطفی در تاریخ مهندسی بازیابی اطلاعات و مبارزه با تکنیک‌های مخرب سئو است. امروزه یک موتور جستجوی پیشرفته، اعتبار واژگان را صرفاً بر مبنای حضور آن‌ها در کدهای **HTML** ارزیابی نمی‌کند؛ بلکه با شبیه‌سازی دقیق محیط کلاینت، ساختار هندسی و دسترسی‌پذیری بصری محتوا را برای چشم انسان می‌سنجد. در لبه‌ی تکنولوژی این معماری دفاعی، کشف تقلب از مرزهای محاسبات مدل درختی (**DOM-based Heuristics**) نیز فراتر رفته است. برای مقابله با اسپم‌های پیشرفته‌ای که از بوم‌های گرافیکی نظیر **Canvas** بهره می‌برند، موتورهای جستجو از قدرت شبکه‌های عصبی پیچشی (**CNNs**) و بینایی ماشین (**Computer Vision**) استفاده می‌کنند. در این مکانیزم هوشمند، از خروجی نهایی رندر شده یک اسکرین‌شات پیکسلی (**Viewport Snapshot**) تهیه شده و به مدل‌های پردازش تصویر سپرده می‌شود. مقایسه تطبیقی میان «آنچه ماشین در کد می‌خواند» و «آنچه چشم انسان در تصویر می‌بیند»، هرگونه ناهنجاری و کلمات کلیدی پنهان را آشکار ساخته و جریمه‌های الگوریتمی سنگینی را برای دامنه‌های خاطی به همراه خواهد داشت.

پاسخنامه مسئله 19.16: محاسبه حراج قیمت دوم تعمیم‌یافته

(GSP - Generalized Second-Price Auction)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، درک مکانیزم پیچیده قیمت‌گذاری و تئوری مزایده‌ها در پلتفرم‌های تبلیغاتی موتورهای جستجو (مانند سیستم **Google Ads**) است. دانشجو باید بتواند الگوریتم «حراج قیمت دوم تعمیم‌یافته» (**Generalized Second-Price Auction**) را به صورت ریاضی روی یک سناریوی رقابتی ساده پیاده‌سازی کند و تفاوت بنیادین بین «مبلغ پیشنهادی» (**Max Bid**) به عنوان یک استراتژی، و «مبلغ پرداختی واقعی» (**Actual CPC**) به عنوان خروجی سیستم را به دقت محاسبه نماید.

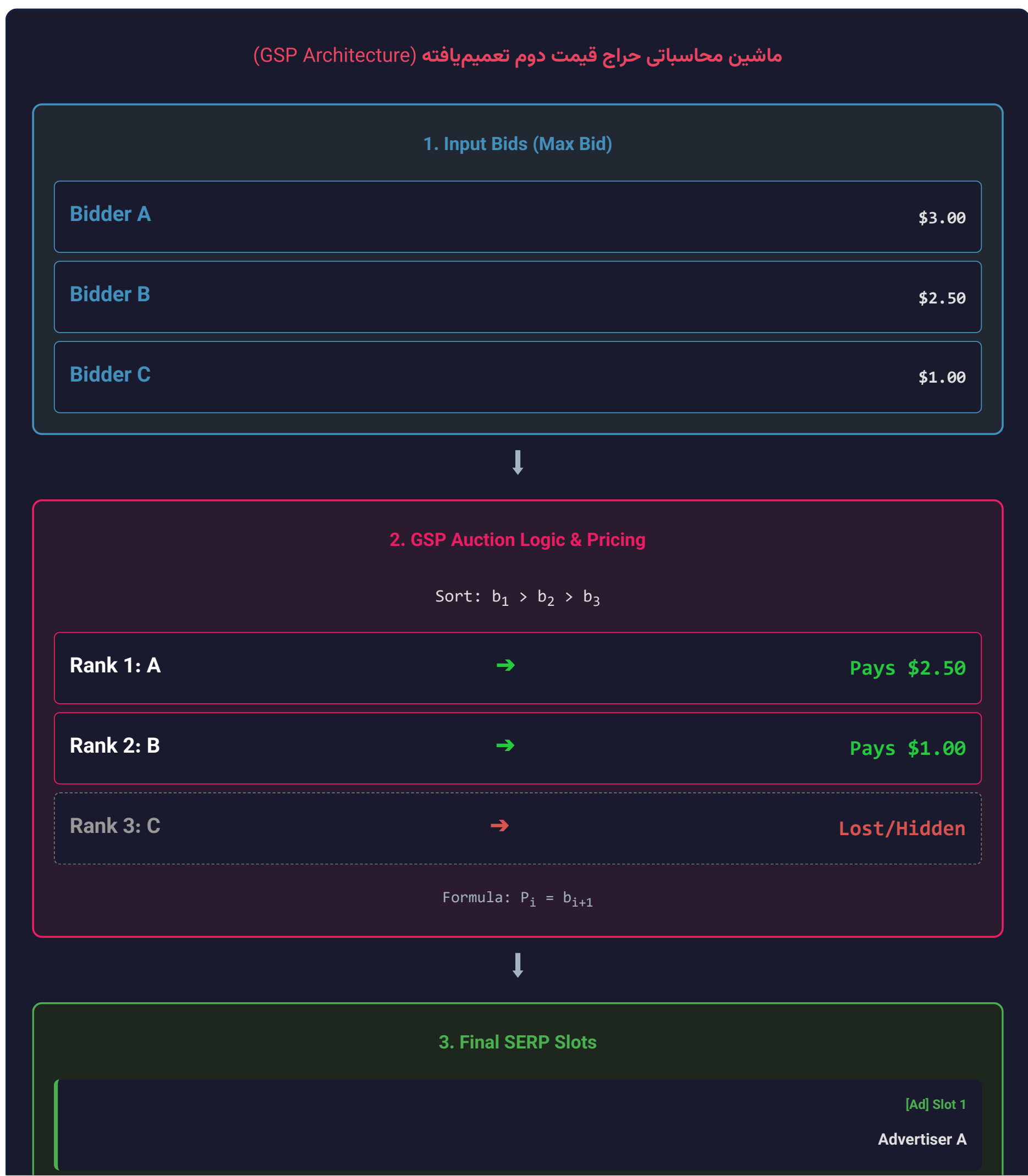
بخش 2: مبانی، پیش‌نیازها و تئوری بازی‌ها

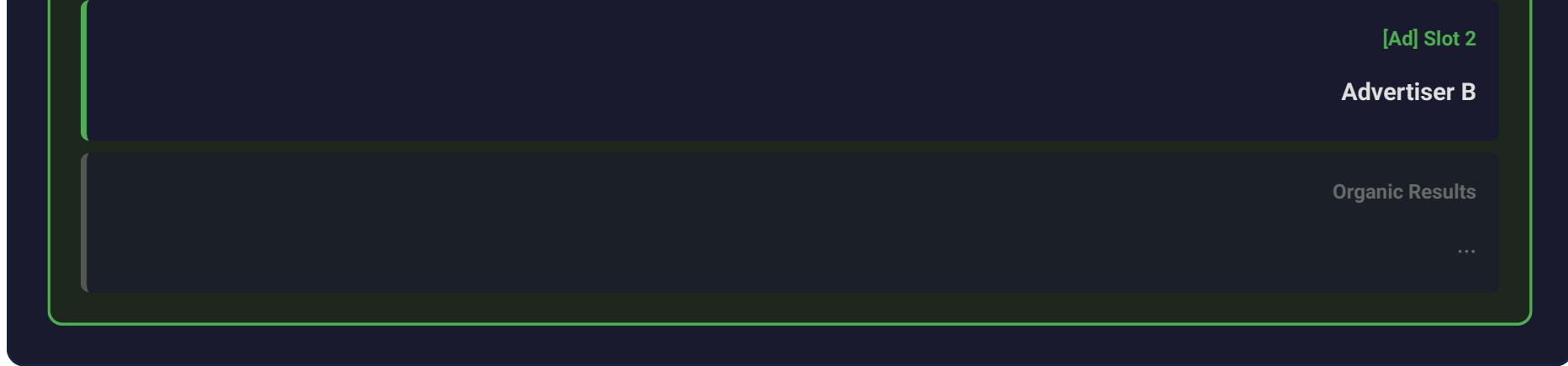
در یک حراج استاندارد و کلاسیک (**First-Price Auction**)، برنده دقیقاً همان مبلغی را می‌پردازد که در پاکت پیشنهاد داده است. این مدل در محیط وب باعث می‌شود تبلیغ‌دهندگان دائماً قیمت خود را تغییر دهند (پدیده **Bid Shading**) تا فقط یک سنت بیشتر از نفر دوم بپردازند. این نوسانات، ثبات سیستم درآمدی موتور جستجو را به شدت مختل می‌کند.

برای حل این مشکل، موتورهای جستجو از مدل ریاضی **GSP** استفاده می‌کنند. در مدل **GSP**، تبلیغ‌دهنده در صورت برنده شدن یک جایگاه، مبلغ پیشنهادی خود را پرداخت نمی‌کند؛ بلکه دقیقاً معادل مبلغ پیشنهادی نفر بعد از خود (نفر پایین‌تر) را پرداخت می‌نماید. این مکانیزم باعث می‌شود بهترین استراتژی برای کاربران، اعلام ارزش واقعی (**Truthful Bidding**) باشد.

نکته آموزشی: در این مسئله پارامتر «نمره کیفیت» (**Quality Score**) ذکر نشده است. بنابراین به صورت پیش‌فرض فرض می‌کنیم نمره کیفیت تمام تبلیغ‌دهندگان با یکدیگر برابر است (معادل 1) و تنها فاکتور تعیین‌کننده، مبلغ **Bid** است.

بخش 3: نمودار شماتیک ماشین محاسباتی حراج





بخش 4: شرح گام به گام محاسبات ریاضی

محاسبه نتایج حراج طی سه گام منطقی زیر انجام می شود:

1. مرتب سازی پیشنهادهای و تخصیص جایگاه (Allocation)

ابتدا تمامی پیشنهادهای را به صورت نزولی مرتب می کنیم تا برندگان مشخص شوند. از آنجا که تنها 2 جایگاه (Slot) تبلیغاتی در صفحه نتایج جستجو وجود دارد، دو پیشنهاد برتر برنده مزایده خواهند بود.

رتبه 1: کاربر A (با پیشنهاد 3.0 دلار) → برنده جایگاه اول

رتبه 2: کاربر B (با پیشنهاد 2.5 دلار) → برنده جایگاه دوم

رتبه 3: کاربر C (با پیشنهاد 1.0 دلار) + حذف از حراج (عدم نمایش تبلیغ در این مزایده)

2. محاسبه قیمت پرداختی برای جایگاه اول

طبق قوانین حراج قیمت دوم، فرمول پرداخت برای کاربری که در رتبه i قرار دارد به صورت تابع زیر تعریف می شود:

$$P_i = b_{i+1}$$

که در آن b_{i+1} پیشنهاد نفر بعدی است. کاربر A در جایگاه اول قرار دارد ($i=1$). بنابراین او باید دقیقاً معادل پیشنهاد نفر دوم (کاربر B) پول پرداخت کند.

مبلغ پرداختی کاربر A به ازای هر کلیک = 2.5 دلار

3. محاسبه قیمت پرداختی برای جایگاه دوم

کاربر B در جایگاه دوم قرار دارد ($i=2$). بنابراین او باید معادل پیشنهاد نفر بعدی خود، یعنی نفر سوم (کاربر C) پول پرداخت کند؛ حتی با وجود اینکه کاربر C برنده هیچ جایگاهی در صفحه نتایج نشده است، اما حضور او در مزایده به عنوان کف رقابت عمل می کند.

مبلغ پرداختی کاربر B به ازای هر کلیک = 1.0 دلار

بخش 5: جدول خلاصه نتایج و وضعیت نهایی حراج

برای درک بهتر، خروجی نهایی حراج به شکل زیر در سیستم ثبت و به مازول صورت حساب (Billing Module) ارسال می گردد:

تبلیغ کننده	پیشنهاد اولیه (Max Bid)	جایگاه تخصیص یافته	مبلغ نهایی پرداختی (Actual CPC)
کاربر A	\$3.00	اول	\$2.50
کاربر B	\$2.50	دوم	\$1.00
کاربر C	\$1.00	برنده نشد (Lost)	\$0.00

بخش 6: جمع بندی

سیستم **GSP** در معماری پلتفرم های تبلیغاتی موتورهای جستجو، یک مکانیزم تدافعی قدرتمند در برابر نوسانات مخرب قیمت گذاری است. این الگوریتم تضمین می کند که هیچ تبلیغ کننده ای بابت دریافت یک کلیک، هزینه ای فراتر از مرز رقابتی نفر بعد از خود پرداخت ننماید. همان طور که در محاسبات ریاضی نشان داده شد، کاربر **A** با وجود ثبت سقف پیشنهادی **3** دلار، به واسطه ساختار حمایتی این مدل، تنها **2.5** دلار (معادل پیشنهاد نفر دوم) پرداخت می کند. در تئوری بازی ها، الگوریتم **GSP** به دلیل همگرایی با مکانیزم های حراج **Vickrey-Clarke-Groves (VCG)**، اکوسیستم مزایده را به سمت یک تعادل پایدار (تعادل نش) هدایت می کند که در آن، استراتژی غالب و بهینه برای هر تبلیغ دهنده، صرفاً اعلام ارزش واقعی است.

علاوه بر این، در سیستم های رتبه بندی مدرن (توسعه یافته پس از سال **2005**)، موتور جستجو صرفاً به یک حراج مالی محض اتکا نمی کند، بلکه با ادغام پارامتر «نمره کیفیت» (**Quality Score**)، ساختار پرداخت را به صورت هوشمندانه ای تعمیم داده است. در این معماری تکامل یافته، رتبه بندی بر اساس حاصل ضرب $Bid \times QS$ استخراج می شود و فرمول قیمت گذاری پرداختی به شکل زیر ارتقا می یابد:

$$P_i = \frac{Ad_Rank_{i+1}}{QS_i} + 0.01$$

این معادله به وضوح نشان می دهد که الگوریتم، ارتباط معنایی و کیفیت محتوای تبلیغ را پاداش می دهد؛ به طوری که یک تبلیغ دهنده با ارتقای نمره کیفیت سایت خود (QS_i)، می تواند با ثابت ماندن شدت رقابت، مبلغ پرداختی واقعی خویش را به طور چشمگیری کاهش داده و نرخ بازگشت سرمایه (**ROI**) خود را بهینه سازد.

پاسخنامه مسئله 19.17: محاسبه بازگشت سرمایه تبلیغات (Ad ROI Scenario)

بخش 1: تحلیل صورت مسئله

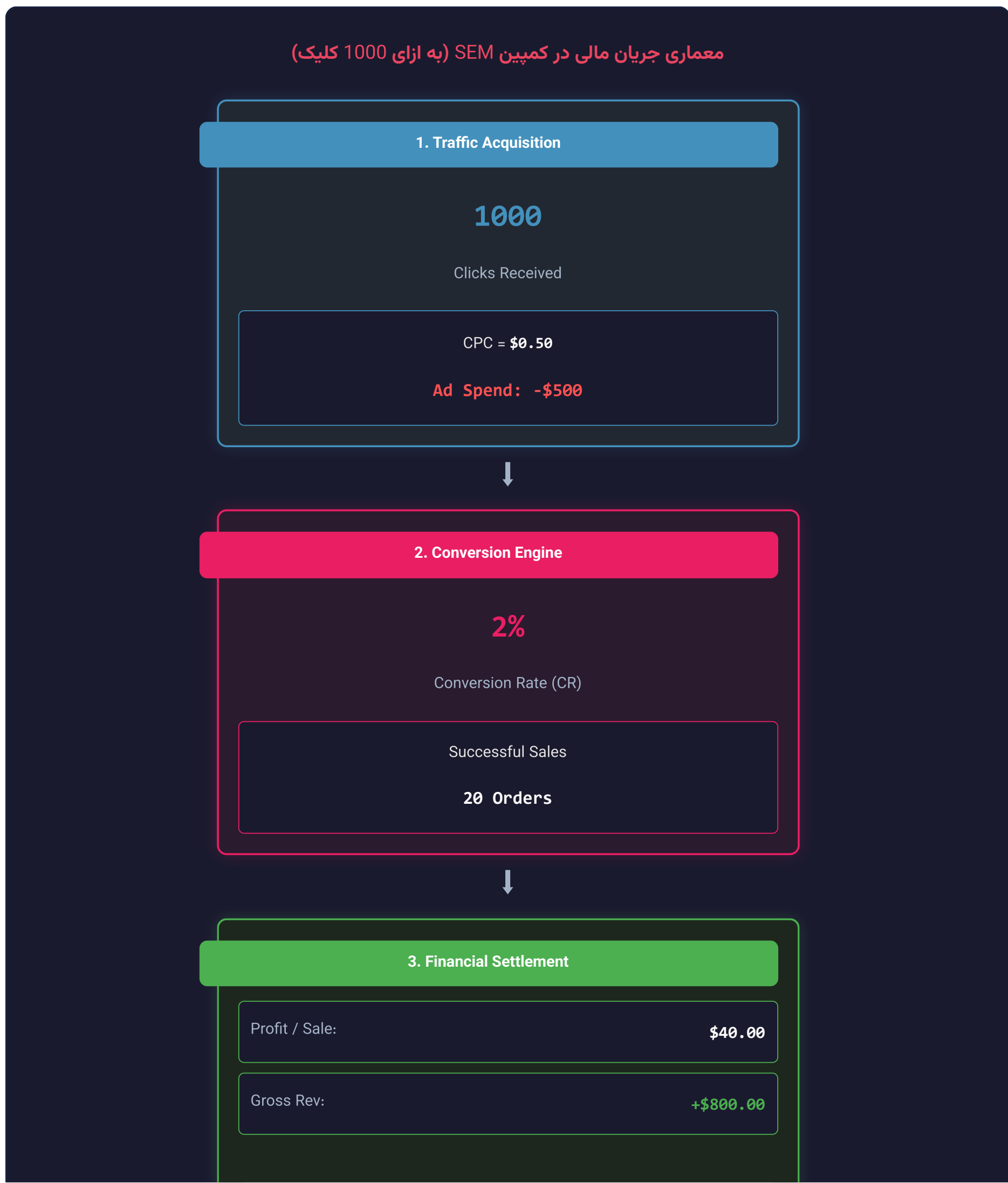
هدف از این سوال، ارزیابی درک دانشجو از اقتصاد بازاریابی موتورهای جستجو (**SEM - Search Engine Marketing**) و تبدیل شاخص های عملکردی کلیدی (**KPIs**) به اعداد مالی واقعی در حسابداری کسب و کار است. در این مسئله، دانشجو باید بتواند متغیرهای هزینه (**CPC**)، نرخ موفقیت تبدیل (**Conversion Rate**) و حاشیه سود خالص را در یک سناریوی عملیاتی ترکیب کرده و سودآور بودن یا نبودن یک کمپین کلیک را به صورت کمی و قطعی اثبات کند.

بخش 2: مبانی و پیش‌نیازهای اقتصاد بازاریابی

در تبلیغات کلیک، صرفِ دریافت کلیک به معنای موفقیت تجاری نیست. بسیاری از کاربرانی که روی آگهی کلیک می‌کنند، به دلایل مختلف (مانند تجربه کاربری ضعیف سایت یا قیمت بالا) بدون انجام خرید، سایت را ترک خواهند کرد (پدیده **Bounce**). برای تحلیل مالی دقیق یک کمپین، باید تفاوت بین «درآمد ناخالص» (**Gross Revenue**) و «هزینه تبلیغات» (**Ad Spend**) را به دقت محاسبه کنیم.

مفهوم کلیدی دیگری که در اینجا پنهان است، **CPA (Cost Per Acquisition)** یا «هزینه جذب یک مشتری موفق» است. منطق بنیادین اقتصاد دیجیتال می‌گوید: اگر هزینه جذب یک مشتری کمتر از سودی باشد که آن مشتری ایجاد می‌کند، کمپین قابلیت مقیاس‌پذیری (**Scalability**) و سودآوری دارد.

بخش 3: نمودار شماتیک پیشرفته از قیف تبدیل و جریان مالی



Net Profit (ROI)

+\$300.00

بخش 4: شرح گام به گام محاسبات ریاضی

برای حل این مسئله، محاسبات را به صورت دقیق بر روی مقیاس ارائه شده در صورت سوال (1000 کلیک) پیاده سازی می کنیم:

گام اول: محاسبه هزینه کل تبلیغات (Total Cost)

فروشگاه به ازای هر کلیک 0.5 دلار پرداخت می کند. بنابراین هزینه تمام شده برای دریافت 1000 کلیک برابر است با پولی که مستقیماً به موتور جستجو پرداخت شده است:

$$Total_Cost = Clicks \times CPC = 1000 \times 0.5 = 500 \text{ USD}$$

گام دوم: محاسبه تعداد خریدهای موفق (Conversions)

نرخ تبدیل 2% به این معناست که از هر 100 نفر بازدیدکننده ورودی، تنها 2 نفر فرآیند خرید را تکمیل می کنند. تعداد خریداران از بین کل ترافیک برابر است با:

$$Conversions = Clicks \times CR = 1000 \times 0.02 = 20 \text{ Orders}$$

گام سوم: محاسبه درآمد کل (Gross Revenue)

هر خرید موفق، 40 دلار سود خالص (بدون در نظر گرفتن هزینه خود تبلیغ) برای فروشگاه به همراه دارد. بنابراین درآمد ناخالص حاصل از این 20 خرید برابر است با:

$$Total_Revenue = Conversions \times Profit_Per_Item = 20 \times 40 = 800 \text{ USD}$$

گام چهارم: محاسبه سود یا زیان خالص نهایی (Net Profit / ROI)

برای یافتن سود نهایی کمپین و ارزیابی موفقیت آن، باید پولی که در گام اول به موتور جستجو پرداخت کرده ایم را از درآمد حاصل از فروش کسر کنیم:

$$Net_Profit = Total_Revenue - Total_Cost = 800 - 500 = 300 \text{ USD}$$

پاسخ صریح به سوالات مسئله:

1. آیا این کمپین تبلیغاتی سودآور است؟ بله، کاملاً سودآور است (بازگشت سرمایه مثبت).

بخش 5: دیدگاه تحلیلی جایگزین (معماری مبتنی بر CPA)

برای درک عمیق‌تر و ارزیابی مقیاس‌پذیری کمپین، مهندسان بازاریابی معمولاً این مسئله را از طریق واحد «هزینه جذب مشتری» (CPA) حل می‌کنند:

برای اینکه فروشگاه یک خرید موفق داشته باشد، با توجه به نرخ تبدیل 2٪، باید به طور میانگین 50 کلیک دریافت کند ($50 = 100 / 2$).

$$CPA = \frac{1}{CR} \times CPC = 50 \times 0.5 = 25 \text{ USD}$$

از آنجایی که هزینه جذب مشتری (25 دلار) به وضوح کمتر از سودی است که آن مشتری برای سیستم تولید می‌کند (40 دلار)، فروشگاه روی هر تک‌فروش 15 دلار حاشیه سود امن دارد. برای 20 فروش (حاصل از 1000 کلیک)، سود کل همان 300 دلار (15×20) خواهد بود.

بخش 6: جمع‌بندی

اقتصاد پلتفرم‌های جستجو و بازاریابی دیجیتال بر پایه رقابتی بی‌رحمانه و بهینه‌سازی پیوسته بنا شده است. در این زیست‌بوم، موفقیت تنها با جذب کلیک تضمین نمی‌گردد، بلکه نیازمند هم‌گرایی دقیق میان کاهش هزینه‌های جذب (CPC) و ارتقای نرخ تبدیل (CR) است. کیفیت صفحه فرود سایت (Landing Page Experience) به عنوان حلقه اتصال این دو شاخص، نقشی حیاتی در تعیین سودآوری ایفا می‌کند. از منظر حسابداری مدیریتی و تئوری‌های پیشرفته ارزیابی کمپین، تحلیل نقطه سر به سر (Breakeven Point) ابزاری استراتژیک برای سنجش پایداری مالی است. کسب‌وکارها پیش از ورود به مزایده جستجو، باید «آستانه تحمل نرخ تبدیل» خود را محاسبه کنند. این فرمول به شرح زیر تعریف می‌گردد:

$$CR_{Breakeven} = \frac{CPC}{Profit_per_Sale}$$

با جایگذاری مقادیر سناریوی فعلی در این معادله:

$$CR_{Breakeven} = \frac{0.5}{40} = 0.0125 \text{ (1.25\%)}$$

این آستانه نشان می‌دهد که نرخ تبدیل تا چه حد می‌تواند افت کند پیش از آنکه کمپین وارد فاز زیان‌دهی مطلق شود. محاسبه دقیق این آستانه نشان می‌دهد که اگر کیفیت تجربه کاربری به افت نرخ تبدیل از 2٪ به زیر 1.25٪ (مثلاً 1٪) منجر گردد، کمپین علیرغم تولید ترافیک بالا، با زیان قطعی مواجه خواهد شد (هزینه 500 دلار در برابر درآمد 400 دلار).

این بینش ریاضی و درک تلورانسِ ریسک، هسته مرکزی تصمیم‌گیری در الگوریتم‌های قیمت‌گذاری خودکار (Automated Bidding) و مقیاس‌پذیری کمپین‌های عملکردمحور (Performance Marketing) را تشکیل می‌دهد.

پاسخنامه مسئله 19.18: ابهام در کوئری کاربر و طراحی SERP

(Query Ambiguity & SERP Diversification)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، بررسی یکی از چالش‌برانگیزترین مفاهیم در سیستم‌های بازیابی اطلاعات، یعنی پدیده «چندمعنایی» (Polysemy) و رفع ابهام کلمات است. زمانی که کاربر یک کوئری کوتاه و کاملاً مبهم مانند "Jaguar" را جستجو می‌کند، موتور جستجو با پدیده «فقدان نیت» (Lack of Intent) روبه‌رو می‌شود. اگر موتور جستجو کل صفحه اول نتایج (SERP) را تنها به یک معنی غالب (مثلاً فقط خودروی لوکس جگوار) اختصاص دهد، کاربرانی که به دنبال اطلاعات حیوان (گره‌سان) یا سیستم‌عامل (Mac OS X Jaguar) بوده‌اند، با نرخ نارضایتی 100% مواجه شده و سایت را فوراً ترک می‌کنند (پدیده Abandonment). دانشجو باید راهکاری مهندسی برای به حداقل رساندن این ریسک در لایه رابط کاربری (UI) و منطق الگوریتم ارائه دهد.

بخش 2: مبانی و پیش‌نیازهای معماری سیستم

در معماری بازیابی اطلاعات، اصلی کلیدی به نام تنوع‌بخشی به نتایج جستجو (Search Result Diversification) وجود دارد. این اصل ریاضی بیان می‌کند: «زمانی که نیت کاربر قطعی و شفاف است، الگوریتم باید روی حداکثر کردن ارتباط (Relevance) تمرکز کند؛ اما زمانی که نیت مبهم است، الگوریتم باید استراتژی خود را تغییر داده و روی حداکثر کردن تنوع (Diversity) متمرکز شود.»

هدف از تنوع‌بخشی این است که سبد نتایج به گونه‌ای چیده شود که حداقل یک نتیجه کاملاً مرتبط برای هر کدام از معانی محتمل، در نیمه بالایی صفحه (ناحیه Above the fold) که بدون اسکرول دیده می‌شود وجود داشته باشد. این رویکرد، ریسک ناامیدی کاربر را توزیع و مینیمم می‌کند.

بخش 3: نمودار شماتیک معماری SERP برای کوئری‌های مبهم

طراحی رابط کاربری و لایه‌بندی الگوریتمی برای کوئری "Jaguar"

Jaguar



لایه 1: ابهام‌زدایی نیت کاربر (Disambiguation)

Jaguar (Car) 🚗

Jaguar (Animal) 🐆

لایه 2: نتایج ارگانیک تنوع یافته (Interleaved Results)

Rank 1 & 2 (Highest Prob: 60%)

Jaguar Cars - Official Website & Models

Explore the latest Jaguar luxury vehicles, SUVs, and sports cars...

Rank 3 & 4 (Prob: 30%)

Jaguar - Wikipedia, Animal Facts

The jaguar (Panthera onca) is a large cat species native to the Americas...

Rank 5 (Prob: 10%)

Mac OS X Jaguar 10.2 - Apple History

Mac OS X Jaguar (version 10.2) is the third major release of macOS...

لایه 3: گراف دانش تفکیک شده (Knowledge Panel)

آیا منظور شما ... است؟

شرکت خودروسازی 

تاسیس: 1922

گره‌سان (حیوان) 

زیستگاه: قاره آمریکا

سیستم عامل مک 

انتشار: 2002

بخش 4: پیشنهاد طراحی صفحه نتایج (SERP Design Layers)

برای مدیریت این ابهام و کاهش ریسک نارضایتی، صفحه نتایج باید از بالا به پایین با معماری زیر طراحی و رندر شود:

1. لایه اول: نوار ابهام زدایی تعاملی (Disambiguation Chips)

در بالاترین بخش صفحه و بلافاصله زیر کادر جستجو، سیستم باید گزینه‌های صریحی برای شفاف سازی نیت کاربر ارائه دهد. این بخش شامل مجموعه‌ای از دکمه‌ها (**Chips**) با عناوین شفاف است: «جستجو برای: خودروی جگوار»، «جستجو برای: حیوان جگوار»، «جستجو برای: Mac OS X».

منطق الگوریتمی: با کلیک کاربر روی هر یک از این گزینه‌ها، یک کوئری جدید و کاملاً شفاف (**Refined Query**) به سرور ارسال می‌شود و صفحه با نتایج قطعی و تخصصی بازخوانی می‌گردد.

2. لایه دوم: تنوع بخشی در نتایج ارگانیک (Interleaved Organic Results)

ستون اصلی نتایج نباید توسط یک مفهوم قبضه شود. موتور جستجو باید احتمالات آماری (**Query Probability**) را بر اساس فرکانس جستجوهای عمومی و تاریخی محاسبه کرده و نتایج را به صورت بلوک های متناوب (**Interleaving**) در هم بیامیزد:

رتبه 1 تا 3 (محتمل ترین نیت): نتایج مربوط به خودروی جگوار (زیرا معمولاً کوئری های تجاری و صنعتی بیشترین حجم ترافیک جستجو را به خود اختصاص می دهند).

رتبه 4 تا 5 (نیت دوم): نتایج مربوط به موجودیت گربه سان (حیوان جگوار) با لینک هایی به منابع معتبر نظیر ویکی پدیا یا سایت های حیات وحش.

رتبه 6 (نیت سوم): لینک های آرشیوی مربوط به تاریخچه سیستم عامل اپل.

3. لایه سوم: بلوک رسانه ای دسته بندی شده (Categorized Image Carousel)

از آنجا که تصاویر با سرعت بسیار بیشتری نسبت به متن توسط مغز انسان پردازش می شوند، یک نوار افقی از تصاویر در میانه صفحه قرار می گیرد. این بخش به جای نمایش تصادفی و درهم ریخته عکس ها، دارای تب های مجزا (**Tabs**) است: تب «خودرو»، تب «حیوان». این کار به کاربر اجازه می دهد نیت خود را به صورت بصری، سریع و بدون نیاز به خواندن عناوین طولانی انتخاب کند.

4. لایه چهارم: پنل گراف دانش تفکیک شده (Partitioned Knowledge Panel)

در ستون کناری صفحه (در نمایش دسکتاپ)، جایی که معمولاً گراف دانش یکپارچه (**Knowledge Graph**) نمایش داده می شود، سیستم باید ابهام موجود در گراف را به رسمیت بشناسد. به جای نمایش یک باکس اطلاعاتی یکپارچه که ممکن است گمراه کننده باشد، پنل به بخش هایی با عنوان: «آیا منظور شما ... است؟» (**Did you mean**) تقسیم می شود و خلاصه کوتاهی از هر سه موجودیت (**Entity**) را با یک آیکون یا تصویر بندانگشتی نمایش می دهد تا کاربر بتواند فوراً روی موجودیت مورد نظر خود سوییچ کند.

بخش 5: جمع بندی

در معماری سیستم های بازیابی اطلاعات، پدیده چندمعنایی (**Polysemy**) به عنوان یکی از بزرگترین موانع در درک دقیق نیت کاربر (**Intent Modeling**) شناخته می شود. در غیاب سیگنال های شخصی سازی (**Personalization**) یا تاریخچه جستجو، موتور جستجو هرگز نمی تواند با قطعیت ذهن کاربر را بخواند. در چنین شرایطی، تنوع بخشی (**Diversity**) به عنوان مستحکم ترین سپر محافظتی الگوریتم وارد عمل می شود تا از ریزش ناگهانی کاربران و کاهش نرخ رضایت جلوگیری نماید.

برای پیاده سازی این تنوع استراتژیک در لایه بک اند، معماری موتورهای جستجو از الگوریتم های پیشرفته ی جریمه تکرار نظیر **MMR (Maximal Marginal Relevance)** بهره می برند. وظیفه اصلی این توابع ریاضی، برقراری تعادل میان دو قطب متضاد یعنی «ارتباط معنایی» و «تازگی مفاهیم» است. الگوریتم **MMR** نتیجه ای را به صفحه اضافه می کند که بیشترین تفاوت معنایی (**Novelty**) را با نتایج قبلاً اضافه شده داشته باشد تا طیف معنایی پوشش داده شده در یک صفحه **SERP** به حداکثر ممکن برسد. فرمول ریاضیاتی این الگوریتم به شرح زیر بسط می یابد:

$$MMR \stackrel{\text{def}}{=} \arg \max_{D_i \in R} \left[\lambda \cdot Sim_1(D_i, Q) - (1 - \lambda) \max_{D_j \in S} Sim_2(D_i, D_j) \right]$$

در این مدل تحلیلی، پارامتر Sim_1 میزان همبستگی سند با کوئری اولیه (سود الگوریتم) و پارامتر Sim_2 میزان شباهت سند جدید با اسناد موجود در صفحه (هزینه تکراری بودن) را می‌سنجد. ضریب λ (لامبدا) در این ساختار، یک پارامتر تنظیم‌گر است که موتور جستجو به صورت پویا و بر اساس «درجه ابهام کوئری»، مقدار آن را کالیبره می‌کند تا سبد نتایج نهایی بهینه‌ترین ترکیب ممکن از دقت و گستردگی را به کاربر ارائه دهد.

پاسخنامه مسئله 19.19: تصحیح املائی بر اساس لاگ جستجو (Query Log based Spell Correction)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، درک گذار پارادایم از «سیستم‌های مبتنی بر دیکشنری ایستا و قواعد سخت‌افزاری» به «سیستم‌های مبتنی بر خرد جمعی» (**Wisdom of the Crowds**) در مهندسی جستجوی مدرن است. یک دیکشنری سنتی (هرچند عظیم) هرگز نمی‌تواند تمام نام‌های خاص، برندهای جدید، غلط‌های رایج کیبوردی، کلمات خارج از واژگان (**OOV - Out of Vocabulary**) یا اصطلاحات عامیانه را به صورت درنگ‌در لحظه پوشش دهد. در این مسئله، دانشجو باید طراحی یک سیستم یادگیری ماشین بدون نظارت (**Unsupervised Machine Learning**) را مدل‌سازی کند که به طور کاملاً خودکار و ارگانیک، از اشتباهات و اصلاحات روزمره میلیون‌ها کاربر برای کشف قوانین املائی جدید استفاده می‌کند.

بخش 2: مبانی و پیش‌نیازهای معماری داده

در معماری بازیابی اطلاعات، هر زمان که کاربری وارد موتور جستجو می‌شود، سیستم یک «نشست» (**Session**) برای او ایجاد می‌کند. در طول این نشست، تمام کوئری‌های وارد شده، زمان دقیق بین جستجوها (**Timestamps**)، و لینک‌هایی که روی آن‌ها کلیک شده است (تحت عنوان **Click-through Data** یا بازخورد ضمنی)، در فایل‌های توزیع‌شده و عظیمی به نام لاگ جستجو (**Search Logs**) ثبت می‌گردند.

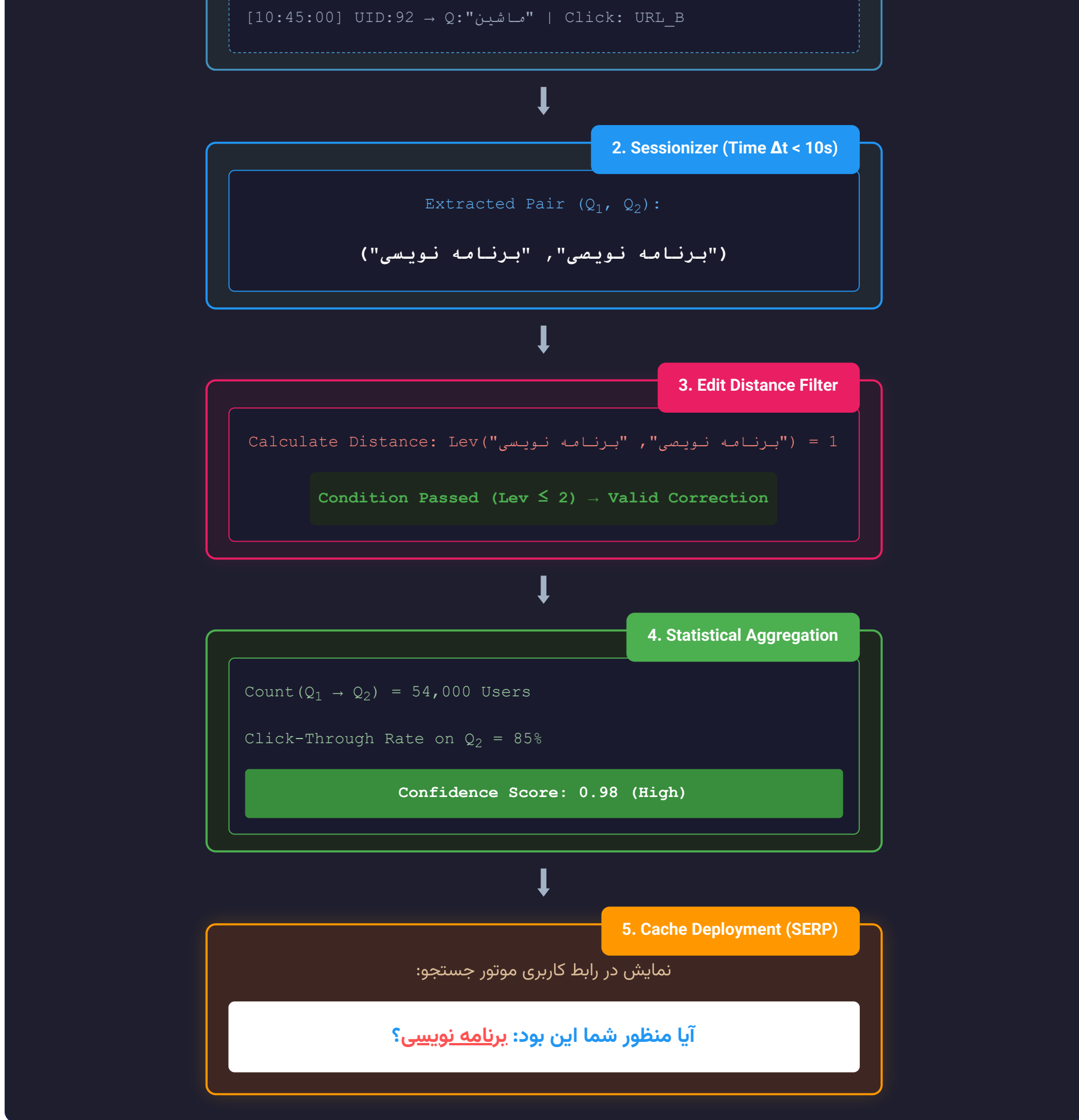
ایده کلیدی و نبوغ‌آمیز این معماری این است: اگر کاربران متعددی یک عبارت غلط را جستجو کنند، با نتایج نامربوط مواجه شوند (عدم کلیک)، و بلافاصله املائی آن را تغییر داده و سپس روی نتایج کوئری جدید کلیک کنند، سیستم می‌تواند این الگو را با استفاده از احتمالات آماری به عنوان یک «قانون تصحیح املائی قدرتمند» استخراج و تثبیت کند.

بخش 3: نمودار شماتیک خط‌لوله پردازش داده (Log Processing Pipeline)

معماری استخراج خودکار الگوهای املائی از خرد جمعی (Wisdom of the Crowds)

1. Raw Search Logs

```
[10:01:05] UID:88 → Q:"برنامه نویسی" | Click: Null  
[10:01:12] UID:88 → Q:"برنامه نویسی" | Click: URL_A
```



بخش 4: شرح گام به گام و سناریوی الگوریتمی

برای تبدیل داده‌های خام لاگ به یک سیستم هوشمند «آیا منظور شما این بود؟» (Did you mean)، مهندسان داده یک پایپ‌لاین پردازشی (Data Pipeline) با معماری 5 مرحله‌ای زیر پیاده‌سازی می‌کنند:

مرحله اول: استخراج نشست‌های کاربری (Session Extraction)

الگوریتم، لاگ‌های خام روزانه را بر اساس شناسه کاربر (User ID) و بازه زمانی، به نشست‌های کوتاه‌مدت (مثلاً پنجره‌های 5 دقیقه‌ای) تفکیک می‌کند. هدف این است که جستجوهای متوالی یک کاربر که به احتمال زیاد حول یک موضوع و هدف واحد هستند، ایزوله و گروه‌بندی شوند.

مرحله دوم: شناسایی جفت‌های اصلاحی (Reformulation Pairs)

در درون هر نشست، الگوریتم به دنبال جفت کوئری‌هایی (Q₁, Q₂) می‌گردد که کاربر Q₁ را جستجو کرده اما روی هیچ نتیجه‌ای کلیک نکرده است (نشانه نارضایتی و پدیده **Abandonment**)، و در فاصله زمانی بسیار کوتاهی (مثلاً 10 ثانیه بعد)، کوئری Q₂ را وارد کرده است.

مثال: کاربر جستجو می‌کند: "برنامه نویسی" (Q_1) ← کلیک نمی‌کند ← جستجو می‌کند: "برنامه نویسی" (Q_2).

مرحله سوم: اعمال فیلتر فاصله ویرایشی (Edit Distance Filter)

هر تغییر کوثری لزوماً به معنای غلط املایی نیست (ممکن است کاربر کلاً موضوع را عوض کرده باشد، مثلاً از «خرید ماشین» به «قیمت دلار» که به آن **Concept Drift** می‌گویند). برای اطمینان، الگوریتم فاصله ویرایشی (**Levenshtein Distance**) بین Q_1 و Q_2 را محاسبه می‌کند. اگر این دو رشته تنها در 1 یا 2 حرف با هم تفاوت داشته باشند (عملیات درج، حذف یا جایگزینی محدود)، سیستم تایید می‌کند که این یک تغییر املایی است، نه یک تغییر مفهومی.

مرحله چهارم: اعتبارسنجی آماری و رفتار کلیک (Statistical Validation)

یک بار رخ دادن این الگو کافی نیست (شاید کاربر دومین بار اشتباه تایپ کرده باشد). سیستم این جفت‌ها را در مقیاس میلیون‌ها کاربر تجمیع (**Aggregate**) می‌کند. اگر هزاران کاربر مستقل، الگوی تغییر Q_1 به Q_2 را تکرار کرده باشند، و پس از وارد کردن Q_2 روی لینک‌های نتایج کلیک‌های موفقی داشته باشند (**Click-through confirmation**)، این جفت تایید نهایی می‌شود.

مرحله پنجم: استقرار در سیستم (Deployment)

جفت استخراج‌شده در یک گراف هدایت‌شده و پرسرعت در حافظه پنهان (**Cache / Redis**) سیستم جستجو ذخیره می‌شود. از این پس، هر کاربری که Q_1 را تایپ کند، موتور جستجو به صورت خودکار پیام «آیا منظور شما برنامه نویسی بود؟» را رندر کرده و ترافیک را به سمت املای صحیح هدایت می‌کند.

بخش 5: جمع‌بندی

عبور از معماری متکی بر دیکشنری‌های ایستا به سمت استفاده از داده‌های لاگ جستجو، نشان‌دهنده بلوغ سیستم‌های بازیابی اطلاعات در بهره‌گیری از هوش مصنوعی و یادگیری ماشین بدون نظارت است. مزیت مهندسی بی‌نظیر این رویکرد، **پویایی و تطبیق‌پذیری مطلق (Adaptability)** آن است. زمانی که یک برند تکنولوژی جدید یا اصطلاحی عامیانه به سرعت ترند می‌شود و در هیچ لغت‌نامه رسمی وجود ندارد، موتور جستجو نیازی به به‌روزرسانی دستی توسط مهندسان زبان‌شناس ندارد. با تحلیل لاگ‌ها مبتنی بر یادگیری آنلاین (**Online Learning**)، سیستم در کسری از روز متوجه املای صحیح کلمه جدید می‌شود؛ زیرا خرد جمعی کاربران، خودش را به صورت خودکار تصحیح نموده و الگوریتم این بازخورد ضمنی را جذب می‌کند.

در معماری پردازش زبان طبیعی (**NLP**)، این فرآیند تکاملی بر پایه مدل احتمالاتی **کانال نویز (Noisy Channel Model)** بنا شده است. موتور جستجو برای یافتن کلمه صحیح متعارف (c) با توجه به کلمه غلط تایپ شده (w)، معادله بیز زیر را بهینه‌سازی و حداکثر می‌کند:

$$c^* = \arg \max_c P(w|c) \times P(c)$$

در این مدل تحلیلی، پارامتر $P(c)$ نمایانگر احتمال مدل زبانی (میزان رایج بودن کلمه c در گراف وب) و پارامتر $P(w|c)$

نشان‌دهنده احتمال مدل خطا (احتمال اینکه کاربر قصد تایپ c را داشته اما به دلیل خطای کیبوردی، اشتباهاً w را تولید کرده باشد) است. لاگ‌های جستجو دقیقاً همان پایگاه داده‌ی طلایی و پویایی هستند که ماتریس احتمالات خطای $P(w|c)$ را در مقیاس پتابایت کالبره و برای هر زبان و منطقه جغرافیایی به صورت لحظه‌ای تنظیم می‌کنند.

پاسخنامه مسئله 19.20: تخمین اندازه وب با روش صید و بازصید (Capture-Recapture Method)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، پیاده‌سازی یک مدل آماری پیشرفته برای حل چالشی است که در مسئله 19.11 (غیرممکن بودن شمارش مستقیم صفحات وب و فضای نامتناهی آدرس‌ها) بررسی کردیم. در این مسئله، دانشجو باید از تکنیک «صید و بازصید» (Capture-Recapture) که در اصل توسط زیست‌شناسان برای تخمین جمعیت حیوانات در حال حرکت در یک جنگل ابداع شده است برای تخمین حجم کل اینترنت (جامعه آماری ناشناخته) با استفاده از میزان همپوشانی (Overlap) ایندکس دو موتور جستجوی مستقل استفاده کند.

بخش 2: مبانی، پیش‌نیازها و روابط ریاضی

اساس این محاسبه بر پایه قضیه احتمالات و «فرض استقلال» (Independence Assumption) استوار است. این فرض بنیادین بیان می‌کند که احتمال ایندکس شدن یک صفحه خاص توسط موتور Y ، کاملاً مستقل از احتمال ایندکس شدن آن توسط موتور X است (خزنده‌های دو شرکت استراتژی‌های تصادفی متفاوتی دارند).

بر اساس این فرض، نسبت «صفحات مشترک به کل صفحات موتور Y »، باید دقیقاً برابر با نسبت «کل صفحات موتور X به کل فضای وب» باشد. فرمول پایه (تخمین‌گر لینکلن-پترسن Lincoln-Petersen Estimator) در این مدل احتمالاتی به شکل زیر استخراج و تعریف می‌شود:

$$N = \frac{N_1 \times N_2}{|N_1 \cap N_2|}$$

شرح متغیرهای معادله فوق:

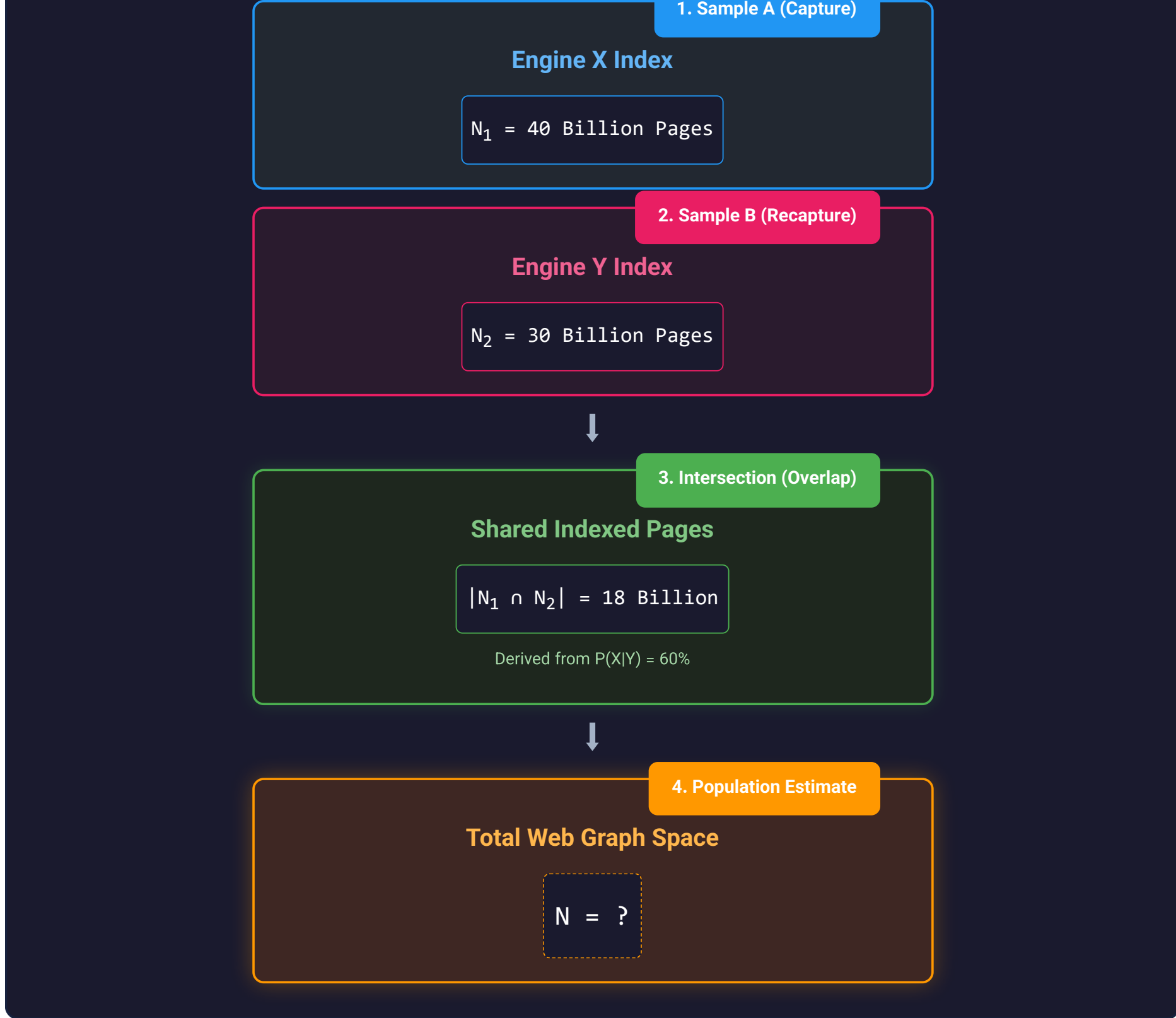
N : حجم کل صفحات وب (مقدار مجهول و هدف نهایی).

N_1 : اندازه ایندکس موتور جستجوی اول (نمونه‌گیری اول).

N_2 : اندازه ایندکس موتور جستجوی دوم (نمونه‌گیری دوم).

$|N_1 \cap N_2|$: تعداد صفحات مشترک کشف‌شده بین هر دو موتور (بازصید یا همپوشانی).

بخش 3: نمودار شماتیک معماری فضای احتمالات



بخش 4: شرح گام به گام و حل ریاضی مسئله

برای رسیدن به تخمین نهایی، اطلاعات داده شده را به صورت مرحله به مرحله استخراج کرده و در فرمول جایگذاری می‌کنیم:

گام اول: استخراج متغیرهای معلوم (Given Data)

بر اساس اطلاعات ورودی صورت مسئله داریم:

$$N_1 = 40 \times 10^9 \text{ (اندازه موتور جستجوی اول)}$$

$$N_2 = 30 \times 10^9 \text{ (اندازه موتور جستجوی دوم)}$$

$$P(X|Y) = 0.60 \text{ (احتمال شرطی حضور یک صفحه از Y در X)}$$

گام دوم: محاسبه تعداد صفحات مشترک (Intersection Size)

می‌دانیم که 60% از صفحاتی که در ایندکس موتور Y وجود دارند، توسط خزنده‌های موتور X نیز ایندکس شده‌اند. بنابراین

حجم دقیق صفحات مشترک (همپوشانی) از حاصل ضرب این احتمال در حجم موتور دوم به دست می‌آید:

$$|N_1 \cap N_2| = P(X|Y) \times N_2 = 0.60 \times (30 \times 10^9) = 18 \times 10^9$$

بنابراین، این دو موتور جستجو دقیقاً 18 میلیارد صفحه مشترک در پایگاه داده خود دارند.

حال مقادیر به دست آمده را در فرمول اصلی **Capture-Recapture** قرار می‌دهیم تا برآوردگر خطی، اندازه جامعه را محاسبه کند:

$$N = \frac{(40 \times 10^9) \times (30 \times 10^9)}{18 \times 10^9} = \frac{1200 \times 10^{18}}{18 \times 10^9} \approx 66.67 \times 10^9$$

پاسخ نهایی:

بر اساس این مدل آماری و با اتکا به فرض استقلال، تخمین اندازه کل صفحات وب برابر با **66.67 میلیارد صفحه** است.

بخش 5: جمع‌بندی

تخمین‌گر لینکلن-پترسن ابزاری قدرتمند برای مدل‌سازی جوامع مجهول در مهندسی وب است، اما در مقیاس‌های کلان، درک ظرافت‌های پنهان آن حیاتی است. در محیط واقعی اینترنت، محاسبه‌ی فوق دارای یک خطای سیستماتیک و ذاتی (**Systematic Bias**) است؛ چرا که فرض «مستقل بودن عملکرد موتورهای جستجو» کاملاً صادق نیست. موتورهای جستجوی تجاری (مانند خزنده‌های گوگل و بینگ)، علیرغم تفاوت در الگوریتم‌های جزئی، استراتژی‌های زمان‌بندی خزش (**Crawl Scheduling**) بسیار مشابهی دارند.

این سیستم‌ها همگی تمایل شدیدی به کشف و ایندکس کردن سریع صفحاتی دارند که دارای **PageRank** بالاتر، اعتبار پیوندی بیشتر و نرخ به‌روزرسانی سریع‌تر (**Update Frequency**) هستند (یعنی صفحاتی که در هسته قویاً همبند مرکزی یا **SCC** گراف وب قرار دارند). این **سوگیری الگوریتمی (Algorithmic Bias)** سبب می‌گردد که درصد همپوشانی و اشتراک واقعی میان دیتابیس این موتورها، بسیار بیشتر از حالت توزیع تصادفی مستقل باشد. از منظر ریاضی، تورم کاذب در مخرج کسر $(|N_1 \cap N_2|)$ منجر به کوچک‌تر شدن حاصل نهایی می‌گردد. در نتیجه، در معماری بازیابی اطلاعات، عدد به‌دست‌آمده از این تکنیک آماری هرگز به عنوان اندازه قطعی وب در نظر گرفته نمی‌شود، بلکه به صورت علمی به عنوان «**مطلق کران پایین (Absolute Lower Bound)**» از حجم گراف وب تفسیر می‌گردد.

پاسخنامه مسئله 19.21: چالش تولید کوئری تصادفی و سوگیری در نمونه‌گیری (Query Sampling Bias)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، بررسی یکی از مهم‌ترین چالش‌های عملی و نظری در ارزیابی سیستم‌های بازیابی اطلاعات (**IR**) است. همان‌طور که در مسئله قبل (تخمین صید و بازصید) دیدیم، محاسبه اندازه وب نیازمند یک نمونه‌گیری تصادفی و کاملاً یکنواخت (**Uniform Random Sampling**) از صفحات است. از آنجایی که موتورهای جستجو به عنوان یک «جعبه سیاه» (**Black Box**) عمل کرده و لیست تمام صفحات خود را به ما نمی‌دهند، تنها راه تعامل پژوهشگران با آن‌ها ارسال عبارات جستجو (**Queries**) از طریق رابط‌های عمومی است. دانشجو باید به صورت الگوریتمی درک کند که چرا تولید کلمات تصادفی و ارسال آن‌ها به موتور

جستجو، به هیچوجه یک صفحه تصادفی و عادلانه را به ما برنمیگرداند و توزیع آماری نتایج را به شدت دچار انحراف (**Bias**) می‌کند.

بخش 2: مبانی، پیش‌نیازها و تئوری نمونه‌گیری

در علم آمار، یک نمونه‌گیری تصادفی یکنواخت به این معناست که اگر اندازه ایندکس موتور جستجو برابر با **N** صفحه باشد، شانس (احتمال) انتخاب شدن هر صفحه خاص (D_i) دقیقاً برابر با $\frac{1}{N}$ باشد:

$$P(D_i) = \frac{1}{N}$$

در سال **1998**، محققانی به نام‌های بهارات و برودر (**Bharat & Broder**) برای تخمین اندازه وب تلاش کردند با انتخاب تصادفی کلمات از یک دیکشنری جامع وب و ارسال آن‌ها (به صورت ترکیب عطفی یا فصلی) به موتورهای جستجو، صفحات وب را نمونه‌گیری کنند. اما آن‌ها به سرعت متوجه شدند که معماری و مکانیزم خود موتور جستجو، مانع از تحقق این احتمال مساوی برای تمام صفحات می‌شود و نمونه‌های استخراج‌شده از نظر ساختاری مخدوش هستند.

بخش 3: نمودار شماتیک فیلترهای سوگیری (Sampling Bias Pipeline)

معماری انحراف آماری در نمونه‌گیری مبتنی بر کوئری (Query-based Sampling)



تولید کوئری تصادفی برای استخراج صفحات، ذاتاً دارای دو سوگیری مهلک و بنیادین است که شانس برابر صفحات را در نظریه احتمالات از بین می‌برد:

1. سوگیری واژگانی و طول سند (Lexical & Length Bias)

مکانیزم فنی: وقتی شما کلماتی را به صورت کاملاً تصادفی از یک دیکشنری استخراج کرده و جستجو می‌کنید، صفحاتی که محتوای متنی بسیار طولانی‌تری دارند، به طور طبیعی کلمات متنوع‌تری را در خود جای داده‌اند و دامنه لغات گسترده‌تری را پوشش می‌دهند.

تأثیر بر نمونه‌گیری: از نظر احتمالات، یک مقاله تحلیلی 10,000 کلمه‌ای شانس بسیار بیشتری برای داشتن آن «کلمات تصادفی» نسبت به یک صفحه کوتاه 100 کلمه‌ای دارد. بنابراین، صفحات طولانی (Long Documents) بیش از حد واقعی خود در نمونه‌ها ظاهر می‌شوند (Over-represented). در مقابل، صفحات کوتاه، صفحاتی که صرفاً حاوی عکس یا ویدیو بدون متن هستند، یا صفحاتی با زبان‌های محلی و اصطلاحات غیررسمی (خارج از دیکشنری پایه)، عملاً شانس برخورد و استخراجشان به صفر میل می‌کند.

2. سوگیری رتبه‌بندی و محدودیت نمایش (Ranking & Visibility Bias)

مکانیزم فنی: موتورهای جستجو یک پایگاه داده تصادفی نیستند. حتی اگر کوئری تولیدی ما کاملاً تصادفی باشد، موتور جستجو صفحات منطبق را به صورت تصادفی بر نمی‌گرداند؛ بلکه آن‌ها را بر اساس الگوریتم‌های پیچیده رتبه‌بندی خود (مانند میزان ارتباط متنی، PageRank و رفتار کاربر) سورت و مرتب می‌کند.

تأثیر بر نمونه‌گیری (پدیده Truncation): اگر کوئری تصادفی ما با 3,000,000 صفحه مطابقت داشته باشد، موتور جستجو به دلایل مربوط به بار پردازشی سرورها، معمولاً فقط 1000 نتیجه برتر را باز می‌گرداند و بقیه صفحات را رسماً پنهان (Truncate) می‌کند. این بدان معناست که سایت‌های معتبر، قدیمی و دارای لینک‌های ورودی زیاد (High Authority Pages) همیشه شانس بالایی برای دیده شدن و ورود به نمونه در این 1000 نتیجه اول دارند. در مقابل، سایت‌های نوپا یا صفحات حاشیه‌ای وب، با وجود اینکه کلمات کوئری را در خود دارند، به دلیل رتبه پایین، هرگز به دست پژوهشگر نمی‌رسند. در نتیجه، نمونه استخراج‌شده پر از سایت‌های مشهور (مثل ویکی‌پدیا یا سایت‌های خبری) خواهد بود.

بخش 5: جمع‌بندی

در دنیای پردازش و بازیابی اطلاعات، «محتوا» و «ارزش‌گذاری شبکه‌ای» در هسته الگوریتم موتورهای جستجو به صورت ذاتی با یکدیگر گره خورده‌اند. شما نمی‌توانید از طریق مکانیزم کوئری، یک سیستم جستجو را مجبور کنید که منطق ارزش‌گذاری و سورتینگ خود را نادیده بگیرد. نمونه‌گیری از طریق کوئری تصادفی، به جای ارائه یک تصویر آماری یکنواخت از «کل پهنه وب»، صرفاً تصویری فیلترشده از «بخش‌های طولانی، غنی و معتبر وب» را منعکس می‌کند که باعث می‌شود تخمین نهایی ما از اندازه و توزیع وب به شدت دچار خطای سیستماتیک گردد.

برای فرار از این بن‌بست کوئری‌محور، محققان مدرن علوم کامپیوتر جهت نمونه‌گیری از وب، به استراتژی‌های مبتنی بر توپولوژی گراف روی آورده‌اند. یکی از قدرتمندترین رویکردها، اجرای یک گشت تصادفی (Random Walk) طولانی بر روی ساختار لینک‌های وب است. با این حال، گشت‌های تصادفی نیز ذاتاً به سمت صفحاتی با درجه ورودی بالا (High In-degree) متمایل می‌شوند. برای جبران و خنثی‌سازی این سوگیری ساختاری، از الگوریتم پیشرفته مارکوف چین

مونت کارلو نظیر متروپولیس-هیستینگز (Metropolis-Hastings Algorithm) استفاده می‌گردد. این الگوریتم با تعدیل پویا و دستکاری احتمالات گذار (Transition Probabilities) در هر گره، شانس ورود گشت به صفحات پرلینک را کاهش می‌دهد. در نهایت و پس از طی کردن میلیون‌ها گام در زنجیره مارکوف، توزیع ثابت (Stationary Distribution) گشت با توزیع کاملاً یکنواخت (Uniform Distribution) همگرا شده و نمونه‌ای عادلانه و فارغ از سوگیری حاصل می‌گردد.

پاسخنامه مسئله 19.22: محاسبه شباهت ژاکارد (Jaccard Similarity Coefficient)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، ارزیابی توانایی دانشجو در محاسبه و درک یکی از مهم‌ترین و پرکاربردترین معیارهای شباهت در علم داده (Data Science) و بازیابی اطلاعات (IR) است. ضریب شباهت ژاکارد برای سنجش میزان همپوشانی (Overlap) بین دو مجموعه (Set) استفاده می‌شود. در سیستم‌های جستجو و خزنده‌های وب، این مجموعه‌ها معمولاً شامل کلمات استخراج‌شده از یک سند در مدل کیسه کلمات (Bag of Words) یا شینگل‌های (Shingles) مجزای آن هستند.

بخش 2: مبانی و پیش‌نیازهای ریاضی

در نظریه مجموعه‌ها در ریاضیات، ضریب ژاکارد به صورت نسبت «اندازه اشتراک دو مجموعه» به «اندازه اجتماع آن دو مجموعه» تعریف و مدل‌سازی می‌شود. فرمول ریاضی ضریب ژاکارد برای دو مجموعه دلخواه **A** و **B** به شکل زیر استخراج می‌گردد:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

تفسیر متغیرهای این معادله:

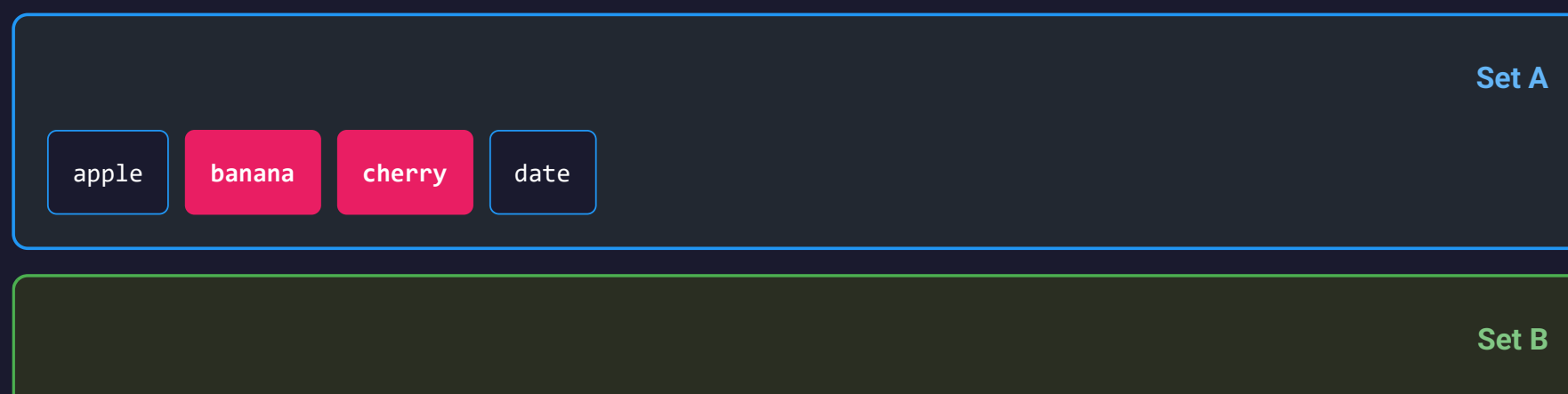
|A ∩ B|: تعداد اعضای مشترک بین دو مجموعه (اندازه تقاطع).

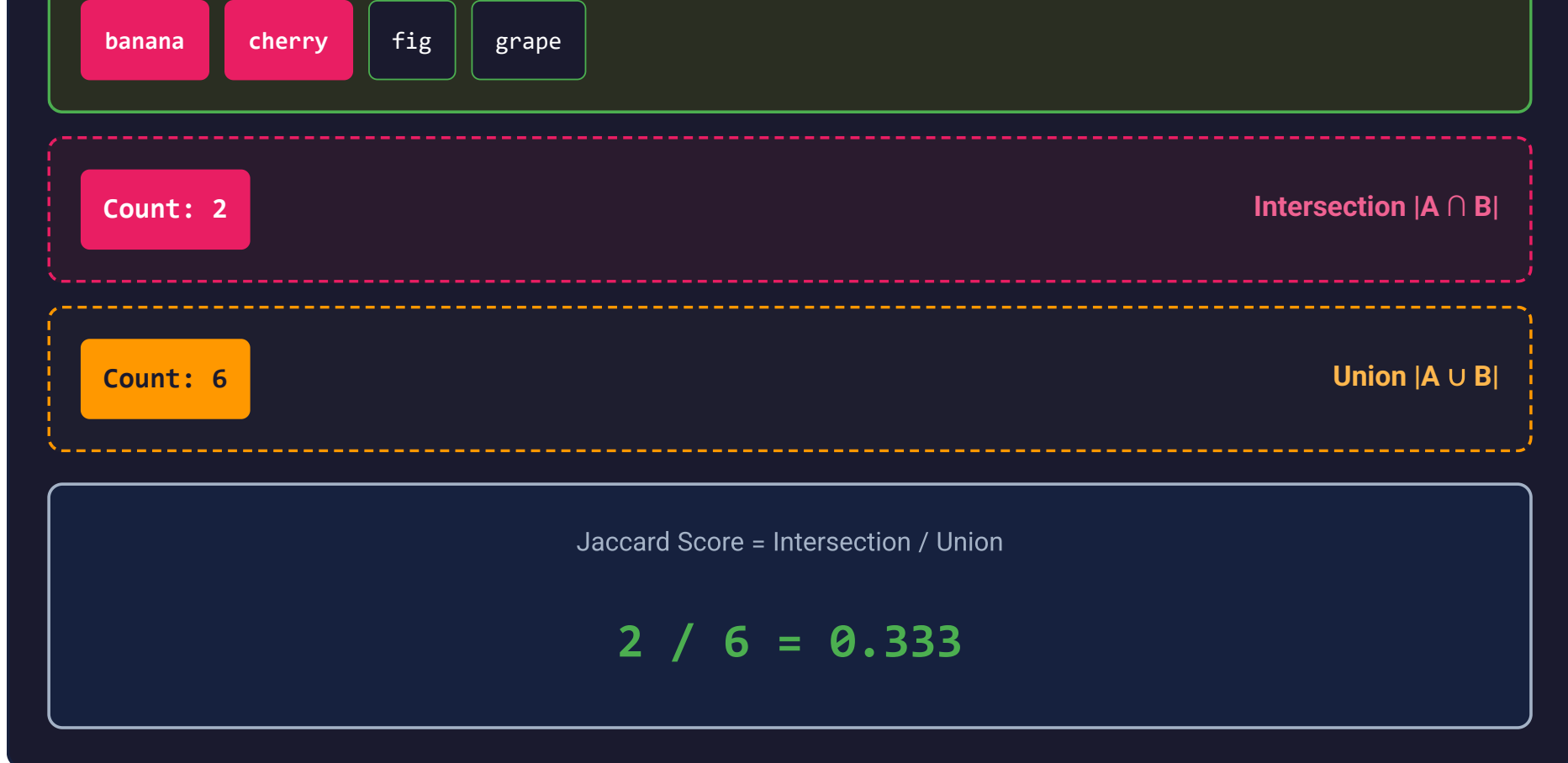
|A ∪ B|: کل اعضای منحصر به فرد موجود در هر دو مجموعه بدون احتساب موارد تکراری (اندازه اجتماع).

دامنه خروجی: مقدار این فرمول همواره یک عدد پیوسته در بازه **[0, 1]** است؛ که **0** نشان‌دهنده استقلال کامل (عدم شباهت) و **1** نشان‌دهنده تطابق صددرصدی و یکسان بودن دو مجموعه است.

بخش 3: نمودار شماتیک پیشرفته و مدل‌سازی بصری مجموعه‌ها

مدل‌سازی ساختاری تقاطع و اجتماع در محاسبه ژاکارد





بخش 4: شرح گام به گام و حل ریاضی مسئله

مجموعه‌های داده شده در صورت مسئله به شکل زیر هستند:

$$A = \{\text{apple, banana, cherry, date}\}$$

$$B = \{\text{banana, cherry, fig, grape}\}$$

برای محاسبه ضریب شباهت بین این دو سند، مراحل ریاضی زیر را به ترتیب طی می‌کنیم:

گام اول: استخراج اعضای مشترک (Intersection)

ابتدا باید کلماتی که در هر دو سند تکرار شده‌اند را پیدا کنیم (تشکیل مجموعه $A \cap B$). با مقایسه دو مجموعه A و B می‌بینیم که توکن‌های **banana** و **cherry** دقیقاً در هر دو موجودیت حضور دارند.

$$A \cap B = \{\text{banana, cherry}\}$$

$$|A \cap B| = 2$$

بنابراین اندازه این مجموعه (صورت کسر) برابر با **2** است.

گام دوم: استخراج کل اعضای منحصر به فرد (Union)

در این مرحله، تمام کلمات هر دو سند را در یک ظرف مجموعه جدید ($A \cup B$) جمع می‌کنیم. بر اساس قوانین جبر مجموعه‌ها، اعضای مشترک تنها یک بار شمرده می‌شوند:

$$A \cup B = \{\text{apple, banana, cherry, date, fig, grape}\}$$

$$|A \cup B| = 6$$

بنابراین اندازه این مجموعه (مخرج کسر) برابر با 6 است.

گام سوم: جایگذاری در فرمول و محاسبه نهایی

حال مقادیر اسکالر به دست آمده را در فرمول ژاکارد قرار می‌دهیم:

$$J(A, B) = \frac{2}{6} = \frac{1}{3} \approx 0.333$$

پاسخ نهایی:

ضریب شباهت ژاکارد بین این دو سند، از نظر ریاضیاتی برابر با **0.333** (یا **33.3%** درصد شباهت محتوایی) است.

بخش 5: جمع‌بندی

معیار شباهت ژاکارد به دلیل استقلال از طول مطلق اسناد و مقیاس‌پذیری نرمالیزه‌شده در بازه صفر تا یک، گزینه‌ای استاندارد در موتورهای جستجو برای حل مسئله شناسایی صفحات تقریباً تکراری (Near-Duplicate Detection) محسوب می‌گردد. با این وجود، این معیار دارای یک محدودیت بنیادین است: ژاکارد به فراوانی کلمات (Term Frequency) در مدل **Bag-of-Words** بی‌تفاوت است. به عنوان مثال، اگر توکن *banana* در سند **A** صد بار تکرار شده باشد و در سند **B** تنها یک بار، از نظر محاسبات مجموعه‌ای ژاکارد (که بر حذف تکرارهای هم‌نام استوار است) هیچ تفاوتی در خروجی ایجاد نمی‌گردد. به همین دلیل، برای تحلیل‌های معنایی عمیق‌تر (**Semantics**)، از معیارهای برداری نظیر شباهت کسینوسی (**Cosine Similarity**) استفاده می‌شود.

علاوه بر این، در معماری کلان داده (**Big Data**)، محاسبه ضریب ژاکارد به صورت جفت‌به‌جفت برای میلیاردها صفحه وب، نیازمند پیچیدگی زمانی از مرتبه $O(N^2)$ است که در عمل غیرقابل اجراست. برای رفع این تنگنای محاسباتی، مهندسان جستجو از الگوریتم درهم‌سازی حساس به محلیت (**MinHash (Locality-Sensitive Hashing)**) بهره می‌برند. این ساختار ریاضیاتی اثبات می‌کند که احتمال برابر بودن کوچک‌ترین هش (**MinHash**) دو مجموعه، تحت توابع هش جایگشتی تصادفی، دقیقاً با ضریب شباهت ژاکارد آن دو مجموعه برابر است:

$$Pr(h_{min}(A) = h_{min}(B)) = J(A, B)$$

این ظرافت در طراحی الگوریتم، به سیستم‌های بازیابی اطلاعات اجازه می‌دهد تا ضریب ژاکارد مجموعه‌های توکنی بسیار عظیم را در زمان خطی تقریب زده و صفحات محتوایی موازی را پیش از ورود به فرآیند نمایه‌سازی، با دقت بالا شناسایی و حذف (**Deduplicate**) نمایند.

پاسخنامه مسئله 19.23: تاثیر اندازه شینگل در کشف کپی‌کاری

(Effect of Shingle Size in Near-Duplicate Detection)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، بررسی رفتار و حساسیت یک «فرایارامتر» (**Hyperparameter**) حیاتی به نام طول شینگل (k) در الگوریتم‌های کشف صفحات تقریباً تکراری (**Near-Duplicate Detection**) است. دانشجو باید بتواند موازنه (**Trade-off**) بین حفظ ساختار نحوی جملات و میزان انعطاف‌پذیری الگوریتم در برابر ویرایش‌های جزئی را به لحاظ ریاضی تحلیل کرده و خطاهای آماری ناشی از انتخاب مقادیر افراطی برای پارامتر k را استدلال نماید.

بخش 2: مبانی و پیش‌نیازهای ریاضی الگوریتم

در تکنیک **k-shingling**، سند متنی (با طول N کلمه) به مجموعه‌ای از توالی‌های پیوسته به طول k کلمه تقسیم می‌شود. پارامتر k در واقع طول «پنجره لغزان» (**Sliding Window**) را مشخص می‌کند. از نظر ریاضی، یک سند با N توکن، دقیقاً دارای $N - k + 1$ شینگل خواهد بود. انتخاب k تعیین می‌کند که الگوریتم تا چه حد باید به ترتیب کلمات و بافتار معنایی (**Context**) اهمیت بدهد.

قانون کلی در این معماری این است: هرچه k بزرگتر باشد، احتمال برخورد تصادفی کاهش یافته و شروط یکسان در نظر گرفتن دو سند سخت‌گیرانه‌تر (دقیق‌تر اما شکننده‌تر) می‌شود.

بخش 3: نمودار شماتیک طیف حساسیت فرایارامتر (Hyperparameter Sensitivity)

موازنه خطای آماری بر اساس طول شینگل (Trade-off Analysis)

مثبت کاذب بالا (Unigrams)	k=1
اسناد کاملاً بی‌ربط صرفاً به دلیل داشتن کلمات ربط یا کلمات توقف مشترک، به اشتباه کپی یکدیگر شناخته می‌شوند.	
تعادل اسناد کوتاه (Short Texts)	k=5
طول مناسب برای استخراج بافتار در بیکره‌های متنی کوچکتر (نظیر طبقه‌بندی ایمیل‌ها یا اخبار کوتاه).	
نقطه بهینه وب (Web Standard)	k=10
استاندارد طلایی موتورهای جستجو برای مقیاس‌های عظیم. برخورد تصادفی شینگل‌ها در این طول عملاً غیرممکن است.	
منفی کاذب بالا (High Brittleness)	k≥15
شکنندگی شدید الگوریتم؛ تغییر تنها یک کلمه توسط اسپمر، باعث تخریب تعداد زیادی شینگل و پنهان ماندن کپی‌کاری می‌شود.	

بخش 4: شرح تفصیلی و تحلیل مقادیر مختلف

برای درک دقیق تاثیر این پارامتر، سه سناریوی مطرح شده در سوال را از دیدگاه بازیابی اطلاعات به دقت بررسی می‌کنیم:

1. عواقب انتخاب $k = 1$ (Unigrams)

از دست رفتن ساختار معنایی: اگر طول شینگل برابر با 1 در نظر گرفته شود، سند عملاً به مجموعه‌ای از کلمات مجزا (Bag of Words) تبدیل می‌شود. در این حالت، هیچ اطلاعاتی از ترتیب، توالی و دستور زبان حفظ نمی‌شود. به عنوان مثال، دو جمله "علی حسن را زد" و "حسن علی را زد" از نظر این الگوریتم 100% مشابه (کپی یکدیگر) خواهند بود.

مشکل اصلی (مثبت کاذب - False Positive): کلمات پرکاربرد توقف (Stop Words) مانند "is"، "the"، و "and" در تمام متون انگلیسی با فرکانس بالا وجود دارند. اگر $k = 1$ باشد، دو مقاله کاملاً نامرتب (مثلاً یکی درباره نجوم و دیگری درباره آشپزی) صرفاً به دلیل داشتن کلمات ربط و حروف اضافه مشترک، ضریب شباهت ژاکارد بسیار بالایی دریافت می‌کنند و سیستم آن‌ها را به اشتباه به عنوان «صفحات کپی» علامت‌گذاری می‌کند.

2. عواقب انتخاب k بسیار بزرگ (مثلاً $k = 15$)

شکنندگی شدید: (Brittleness) در این حالت، هر شینگل یک عبارت طولانی 15 کلمه‌ای است. هدف اصلی این الگوریتم پیدا کردن صفحاتی است که دقیقاً کپی نیستند، بلکه تقریباً کپی هستند (مثلاً یک پاراگراف یا چند کلمه در آن‌ها توسط اسپر تغییر کرده است). از نظر ریاضی، تغییر، حذف یا اضافه کردن تنها یک کلمه در متن، باعث از بین رفتن و تخریب دقیقاً k شینگل مجاور می‌شود.

مشکل اصلی (منفی کاذب - False Negative): اگر $k = 15$ باشد، یک ویرایش بسیار کوچک (مثلاً اصلاح یک غلط املایی یا تغییر تاریخ)، 15 شینگل از مجموعه شینگل‌های سند را به طور کامل تغییر می‌دهد. این افت شدید و نامتناسب در اعضای مشترک دو مجموعه، باعث کاهش چشمگیر ضریب ژاکارد می‌شود. در نتیجه، الگوریتم فریب خورده و دو سندی را که 99% شبیه هم هستند، به عنوان اسناد کاملاً متفاوت دسته‌بندی می‌کند.

3. اندازه استاندارد و بهینه برای متون انگلیسی

انتخاب مقدار مناسب برای k ، وابستگی مستقیمی به «حجم جامعه آماری» (Corpus Size) دارد تا احتمال تولید تصادفی یک شینگل یکسان در دو سند کاملاً نامرتب به حداقل ممکن میل کند:

برای اسناد کوتاه: در پیکره‌های متنی کوچکتر (مانند طبقه‌بندی ایمیل‌ها یا متون خبری کوتاه)، معمولاً مقدار $k=5$ پیشنهاد می‌شود. توالی 5 کلمه‌ای به اندازه کافی طولانی است که معنا را حفظ کند و به ندرت به صورت تصادفی در دو متن کاملاً بی‌ربط تکرار می‌شود.

برای مقیاس عظیم وب: در محیط وب (با میلیاردها صفحه مستند و مقالات طولانی)، احتمال تصادف (Collision) بسیار بالاتر است. در این شرایط بحرانی، مهندسان سیستم‌های جستجو معمولاً از مقادیر $k=9$ یا $k=10$ استفاده می‌کنند تا از نظر آماری مطمئن شوند که اشتراک شینگل‌ها قطعاً ناشی از کپی‌برداری است، نه یک تصادف آماری رایج.

بخش 5: جمع‌بندی

طراحی الگوریتم Shingling یک بازی پیچیده با نقطه تعادل است. کوچک بودن k باعث می‌شود اسناد کاملاً نامرتب به اشتباه به عنوان کپی شناخته شوند (بروز خطای سیستماتیک False Positive) و بزرگ بودن k باعث می‌شود اسنادی که توسط اسپرها با زیرکی و تغییرات اندک ویرایش شده‌اند، از دید سیستم پنهان بمانند (بروز خطای False Negative). در معماری سیستم‌های بازایی اطلاعات مقیاس‌پذیر، استخراج شینگل‌ها صرفاً نیمی از مسیر مهندسی است. انتخاب پارامتر k مستقیماً روی حساسیت توابع درهم‌ساز (Hash Functions) در معماری پیشرفته MinHash تاثیر می‌گذارد. به

عنوان مثال، اگر $k = 10$ تنظیم شود، هر رشته‌ی **10** کلمه‌ای به یک هش **64** بیتی فشرده تبدیل می‌گردد. موتورهای جستجو برای سرعت بخشیدن به این فرآیند سنگین، از تکنیک **درهم‌سازی غلتان (Rolling Hashing)**، نظیر الگوریتم بهینه‌ی **Rabin-Karp** بهره می‌برند. در این چارچوب ریاضیاتی، برای محاسبه هش شینگل بعدی، نیازی به پردازش مجدد کل k کلمه نیست؛ بلکه تنها کلمه‌ی خارج‌شده از پنجره کسر شده و کلمه‌ی جدید واردشده به پنجره، به فرمول درهم‌ساز اضافه می‌شود. این معماری تضمین می‌کند که پیچیدگی زمانی فرآیند استخراج شینگل‌ها، کاملاً فارغ از بزرگی پارامتر k ، همواره از مرتبه خطی $O(N)$ باقی بماند و پردازش در سطح کلان داده عملیاتی شود.

پاسخنامه مسئله 19.24: تله‌های عنکبوتی (Spider Traps & Infinite URL Spaces)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، بررسی یکی از مهم‌ترین چالش‌های عملیاتی و امنیتی در معماری خزنده‌های وب (**Web Crawlers / Spiders**) است. در محیط پویای وب، خزنده‌ها به گونه‌ای برنامه‌ریزی شده‌اند که هر لینک جدیدی را کشف کرده و به صف پردازش خود (**URL Frontier**) اضافه کنند. دانشجو باید به لحاظ الگوریتمی درک کند که چگونه تولید خودکار و داینامیک لینک‌ها می‌تواند از این مکانیزم پایه (پیمایش گراف) سوءاستفاده کرده و منابع پردازشی، حافظه موقت و پهنای باند موتور جستجو را در یک حلقه بی‌نهایت (**Infinite Loop**) به هدر دهد.

بخش 2: مبانی و پیش‌نیازهای معماری خزش

همان‌طور که در مسائل مربوط به وب داینامیک بررسی شد، URLها امروزه دیگر صرفاً آدرس یک فایل ثابت و استاتیک روی هارد دیسک سرور نیستند، بلکه پارامترهایی برای کوئری زدن به پایگاه داده محسوب می‌شوند (مسیردهی مبتنی بر متغیر). تله‌های عنکبوتی در این فضا می‌توانند به دو شکل ظاهر شوند: **عمدی و مخرب (Malicious)** که توسط اسپمرها برای از کار انداختن خزنده‌ها یا حمله **DoS** طراحی می‌شوند، و یا **سهوی و تصادفی (Accidental)** که به دلیل طراحی ضعیف سیستم‌های مدیریت محتوا (**CMS**) نظیر تقویم‌های بی‌نهایت، فیلترهای فروشگاهی چندلایه یا سشن‌آیدی‌ها (**Session IDs**) در URL ایجاد می‌گردند.

بخش 3: نمودار شماتیک پیشرفته از معماری تله عنکبوتی و مکانیزم‌های دفاعی



Depth Limit Trigger

IF Depth > 10 → DROP

Domain Quota Trigger

IF Pages > 500 → HALT

URL Pattern Filter

Regex Anomaly Detect

بخش 4: شرح گام به گام و کالبدشکافی چالش

برای بررسی دقیق این مسئله، ابتدا مفهوم پایه را تعریف کرده، سپس مکانیزم سقوط در سناریوی تقویم را تحلیل نموده و در نهایت راهکارهای مهندسی معماری جستجو را بسط می‌دهیم:

1. تعریف تله عنکبوتی (Spider Trap)

تله عنکبوتی (یا **Crawler Trap**) به مجموعه‌ای از صفحات وب گفته می‌شود که یک فضای آدرس (**URL Space**) بی‌نهایت یا به شدت متراکم و بزرگ را به صورت پویا تولید می‌کنند. این صفحات به گونه‌ای به یکدیگر لینک شده‌اند که خزنده وب را در یک چرخه بی‌پایان از درخواست‌های متوالی گرفتار می‌کنند و مانع از خروج آن و کشف سایر بخش‌های مفید وبسایت یا اینترنت می‌شوند.

2. نحوه گرفتار شدن خزنده در سناریوی تقویم

وبسایت مذکور یک تقویم داینامیک دارد. مکانیزم به دام افتادن خزنده به صورت متوالی و بازگشتی به شکل زیر رخ می‌دهد:

گام 1: کشف لینک اولیه: خزنده وارد صفحه اصلی وبسایت شده و لینکی با آدرس تقویم (مثلاً

example.com/calendar?month=07&year=2026) را پیدا می‌کند. این آدرس به عنوان یک URL جدید و معتبر

در صف خزش (**Frontier**) ثبت و سپس دانلود می‌شود.

گام 2: تولید لینک‌های جدید در لحظه: در کد HTML صفحه دانلود شده، اسکریپت سمت سرور به صورت خودکار

لینکی برای «ماه بعد» تولید کرده است (یعنی **example.com/calendar?month=08&year=2026**). خزنده این

لینک را استخراج کرده و به عنوان یک مسیر کشف‌نشده به صف اضافه می‌کند.

گام 3: ورود به حلقه بی‌نهایت (Infinite Loop): خزنده روی لینک جدید کلیک می‌کند. صفحه جدید دوباره لینکی برای

ماه بعد (ماه 09) تولید می‌کند. از آنجا که بُعد زمان در برنامه‌نویسی حد یقفی ندارد، سایت می‌تواند این URL‌های کاملاً

یکتا را تا سال 9999 و فراتر از آن تولید کند.

گام 4: انسداد منابع (Resource Exhaustion): خزنده که بر اساس تطابق رشته‌ای (**Exact String Matching**) کار

می‌کند، هر URL را کاملاً یکتا می‌پندارد و متوجه تکراری بودن ماهیت محتوا نمی‌شود. تمام پهنای باند شبکه و حافظه

موقت صف خزنده به دانلود ماه‌های آینده اختصاص می‌یابد و عملاً در این دامنه گیر می‌افتد. این امر همچنین باعث

فشار سنگین بر سرور مقصد شده و قانون ادب خزش (**Politeness Policy**) را نقض می‌کند.

3. محدودیت‌های مهندسی برای جلوگیری از تله

برای جلوگیری از سقوط قطعی خزنده در این سیاه‌چاله‌های اطلاعاتی، طراحان موتور جستجو باید مکانیزم‌های دفاعی ابتکاری

(**Heuristics**) موثری را در کد خزنده تعبیه کنند. سه محدودیت استاندارد در معماری موتورهای جستجو عبارتند از:

محدودیت عمق خزش (Maximum Crawl Depth): خزنده باید فاصله هر صفحه را از صفحه اصلی (یا **Seed URL**)

در گراف محاسبه کند. طراح می‌تواند یک شرط قطعی تعیین کند: «هیچ لینکی که فاصله آن از ریشه بیش از N کلیک

(مثلاً عمق 10) باشد را دنبال نکن.» در سناریوی تقویم، وقتی خزنده به ماه دهم برسد، الگوریتم متوقف شده و به صورت امن از شاخه تقویم خارج می‌شود.

سهمیه تعداد صفحات برای هر دامنه: (Domain Page Quota) سیستم به هر دامنه (Domain / Host) بر اساس اعتبار اولیه آن (مثلاً نمره PageRank) یک سهمیه مشخص اختصاص می‌دهد (مثلاً حداکثر 500 صفحه برای سایت‌های ناشناخته). اگر تقویم شروع به تولید بی‌نهایت صفحه کند، به محض رسیدن به سقف 500 صفحه، خزنده عملیات را **Halt** کرده و به سراغ سایت و دامنه بعدی می‌رود.

نرمال‌سازی و فیلتر الگوهای پارامتریک: (URL Pattern Normalization) سیستم جستجو الگوهای تکراری در پارامترها را شناسایی می‌کند. اگر متوجه شود که متغیرهای **?month=** و **&year=** ساختار صفحه را تغییر بنیادین نمی‌دهند، آن‌ها را از دید خزش فیلتر می‌کند.

بخش 5: جمع‌بندی و نکات

تله‌های عنکبوتی اثبات می‌کنند که در مهندسی وب در مقیاس اینترنت، یک خزنده هرگز نمی‌تواند و نباید «کورکورانه» هر لینکی را دنبال کند. وجود پارامترهای داینامیک در URL (مانند علامت **?** و متغیرهای **GET**) اولین زنگ خطر برای الگوریتم‌های خزش است تا محدودیت‌های پردازشی سخت‌گیرانه‌تری اعمال کنند.

نکته پیشرفته استفاده از یادگیری ماشین برای تشخیص تله:

الگوریتم‌های مدرن موتورهای جستجو برای فرار از تله‌های پیچیده‌تر، صرفاً به شروط ایستا (مثل عمق 10) تکیه نمی‌کنند؛ زیرا یک سایت ارزشمند ممکن است واقعاً ۱۱ سطح عمق داشته باشد. امروزه از مدل‌های طبقه‌بند یادگیری ماشین (**Machine Learning Classifiers**) برای تحلیل گراف ساختاری مسیر (**Path Topology**) استفاده می‌شود. این سیستم‌ها با رصد کردن سرعت رشد یک شاخه از درخت URL و تحلیل میزان تشابه محتوایی صفحاتِ دانلود شده با الگوریتم ژاکارد (اگر محتوای تقویم‌های متوالی بیش از حد شبیه هم باشند)، به صورت پویا (**Dynamically**) تصمیم می‌گیرند که آیا این شاخه یک تله است که باید هرس (**Prune**) شود یا یک مسیر معتبر است.

پاسخنامه مسئله 19.25: معماری خزنده مقاوم در برابر گراف‌های خصمانه

(Resilient Crawler Architecture)

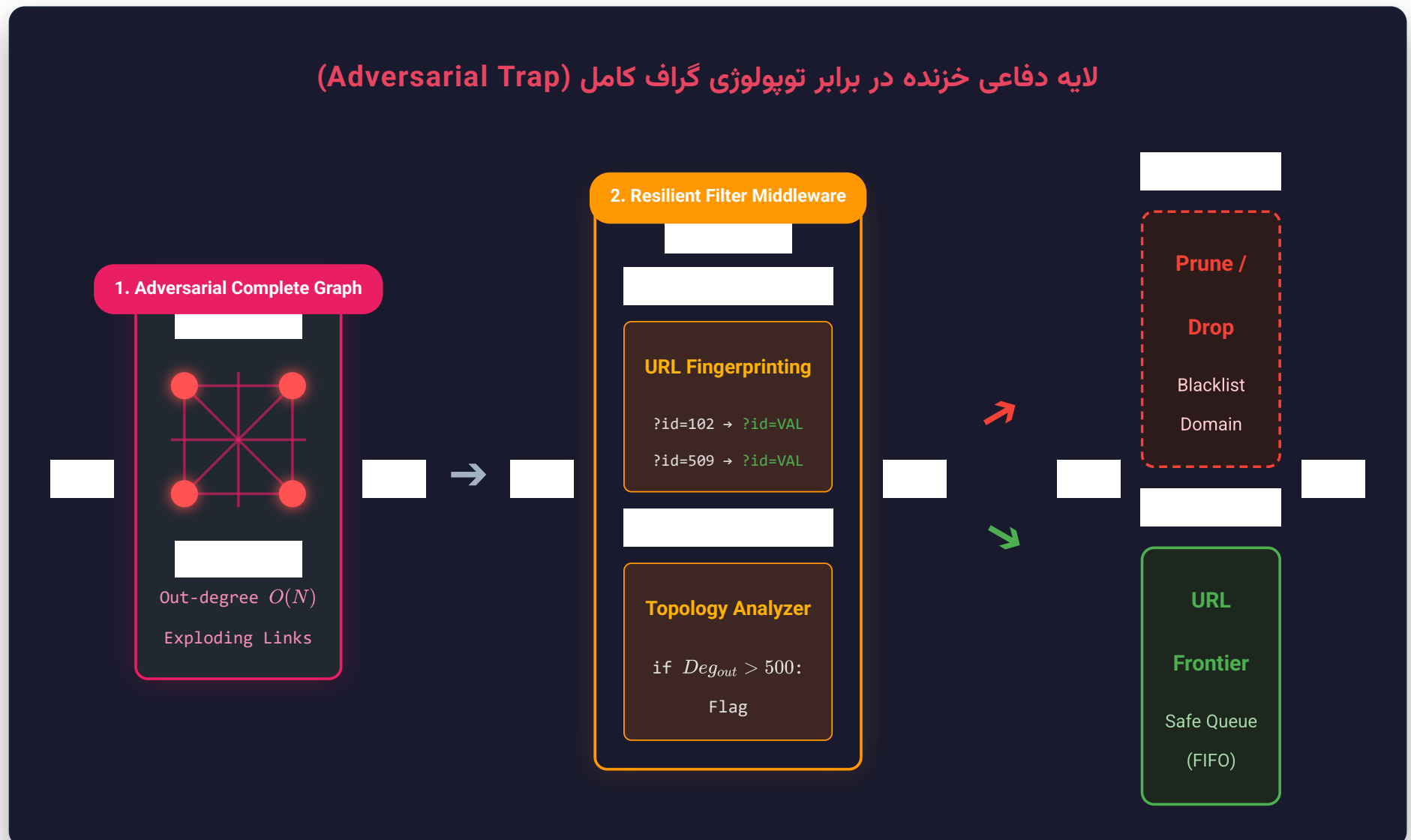
بخش 1: تحلیل صورت مسئله

این مسئله به بررسی پیشرفته‌ترین نوع از حملات «بازیابی خصمانه» (**Adversarial IR**) می‌پردازد. اسپمرها با ایجاد یک گراف کامل (**Complete Graph / Clique**) از صفحات داینامیک، قصد دارند یک حمله محروم‌سازی از سرویس (**DoS - Denial of Service**) علیه خزنده‌های موتور جستجو ترتیب دهند. در یک گراف کامل با N گره، هر صفحه دقیقاً $N - 1$ لینک خروجی به سایر صفحات دارد؛ به این معنا که تعداد کل یال‌ها از مرتبه $O(N^2)$ رشد می‌کند. بنابراین اگر خزنده وارد این دامنه شود، با حجم تصاعدی و بی‌نهایتی از لینک‌های متقاطع روبه‌رو می‌شود که به سرعت حافظه صف (**URL Frontier**) و پهنای باند شبکه را می‌بلعد. هدف ما طراحی یک معماری درنگام (**Real-time**) است که این توپولوژی مخرب را پیش از هدررفت منابع کشف و هرس (**Prune**) کند.

بخش 2: مبانی و پیش‌نیازهای معماری

خزنده‌های سنتی بر اساس یک صف ساده (FIFO یا اولویت‌دار) کار می‌کنند و تنها پس از دانلود کامل صفحه و پردازش درخت DOM، متوجه محتوای آن و توپولوژی لینک‌ها می‌شوند. اما یک خزنده مقاوم (Resilient Crawler) نیازمند یک لایه میانی (Middleware) برای «تحلیل الگوها» پیش از مرحله دانلود فیزیکی است. از آنجایی که اسپمرها برای تولید میلیون‌ها صفحه داینامیک متصل به هم، مجبورند از یک اسکریپت واحد (Template) و پایگاه داده مشترک استفاده کنند، آدرس‌های اینترنتی تولیدشده دارای الگوهای ساختاری و پارامتریک کاملاً مشابهی خواهند بود. کشف این تقارن ساختاری، کلید خنثی‌سازی حمله است.

بخش 3: نمودار شماتیک پیشرفته از معماری فیلتر درنگام (Real-time Pruning Pipeline)



بخش 4: شرح گام‌به‌گام معماری و استراتژی هرس

برای خنثی‌سازی این تله پیچیده، معماری سیستم باید از یک رویکرد دفاعی دوگانه (تحلیل ساختار آدرس + تحلیل توپولوژی محلی) استفاده کند:

1. استخراج الگوی ساختاری آدرس‌ها (URL Fingerprinting)

اسکریپت‌های داینامیک با تغییر مقادیر پارامترها (مانند `?id=102` و `?id=509`) صفحات ظاهراً جدیدی می‌سازند. سیستم باید پیش از اضافه‌کردن لینک به صف رم (RAM Queue)، مقادیر متغیر پارامترها را با عبارات باقاعده (Regex) حذف کرده و یک «اثر انگشت ساختاری» (Structural Pattern) از URL بسازد. به این ترتیب، میلیون‌ها آدرس ظاهراً متفاوت، فوراً به یک الگوی واحد و قابل شمارش تقلیل می‌یابند.

2. پایش تراکم یال‌ها و درجه خروجی (Out-degree Anomaly Detection)

در یک سایت طبیعی، درجه خروجی (تعداد لینک‌های داخل یک صفحه) معمولاً از توزیع قانون توان (Power Law Distribution) پیروی می‌کند و میانگین آن عددی معقول است. در گراف کامل اسپمرها، هر صفحه

جدید حاوی هزاران لینک خروجی به تمام صفحات مشابه دیگر است. سیستم باید میانگین درجه خروجی هر دامنه را به صورت درنگام محاسبه و مانیتور کند.

3. اعمال سیاست هرس درنگام (Real-time Pruning)

زمانی که یک خزنده از طریق لایه **Middleware** متوجه شود که الگوی ساختاری یکسانی از یک دامنه به دفعات غیرطبیعی (مثلاً بیش از **100** بار) تکرار شده و همزمان صفحات این دامنه دارای درجه خروجی به شدت بالایی هستند، سیستم وضعیت قرمز اعلام می‌کند. در این لحظه، تمام لینک‌های موجود در صف که دارای آن اثر انگشت مخرب هستند به صورت گروهی حذف (**Drop**) شده و دامنه به لیست سیاه (**Blacklist**) منتقل می‌شود.

بخش 5: پیاده‌سازی الگوریتمی و منطق ارزیابی (پایتون)

برای پیاده‌سازی این منطق در هسته یک موتور جستجو (و ارزیابی پاسخ‌های مبتنی بر کد دانشجویان)، معماری پیشنهادی باید بتواند الگوهای داینامیک را در مرتبه زمانی $O(1)$ با استفاده از جداول درهم‌سازی ردیابی کند. پیاده‌سازی ساختاریافته زیر، هسته این الگوریتم دفاعی را نشان می‌دهد:

```
resilient_crawler.py

import re
from collections import defaultdict
from urllib.parse import urlparse

class AdversarialTrapDetector:
    """
    سیستم درنگام شناسایی تله‌های خصمانه مبتنی بر گراف‌های متراکم وب
    """

    def __init__(self, pattern_threshold: int = 100, out_degree_threshold: int = 500):
        """
        نگاشت الگوهای استخراج شده به تعداد دفعات مشاهده
        """
        self.url_patterns = defaultdict(int)
        self.pattern_threshold = pattern_threshold
        self.out_degree_threshold = out_degree_threshold

    def _extract_structural_pattern(self, url: str) -> str:
        """
        های داینامیک به یک الگوی ایستا با حذف مقادیر متغیر URL تبدیل
        مثال: domain.com/page?id=123&sort=asc به domain.com/page?id=VAL&sort=VAL تبدیل
        """
        parsed = urlparse(url)
        # جایگزینی تمام مقادیر پس از علامت مساوی با یک مقدار ثابت
        normalized_query = re.sub(r'=[^&]+', '=VAL', parsed.query)
        return f"{parsed.netloc}/{parsed.path}?{normalized_query}"

    def should_prune_node(self, url: str, out_degree: int) -> bool:
        """
        تصمیم‌گیری برای هرس کردن گره پیش از ورود به صف خزش
        """
        # شرط اول: شناسایی تراکم پال غیرطبیعی (مشخصه گراف کامل خصمانه)
        if out_degree > self.out_degree_threshold:
            return True

        pattern = self._extract_structural_pattern(url)
        self.url_patterns[pattern] += 1

        # های داینامیک با الگوی تکراری URL شرط دوم: شناسایی انفجار
        if self.url_patterns[pattern] > self.pattern_threshold:
            return True

        return False
```

بخش 6: جمع‌بندی و نکات

ترکیب تجزیه و تحلیل رشته‌ای (URL Fingerprinting) با تحلیل توپولوژی (Out-degree Monitoring)، راه‌حل قطعی برای خنثی‌سازی حملات گراف کامل است. این روش نیازی به بارگذاری تمام گراف در حافظه ندارد و می‌تواند به عنوان یک فیلتر سبک (**Lightweight Filter**) در لایه ورودی خزنده‌ها مستقر شود تا پایداری شبکه و منابع محاسباتی موتور جستجو را تضمین کند.

نکته مهم (محدودیت‌های RAM و داده‌ساختارهای احتمالاتی):

در کد پایتون ارائه‌شده، از دیکشنری استاندارد **defaultdict** برای شمارش الگوها استفاده شده است. در یک موتور جستجوی واقعی در مقیاس اینترنت (مانند گوگل یا بینگ)، ذخیره میلیاردها الگوی متنی در حافظه RAM به سرعت باعث سرریز حافظه و قطعی سیستم (**OOM Exception**) می‌شود. برای حل این مشکل، مهندسان سیستم از داده‌ساختارهای احتمالاتی بسیار فشرده مانند **Count-Min Sketch** یا **فیلتر بلوم (Bloom Filters)** استفاده می‌کنند. این ساختارها می‌توانند فرکانس تکرار الگوهای مخرب را با دقت **99%** و تنها با استفاده از چند مگابایت حافظه ثابت (با بهره‌گیری از توابع هش چندگانه) محاسبه کنند و مقیاس‌پذیری الگوریتم دفاعی را تضمین نمایند.

پاسخنامه مسئله 19.26: تقلب در کلیک و سیستم‌های تشخیص مبتنی بر یادگیری ماشین

(Click Fraud & ML-based Detection)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، ورود عمیق به حوزه امنیت سیستم‌های تبلیغاتی (**Ad Fraud**) و طراحی سیستم‌های تشخیص ناهنجاری (**Anomaly Detection**) در مقیاس کلان داده (**Big Data**) است. در شبکه‌های تبلیغاتی مبتنی بر پرداخت به ازای کلیک (**CPC**)، رقبای تجاری یا سندیکاهای کلاهبرداری با استفاده از هزاران سیستم آلوده (بات‌نت‌ها)، روی تبلیغات کلیک می‌کنند تا بودجه تبلیغ‌دهنده را به سرعت تخلیه کرده و اصطلاحاً بسوزانند. از آنجا که بات‌نت‌های مدرن از میلیون‌ها آدرس **IP** پراکنده و مسکونی استفاده می‌کنند، مسدودسازی ساده مبتنی بر **IP** (مانند فایروال‌های سنتی در لایه شبکه) کاملاً ناکارآمد است. دانشجو باید بتواند به عنوان یک مهندس داده، ویژگی‌هایی (**Features**) را استخراج کند که الگوهای پنهان و محاسباتی ماشین را از رفتار ارگانیکی انسان تفکیک کند.

بخش 2: مبانی، پیش‌نیازها و تئوری یادگیری ماشین

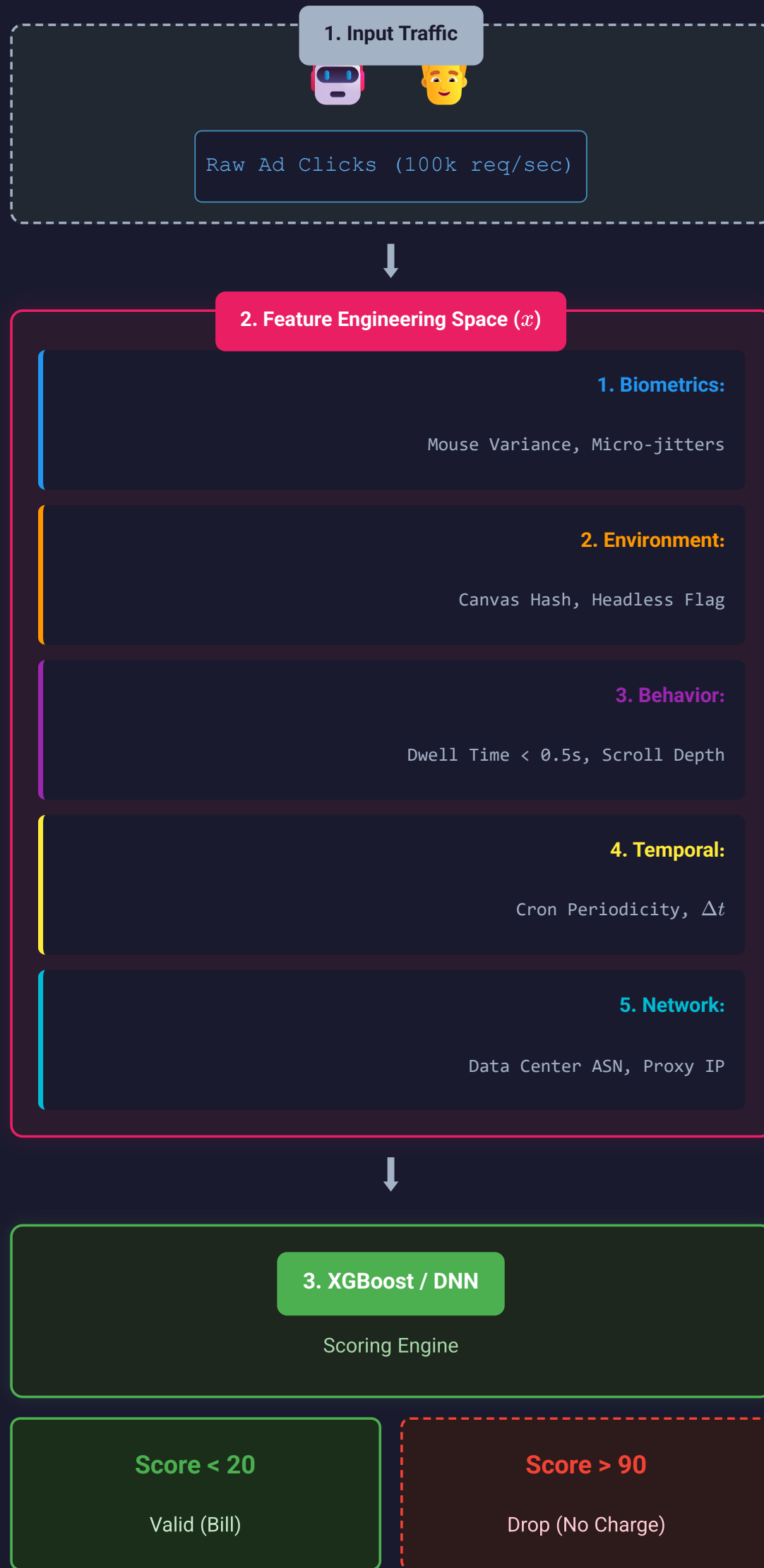
در معماری یادگیری ماشین، مسئله تشخیص کلیک‌های تقلبی یک مدل دسته‌بندی دودویی (**Binary Classification**) است که در آن خروجی الگوریتم، تعلق کلیک به یکی از دو کلاس را پیش‌بینی می‌کند (کلاس **0**: انسان معتبر، کلاس **1**: ربات مخرب). موفقیت این مدل کاملاً به کیفیت مرحله **مهندسی ویژگی (Feature Engineering)** بستگی دارد. ما باید سیگنال‌هایی را به مدل (نظیر **Random Forest** یا شبکه‌های عصبی عمیق) تزریق کنیم که تقلید آن‌ها برای توسعه‌دهندگان ربات‌ها از نظر محاسباتی بسیار هزینه‌بر، غیرممکن یا فاقد توجیه اقتصادی باشد.

رابطه ریاضی احتمال تقلب (**Fraud Probability**) به فرم تابع سیگموئید یا لجستیک زیر تعریف می‌شود:

$$P(y = 1|x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

در این فرمول، بردار x دقیقاً همان فیچرهای استخراج‌شده از رفتار کاربر در لایه‌های مختلف است که در ادامه به تفصیل کالبدشکافی می‌شوند.

معماری سیستم درنگام ارزیابی ترافیک و استخراج فیچر (Real-time Feature Extraction)



بخش 4: شرح تفصیلی 5 ویژگی (Feature) استراتژیک برای متمایزسازی

برای ساخت یک مدل قدرتمند (مانند **XGBoost** یا **Random Forest**)، 5 فیچر حیاتی در لایه‌های تعاملی، زمانی و شبکه‌ای استخراج می‌شوند:

1. لایه تعاملی: بیومتریک حرکت و مختصات کلیک (Interaction Biometrics)

تعریف فیچر: بررسی مختصات دقیق پیکسل کلیک شده روی بئر تبلیغاتی و محاسبه واریانس مسیر حرکت نشانگر موس (Mouse Trajectory Variance).

رفتار ماشین: ربات‌های ساده معمولاً با ارسال مستقیم کدهای جاوا اسکریپت (مانند `element.click()`) کلیک می‌کنند؛ بنابراین در طول زمان، هزاران کلیک دقیقاً روی مختصات هندسی ثابتی (مثلاً پیکسل **X:50, Y:50** که مرکز ثقل دکمه است) ثبت می‌شود. اگر حرکت موسی شبیه‌سازی شود، معمولاً برداری خطی و کاملاً صاف است.

رفتار انسان: انسان‌ها دارای لرزش دست (**Micro-jitters**) هستند، مسیر حرکت موس آن‌ها منحنی و حاوی نویز است و هرگز دو بار دقیقاً روی یک پیکسل واحد از یک دکمه کلیک نمی‌کنند. پراکندگی بالای مختصات کلیک‌ها یک سیگنال قدرتمند انسانی است.

2. لایه مرورگر: عدم تطابق اثر انگشت و محیط (Environment Fingerprint Mismatch)

تعریف فیچر: استخراج ویژگی‌های پنهان مرورگر (مانند **Canvas Fingerprinting**) و مقایسه آن‌ها با مشخصات ادعا شده در شبکه.

رفتار ماشین: بات‌نت‌ها معمولاً از مرورگرهای بدون رابط کاربری (**Headless Browsers**) مانند **Puppeteer** یا **Selenium** استفاده می‌کنند. فیچرهایی مانند وجود متغیر `navigator.webdriver == true`، فقدان پلاگین‌های رایج مرورگر، یا تضاد بین زبان سیستم عامل و منطقه جغرافیایی **IP** (مثلاً **IP** متعلق به ایران است اما مرورگر کاملاً با زبان روسی تنظیم شده) نشان‌دهنده یک محیط ماشینی و جعلی است.

3. لایه رفتار پس از کلیک: نرخ پرش و زمان توقف (Dwell Time & Bounce Behavior)

تعریف فیچر: محاسبه دقیق مدت زمانی که کاربر پس از کلیک روی تبلیغ، در صفحه فرود (**Landing Page**) باقی می‌ماند و تحلیل عمق پیمایش (**Scroll Depth**).

رفتار ماشین: هدف ربات فقط «ثبت کلیک و تریگر شدن پیکسل» برای کسر هزینه است. آن‌ها معمولاً بلافاصله پس از بارگذاری اولیه، نشست (**Session**) را می‌بندند (زمان توقف نزدیک به صفر). همچنین هیچ‌گونه تعامل ثانویه‌ای مانند اسکرول کردن صفحه به پایین یا کلیک روی لینک‌های داخلی انجام نمی‌دهند.

رفتار انسان: انسان‌ها پس از ورود، حداقل چند ثانیه متن را اسکرول کرده، متوقف می‌شوند (برای خواندن محتوا) و الگوهای زمانی تصادفی توقف و حرکت ایجاد می‌کنند.

4. لایه زمانی: سرعت و الگوهای پریودیک (Temporal Velocity & Periodicity)

تعریف فیچر: استفاده از تبدیل فوریه و تحلیل طیفی برای اندازه‌گیری فاصله زمانی (**Time Delta / Δt**) بین کلیک‌های متوالی از یک کاربر خاص یا یک خوشه **IP**.

رفتار ماشین: ربات‌ها توسط اسکریپت‌های زمان‌بندی شده (**Cron Jobs**) در سرور کنترل می‌شوند. اگر سیستم ماشین‌لرنینگ متوجه شود که از یک بلوک **IP**، دقیقاً هر **300** میلی‌ثانیه یا دقیقاً راس ساعت **3** بامداد یک کلیک ثبت می‌شود، این نظم ریاضیاتی و واریانس صفر زمانی، نشان‌دهنده فعالیت ماشینی است.

رفتار انسان: ترافیک انسانی در طول روز دارای توزیع طبیعی (زنگوله‌ای منطبق بر چرخه بیداری و خواب) است و فواصل بین کلیک‌ها کاملاً بی‌نظم است.

5. لایه شبکه: سیستم‌های خودمختار و شهرت IP (ASN Reputation)

تعریف فیچر: ترجمه آدرس IP به شماره سیستم خودمختار (ASN - Autonomous System Number) با استفاده از جداول مسیریابی BGP و بررسی نوع تامین‌کننده اینترنت.

رفتار ماشین: ساخت بات‌نت متمرکز نیازمند سرور است. بنابراین بخش عظیمی از کلیک‌های تقلبی، از سوی IP‌های متعلق به دیتاسنترها و ارائه‌دهندگان خدمات ابری (مانند AWS، Hetzner یا DigitalOcean) ساطع می‌شوند.

رفتار انسان: کاربران واقعی معمولاً از اینترنت‌های خانگی (Residential ADSL) یا شبکه‌های تلفن همراه (Mobile ISP) استفاده می‌کنند. مدل یادگیری ماشین به کلیک‌هایی که از یک ASN تجاری (دیتاسنتر) می‌آیند، به طور خودکار وزن مشکوک بالاتری اختصاص می‌دهد.

بخش 5: جدول مقایسه‌ای سریع و تطبیقی

لایه تحلیلی	الگوی انسان معتبر (کلاس 0)	الگوی بات مخرب (کلاس 1)
هندسه تعامل	مسیر موس منحنی با لرزش طبیعی	کلیک دقیق روی پیکسل مرکزی (مرتباً)
زمان توقف	چندین ثانیه مکث و اسکرول محتوا	نزدیک به 0 (بستن سریع سشن)
توزیع زمانی	نامنظم با واریانس بالا (Δt تصادفی)	دوره‌ای و منظم منطبق بر اسکریپت
رده شبکه (ASN)	شبکه مسکونی و موبایل (Residential)	دیتاسنترها و پروکسی‌های ابری

بخش 6: جمع‌بندی

سیستم‌های مدرن مقابله با تقلب در تبلیغات (Anti-Fraud Systems)، از پارادایم مسدودسازی مبتنی بر قواعد سخت و ایستای شبکه‌ای (Rule-based) فراتر رفته‌اند؛ چرا که این قواعد سنتی به راحتی توسط بات‌نت‌های توزیع‌شده دور زده می‌شوند. امروزه این سیستم‌ها با تجمیع صدها ویژگی رفتاری، شبکه‌ای و بیومتریک در یک فضای چندبعدی، الگوریتم‌های یادگیری ماشین را برای محاسبه یک امتیاز ریسک پیوسته (Fraud Score) از 0 تا 100 آموزش می‌دهند. کلیک‌هایی که از آستانه قطع (Threshold) الگوریتم عبور کنند، به عنوان ترافیک نامعتبر طبقه‌بندی شده و در لحظه فیلتر می‌گردند تا هزینه‌ای از کمپین تبلیغ‌دهنده کسر نگردد.

در معماری‌های نوین و مقیاس‌پذیر، با توجه به حجم نجومی ترافیک (صدها هزار کلیک در ثانیه برای پلتفرم‌هایی مانند گوگل)، استخراج این ویژگی‌ها و اجرای مدل‌های هوش مصنوعی به صورت آفلاین و بر روی پایگاه‌های داده رابطه‌ای امکان‌پذیر نیست. از این رو، مهندسی این سیستم‌ها بر پایه پردازش جریان داده‌های درنگام (Real-time Stream Processing) با استفاده از فناوری‌هایی نظیر Apache Kafka و Apache Flink استوار شده است. در این فرآیند، داده‌های خام کلیک وارد صف‌های پیام توزیع‌شده گشته و توسط موتورهای پردازش جریانی، در پنجره‌های زمانی چند میلی‌ثانیه‌ای تجمیع و ویژگی‌سازی می‌شوند. در نهایت، این بردارهای ویژگی زنده به یک مدل استنتاج (Inference Model) مستقر در حافظه اصلی (RAM) تزریق شده تا فرآیند طبقه‌بندی دودویی و مسدودسازی پیش از صدور صورت‌حساب قطعی، با دقتی بی‌نظیر انجام پذیرد.

پاسخنامه مسئله 19.27: اثبات سوگیری در تخمین اندازه وب (Proving Bias in Web Size Estimation)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، نقد دقیق و ریاضیاتی روش‌های آماری کلاسیک (مانند روش **Capture-Recapture**) هنگام مواجهه با کلان‌داده‌های شبکه‌ای در محیط وب است. دانشجو باید بتواند با استفاده از تئوری گراف وب (به‌ویژه ویژگی اتصال ترجیحی یا **Preferential Attachment**) اثبات کند که چرا خزنده‌های موتورهای جستجو، نمونه‌بردارهای آماری مستقلی نیستند. در نهایت، با استفاده از جبر پایه‌ای در کسرها، باید جهت این سوگیری و انحراف (**Bias Direction**) یعنی متمایل شدن پاسخ به سمت کم‌برآورد بودن نسبت به مقدار واقعی – اثبات شود.

بخش 2: مبانی، پیش‌نیازها و فرض استقلال

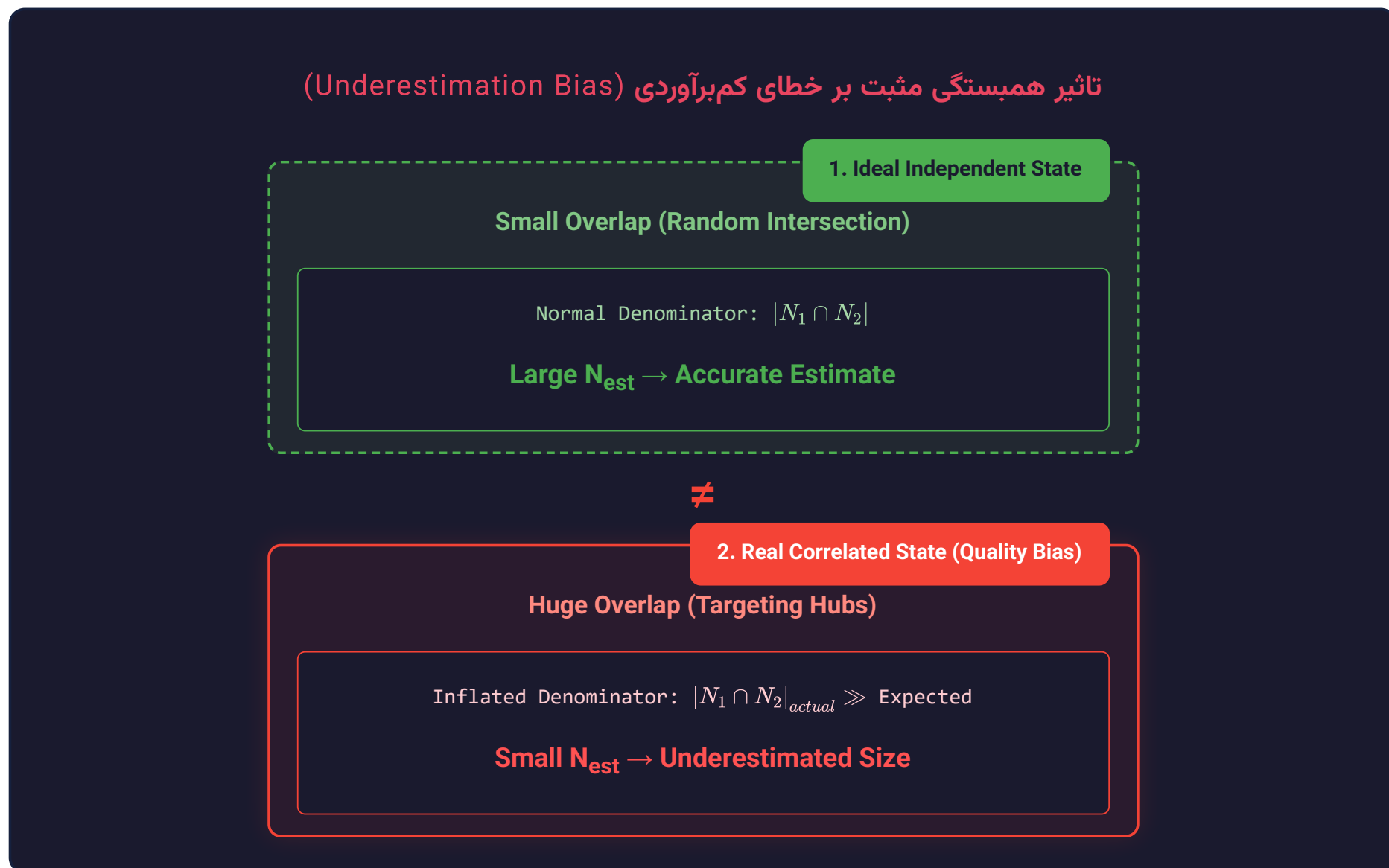
در مدل کلاسیک صید و بازصید که با فرمول تخمین‌گر لینکلن-پترسن محاسبه می‌شود ($N = \frac{N_1 \times N_2}{|N_1 \cap N_2|}$), فرض بنیادین و ستون فقرات محاسبات، «استقلال پیشامدهای آماری» (**Independence Assumption**) است.

از نظر ریاضی در تئوری احتمالات، استقلال به این معناست که احتمال حضور یک صفحه به صورت همزمان در اشتراک دو موتور جستجوی **X** و **Y**، باید دقیقاً برابر با حاصل ضرب احتمال‌های فردی ایندکس شدن آن در هر کدام از موتورها باشد:

$$P(X \cap Y) = P(X) \times P(Y)$$

اگر این معادله در سیستم برقرار نباشد، به این معناست که متغیرها به هم وابسته‌اند و مدل آماری دچار سوگیری سیستماتیک (**Systematic Bias**) است و پاسخی نهایی فاقد اعتبار مطلق خواهد بود.

بخش 3: نمودار شماتیک تورم همپوشانی و انحراف کسر



بخش 4: شرح گام‌به‌گام و اثبات تحلیلی

برای پاسخ به این مسئله، دو بخش (الف) و (ب) را به صورت متوالی و با استدلال ریاضی اثبات می‌کنیم:

بخش الف: اثبات نقض فرض استقلال (وابستگی مثبت)

تحلیل توپولوژی: در وب واقعی، صفحات دارای اهمیت و اتصال مساوی نیستند. توزیع لینک‌ها از قانون توان (Power Law) و مکانیزم اتصال ترجیحی (Preferential Attachment) پیروی می‌کند؛ به این معنا که سایت‌هایی که لینک‌های بیشتری دارند (سایت‌های ثروتمند و معتبر)، با سرعت بسیار بیشتری لینک‌های جدید دریافت می‌کنند (پدیده Rich get richer).

رفتار خزنده‌ها: موتورهای جستجو (X و Y) هر دو الگوریتم‌های خزش خود را بر پایه دنبال کردن لینک‌ها و اولویت‌دهی به صفحات با PageRank بالاتر بهینه‌سازی می‌کنند. بنابراین، یک سایت معروف (مثل ویکی‌پدیا) به دلیل داشتن میلیون‌ها پیوند ورودی، با احتمال نزدیک به 100% توسط هر دو موتور جستجو به سرعت کشف و ایندکس می‌شود. در مقابل، یک سایت منزوی، تاریک و بدون لینک، توسط هیچ‌کدام کشف نمی‌شود.

نتیجه‌گیری منطقی بخش الف: از آنجا که هر دو موتور جستجو دارای «سوگیری کیفیت» (Quality Bias) کاملاً مشترکی هستند، احتمال پیدا شدن یک صفحه توسط موتور Y، در صورتی که بدانیم موتور X قبلاً آن را پیدا کرده است، بسیار بیشتر از حالت تصادفی است. در بیان ریاضی، این دو پیشامد دارای همبستگی مثبت (Positive Correlation) هستند:

$$P(X \cap Y) > P(X) \times P(Y)$$

در نتیجه، فرض استقلال مدل به طور کامل نقض می‌شود.

بخش ب: اثبات کم‌برآورد بودن فرمول (Underestimation)

حال با استفاده از جبر کسرها بررسی می‌کنیم که این وابستگی و همبستگی مثبت چه تاثیری بر نتیجه نهایی فرمول می‌گذارد:

محاسبه حالت ایده‌آل: فرض کنید اندازه واقعی و قطعی وب N_{true} باشد. اگر موتورها کاملاً مستقل و تصادفی عمل می‌کردند، اندازه تقاطع (همپوشانی مورد انتظار) آن‌ها به این صورت محاسبه می‌شد:

$$|N_1 \cap N_2|_{expected} = \frac{N_1 \times N_2}{N_{true}}$$

تاثیر سوگیری در واقعیت: اما همان‌طور که در بخش (الف) اثبات کردیم، موتورها تمایل شدیدی به کشف مشترک صفحات معروف دارند. بنابراین، تعداد صفحاتی که در واقعیت در هر دو ایندکس مشترک است، بسیار بزرگتر و متورم‌تر از حالت تصادفی است:

$$|N_1 \cap N_2|_{actual} > |N_1 \cap N_2|_{expected}$$

نتیجه‌گیری ریاضی بخش ب: فرمول تخمین اندازه وب به صورت $N_{estimated} = \frac{N_1 \times N_2}{|N_1 \cap N_2|_{actual}}$ است. در قوانین جبر ریاضی، اگر مخرج یک کسر بزرگتر از حد استاندارد خود شود (تورم حجم تقاطع به دلیل صفحات مشترک)، حاصل کل کسر کوچکتر خواهد شد.

از آنجا که مخرج کسر به صورت مصنوعی بزرگ شده است، مقدار نهایی محاسبه شده ($N_{estimated}$) عدد کوچکتري از N_{true} را نشان می‌دهد.

$$N_{estimated} < N_{true}$$

در نتیجه، این فرمول حجم وب را به شدت کم‌برآورد (Underestimate) می‌کند و فقط حداقل ممکن صفحات را محاسبه می‌نماید.

بخش 5: جمع‌بندی

در مهندسی سیستم‌های توزیع‌شده و ارزیابی معماری موتورهای جستجو، نادیده گرفتن همبستگی الگوریتمی (Algorithmic Correlation) می‌تواند منجر به بروز خطاهای فاحش آماری در تحلیل کلان‌داده‌ها گردد. موتورهای جستجو رقابایی نیستند که در محیط‌های تصادفی و کاملاً مجزا از یکدیگر فعالیت کنند؛ بلکه آن‌ها همگی بر روی یک گراف واحد با یک توپولوژی متمرکز و به شدت نامتقارن، فرآیند خزش (Crawling) را پیش می‌برند. بر این اساس، همپوشانی نجومی در ایندکس‌های آن‌ها ناشی از «تصادف در نمونه‌گیری» نیست، بلکه به طور مستقیم معلول «طراحی همسان الگوریتم‌ها» برای رسیدن هرچه سریع‌تر به گره‌های مرکزی (هاب‌ها و مراجع معتبر یا Hubs & Authorities مبتنی بر الگوریتم HITS) در گراف وب است. این همبستگی مثبت، تمامی برآوردگرهای کلاسیک آماری را در مواجهه با شبکه‌های بی‌مقیاس (Scale-free Networks) که قویاً از مدل اتصال ترجیحی (Preferential Attachment) پیروی می‌کنند، با شکست مواجه می‌سازد. از منظر ریاضیاتی، تورم کاذب در مخرج کسر محاسباتی باعث می‌گردد که حجم برآوردشده از طریق تکنیک صید و بازصید، همواره به عنوان یک کران پایین مطلق (Absolute Lower Bound) در نظر گرفته شود و هرگز بازتاب دقیق و بی‌نقصی از اندازه واقعی و نامتناهی شبکه وب نباشد.

پاسخنامه مسئله 19.28: طراحی مکانیزم حراج با احتساب CTR (CTR-based Auction Mechanism)

بخش 1: تحلیل صورت مسئله

هدف از این سوال، درک تکامل سیستم‌های مزایده از یک مدل صرفاً مالی به یک مدل مبتنی بر کارایی است. در محیط وب، فضای نمایش (Screen Real Estate) محدود است. اگر موتور جستجو یک فضای تبلیغاتی را به کاربری بدهد که پول زیادی پیشنهاد کرده (ثبت Bid بالا) اما هیچ‌کس روی تبلیغ او کلیک نمی‌کند (CTR پایین)، موتور جستجو در نهایت درآمدی کسب نخواهد کرد (چون مدل پرداخت به صورت CPC است). بنابراین، دانشجو باید بتواند ارزش تئوریک یک پیشنهاد را با احتمال وقوع آن ترکیب کند تا «ارزش واقعی و مورد انتظار» برای پلتفرم مشخص شود.

بخش 2: مبانی و پیش‌نیازها

در مدل GSP استاندارد (که در مسئله 19.16 بررسی کردیم)، رتبه‌بندی تنها بر اساس متغیر Bid (مبلغ پیشنهادی) انجام می‌شد. اما در مزایده‌های پیشرفته و واقعی (مانند الگوریتم پایه‌ای پلتفرم Google Ads)، رتبه‌بندی بر اساس متغیری ترکیبی به نام رتبه تبلیغ (Ad Rank) صورت می‌گیرد.

در این مکانیزم، هدف موتور جستجو بیشینه‌سازی «درآمد مورد انتظار به ازای هر بار نمایش تبلیغ» (Expected Revenue per Impression) است.

جریان محاسباتی رتبه‌بندی و قیمت‌گذاری (Modified GSP Pipeline)

Bidder	Max Bid	Expected CTR	Ad Rank Score	Final Position	Actual CPC (Pricing)
User B	2.00 USD	3% (0.03)	0.06	Rank 1 🏆	Based on A's Rank 1.00 USD
User A	3.00 USD	1% (0.01)	0.03	Rank 2	Based on C's Rank 2.00 USD
User C	1.00 USD	2% (0.02)	0.02	Rank 3	Floor Price -

بخش 4: شرح گام‌به‌گام و محاسبات ریاضی

پاسخ به سه بخش مسئله را به ترتیب و با استفاده از معادلات مربوطه پیش می‌بریم:

الف) متریک و فرمول رتبه‌بندی (Ranking Metric)

موتور جستجو تبلیغات را بر اساس فرمول **Ad Rank** مرتب می‌کند. این متریک از حاصل ضرب مبلغ پیشنهادی در احتمال کلیک شدن تبلیغ به دست می‌آید. فرمول ریاضی حاکم بر این مرتب‌سازی به شکل زیر است:

$$Ad_Rank = Max_Bid \times Expected_CTR$$

این فرمول نشان می‌دهد که موتور جستجو به طور میانگین از هر بار نمایش این تبلیغ، چقدر درآمد انتظار دارد.

ب) محاسبه رتبه‌ها و تعیین ترتیب نمایش (Ad Positioning)

با استفاده از فرمول فوق، ارزش مورد انتظار (**Ad Rank**) را برای هر سه تبلیغ‌دهنده محاسبه می‌کنیم:

$$Ad_Rank_A = 3 \times 0.01 = 0.03$$

$$Ad_Rank_B = 2 \times 0.03 = 0.06$$

$$Ad_Rank_C = 1 \times 0.02 = 0.02$$

با مرتب‌سازی مقادیر به‌دست‌آمده از بیشترین به کمترین، ترتیب نمایش نتایج در صفحه (**SERP**) به این شرح خواهد بود:

جایگاه اول: کاربر B (با امتیاز 0.06)

ج) محاسبه هزینه پرداختی جایگاه اول (Modified GSP Pricing)

در مدل **GSP** اصلاح شده، برنده مزایده مبلغ پیشنهادی خود را پرداخت نمی‌کند؛ بلکه حداقل مبلغی را می‌پردازد که برای حفظ جایگاه فعلی‌اش (یعنی شکست دادن **Ad Rank** نفر بعدی) نیاز دارد. فرمول محاسبه هزینه پرداختی واقعی (**Actual CPC**) برای جایگاه i به صورت زیر است:

$$Actual_CPC_i = \frac{Ad_Rank_{i+1}}{Expected_CTR_i}$$

در این مسئله، می‌خواهیم مبلغ پرداختی نفر اول (کاربر B) را محاسبه کنیم. نفر بعدی او در جایگاه دوم، کاربر A است. متغیرهای لازم را با دقت جایگذاری می‌کنیم:

امتیاز نفر بعدی (کاربر A): $Ad_Rank(A) = 0.03$

نرخ کلیک کاربر اول (کاربر B): $Expected_CTR(B) = 0.03$

با جایگذاری این مقادیر در فرمول محاسبه پرداختی داریم:

$$Actual_CPC_B = \frac{0.03}{0.03} = 1.00$$

پاسخ نهایی بخش ج:

تبلیغ‌دهنده جایگاه اول (کاربر B)، بابت هر کلیک دقیقاً **1.00 دلار (USD)** پرداخت خواهد کرد.

بخش 5: جمع‌بندی

ساختار ریاضیاتی این مسئله، یکی از زیباترین مفاهیم در اقتصاد الگوریتمی و نظریه بازی‌ها را به تصویر می‌کشد: **پاداش برای کیفیت (Quality Premium)** در مدل‌های اصلاح‌شده **GSP**. تحلیل نتایج نشان می‌دهد که کاربر B با وجود اینکه سقف بودجه‌ی بسیار کمتری نسبت به کاربر A به سیستم پیشنهاد داده بود (2.00 دلار در برابر 3.00 دلار)، اما به دلیل جذابیت محتوای تبلیغش برای کاربران (که منجر به **CTR** سه‌برابری شد)، توانست از طریق اهرم درآمد مورد انتظار (**Expected Revenue**) نه تنها جایگاه اول را در رتبه‌بندی **Ad Rank** فتح نماید، بلکه هزینه‌ای به مراتب کمتر از بودجه اعلامی‌اش (تنها 1.00 دلار) پرداخت کند. در حقیقت، موتور جستجو با کالیبره کردن قیمت‌ها بر اساس احتمال کلیک، به شبکه تبلیغ‌دهندگان این پیام استراتژیک را مخابره می‌کند که تولید محتوای مرتبط‌تر و کاربرپسندتر، مستقیماً به کاهش هزینه‌های پرداختی و ارتقای حاشیه سود آن‌ها منجر خواهد شد.

پاسخنامه مسئله 19.29: پیاده‌سازی مفهومی MinHash و کاهش ابعاد

(MinHash & Dimensionality Reduction)

بخش 1: تحلیل بنیادین و معماری الگوریتم

محاسبه مستقیم شباهت ژاکارد برای $N = 10^6$ سند، نیازمند انجام محاسبات ترکیبی $\frac{N(N-1)}{2}$ یا حدود 500 میلیارد مقایسه جفتی روی بردارهای بسیار طولانی است که پیچیدگی زمانی آن $O(N^2)$ بوده و از نظر محاسباتی عملاً غیرممکن است. الگوریتم MinHash با کاهش ابعاد فضای ویژگی‌ها (Dimensionality Reduction) از طریق توابع درهم‌ساز حافظ محلیت (Locality-Preserving)، بردارهای پراکنده و عظیم را به «امضاهای فشرده» (Compact Signatures) تبدیل می‌کند.

فرآیند استخراج امضا در الگوریتم MinHash طی 4 مرحله متوالی زیر انجام می‌شود:

مرحله 1: تشکیل ماتریس مشخصه (Boolean Feature Matrix)

ابتدا فضای کل شینگل‌های موجود در مجموعه دادگان استخراج می‌شود. یک ماتریس بولی M تشکیل می‌گردد که سطرها نشان‌دهنده اندیس شینگل‌های یکتا و ستون‌ها نشان‌دهنده اسناد هستند. اگر شینگل i در سند j وجود داشته باشد، $M[i][j] = 1$ و در غیر این صورت 0 خواهد بود.

مرحله 2: تعریف توابع درهم‌ساز جهانی (Universal Hashing)

به جای جابه‌جا کردن فیزیکی سطرهای ماتریس عظیم M (تولید جایگشت تصادفی که بسیار پرهزینه است)، K تابع هش مستقل به فرم $h(x) = (ax + b) \pmod{p}$ تولید می‌شود. این توابع، اندیس هر سطر را به یک مقدار تصادفی جدید نگاشت می‌کنند و عملاً اثر یک جایگشت تصادفی (Random Permutation) را شبیه‌سازی می‌نمایند.

مرحله 3: ساخت ماتریس امضا (Signature Matrix Construction)

یک ماتریس خالی امضا (Signature Matrix) با ابعاد $K \times N$ (تعداد توابع هش $\times$ تعداد اسناد) با مقدار اولیه بی‌نهایت (∞) مقداردهی می‌شود. به ازای هر سطر r از ماتریس M و هر ستون c (که $M[r][c] = 1$ است)، مقدار $h_i(r)$ محاسبه شده و ماتریس امضا با رابطه $\min(\text{Sig}[i][c], h_i(r))$ به‌روزرسانی می‌شود.

مرحله 4: تخمین شباهت ژاکارد (Jaccard Estimation)

پس از پایان پردازش، هر سند با یک بردار فشرده به طول K (مثلاً 100 یا 200) نمایش داده می‌شود. میزان شباهت ژاکارد بین دو سند، به سادگی برابر است با کسری از این K مولفه که در امضای هر دو سند دقیقاً یکسان هستند.

بخش 2: نمودار شماتیک معماری کاهش ابعاد (Dimensionality Reduction)

تبدیل فضای پراکنده (Sparse) به امضای فشرده (Dense)

1. Sparse Boolean Matrix (M)

Dimensions: 10^6 Docs $\times$ Millions of Shingles

Doc Vector:

0 1 0 0 1 0 ... (Mostly Zeros)

Apply K Random Hash Functions

$h_1(x), h_2(x), \dots, h_k(x)$

2. Dense Signature Matrix (Sig)

Dimensions: 10^6 Docs $\times$ Only $K=100$ Rows

Signature Vector:

14 92 5 37 ... (Dense Integers)

بخش 3: پیاده‌سازی بهینه (Vectorized Python)

minhash_estimator.py

```
import numpy as np

def generate_minhash_signatures(boolean_matrix: np.ndarray, num_hashes: int) -> np.ndarray:
    num_shingles, num_docs = boolean_matrix.shape
    signatures = np.full((num_hashes, num_docs), np.inf)
    # p must be a prime number greater than num_shingles
    p = 4294967311
    a = np.random.randint(1, p, num_hashes).reshape(-1, 1)
    b = np.random.randint(0, p, num_hashes).reshape(-1, 1)
    # Process each row (shingle)
    for row_idx in range(num_shingles):
        # Find which documents contain this shingle
        docs_with_shingle = np.where(boolean_matrix[row_idx, :] == 1)[0]
        if docs_with_shingle.size == 0:
            continue
        # Compute all K hash values for the current row index
        hash_vals = (a * row_idx + b) % p
        # Vectorized min update across all K hashes for the active documents
        signatures[:, docs_with_shingle] = np.minimum(
            signatures[:, docs_with_shingle],
            hash_vals
        )
    return signatures
```

بخش 4: اثبات ریاضی مکانیزم MinHash

برای درک اینکه چگونه مقایسه این بردارهای فشرده، شباهت ژاکارد فضای اصلی را تخمین می‌زند، باید رفتار سطرها را در یک جایگشت تصادفی بررسی کنیم.

دو سند A و B (دو ستون از ماتریس) را در نظر بگیرید. هنگام بررسی سطرهای این دو ستون، تنها سه حالت وجود دارد:

نوع X: سطر در هر دو سند مقدار 1 دارد (اشتراک).

نوع Y: سطر در یکی از اسناد 1 و در دیگری 0 است (تفاوت).

نوع Z: سطر در هر دو سند 0 است (توسط الگوریتم MinHash نادیده گرفته می‌شود).

تابع درهم‌ساز (جایگشت تصادفی)، سطرها را با ترتیبی کاملاً تصادفی به ما نشان می‌دهد. اولین سطر که در این پیمایش تصادفی دارای حداقل یک مقدار 1 باشد (خروج از حالت Z)، مقدار MinHash را برای این اسناد تعیین می‌کند.

احتمال اینکه این «اولین سطر دارای مقدار 1»، در هر دو سند برابر با 1 باشد (یعنی سطر از نوع X باشد)، دقیقاً برابر است با نسبت تعداد سطرهای نوع X به کل سطرهای نوع X و Y:

$$P(h_{min}(A) = h_{min}(B)) = \frac{X}{X + Y}$$

از سوی دیگر، بر اساس تعریف شباهت ژاکارد می‌دانیم که اشتراک دو مجموعه برابر با X و اجتماع آن‌ها برابر با X + Y است:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{X}{X + Y}$$

بخش 5: جمع‌بندی

تکنیک MinHash در هسته مرکزی خود، از یک مکانیزم هوشمندانه در نظریه احتمالات استفاده می‌کند. استفاده از K تابع هش تصادفی و مستقل، به معنای انجام K آزمایش آماری مجزا بر روی توزیع سطرهاست. بر اساس **قانون اعداد بزرگ** (Law of Large Numbers)، با محاسبه کسر تطابق‌ها در ماتریس امضا (بدون نیاز به پردازش مجدد ماتریس پراکنده و غول‌پیکر اولیه)، می‌توانیم به یک برآوردگر بدون سوگیری (**Unbiased Estimator**) از شباهت ژاکارد دست یابیم. این معماری کاهش ابعاد، بن‌بست پیچیدگی زمانی را شکسته و پردازش و شناسایی صفحات تقریباً تکراری (**Near-Duplicate Detection**) را در مقیاس‌های کلان‌داده (**Big Data**) عملیاتی می‌سازد.

پاسخنامه مسئله 19.30: مکانیزم هش کردن محلی با LSH (Locality Sensitive Hashing)

بخش 1: تحلیل صورت مسئله

چالش اصلی در مقیاس وب این است که حتی اگر اندازه هر سند را با MinHash به یک آرایه 100 تایی کاهش دهیم، مقایسه دو به دو (**Pairwise**) یک میلیارد صفحه نیازمند حجم وحشتناکی از عملیات است:

$$\frac{10^9 \times 10^9}{2} = 5 \times 10^{17}$$

یعنی نیم کوینتیلیون مقایسه! الگوریتم **LSH** به عنوان یک «فیلتر پیش پردازش» هوشمند عمل می‌کند تا به جای مقایسه همه اسناد با هم، تنها اسنادی را با یکدیگر مقایسه کنیم که احتمال شباهت آن‌ها بسیار بالاست. با این تکنیک، پیچیدگی زمانی از مرتبه $O(N^2)$ به مرتبه خطی $O(N)$ تقلیل می‌یابد و بار پردازشی سیستم به شدت کاهش پیدا می‌کند.

بخش 2: مبانی و پیش‌نیازهای ریاضی

در توابع هش کلاسیک و رمزنگاری (مانند **SHA-256** یا **MD5**)، اگر حتی یک بیت از ورودی تغییر کند، خروجی هش کاملاً دگرگون می‌شود (اثر بهمن یا **Avalanche Effect**). این ویژگی برای امنیت عالی است، اما برای یافتن شباهت‌ها فاجعه‌بار است.

اما توابع **LSH** کاملاً برعکس عمل می‌کنند؛ این توابع به گونه‌ای طراحی شده‌اند تا حافظ محلیت (**Locality-Preserving**) باشند. یعنی اگر دو ورودی به هم شبیه باشند، احتمال اینکه خروجی هش آن‌ها دقیقاً یکسان (**Collision**) شود، بسیار بالا است. در فضای **MinHash**، این کار با تکنیک قدرتمندی به نام **باندینگ (Banding)** انجام می‌شود که اسناد مشابه را در یک «سطل» مشترک می‌اندازد.

بخش 3: نمودار شماتیک معماری Banding در الگوریتم LSH



بخش 4: شرح گام به گام الگوریتم (Banding تکنیک)

فرض کنید ماتریس امضا (Signature Matrix) حاصل از MinHash دارای n سطر (طول امضا) و N ستون (تعداد کل اسناد) است. مکانیزم کشف نامزدها (Candidates) طی 4 مرحله دقیق زیر انجام می‌شود:

مرحله اول: تقسیم امضاها به باندها (Banding)

ماتریس امضای هر سند به b باند (Band) متوالی تقسیم می‌شود که هر باند شامل r ردیف (Row) است. بنابراین طول کل امضای هر سند از معادله $n = b \times r$ به دست می‌آید. مثلاً اگر امضای MinHash دارای 100 ردیف باشد، می‌توانیم آن را به 20 باند که هر کدام 5 ردیف دارند، تفکیک کنیم.

مرحله دوم: هش کردن باندها به سطل‌ها (Bucketing)

برای هر باند به صورت مجزا، یک تابع هش استاندارد در نظر می‌گیریم. این تابع، بردار r تایی از اعداد (ردیف‌های موجود در آن باند) را به عنوان یک کلید واحد دریافت کرده و آن را به شماره یک سطل (Bucket) در حافظه هش می‌کند.

نکته حیاتی: دو سند تنها در صورتی در یک باند به یک سطل مشترک می‌افتند که دقیقاً تمام r ردیف آن‌ها در آن باند با هم برابر باشند.

مرحله سوم: استخراج نامزدهای شباهت (Candidate Selection)

قانون طلایی LSH این است: اگر دو سند در حداقل یک باند (از مجموع b باند موجود) به یک سطل یکسان هش شوند، آن‌ها فوراً به عنوان یک «جفت نامزد» (Candidate Pair) در نظر گرفته می‌شوند. این یعنی نیازی نیست دو سند در تمام ابعاد ماتریس با هم مو بزنند؛ یافتن تنها یک قطعه r تایی کاملاً مشابه کافی است تا آن‌ها را برای بررسی دقیق‌تر از بقیه جدا کنیم.

مرحله چهارم: ارزیابی نهایی (False Positive Filtering)

الگوریتم اکنون لیستی از جفت‌های کاندیدا دارد که طول آن بسیار کوچکتر از تمام ترکیبات ممکن یعنی $O(N^2)$ است. در این مرحله، شباهت ژاکارد واقعی (یا شباهت امضای کامل MinHash) صرفاً برای همین کاندیداها محاسبه می‌شود. اگر شباهت نهایی آن‌ها از آستانه (Threshold) مدنظر بیشتر بود، به عنوان صفحات تکراری ثبت می‌شوند و جفت‌های اشتباه که تصادفاً در یک سطل افتاده‌اند (False Positives) دور ریخته می‌شوند.

بخش 5: تحلیل ریاضی و نمودار منحنی S (S-Curve)

جادوی LSH در احتمالات نهفته است. بیایید قدم به قدم محاسبه کنیم که احتمال اینکه دو سند با شباهت ژاکارد s به عنوان نامزد انتخاب شوند، دقیقاً چقدر است:

1. احتمال برابری در یک ردیف مشخص: طبق تئوری اثبات شده MinHash، این احتمال دقیقاً برابر با مقدار شباهت یعنی s است.

2. احتمال برابری در تمام r ردیف از یک باند: برابر است با s^r (زیرا توابع هش در ردیف‌های مختلف از هم مستقل‌اند).

3. احتمال عدم تطابق در یک باند (حداقل یک ردیف فرق کند): مکمل حالت قبل، یعنی برابر است با $1 - s^r$.

4. احتمال اینکه دو سند در هیچ‌کدام از b باند با هم تطابق نداشته باشند: برابر است با $(1 - s^r)^b$.

5. احتمال نهایی (احتمال کاندیدا شدن): مکمل حالت 4؛ یعنی احتمال اینکه دو سند حداقل در یک باند با هم تطابق داشته باشند و به Candidate تبدیل شوند:

$$P = 1 - (1 - s^r)^b$$

این معادله غیرخطی، یک منحنی به شکل حرف S (معروف به S-Curve) تولید می‌کند. برای درک بهتر، بیایید یک مثال عددی با پارامترهای $b = 20$ و $r = 5$ را در جدول زیر بررسی کنیم:

شباهت واقعی (s)	محاسبه فرمول $P = 1 - (1 - s^5)^{20}$	احتمال کاندیدا شدن (P)
0.2 (شباهت کم)	$1 - (1 - 0.00032)^{20} \approx 1 - 0.9936$	0.006 (0.6%)
0.5 (شباهت متوسط)	$1 - (1 - 0.03125)^{20} \approx 1 - 0.5307$	0.469 (46.9%)
0.8 (شباهت زیاد)	$1 - (1 - 0.32768)^{20} \approx 1 - 0.0003$	0.999 (99.9%)

همان‌طور که جدول نشان می‌دهد، اسنادی که شباهت کمی دارند (مثلاً $s = 0.2$)، احتمال Candidate شدنشان به شدت به سمت 0 سقوط می‌کند و اسنادی که شباهت بالایی دارند (مثلاً $s = 0.8$)، با احتمال نزدیک به 1 (100%) کشف می‌شوند.

نقطه عطف این منحنی (تندترین شیب S) همان آستانه شباهت (Threshold - t) است که مرز بین دفع و پذیرش اسناد را مشخص می‌کند. این آستانه به صورت تقریبی با فرمول زیر تخمین زده می‌شود:

$$t \approx \left(\frac{1}{b} \right)^{\frac{1}{r}}$$

Example: $t \approx (1/20)^{1/5} \approx 0.55$

بخش 6: جمع‌بندی

در معماری سیستم‌های توزیع‌شده و کلان‌داده، الگوریتم درهم‌سازی حساس به محلّیت (LSH) نقشی بسیار فراتر از یک فیلتر ساده ایفا می‌کند. این مکانیزم با بهره‌گیری از تکنیک باندینگ (Banding)، یک نگاشت احتمالاتی و غیرخطی (S-Curve) ایجاد می‌نماید که به مهندسان سیستم اجازه می‌دهد با کالیبراسیون دقیق فرآپارامترهای b (تعداد باندها) و r (تعداد ردیف‌ها در هر باند)، مرز تصمیم‌گیری (Threshold) را به صورت ریاضیاتی تنظیم کنند. این تنظیمات دقیق،

موازنه ایده‌آلی میان خطای مثبت کاذب (**False Positive**) و منفی کاذب (**False Negative**) به وجود می‌آورد. در نهایت، **LSH** به عنوان یک «غربالگر ماتریسی بی‌رحم و فوق‌سریع» عمل کرده و پیچیدگی زمانی وحشتناک و غیرعملی ($O(N^2)$) در مقایسه جفت‌به‌جفت اسناد را به یک جستجوی خطی و مقیاس‌پذیر ($O(N)$) تقلیل می‌دهد. این تغییر پارادایم، تضمین می‌کند که توان پردازشی گران‌قیمت خوشه‌های سرور (**Server Clusters**)، تنها برای پردازش عمیق و قطعی جفت‌اسنادی که دارای پتانسیل بالای کپی‌برداری هستند، مصرف گردد.

پاسخنامه مسئله 19.31: اسپم توزیع شده و الگوریتم TrustRank (Distributed Spam & TrustRank)

بخش 1: تحلیل صورت مسئله

الگوریتم **PageRank** با وجود قدرت و کارایی بالا در روزهای اولیه وب، یک نقطه ضعف اساسی و ساختاری دارد: «**دموکراتیک بودن بی‌قیدوشرط**». در الگوریتم **PageRank**، هر صفحه جدیدی که به گراف وب اضافه می‌شود، به صورت پیش‌فرض سهمی از اعتبار (هرچند کوچک) دریافت می‌کند.

اسپمرها با سوءاستفاده از این ویژگی و ساخت میلیون‌ها صفحه داینامیک که به صورت متقاطع به هم لینک داده‌اند (مزارع لینک یا **Link Farms**)، این اعتبارهای خرد را در یک چرخه بسته به نام سیاه‌چاله رتبه (**RankSink**) جمع‌آوری کرده و رتبه یک سایت هدف را به صورت مصنوعی بالا می‌برند. الگوریتم **TrustRank** پاسخی مهندسی برای تغییر این رویکرد دموکراتیک به یک «الیگارش مبتنی بر اعتماد» است تا جلوی تولید اعتبار از هیچ را بگیرد.

بخش 2: الف) ایده اصلی TrustRank و انتخاب مجموعه هسته (Seed Set)

ایده اصلی:

الگوریتم **TrustRank** بر پایه یک اصل بنیادین و جامعه‌شناختی در گراف وب بنا شده است: «**صفحات خوب و معتبر، به ندرت به صفحات اسپم و مخرب لینک می‌دهند.**»

در این الگوریتم، به جای اینکه فرض کنیم همه صفحات گراف در ابتدا دارای اعتباری برابر هستند، اعتماد تنها از یک مجموعه بسیار کوچک و کاملاً تاییدشده به نام «مجموعه هسته» (**Seed Set**) سرچشمه می‌گیرد و از طریق لینک‌های خروجی در طول گراف جریان می‌یابد.

نحوه انتخاب Seed Set:

انتخاب این مجموعه نمی‌تواند صرفاً خودکار باشد و نیازمند یک ماشین ارزیاب انسانی (Oracle) است. اما از آنجایی که انسان نمی‌تواند کل وب (با میلیاردها صفحه) را دستی بررسی کند، یک فرآیند دو مرحله‌ای ترکیبی طی می‌شود:

1. فیلتر ماشینی: (**Inverse PageRank**) ابتدا الگوریتم صفحاتی را پیدا می‌کند که به بیشترین صفحات دیگر در وب

لینک داده‌اند (یعنی دسترسی و نفوذ بالایی به گراف دارند و به عنوان هاب‌های اطلاعاتی شناخته می‌شوند).

2. بازبینی انسانی: متخصصان انسانی این لیست فیلترشده را بررسی کرده و سایت‌های کاملاً معتبر (مانند دامنه‌های

.edu، **.gov**، دانشگاه‌های معتبر، خبرگزاری‌های رسمی و دایرکتوری‌های تاییدشده) را به عنوان دانه‌های اعتماد

(**Seed Set**) انتخاب و به سیستم معرفی می‌کنند.

بخش 3: نمودار شماتیک مقایسه‌ای (PageRank در برابر TrustRank)

تفاوت بنیادین در نحوه تولید و جریان اعتبار

ویژگی سیستم	الگوریتم PageRank (دموکراتیک)	الگوریتم TrustRank (الیگارش)
مقداردهی اولیه	همه صفحات وب مقدار $1/N$ دریافت می‌کنند	فقط صفحات Seed Set مقدار می‌گیرند، بقیه صفر
بردار پرش تصادفی	کاربر می‌تواند به هر صفحه‌ای در کل وب بپرد	کاربر فقط به صفحات تاییدشده (هسته) تله‌پورت می‌کند
آسیب‌پذیری	مزارع لینک به راحتی تولید اعتبار کاذب می‌کنند	مزارع لینک ایزوله شده و اعتبارشان صفر می‌ماند

بخش 4: مکانیزم انتشار اعتماد و خنثی‌سازی مزارع لینک

جریان یافتن اعتماد در گراف وب، از نظر ریاضی شباهت بسیار زیادی به **PageRank** دارد، با این تفاوت کلیدی که بردار پرش تصادفی (**Teleportation Vector**) تنها محدود به مجموعه هسته است. مکانیزم انتشار طی قوانین متوالی زیر عمل می‌کند:

1. مقداردهی اولیه (Initialization)

به تمام صفحات موجود در **Seed Set** مقدار اعتماد مشخصی (مثلاً 1) داده می‌شود و مقدار اعتماد (**Trust**) تمامی صفحات دیگر وب در لحظه صفر، مطلقاً 0 در نظر گرفته می‌شود.

2. تسهیم و توزیع اعتماد (Trust Splitting)

هر صفحه معتبر، اعتبار خود را به طور مساوی بین لینک‌های خروجی‌اش تقسیم می‌کند. اگر سایت دانشگاه هاروارد (با کل اعتماد **T**) به 10 سایت مختلف لینک داده باشد، به هر کدام از آن‌ها دقیقاً اعتماد **T/10** منتقل می‌شود.

3. میرایی در طول مسیر (Trust Attenuation)

اعتماد در وب با هر پرش (**Hop**) کاهش می‌یابد. در هر مرحله از انتشار، مقدار اعتماد عبوری در یک «ضریب میرایی» (**Damping Factor**) که معمولاً 0.85 در نظر گرفته می‌شود، ضرب می‌گردد. این یعنی هرچه یک سایت فاصله کلیک (**Distance**) بیشتری از هسته مورد اعتماد داشته باشد، در نهایت اعتماد کمتری به آن می‌رسد.

فرمول ریاضی TrustRank:

با توجه به قوانین سه‌گانه بالا، ساختار ریاضی این الگوریتم به فرم زیر کامل می‌شود:

$$TR(p) = \alpha \sum_{q \in IN(p)} \frac{TR(q)}{|OUT(q)|} + (1 - \alpha)d(p)$$

در این معادله تعادل، متغیر **d(p)** بردار توزیع اولیه است و تنها در صورتی مقداری بزرگتر از صفر دارد که صفحه **p** مستقیماً عضو مجموعه هسته (**Seed Set**) باشد؛ در غیر این صورت مقدار آن 0 خواهد بود. متغیر α نیز همان ضریب میرایی است.

بخش 5: چرا سایتی با هزاران بک‌لینک اسپم رتبه پایینی می‌گیرد؟

در یک مزرعه لینک (**Link Farm**)، اسپمرها میلیون‌ها لینک داخلی بین صفحات خود ایجاد می‌کنند. در **PageRank** عادی، این چرخه باعث هم‌افزایی و تولید اعتبار از هیچ می‌شود. اما در الگوریتم **TrustRank**، معادله به دلایل زیر تغییر می‌کند:

1. **فقدان اعتماد اولیه:** هیچ‌کدام از صفحات اسپم در **Seed Set** قرار ندارند، بنابراین مقدار اولیه اعتماد آن‌ها **صفر** است.

2. **ایزوله بودن گراف:** از آنجا که سایت‌های معتبر (نظیر سایت‌های دولتی یا دانشگاهی) هرگز به مزارع لینکِ داروی تقلبی یا سایت‌های قمار لینک نمی‌دهند، هیچ اعتباری از خارجِ سیستم وارد این چرخه اسپم نمی‌شود.

3. **حاصل‌ضرب صفر:** در نتیجه، الگوریتم **TrustRank** در حال محاسبه توزیع اعتمادی است که ورودی آن صفر است. حتی اگر اسپمرها 100 میلیون لینک متقاطع (**Cross-link**) به یکدیگر بدهند، حاصل‌ضربِ میلیون‌ها لینک در مقدار اعتماد 0، باز هم **صفر** خواهد بود و سایت هدف رتبه‌ای کسب نمی‌کند.

نکته کلیدی و دفاع لایه‌ای:

حتی اگر اسپمر موفق شود با نفوذ به سایت‌های دیگر، یک بک‌لینک از بخش نظرات (**Comments**) یک وبلاگ معتبر دریافت کند، به دلیل مکانیزم **میرایی (Attenuation)** و مهم‌تر از آن **تسهیم (Splitting)** در بین هزاران کامنتِ دیگرِ موجود در آن وبلاگ، مقدار اعتمادِ بسیار ناچیز و رقیقی به مزرعه لینک می‌رسد که برای بالا بردن رتبه سایت هدف در نتایج جستجو به هیچ‌وجه کافی نیست.

پاسخنامه مسئله 19.32: معماری موتور جستجوی درنگام (Real-time Search Architecture)

بخش 1: تحلیل صورت مسئله

در رویدادهای زنده (**Breaking News**)، معماری سنتی موتورهای جستجو (که بر پایه خزش دوره‌ای، پردازش دسته‌ای یا **Batch Processing** و محاسبه گراف عظیم **PageRank** استوار است) به طور کامل شکست می‌خورد. وقتی زلزله‌ای رخ می‌دهد، کاربران به دنبال اخباری هستند که ثانیه‌هایی پیش منتشر شده‌اند، در حالی که این صفحات هنوز بک‌لینکی دریافت نکرده‌اند تا **PageRank** آن‌ها محاسبه و تثبیت شود.

برای حل این تضادِ ساختاری بین «تازگی» و «اعتبار»، مهندسان سیستم‌های بازیابی اطلاعات از معماری دوگانه (شبیه به معماری لاندا - **Lambda Architecture**) استفاده می‌کنند. در این الگو، ما یک سیستم یکپارچه نداریم، بلکه دو موتور جستجوی کاملاً مجزا داریم که به صورت موازی کار می‌کنند و نتایج آن‌ها در لایه کاربری با یکدیگر ادغام می‌شود.

بخش 2: طراحی دو لایه ایندکس (Dual-Index Architecture)

قبل از بررسی مکانیزم ادغام، باید تفاوت ساختاری این دو لایه را به عنوان دو پایگاه داده با ماموریت‌های کاملاً متفاوت درک کنیم:

1. لایه عمیق (Deep / Historical Layer)

وظیفه: پردازش و نگهداری میلیاردها صفحه از تاریخچه کامل وب.

ذخیره‌سازی: کاملاً مبتنی بر دیسک (مانند سیستم فایل توزیع‌شده **HDFS**) و حفظ ایندکس‌های معکوس توزیع‌شده و بسیار حجیم.

الگوریتم رتبه‌بندی: محاسبه کامل الگوریتم‌های **PageRank**، **TrustRank** و تحلیل عمیق محتوا با استفاده از مدل‌های **TF-IDF** و **BM25**.

تأخیر به‌روزرسانی (**Latency**): چرخه‌های پردازشی روزانه تا هفتگی.

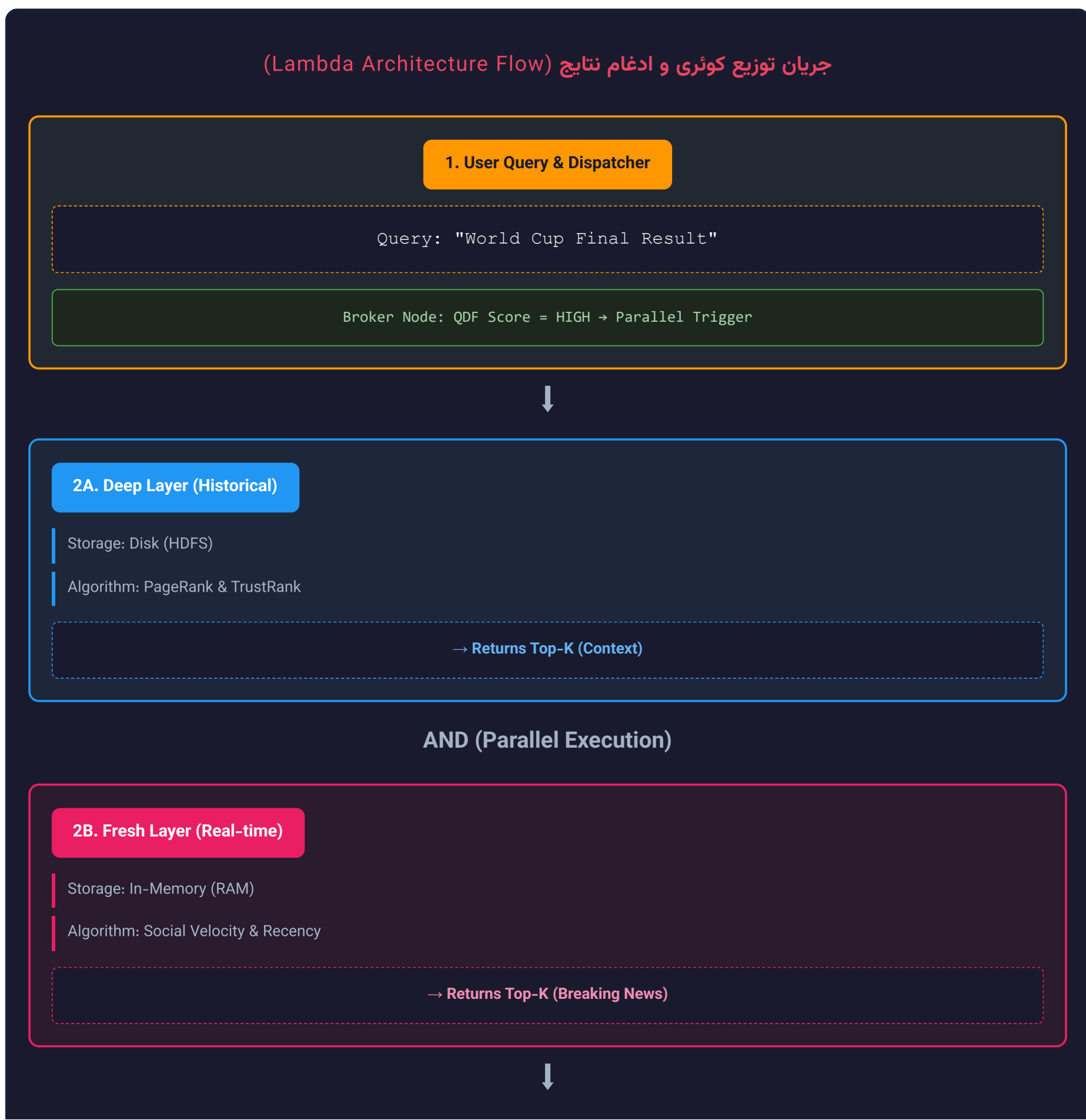
2. لایه تازگی (Freshness / Real-time Layer)

وظیفه: ایندکس کردن محتوای در حال تولید (مانند فیدهای **RSS** خبرگزاری‌ها، شبکه‌های اجتماعی و خروجی‌های پروتکل **WebSub**) در کمتر از **1** ثانیه.

ذخیره‌سازی: تماماً مبتنی بر حافظه اصلی (**In-Memory RAM**) برای رسیدن به سرعت خواندن و نوشتن حداکثری. داده‌های این لایه معمولاً پس از گذشت **24** تا **72** ساعت تخلیه شده و به لایه عمیق منتقل می‌گردند.

الگوریتم رتبه‌بندی: در این محیط پرسرعت، مفهوم بک‌لینک تقریباً فاقد کارایی است. رتبه‌بندی تماماً بر اساس «تازگی زمانی» (**Recency**) و «سیگنال‌های شبکه‌های اجتماعی» (**Social Velocity**) (مانند نرخ بازنشر شدن یک خبر در هر دقیقه) کالیبره می‌شود.

بخش 3: نمودار شماتیک جریان توزیع در معماری دوگانه (Federated Pipeline)



3. Blending (LTR Engine)

Score Calibration & Exponential Time-Decay



4. Final SERP Rendering

News Carousel (Top Priority)

Historical/Wiki Context (Bottom)

بخش 4: مکانیزم ادغام درنگام (Sub-second Merging Mechanism)

هنگامی که کاربر دکمه جستجو را می‌فشارد، سیستم فرصت ندارد تا دو دیتابیس عظیم را به صورت فیزیکی با هم **Join** کند. در عوض، از معماری **جستجوی یکپارچه / نماینده (Federated Search)** استفاده می‌شود. یک قطعه نرم‌افزاری توزیع‌کننده به نام **Aggregator** (یا **Broker**) فرآیند ادغام را طی **4** گام زیر مدیریت می‌کند:

گام اول: تشخیص (QDF (Query Deserves Freshness)

به محض دریافت کوئری، یک مدل یادگیری ماشین سبک بررسی می‌کند که آیا این کلمه کلیدی در حال حاضر دچار «جهش ناگهانی جستجو» (**Spike**) شده است یا خیر. مثلاً کوئری «نتیجه فینال جام جهانی» دارای نمره **QDF** بالایی است، اما کوئری «تاریخچه پیدایش خط» نمره **QDF** پایینی دارد. این نمره تعیین می‌کند که در گام‌های بعدی، وزن لایه **Freshness** چقدر باید افزایش یابد.

گام دوم: جستجوی موازی (Parallel Dispatch)

سیستم **Broker**، کوئری کاربر را به صورت همزمان (**Asynchronous**) برای هر دو لایه **Deep** و **Freshness** ارسال می‌کند. هر لایه، به طور کاملاً مستقل در دیتابیس خود جستجو کرده و **Top-K** نتیجه برتر خود را (مثلاً **20** نتیجه اول) به همراه امتیاز (**Score**) به **Broker** برمی‌گرداند.

گام سوم: نرمال‌سازی و اعمال تابع میرایی (Time-Decay)

امتیاز لایه **Deep** بر اساس **PageRank** است، اما امتیاز لایه **Freshness** بر اساس زمان است؛ این دو مقیاس کاملاً متفاوت باید یکسان شوند (**Score Calibration**). در این مرحله، یک تابع میرایی نمایی (**Exponential Decay**) روی نتایج اعمال می‌شود. فرمول پایه ریاضی برای کالیبره کردن امتیازات معمولاً به شکل زیر است:

$$Final_Score = Initial_Score \times e^{-\lambda \Delta t}$$

که در آن Δt عمر صفحه (زمان سپری‌شده از انتشار) و λ ضریبی است که مستقیماً توسط نمره **QDF** (از گام اول) تعیین می‌شود. برای رویدادهای زنده، λ بسیار بزرگ تنظیم می‌شود تا صفحاتی که حتی چند ساعت قدیمی‌تر هستند، به شدت

گام چهارم: ترکیب نهایی (Blending) و رندر SERP

مدل یادگیری رتبه‌بندی نهایی (**Learning to Rank - LTR**) در لایه **Broker**، نتایج نرمال‌شده از هر دو لایه را در یک صف اولویت واحد (**Priority Queue**) در هم می‌آمیزد. در صورت بالا بودن نمره **QDF**، نتایج لایه **Freshness** در صدر بلوک‌های خبری (مانند کاروسل‌های **Top Stories**) قرار گرفته و نتایج لایه **Deep** به عنوان محتوای پس‌زمینه و مرجع (**Background Context**) در پایین صفحه نمایش داده می‌شوند.

بخش 5: جمع‌بندی

طراحی یک سیستم بازیابی اطلاعات دوگانه (**Dual-Index Architecture**) که بر پایه الگوهای پردازشی نظیر معماری لاندا (**Lambda Architecture**) بنا شده باشد، تلاشی مهندسی برای آشتی دادن دو قطب متضاد «کیفیت عمیق تاریخی» و «واکنش‌پذیری درنگام» است. در این زیست‌بوم، معماری لایه تازگی (**Freshness Layer**) نیازمند مدیریت فوق‌العاده حساس منابع سخت‌افزاری است؛ چرا که حافظه موقت **RAM** در مقایسه با دیسک‌های سنتی بسیار گران‌قیمت بوده و ظرفیت محدودی دارد. در نتیجه، مهندسان سیستم باید استراتژی‌های پیشرفته‌ای برای سیاست‌های تخلیه حافظه (**Eviction Policies**)، نظیر مکانیزم‌های **LRU (Least Recently Used)** تدوین کنند تا داده‌های کهنه بدون ایجاد وقفه به لایه ذخیره‌سازی دیسک (**Cold Storage**) سرازیر شوند. از سوی دیگر، هنر واقعی یک موتور جستجوی مدرن در منطق ادغام‌گر یا همان گره توزیع‌کننده (**Broker / Federated Search**) نهفته است. بهره‌گیری از توابع میرایی نمایی (**Exponential Time-Decay Functions**) در کنار مدل‌های پیش‌بینی **QDF**، به موتور جستجو این هوشمندی را می‌بخشد که در کسری از ثانیه تشخیص دهد آیا کاربر به دنبال هیجانات لحظه‌ای یک رویداد ورزشی/سیاسی است، یا نیازمند مقالات مرجع و آکادمیک در لایه‌های عمیق تاریخ وب است.

پاسخنامه مسئله 19.33: اسپم معنایی و عبور از سد شینگلینگ (Semantic Spam & Shingling Bypass)

بخش 1: تحلیل شکست الگوریتم‌های واژگانی در برابر Content Spinning

پاسخ به بخش اول: خیر.

الگوریتم‌های کلاسیک مبتنی بر **k-shingling**، ژاکارد و **MinHash** در برابر این نوع از اسپم پیشرفته یعنی چرخش محتوا (**Content Spinning**) کاملاً خلع سلاح و نابینا هستند.

دلیل ریاضی و ساختاری این شکست به ماهیت «تطابق دقیق واژگانی» (**Exact Lexical Matching**) در شینگلینگ برمی‌گردد. زمانی که یک متن با طول شینگل k (مثلاً $k = 5$) قطعه‌بندی می‌شود، تغییر دادن حتی یک کلمه با مترادف آن، تمام 5 شینگلی را که آن کلمه در آن‌ها حضور داشته است به طور کامل تخریب می‌کند.

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \approx 0$$

در مثال شما، شینگل "the fast car" از نظر الگوریتم رشته‌ای کاملاً و 100% متفاوت از "the quick automobile" است. اسپرها با جایگذاری سیستمی مترادف‌ها در فواصل منظم، باعث می‌شوند اشتراک مجموعه‌ها یعنی $|A \cap B|$ به شدت و با نرخ نمایی افت کند. در نتیجه، ضریب ژاکارد به مقادیر بسیار پایینی سقوط کرده و سیستم فریب می‌خورد؛ به طوری که 2 مقاله با معنای کاملاً یکسان، توسط موتور جستجو به عنوان 2 محتوای کاملاً مستقل و اورجینال دسته‌بندی می‌شوند.

بخش 2: نمودار شماتیک تفاوت فضای واژگانی و فضای معنایی

مقایسه رفتار الگوریتم‌ها در برابر اسپم جایگزینی مترادف

رویکرد سنتی: فضای نحوی (MinHash / Jaccard)

String Inputs:

A: "The fast car"

B: "The quick automobile"

Jaccard = 0.00

(شکست سیستم - صفحات نامرتبط تشخیص داده شدند)

VS

رویکرد مدرن: فضای معنایی (Dense Embeddings)

Dense Vector Inputs (768-D):

v_A: [0.12, 0.45, -0.89, ...]

v_B: [0.11, 0.47, -0.85, ...]

Cosine Similarity = 0.96

(موفقیت سیستم - کپی معنایی کشف شد)

بخش 3: معماری جایگزین، رویکرد مبتنی بر NLP و بردارهای متراکم

برای کشف تقلب در «فضای معنایی» (Semantic Space) به جای «فضای نحوی» (Syntactic Space)، معماری موتور جستجو باید از پردازش کلمات گسسته عبور کرده و به درک پیوسته‌ی زبان دست یابد. معماری پیشنهادی برای جایگزینی پایپ‌لاین MinHash، سیستم Vector-based Semantic Deduplication است که در 3 گام اصلی پیاده‌سازی می‌شود:

گام 1: تولید بردارهای متراکم (Embedding Generation)

به جای خرد کردن سند به شینگل‌ها، متن صفحه به یک مدل زبانی مبتنی بر معماری ترانسفورمر (Transformer) نظیر Sentence-BERT یا معماری‌های نوین تعبیه‌سازی (Document Embeddings) داده می‌شود.

این مدل‌ها، کل یک پاراگراف یا سند را به یک بردار متراکم با طول ثابت (مثلاً یک بردار **768** بُعدی از اعداد اعشاری پیوسته) تبدیل می‌کنند. جادوی این مدل‌ها در این است که کلمات و جملات هم‌معنی در فضای برداری به هم نزدیک هستند. در نتیجه، بردار تولید شده برای **"The fast car"** و **"The quick automobile"** تقریباً با هم یکسان (دارای همپوشانی برداری بالا) خواهند بود، صرف‌نظر از اینکه از چه کلمات ظاهری استفاده شده است.

گام 2: نمایه‌سازی برداری و جستجوی تقریبی (Vector Indexing & ANN)

همانند الگوریتم **MinHash** که از **LSH** برای فرار از مقایسه $O(N^2)$ استفاده می‌کرد، در فضای برداری نیز می‌توانیم میلیاردها بردار را دو به دو مقایسه کنیم. در این لایه، از الگوریتم‌های پیشرفته **جستجوی نزدیک‌ترین همسایه تقریبی (Approximate Nearest Neighbor - ANN)** استفاده می‌شود.

ساختارهای گراف‌محور مانند **HNSW** (Hierarchical Navigable Small World) یا فریم‌ورک‌هایی نظیر **FAISS** (توسعه‌یافته توسط فیسبوک)، بردارهای مشابه را در خوشه‌ها و گراف‌های چندلایه‌ای سازمان‌دهی می‌کنند تا جستجوی صفحات مشابه در زمان لگاریتمی یعنی $O(\log N)$ انجام شود.

گام 3: جایگزینی کسینوس به جای ژاکارد (Cosine Similarity Metric)

در مرحله نهایی ارزیابی، معیار شباهت از اشتراک مجموعه‌های گسسته (**Jaccard**) به شباهت کسینوسی پیوسته (**Cosine Similarity**) تغییر می‌کند. سیستم، زاویه بین **2** بردار را در فضای **768** بُعدی با فرمول زیر محاسبه می‌کند:

$$\text{Cosine}(\vec{v}_1, \vec{v}_2) = \frac{\vec{v}_1 \cdot \vec{v}_2}{|\vec{v}_1| |\vec{v}_2|}$$

اگر مقدار خروجی این فرمول بزرگتر از یک آستانه مشخص (مثلاً **0.92**) باشد، الگوریتم به طور قطعی تایید می‌کند که این **2** سند دارای یک «روح معنایی مشترک» هستند و یکی از آن‌ها کپی بازنویسی‌شده (**Spun Content**) محسوب شده و از گراف ایندکس هرس (**Prune**) می‌شود.

بخش 4: جمع‌بندی

گذر از الگوریتم‌های سنتی تطابق واژگانی نظیر شینگلینگ به معماری‌های نوین مبتنی بر معنا، یکی از بزرگترین شیفت‌های پارادایمی در مهندسی سیستم‌های بازیابی اطلاعات محسوب می‌گردد. در ارزیابی پروژه‌های دانشگاهی و سیستم‌های داوری خودکار (**Autograders**)، تفکیک دقیق مفاهیمی چون «سرقت علمی عینی» و «بازنویسی ساختاری» ضرورتی اجتناب‌ناپذیر است. الگوریتم‌هایی مانند **Jaccard** و **MinHash** اگرچه برای حذف کپی‌های قطعی و دقیق در مقیاس‌های پتابایتی بی‌نظیر عمل می‌کنند، اما در برابر تکنیک‌های چرخش محتوا (**Content Spinning**) دچار نابینایی ساختاری هستند.

به همین منظور، معماری موتورهای جستجوی مدرن امروزی از یک خط‌لوله ترکیبی پیچیده بهره می‌برد. ابتدا فیلترهای فوق‌سریع واژگانی برای دفع نسخه‌های تکراری قطعی (**Exact Duplicates**) با پیچیدگی پردازشی کم وارد عمل می‌شوند. سپس در لایه عمیق‌تر، پایگاه‌داده‌های برداری (**Vector Databases**) و مدل‌های زبانی ترانسفورمر، وظیفه‌ی کشف

تقلب‌های معنایی پیشرفته (**Semantic Near-Duplicates**) را با بهره‌گیری از معیارهای هندسی نظیر شباهت کسینوسی بر عهده می‌گیرند تا کیفیت و اصالت خروجی جستجو، در بالاترین سطح ممکن تضمین گردد.

پاسخنامه مسئله 19.34: مهندسی معکوس کوئری‌های تصادفی و اصلاح سوگیری نمونه‌گیری

(Query Sampling Bias Correction)

بخش 1: تحلیل صورت مسئله و رویکرد ارزیابی

این سوال یکی از بهترین محک‌ها برای سنجش درک تحلیلی از «تئوری نمونه‌گیری احتمالی» و آشنایی با «توزیع‌های آماری کلان‌داده‌ها» در معماری سیستم‌های بازیابی اطلاعات است.

در روش کلاسیک صید و بازصید (**Capture-Recapture**) برای تخمین اندازه وب، فرض بنیادین این است که هر صفحه وب دارای شانس برابری برای استخراج (**Uniform Sampling**) باشد. دانشجو باید بتواند به صورت ریاضی نشان دهد که تضاد بین «توزیع یکنواخت کلمات در فرهنگ لغت» و «توزیع توانی کلمات در دنیای واقعی»، چگونه این فرض را کاملاً نابود می‌کند و چگونه می‌توان با استفاده از منطق وزن‌دهی معکوس (**Inverse Weighting**) این خطای آماری را جبران کرد.

بخش 2: تحلیل خطای نمونه‌گیری (Sampling Bias Error)

اگر کلمات به صورت کاملاً تصادفی و با احتمال برابر (**Uniform**) از یک فرهنگ لغت انتخاب شوند، سیستم دچار یک سوگیری مهلک و سیستماتیک به نام **سوگیری انتخاب (Selection Bias)** ناشی از نقض قانون زیف (**Zipf's Law**) می‌شود. این خطا به شکل مخرب خود را در نتایج موتور جستجو نشان می‌دهد:

1. تورم صفحات حاشیه‌ای (Marginal Page Over-representation)

در یک فرهنگ لغت استاندارد، اکثریت قریب به اتفاق کلمات را واژگان تخصصی، مهجور و بسیار کم‌کاربرد تشکیل می‌دهند. اگر احتمال انتخاب هر کلمه $1/|V|$ (اندازه کل فرهنگ لغت) باشد، کوئری‌های تولید شده اکثراً شامل کلمات بسیار نادر خواهند بود.

در نتیجه، موتور جستجو صفحاتی را برمی‌گرداند که شامل این واژگان نادر هستند (مانند دیکشنری‌های آنلاین، مقالات بسیار تخصصی یا صفحات اسپم تولیدشده توسط ماشین)، و صفحات وبِ نرمال و کاربردی که با کلمات متداول نوشته شده‌اند، عملاً از چرخه نمونه‌گیری حذف و ایزوله می‌شوند.

2. سوگیری طول سند (Document Length Bias)

صفحات طولانی‌تر، کلمات منحصربه‌فرد (**Unique Terms**) به مراتب بیشتری در خود دارند. اگر کوئری ما تصادفی باشد، شانس اینکه یک مقاله 10,000 کلمه‌ای حداقل یکی از آن کلمات تصادفی را در خود داشته باشد، بسیار بیشتر از یک صفحه وبلاگ کوتاه 200 کلمه‌ای است. این موضوع باعث می‌شود نمونه‌گیری به سمت اسناد بسیار طولانی متمایل شود.

بخش 3: نمودار شماتیک تقابل توزیع یکنواخت و قانون زیف

تضاد ساختاری در مدل‌های تولید کوئری تصادفی

فضای اسناد وب (قانون توانی / زیف)

رخداد در وب: $P(w) \propto 1/rank(w)$

- تعداد کمی کلمه متداول کل وب را ساختند.
- کلمات نادر در صفحات بسیار کمی هستند.
- خروجی: استخراج صفحات غیرطبیعی و حاشیه‌ای.

فضای فرهنگ لغت (توزیع یکنواخت)

احتمال انتخاب کلمه: $P(w) = 1/|V|$

- همه کلمات شانس برابری دارند.
- 80% فرهنگ لغت کلمات نادر است.
- کوئری‌ها مملو از لغات مهجور می‌شوند.



نتیجه نهایی این تضاد: Selection Bias
ضرورت استفاده از وزن‌دهی معکوس برای بازگرداندن تعادل آماری

بخش 4: پیشنهاد فرمول اصلاحی (Correction Weighting - IPW)

برای خنثی کردن این سوگیری‌ها و تبدیل یک «نمونه‌گیری دارای تورش» به یک «نمونه‌گیری یکنواخت شبه‌تصادفی»، باید از تکنیک آماری وزن‌دهی احتمال معکوس (Inverse Probability Weighting - IPW) استفاده کنیم. در این مکانیزم، ما به صفحاتی که شانس انتخاب شدن بالایی داشته‌اند وزن کمتری می‌دهیم و برعکس.

این پروسه در 4 گام تحلیلی زیر تکمیل می‌شود:

گام 1: تعریف متغیرهای پایه

- $P_{dict}(w)$: احتمال انتخاب کلمه w از فرهنگ لغت (معمولاً توزیع یکنواخت $1/|V|$).
- $P_{web}(w)$: احتمال واقعی رخداد کلمه w در کل اسناد وب (بر اساس توزیع زیف).
- D : سند استخراج شده توسط موتور جستجو.

گام 2: محاسبه احتمال شمول سند (Inclusion Probability)

احتمال اینکه سند D در فرآیند نمونه‌گیری ما توسط یکی از کلمات تصادفی شکار شود، متناسب است با مجموع احتمالات انتخاب کلماتی که درون آن سند وجود دارند. بنابراین احتمال شکار شدن سند D (با فرض استقلال نسبی) برابر است با:

$$P_{inclusion}(D) \approx \sum_{w \in D} P_{dict}(w)$$

گام 3: اعمال فرمول وزن اصلاحی (Correction Weight)

برای اینکه تاثیر سوگیری طول سند و فراوانی کلمات خنثی شود، به هر سند D که از طریق این کوئری‌ها استخراج می‌شود، یک وزن اصلاحی $W(D)$ اختصاص می‌دهیم. این وزن، به سادگی معکوس احتمال شمول آن سند است:

$$W(D) = \frac{1}{P_{inclusion}(D)} = \frac{1}{\sum_{w \in D} P_{dict}(w)}$$

گام 4: تصحیح نهایی در فرمول Capture-Recapture

حال به جای اینکه در فرمول صید و بازصید، اسناد استخراج شده را به عنوان "1 واحد" بشماریم، آن‌ها را با وزن تعدیل‌شده $W(D)$ در محاسبات دخالت می‌دهیم.

اگر صفحه‌ای بسیار طولانی باشد یا شامل کلمات بسیار متداول باشد (که شانس استخراج بالایی دارد)، مخرج کسر در گام 3 بزرگ شده و وزن سند به شدت کاهش می‌یابد (مثلاً 0.01). در مقابل، به یک صفحه کوتاه که شانس استخراج پایینی داشته، وزن جبرانی بیشتری (مثلاً 5.0) داده می‌شود تا توزیع نهایی صفحات کاملاً متعادل و نماینده واقعی وب گردد.

بخش 5: جمع‌بندی و نکات کلیدی

در مهندسی ارزیابی سیستم‌های جستجو، طراحی کوئری کاملاً بدون سوگیری در مقیاس وب تقریباً غیرممکن است.

راه‌حل مهندسی این نیست که به صورت فیزیکی جلوی سوگیری کوئری‌ها را بگیریم، بلکه راه‌حل هوشمندانه این است که سوگیری را با استفاده از توزیع پیشین (Prior Distribution) کلمات در کل گراف وب به صورت ریاضی مدلسازی کرده و نتایج را در لایه پردازش نهایی با اعمال وزن‌های کسری (Fractional Weights) کالیبره و اصلاح کنیم.

پاسخنامه مسئله 19.35: شخصی‌سازی نتایج جستجو و شکستن حباب فیلتر

(Filter Bubble & Personalization)

بخش 1: تحلیل صورت مسئله

پدیده «حباب فیلتر» (Filter Bubble) زمانی رخ می‌دهد که الگوریتم‌های یادگیری ماشین به شدت روی داده‌های تاریخی کاربر (User Profile) بیش‌برازش (Overfit) می‌شوند. در این حالت، موتور جستجو برای حداکثر کردن نرخ کلیک (CTR) در کوتاه‌مدت، تنها محتوایی را به کاربر نشان می‌دهد که با سوگیری‌های تایید قبلی او (Confirmation Bias) همسو باشد و او را در یک اتاق پژواک (Echo Chamber) اطلاعاتی محبوس می‌کند.

به عنوان یک طراح الگوریتم، وظیفه ما تبدیل این مسئله از یک «بهینه‌سازی تک‌هدفه» (که در آن فقط ارتباط یا Relevance مهم است) به یک مسئله پیچیده‌تر «موازنه اکتشاف و بهره‌برداری» (Exploration vs. Exploitation) است. ما باید تابع هدفی طراحی کنیم که همزمان با پاسخ دقیق به نیاز اصلی کاربر، او را با دیدگاه‌های جدید یا محتوای غیرمنتظره اما مفید (Serendipity) مواجه کند.

بخش 2: نمودار شماتیک معماری موازنه (کشف در برابر بهره‌برداری)

تجزیه و تحلیل بلوک‌های سازنده تابع مطلوبیت (Utility Function Components)

بلوک الگوریتمی	نماد ریاضی	وظیفه در سیستم	خطر در صورت افراط
بهره‌برداری (Exploitation)	$Rel(d, q, u)$	یافتن دقیق‌ترین پاسخ: تمرکز بر تاریخچه کاربر و ارتباط مستقیم با کوئری برای افزایش CTR.	ایجاد حباب فیلتر
اکتشاف / نوآوری (Novelty)	$Nov(d, u)$	شکستن حباب: پیشنهاد محتوای دور از تاریخچه کاربر (اما مرتبط با کوئری) برای کشف دیدگاه‌های جدید.	کاهش ارتباط (Relevance)
تنوع درون‌صفحه‌ای (Diversity)	$- Sim(d, d')$	جریمه تکرار: جلوگیری از نمایش نتایجی که شبیه به نتایج رتبه‌های بالاتر همان صفحه هستند.	حذف منابع تاییدکننده مهم

بخش 3: طراحی تابع هدف ریاضی (Objective Function)

در سیستم‌های رتبه‌بندی مدرن، به جای تابع هزینه (که باید حداقل شود)، معمولاً از یک تابع مطلوبیت (Utility Function) استفاده می‌شود که الگوریتم ماشین لرنینگ در تلاش برای بیشینه‌سازی (Maximize) آن است.

ما تابع رتبه‌بندی $Score(d)$ را برای ارزیابی و امتیازدهی سند d جهت اضافه شدن به مجموعه صفحه نتایج S ، به صورت ترکیبی از 3 مولفه اصلی که در جدول بالا معرفی شد، به شکل ریاضی زیر تعریف می‌کنیم:

$$Score(d) = \alpha \cdot Rel(d, q, u) + \beta \cdot Nov(d, u) - \gamma \cdot \max_{d' \in S} Sim(d, d')$$

برای درک دقیق این معماری، این تابع را به 3 بلوک پردازشی زیر تجزیه می‌کنیم:

1. بلوک اول: بهره‌برداری و ارتباط شخصی‌سازی شده (Exploitation)

$$\alpha \cdot Rel(d, q, u)$$

این بخش نمایانگر الگوریتم‌های کلاسیک شخصی‌سازی است. تابع Rel احتمال کلیک کاربر روی سند d را با توجه به کوئری فعلی q و پروفایل تاریخی کاربر u محاسبه می‌کند.

اگر کوئری از نوع ناوبری (Navigational) باشد (مثلاً کاربر کلمه "Github" را سرچ کند)، این بخش باید بیشترین وزن (یعنی ضریب α بالا) را داشته باشد، زیرا کاربر دقیقاً به دنبال یک سایت مشخص است و نیازی به کشف عقاید جدید در مورد گیت‌هاب ندارد.

2. بلوک دوم: کشف و شکستن حباب (Exploration / Serendipity)

$$\beta \cdot Nov(d, u)$$

این بخش، قلب تپنده مبارزه با حباب فیلتر است. تابع Nov (مخفف Novelty) فاصله معنایی سند d را با «مرکز ثقل» تاریخچه مطالعاتی کاربر در فضای برداری (Embedding Space) محاسبه می‌کند.

هرچه یک سند از نظر دیدگاه سیاسی، اجتماعی یا علمی از تاریخچه کاربر دورتر باشد (اما همچنان به موضوع کوئری q مرتبط بماند)، امتیاز Nov بالاتری می‌گیرد. ضریب β به کاربر اجازه می‌دهد محتوایی را ببیند که «نمی‌دانست به آن نیاز دارد» و او را با ابعاد دیگر یک مسئله آشنا می‌کند.

3. بلوک سوم: جریمه تکرار و تنوع درون صفحه‌ای (Diversity Penalty)

$$- \gamma \cdot \max \text{Sim}(d, d')$$

این بخش برگرفته از منطق قدرتمند الگوریتم **MMR (Maximal Marginal Relevance)** است. تابع Sim شباهت سند جدید d را با اسنادی (d') که قبلاً برای نمایش در صفحه نتایج (S) انتخاب شده‌اند، می‌سنجد. علامت منفی ($-$) در این فرمول به عنوان یک «تابع جریمه» (**Penalty**) عمل می‌کند. اگر سند جدید حرف تکراری بزند و کاملاً شبیه لینک اول باشد (حتی اگر از نظر تابع Rel خیلی به کاربر مرتبط باشد)، جریمه شده و به رتبه‌های پایین‌تر سقوط می‌کند تا فضا برای نمایش دیدگاه‌های متنوع‌تر و لینک‌های غنی‌تر در همان صفحه باز شود.

بخش 4: پویایی ضرایب (Dynamic Parameter Tuning)

بزرگترین اشتباه مهندسی در طراحی چنین سیستمی، ثابت در نظر گرفتن ضرایب α, β, γ (Hardcoding) است. موتور جستجو باید با استفاده از یک طبقه‌بندِ نیتِ کوئری (**Query Intent Classifier**)، این ضرایب را به صورت درنگام و بلادرنگ (**Real-time**) تنظیم کند:

برای کوئری‌های قطعی و علمی: (Fact-finding)

مثل کوئری "دمای جوش آب". در اینجا باید تنظیمات به صورت $\alpha = 1$ و $\beta = 0$ اعمال شود، زیرا تنوع و کشف دیدگاه‌های جدید در حقایق مطلق علمی کاملاً بی‌معناست.

برای کوئری‌های مناقشه‌برانگیز: (Controversial / Exploratory)

مثل کوئری "مزایا و معایب انرژی هسته‌ای". در این سناریو، الگوریتم باید وزن β (نوآوری) و γ (تنوع) را به شدت افزایش دهد تا کاربر مقالاتی از هر 2 طیف موافق و مخالف دریافت کند و در یک حباب فکری ایزوله محبوس نشود.

بخش 5: جمع‌بندی و نکات کلیدی برای ارزیابی

حباب فیلتر صرفاً یک مشکل اخلاقی یا رسانه‌ای نیست، بلکه از نظر ریاضی یک **خطای بهینه‌سازی محلی (Local Optima)** در الگوریتم‌های یادگیری ماشین است. سیستمی که فقط محتوای آشنا و تکراری به کاربر نشان دهد، در درازمدت توانایی یادگیری از تغییراتِ سلیقه کاربر را به طور کامل از دست می‌دهد.

تزریق حساب‌شده‌ی «تصادفی‌بودن» (**Randomness**) و «نوآوری مطلوب» (**Serendipity**) در فرمول بالا، نه تنها حباب را می‌شکند، بلکه نرخ درگیری بلندمدت (**Long-term Engagement**) کاربر را با موتور جستجو به شکل پایداری افزایش می‌دهد.

کلمات کلیدی که باید به خاطر بسپارید:

حباب فیلتر (**Filter Bubble**)، موازنه اکتشاف/بهره‌برداری (**Exploration vs. Exploitation**)، یافتِ تصادفیِ مطلوب (**Serendipity**)، الگوریتم **MMR**.

پاسخنامه مسئله 19.36: پیاده‌سازی و محاسبات ماتریس MinHash (MinHash Matrix Computation)

بخش 1: تحلیل و معماری حل مسئله

هدف اصلی الگوریتم **MinHash** کاهش ابعادِ (**Dimensionality Reduction**) بردارهای عظیم بولی و تبدیل آن‌ها به امضاهای فشرده و کارآمد است، در حالی که تقریبِ دقیقی از شباهت ژاکارد (**Jaccard Similarity**) بین اسناد حفظ شود.

برای ساخت ماتریس امضا (**Signature Matrix**) به صورت دستی، به جای رویکرد رایج «پیمایش سطر به سطر» که در برنامه‌نویسی استفاده می‌شود، از «روش مجموعه‌های فعال» (**Active Sets Method**) استفاده می‌کنیم که برای انسان شهودی‌تر و سریع‌تر است. در این روش، برای هر سند فقط ردیف‌هایی (شینگل‌هایی) را در نظر می‌گیریم که مقدار آن‌ها 1 است (سند شامل آن ویژگی است) و سپس به سادگی **کمترین مقدار خروجی** هر تابع هش را برای آن مجموعه از ردیف‌های فعال محاسبه می‌کنیم.

بخش 2: محاسبات گام‌به‌گام و استخراج امضاها

فرآیند استخراج امضاها و تشکیل ماتریس نهایی در 4 گامِ تحلیلی زیر انجام می‌پذیرد:

گام 1: محاسبه مقادیر توابع هش برای تمامی اندیس‌ها

برای جلوگیری از محاسبات تکراری در مراحل بعد، ابتدا مقدار هر 3 تابع هش را برای تمامی شماره ردیف‌های موجود (اندیس x از 0 تا 4) محاسبه می‌کنیم تا از آن به عنوان یک **جدول جستجوی مرجع** (**Lookup Table**) استفاده کنیم. توابع داده شده در صورت مسئله عبارتند از:

- $h_1(x) = (x + 1) \bmod 5$
- $h_2(x) = (3x + 1) \bmod 5$
- $h_3(x) = (2x + 2) \bmod 5$

جدول مرجع: مقادیر محاسبه‌شده توابع هش

ردیف (x)	$h_1(x)$	$h_2(x)$	$h_3(x)$
0	1	1	2
1	2	4	4
2	3	2	1
3	4	0	3
4	0	3	0

گام 2: استخراج امضای سند اول (D_1)

با بررسی ماتریس اولیه متوجه می‌شویم که سند D_1 تنها در ردیف‌های $\{0, 2, 3\}$ دارای مقدار 1 است (مجموعه فعال). برای یافتن امضای این سند، با مراجعه به جدول مرجع، **کمترین مقدار** هر تابع هش را منحصراً در میان این سه ردیف فعال پیدا می‌کنیم:

$$Sig(h_1, D_1) = \min(h_1(0), h_1(2), h_1(3)) = \min(1, 3, 4) = 1$$

$$Sig(h_2, D_1) = \min(h_2(0), h_2(2), h_2(3)) = \min(1, 2, 0) = 0$$

$$Sig(h_3, D_1) = \min(h_3(0), h_3(2), h_3(3)) = \min(2, 1, 3) = 1$$

امضای استخراج شده برای سند D1: $[1, 0, 1]^T$

گام 3: استخراج امضای سند دوم (D2)

سند D_2 مجموعه فعال بزرگتری دارد و در ردیف‌های $\{0, 1, 3, 4\}$ دارای مقدار 1 است. مجدداً با استفاده از مقادیر جدول مرجع، کمترین مقدار هشت در میان این ردیف‌ها محاسبه می‌شود:

$$Sig(h_1, D_2) = \min(1, 2, 4, 0) = 0$$

$$Sig(h_2, D_2) = \min(1, 4, 0, 3) = 0$$

$$Sig(h_3, D_2) = \min(2, 4, 3, 0) = 0$$

امضای استخراج شده برای سند D2: $[0, 0, 0]^T$

گام 4: استخراج امضای سند سوم (D3)

در نهایت، سند D_3 تنها در ردیف‌های $\{2, 3\}$ دارای مقدار 1 است (یک مجموعه فعال کوچک). عملیات یافتن حداقل را برای این 2 ردیف تکرار می‌کنیم:

$$Sig(h_1, D_3) = \min(3, 4) = 3$$

$$Sig(h_2, D_3) = \min(2, 0) = 0$$

$$Sig(h_3, D_3) = \min(1, 3) = 1$$

امضای استخراج شده برای سند D3: $[3, 0, 1]^T$

بخش 3: ماتریس امضا نهایی (Final Signature Matrix)

با کنار هم قرار دادن ستونی بردارها و امضاهای به دست آمده، ماتریس فشرده 3×3 نهایی شکل می‌گیرد. این ماتریس جدید به طور کامل جایگزین ماتریس عظیم و اولیه سیستم خواهد شد:

ماتریس امضای فشرده MinHash

توابع هش	Doc 1	Doc 2	Doc 3
h_1	1	0	3
h_2	0	0	0
h_3	1	0	1

* ردیف‌های رنگی نشان‌دهنده تطابق در امضاها برای D1 و D3 است.

بخش 4: مقایسه شباهت ژاکارد (تقریبی در برابر واقعی)

مهمترین گام برای ارزیابی عملکرد الگوریتم **MinHash**، اعتبارسنجی آن است. اکنون شباهت ژاکارد بین اسناد D_1 و D_3 را از هر 2 روش تحلیلی (از فضای فشرده و از فضای اصلی) محاسبه و با دقت مقایسه می‌کنیم:

1. شباهت ژاکارد تقریبی (از روی ماتریس امضا):

برای محاسبه این شباهت تخمینی، تنها کفایت تعداد درایه‌هایی که مقادیر دقیقاً برابری دارند را شمرده و بر تعداد کل توابع هش (طول بردار امضا) تقسیم کنیم. ستون‌های D_1 و D_3 را سطر به سطر در ماتریس بالا مقایسه می‌کنیم:

- مقایسه تابع اول (h_1): $3 \neq 1 \rightarrow$ (عدم تطابق)
- مقایسه تابع دوم (h_2): $0 = 0 \rightarrow$ (تطابق کامل)
- مقایسه تابع سوم (h_3): $1 = 1 \rightarrow$ (تطابق کامل)

$$J_{approx}(D_1, D_3) = \frac{2 \text{ (Matches)}}{3 \text{ (Total Hashes)}} \approx 0.667$$

2. شباهت ژاکارد واقعی (از روی ماتریس اولیه):

برای مقایسه صحت الگوریتم، به سراغ تعریف بنیادین و اصلی ژاکارد می‌رویم. با استفاده از فرمول اصلی روی مجموعه‌های اولیه (ردیف‌های فعال) بررسی را انجام می‌دهیم:

▪ مجموعه D_1 : اعضای $\{3, 2, 0\}$

▪ مجموعه D_3 : اعضای $\{3, 2\}$

▪ اشتراک یا $|D_1 \cap D_3|$: عناصر مشترک $\{3, 2\} \rightarrow$ تعداد اعضا = 2

▪ اجتماع یا $|D_1 \cup D_3|$: تمام عناصر بدون تکرار $\{3, 2, 0\} \rightarrow$ تعداد کل اعضا = 3

$$J_{real}(D_1, D_3) = \frac{|D_1 \cap D_3|}{|D_1 \cup D_3|} = \frac{2}{3} \approx 0.667$$

بخش 5: جمع‌بندی نهایی

نتیجه‌گیری و اثبات کارایی:

همان‌طور که نتایج ریاضی نشان می‌دهد، $J_{approx} = J_{real} = 2/3$. در این سناریوی خاص آموزشی، الگوریتم **MinHash** با کاهش ابعاد مسئله از 5 (تعداد کل شینگل‌ها) به تنها 3 ویژگی فشرده، توانسته است شباهت ساختاری ژاکارد را با دقت 100% و بدون هیچ‌گونه خطای تقریب یا تلفات اطلاعاتی استخراج کند.

در سیستم‌های مقیاس‌عظیم (**Large-Scale Systems**) در دنیای واقعی وب، با وجود میلیون‌ها کلمه و شینگل متمایز، با استفاده از تنها حدود 100 تا 200 تابع هش مستقل، شباهت تقریبی محاسبه‌شده با خطای بسیار ناچیز و قابل قبولی به شباهت واقعی همگرا می‌شود و بار محاسباتی سرورها را به صورت نمایی کاهش می‌دهد.